

Gilbert Strang 《Linear Algebra and Learning From Data》part004 双语翻译

范围：书页 147–197（OCR 页 151–200，共 50 页）。§II.4 Randomized Linear Algebra 收尾（随机采样 / 随机投影 / 四步分解）→ Part III Low Rank and Compressed Sensing（§III.1 扰动下 A^{-1} 的变化与 Sherman–Morrison–Woodbury → §III.2 特征值交错与低秩信号 → §III.3 快速衰减的奇异值 → §III.4 分裂算法 ADMM / prox / 软阈值 → §III.5 压缩感知与矩阵补全）。格式：行内双语（中文主体 + English term（中文翻译）行内嵌入），习题题干保留英文原文；公式行内 ...、块级 $\mathcal{M}$ 围栏（无 $\mathcal{M}$ ）。页映射：page_151–160 = 书页 147–156（偏移 −4）；page_161 = 书页 157（Part III 章首标题页）；page_162 = 书页 159（书页 158 为空白背面，未入扫描）；page_163–200 = 书页 160–197（偏移 −3）。映射由 17 个印页码锚点实测确定。本合并文件由 50 个分页文件拼合而成，每页以 <!-- page NNN --> 注释起始，页面以 --- 分隔；单页原文见 pages/page_NNN.bilingual.md。

本页为书页 147，属 §II.4 Randomized Linear Algebra（随机化线性代数）正文；页眉与页码为重复元素，不译。

[section]

4. Randomized Linear Algebra（随机化线性代数）

[note] 译者注（OCR 修正与术语准备）：本页 OCR 把矩阵 A 误识为 盖、风，把 from 误识为 什 om，把求和号 $\sum$ 误识为 丕，把下标 k 误识为 θ ，把 s_k^2 误识为 sz，把 s 误识为 5，把 $v = (a, b)$ 误识为 = @, b)；Our two-sample experiment was ... 一句句末无法辨认，按上下文补出。式 (2) 按“一列乘一行”的语境重建为分块外积，编号与原文一致。此外请先记住三个全文通用的词：**thin（薄）**指矩阵的行数远多于列数，形如“高而窄”的长条；**rank-one（秩一）**指形如 uv^T 的矩阵，它的每一列都是同一个向量 u 的倍数，所以整张矩阵的信息量只相当于两个向量；**preconditioner（预条件子）**指一个容易求逆的辅助矩阵 M ，用来把难解方程组 $Ax = b$ 换成“条件数更好”的 $M^{-1}Ax = M^{-1}b$ ，使迭代法收敛更快。这三个词在本章后面会被反复使用，读者现在只需建立直觉，不必掌握构造方法。

[section]

1. Column-row sampling（列-行采样）

S 的每一列都只有一个非零元，记作 s_k 。于是 AS 直接从 A 中挑出若干个单独的列 a_k ，而 S^TB 挑出 B 中相应的行 b_k^T 。用 AS 乘以 S^TB ，得到的是以数 s_k^2 加权的**列-行乘积（column-row products）** $a_k b_k^T$ 之和。

其中一个乘积 $s_k^2 a_k b_k^T$ 来自 AS 的一列与 S^TB 的一行：

$$\begin{bmatrix} \cdot & a_k & \cdot \end{bmatrix} \begin{bmatrix} 0 \\ s_k \\ 0 \end{bmatrix} = s_k a_k \qquad \begin{bmatrix} 0 & s_k & 0 \end{bmatrix} \begin{bmatrix} \cdot \\ b_k^T \\ \cdot \end{bmatrix} = s_k b_k^T \tag{2}$$

把这些合在一起， AB 就被*随机抽取的 s 个秩一矩阵（rank-one matrices）*的加权和 $CR = \sum_k s_k^2 a_k b_k^T$ 所逼近。选取是随机的，但权重由我们选择。

[note] 译者注（为什么“一列乘一行”能还原整个矩阵乘积）：这一段的的全部内容其实只是矩阵乘法的一种写法，但它值得慢慢拆开，因为整章的低秩思想都从这里长出来。设 A 是 $m \times n$ 矩阵， B 是 $n \times p$ 矩阵，把 A 按列分块写成 $A = [a_1 \cdots a_n]$ ，把 B 按行分块写成 $B = [b_1^T; \cdots; b_n^T]$ （这里 b_k^T 表示 B 的第 k 行，写成行向量）。标准定义 $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ 可以逐项重排为
先回答它在解决什么设计问题：教科书通常先给逐元素定义 $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ ，这个定义只告诉我们“怎么算”，却完全没有暴露“结果往哪里去”。而这一章后面要做的是秩、列空间、低秩近似、最小二乘，这些问题的答案都藏在“列与行”的语言里，不在“下标与求和”的语言里。所以必须把同一个乘积换一副骨架来讲，让 AB 的每一列、每一行的来源可见。
从第一性原理推：乘积的第 j 列是 $(AB)_{\cdot j}$ ，它的第 i 个分量是 $\sum_k A_{ik} B_{kj}$ 。把求和下标 k 挪到外面，就得到 $(AB)_{\cdot j} = \sum_k B_{kj} a_k$ 。这句话的含义是： **AB 的第 j 列，就是 A 的诸列以 B 的第 j 列为系数的一次线性组合**。同理，第 i 行是 A 的第 i 行与 B 的诸行的线性组合。把这两件事同时按 k 拆开，就得到矩阵层面的分解： $AB = \sum_{k=1}^n a_k b_k^T$ 。右边每一项 $a_k b_k^T$ 是 $m \times p$ 的外积，它的秩至多为 1（所有列都是 a_k 的倍数）；左边是 n 个秩一矩阵之和。这一步换来的东西极其值钱：矩阵乘法被拆成“ n 个秩一块，逐个叠加”。

用最小的例子验算， 2×2 就够。取 $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ ， $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ 。按逐元素定义算 AB 的第 (1,2) 元： $1 \times 6 + 2 \times 8 = 22$ 。再按外积分解算： $a_1 b_1^T = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 5 & 6 \end{bmatrix} = \begin{bmatrix} 5 & 6 \\ 15 & 18 \end{bmatrix}$ ， $a_2 b_2^T = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \begin{bmatrix} 7 & 8 \end{bmatrix} = \begin{bmatrix} 14 & 16 \\ 28 & 32 \end{bmatrix}$ ，相加得 $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$ ，第 (1,2) 元正是 22，与定义完全一致。你可以只用两步手算把它复算一遍：先算两对“列乘行”，再逐元素相加；然后再用标准定义算同一个位置，看到两条路汇到同一个数，这个分解就不再是记忆负担，而是一个可检验的事实。

跨课程连接：在 6.042J 离散数学里，你学过线性映射与基的对应，也学过等价关系把一个大集合划分成若干类；这里恰好是同一件事——换一组“生成元”（由“列”生成，而不是由“逐元素”生成）就换了描述方式，但映射本身没变。更直接的连接在 CLRS 与 6.006：分治把一个大问题拆成若干可独立处理的小块再合并，而 $AB = \sum_k a_k b_k^T$ 就是“把一次笨重的乘法拆成 n 个极简的秩一操作再求和”，只不过这里的“小块”之间有天然的并行性（互不依赖，可以乱序相加）。与 Strang 线性代数、18.065 的连接最要紧：后面讲 SVD 时会说 $A = \sum_k \sigma_k u_k v_k^T$ ，那正是把外积分解“按重要性排序”之后的版本，区别在于本注的分解中每一项权重为 1 且不保证 $a_k \perp a_l$ ，而 SVD 要求左右两组向量各自正交、并按 σ_k 从大到小排列，因此 SVD 的截断才

有最优性保证。与在学的概率论（Bertsekas）也有一个干净的连接： $a_k b_k^T$ 的“大小”可以用 Frobenius 范数衡量，而 $\|a_k b_k^T\|_F = \|a_k\|_2 \|b_k\|_2$ ，于是 $\|AB\|_F^2$ 不存在简单的逐项平方和（交叉项一般不为零），只有当所有 a_k 两两正交时才有 $\|AB\|_F^2 = \sum_k \|a_k\|_2^2 \|b_k\|_2^2$ ——这与概率论里“和的方差等于方差之和，仅当不相关时”的直觉完全同构。

命名模板：可以把这套动作叫**秩一外积累加法**（分四步：按列切 A 、按行切 B 、逐对做外积、求和后与原定义对表）。配套有一个**能量检验法**：把 $\|AB\|_F^2$ 按 k 分组逐项估计，若某一项的 $\|a_k\|_2 \|b_k\|_2$ 远小于其他项，说明这一列在这一乘积中几乎不贡献能量，这正是后面低秩近似“留哪些、砍哪些”的判据来源。

$$AB = \sum_{k=1}^n a_k b_k^T.$$

右端每一项 $a_k b_k^T$ 是 $m \times 1$ 的列向量乘 $1 \times p$ 的行向量，得到一个 $m \times p$ 矩阵，恰好与 AB 同型， n 项相加正好还原 AB 。这就是外积展开（outer product expansion）：把一次大型矩阵乘法换成 n 次“小得多的”秩一乘法之和。它在计算上的意义是：如果这 n 项里只有少数几项的系数很大，其余很小，就可以只保留少数几项，用 $CR = \sum_{k \in \mathcal{K}} s_k^2 a_k b_k^T$ 近似 AB ，存储量从 $mn + np$ 降到 $|\mathcal{K}|(m + p)$ 。这正是本章“低秩逼近”的最朴素版本。读者可以动手验算一个 2×2 的例子： $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ ， $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ ，则 $AB = a_1 b_1^T + a_2 b_2^T = \begin{bmatrix} 5 & 6 \\ 15 & 18 \end{bmatrix} + \begin{bmatrix} 14 & 16 \\ 28 & 32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$ ，与直接相乘一致。

再说 S 的角色。 S 是 $n \times s$ 的“采样矩阵”，它并非任意矩阵，而是每一列恰有一个非零元、其余全为零。这种结构使它在乘法中只起“选择器”的作用： AS 的每一列恰好是 A 的某一行乘上那个非零元， $S^T B$ 的每一行恰好是 B 的某一行乘上一个非零元。于是 $(AS)(S^T B) = \sum_k (s_k a_{c(k)}) (s_k b_{c(k)})^T = \sum_k s_k^2 a_{c(k)} b_{c(k)}^T$ ，权重自动变成平方 s_k^2 ，而不是 s_k ——这一点最容易抄错，务必记住“两边各乘一次”。式 (2) 的画法用零把“被选中”的位置空出来，正说明一个采样列只与一个采样行配对，其余位置在相乘时贡献为零。

最后要提醒一个容易误解的地方： $\sum_k s_k^2 a_k b_k^T$ 与 AB 一般不相等， CR 只是 AB 的一个**随机估计**。它的价值在于“期望正确”：若第 k 列被选中的概率为 p_k ，并取 $s_k = 1/\sqrt{s p_k}$ （下文将给出），则 $E[CR] = AB$ ，即无偏（unbiased）。这就是随机化线性代数区别于普通近似（如截断 SVD 的有偏低秩逼近）的关键：它不是丢掉小奇异值，而是用随机抽样的方式，让“平均而言”还原真值。至于抽样带来的偏差有多大，则由方差控制，本章后续会给出集中不等式。这与概率论中“用样本均值估计总体均值”完全同构，读者在《概率论与数理统计》课上学到的“无偏估计”“估计的方差”正是这套语言的原型。

[section]

2. Sampling by random projections（用随机投影采样）

矩阵 S 仍然是 thin（薄）的，所以 AS 的列数远少于 A 的列数。此时 S 的各列含有多个非零元，于是 AS 在把列空间 $C(A)$ 投影到更低维时混合了各列。这样一来， $AS S^T B$ 就是对 AB 更均匀的逼近。

我们从随机采样（random sampling）开始，稍后再描述随机投影（random projections）。后者能为原始矩阵 AB 产生快速而有用的预条件子（preconditioner），并且可能降低图聚类（graph clustering）的计算代价。

[note] 译者注（“选择列”与“混合列”是两种不同的随机化）：这一节只有两段话，却包含一个容易被跳过的重要对照。第 1 小节的 S 每列只有一个非零元， AS 做的事情是**挑出若干整列**，列空间 $C(AS)$ 是 $C(A)$ 的一个**子空间**——它能张成的方向绝不会超出 A 原有的列方向。第 2 小节的 S 每列允许有多个非零元（例如各元素独立取 ± 1 再除以 $\sqrt{s}$ 的随机矩阵）， AS 做的事情是**线性组合**原有的列： AS 的每一列都是 A 若干列的混合，于是 $C(AS)$ 一般**不再是** $C(A)$ 的子空间，而是一个“旋转”过的低维子空间，它把 A 的列空间压缩到更低维度，同时尽量不丢掉任何方向的能量。这个区别就是“随机采样（random sampling）”与“随机投影（random projections）”的分水岭，也是本页标题从“1.”过渡到“2.”的原因。

它在解决什么设计问题：当 A 的列数 n 极大（例如 $n = 10^6$ ），我们想用一个小矩阵去“概括” A ，使得后面所有的计算（QR、SVD、最小二乘）只在小矩阵上做。问题在于“概括”有两种本质不同的做法——挑现成的列，还是造新的列。必须把这两者分开，因为它们的误差来源、可实现性和适用条件完全不同：挑列受制于“ A 里是否已经存在足够好的列”，造列则把选择权交给随机性本身。

从第一性原理展开。 S 是一个 $n \times s$ 的矩阵， AS 的每一列是 A 与 S 的一列相乘，即 A 各列的线性组合，系数就是 S 那一列。若 S 的每一列只有一个 1，那么 AS 的每一列就是 A 的某一列的复制（可能带符号）， AS 的列集合是 A 列集合的子集，因此 $C(AS) \subseteq C(A)$ 是集合意义上的包含关系，秩 $\text{rank}(AS) \leq \text{rank}(A)$ 且 AS 的元素全部直接来自 A （这在需要“可解释性”时是优点：小矩阵的每一列都能指回原数据）。若 S 的每列有多个非零元，新的列 $\sum_k S_{kj} a_k$ 一般落在 $C(A)$ 内却不在原有列集合中， $C(AS)$ 是 $C(A)$ 里一个由 S 的列所决定的 s 维子空间；此时“包含关系”失效，取而代之的是“距离关系”——我们要的是 $C(AS)$ 到 $C(A)$ 的**投影误差**小，而不是 $C(AS)$ 被 $C(A)$ 容纳。把这两句话并排写出来，就是本页标题分段的原因。

一个可手算的 2 维小例子能把区别摆到桌面上。取 $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$ ，它的列空间 $C(A)$ 是 $(x, y, 0)$ 这张平面。第 1 小节令 $S_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ，则 $AS_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ ，这是 A 的第一列，仍在 $C(A)$ 内，是一条一维子空间。第 2 小节令 $S_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ （混合两列），则 $AS_2 = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}$ ，方向是“对角线”，它依然在 $C(A)$ 内但已不是任何一列的原方向，能量 $\|AS_2\|_2^2 = 2$ 与 $\|AS_1\|_2^2 = 1$ 不同——混合把两列的“长度”合并利用了。如果你把 S_2 换成 $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 之外的某个更一般的矩阵，例如让 A 为 3×2 而 S 为 2×1 且元素为 $\pm 1/\sqrt{1}$ ，你会发现同一个 $s = 1$ 的预算下，混合得到的子空间方向可以落在 $C(A)$ 里任意位置，而选择只能落在两列之一——这就是“旋转过的低维子空间”在最小例子上的样子。

跨课程连接：在 6.042J 离散数学里， S 每列只有一个 1 的情形等价于在列指标集合上取一个子集（并可以看成从 A 的列集合到一个更小集合的满射），而混合列则对应一个真正的线性映射，二者在“是选择还是组合”上有本质差别；同样的语言在 CSAPP 里对应“直接映射缓存”与“全相联缓存”：前者只能命中已有行，后者靠计算把数据放到新位置。与在学的概率论（Bertsekas）连接更紧：选择列时 $\mathbb{E}[AS] = A$ （抽到第 k 列的概率为 p_k 就加权 $1/p_k$ ），方差只涉及单个 $\|a_k\|^2$ ；混合列时估计量仍是 A 的无偏估计，方差却由多个 $\|a_k\|$ 的协方差与 ± 1 的独立性共同决定，因此“把别人的信息匀过来”正是在降低方差。与 CLRS、6.006 的连接在于随机化算法的复杂度论证方式：采样要证“抽到好列的概率”，投影要证“随机方向几乎不可能与任何固定方向对齐”（这正是 Johnson–Lindenstrauss 引理用随机投影而非随机采样的原因）。与 Strang 线性代数、18.065 的连接是：选择列给出的是 A 的一个列子矩阵 $A_{:, \Omega}$ ，混合列给出的是一般的 AS ，后者的 QR 或 SVD 才需要按正交化流程重算；区别在于前者零空间结构被原样继承，后者零空间被 S 换掉。

命名模板：这套对照可以叫**子空间归属检验法**（三问：新空间是旧空间的子集吗？误差来源是“缺好列”还是“方向对齐”？零空间是被继承还是被替换？）。配套一个**预算对照法**：固定 s ，分别记录选择与混合的投影误差 $\|A - AS(AS)^+A\|_F$ ，谁小就用谁——这一步把两段话里的定性区别变成了可复算的数字比较。

为什么混合会比挑选更“均匀”？可以这样建立直觉。若 A 的列长度差异极大，纯挑选的策略会被少数长列主导：抽中短列时补不回来，抽不中长列时误差很大，估计的方差因此很高。而随机投影让每一列都以相近的权重参与进来，相当于在压缩维度之前先把各个方向“拌匀”。在概率论里这类似于“简单随机抽样”与“分层抽样 / 加权抽样”的差别：前者简单但方差大，后者用权重或结构换取更小的方差，代价是需要事先知道 p_j 或做更多计算。注意这里的“投影”是字面意义的： S 的列若近似正交归一，则 SS^T 近似于到某个 s 维子空间的正交投影矩阵。

术语上还有三点需要读者提前分清，否则读英文文献时极易混淆。第一， $C(A)$ 是 **column space (列空间)**，即 A 的所有列的线性组合构成的集合，也叫值域 (range)，不要与“行空间”混用； $C(A)$ 的维数就是 A 的秩。第二，**preconditioner (预条件子) 不是用来求精确解的，而是用来加速迭代**：把 $Ax = b$ 改写为 $M^{-1}Ax = M^{-1}b$ 后，迭代法（如共轭梯度法）收敛速度主要由 $M^{-1}A$ 的条件数决定，一个便宜且“接近 A^{-1} ”的 M 就能显著减少迭代步数；随机投影之所以有用，是因为它能在 $O(ns)$ 级别的代价下造出这样的 M 。第三，**graph clustering (图聚类)**指把图的顶点分成若干组，使组内连接紧密、组间连接稀疏；它的代数形式依赖于图的邻接矩阵或拉普拉斯矩阵的低秩结构，所以低秩逼近与随机化方法能直接降低这一任务的计算代价。这句话原文只是“may reduce the computational cost”，中文也保留“可能”，不宜写成确定结论。

跨课程的连接可以精确地记成这样一条路径：《线性代数》中“列空间—秩—线性组合”的概念 → 把列空间用随机矩阵压缩到低维 → 在低维空间里做乘法与分解 → 把结果映回原空间得到 AB 的近似。与《数值分析》里的“矩阵分解 + 误差分析”相比，这里的误差不再是确定性的界，而是**高概率的界** (with high probability)，这是随机数值线性代数最本质的方法论转变：接受“极小概率失败”，换取数量级更快的算法。

[section]

Practice with Computing Mean and Variance (计算均值与方差的练习)

下面是一个非常简化的采样问题。这里不从矩阵出发，而是从一个向量 $v = (a, b)$ 出发。我们要对它采样两次（2 次独立试验），然后计算均值 m 与方差 σ^2 。第 V.1 节 (Section V.1) 将给出关于 m 与 σ^2 的更多例子。

第一次采样：以概率 $\frac{1}{2}$ 与 $\frac{1}{2}$ 选择 $(a, 0)$ 或 $(0, b)$ 。第二次采样：完全重复一次。然后把两个样本相加，得到 (x_1, x_2) 。

计算均值 $m = E[(x_1, x_2)]$ = **期望值 (expected value)** = **输出的平均值** (x_1, x_2) 。

第一种方式：样本 1 的平均值是 $\frac{1}{2}(a, 0) + \frac{1}{2}(0, b) = \frac{1}{2}(a, b)$ 。我们有两个 independent identical trials (独立同分布试验)，也就是两个样本，把它们的均值相加：

$$\text{Overall mean } m = E[(x_1, x_2)] = \frac{1}{2}(a, b) + \frac{1}{2}(a, b) = (a, b)$$

我们这次双样本试验是**无偏的 (unbiased)**，所求的均值确实达到了。

第二种方式：该试验有下面 4 种结果，每种的概率都是 $\frac{1}{4}$ ：

$$(a, 0) + (a, 0) = (2a, 0) \quad (a, 0) + (0, b) = (a, b) = (0, b) + (a, 0) \quad (0, b) + (0, b) = (0, 2b)$$

均值就是全部四个输出按其概率加权之和（四个概率都是 $\frac{1}{4}$ ）：

$$\text{Overall mean } m = \frac{1}{4}(2a, 0) + \frac{1}{4}(a, b) + \frac{1}{4}(a, b) + \frac{1}{4}(0, 2b) = (a, b) \text{ as before}$$

[note] 译者注（把“期望”算成一次实验，而不是一句口号）：这一段是全书最朴素的一页，却值得逐字落实，因为“无偏”这个说法只有在算过一遍之后才真的可信。先把记号说清楚： $v = (a, b)$ 是我们“采样”的目标向量， x 是一次完整试验的输出，即两次独立试验之和 (x_1, x_2) ，它是**随机向量**（取值随机）； $m = E[x]$ 是期望值，即把所有可能输出按概率加权求和； σ^2 是方差，本页只用到它的一个等价形式 $\sigma^2 = E[\|x - m\|^2] = E[\|x\|^2] - \|m\|^2$ ，下一页会用两种算法各算一遍来验证这个等式。注意这里的 $\|x\|^2$ 就是把向量各分量平方再相加，例如 $\|(a, -b)\|^2 = a^2 + b^2$ ，它与“逐元素平方”的写法 $(a, -b)^2$ 是同一件事的两种记号，本译文统一用 $\|\cdot\|^2$ ，以免读者误以为是对向量做逐项平方后再配对。

先问机制：这段话在设计上要解决什么问题？随机化算法每跑一次给出的答案都不一样，我们无法承诺“某一次的输出等于 v ”，只能承诺“平均起来等于 v ”。于是“正确性”这个词被换掉了：从逐次相等，换成期望意义上的相等。这一步不是修辞，而是把概率论里的期望当作一份**验收合同**：只要每个可能的输出都按它出现的概率被计一次账，加权和必须正好落在目标上。为什么必须这样做？因为随机化算法唯一能被证明的东西就是分布，而期望是分布最粗、也最容易计算的一个数字特征；先把一阶矩钉死，后面的方差、集中不等式才有立足点。换句话说，本页负责“不偏”，下一页负责“有多散”，顺序不能颠倒。

把它算成一次实验。取 $a = b = 1$ ，于是每次试验等概率地输出 $(1, 0)$ 或 $(0, 1)$ ，两次试验相互独立。四种组合各占 1/4：两次都得 $(1, 0)$ 时 $x = (2, 0)$ ；一次 $(1, 0)$ 一次 $(0, 1)$ 时得到 $(1, 1)$ ；两次都得 $(0, 1)$ 时得到 $(0, 2)$ 。于是 $E[x] = \frac{1}{4}(2, 0) + \frac{1}{4}(1, 1) + \frac{1}{4}(1, 1) + \frac{1}{4}(0, 2) = (1, 1) = v$ 。因为 x_1, x_2 独立同分布且 $E[x_1] = E[x_2] = \frac{1}{2}v$ ，所以 $E[x] = E[x_1] + E[x_2] = v$ ，与上面的穷举完全对上。注意这里还有一个常被忽略的细节：**算期望的过程根本不需要独立性**。把 x_1 与 x_2 的联合分布摊开来加权求和，无论相关系数是多少，期望的线性性都成立。独立性是下一节用来控制方差、让误差缩小的，不是用来保证无偏的；把这两件事分开，才能明白为什么“无偏”比“误差小”更容易成立。

顺手验算方差那一半。仍取 $a = b = 1$ ，只需算 x 这一个随机向量。四个等概率取值 $(2, 0), (1, 1), (1, 1), (0, 2)$ ，逐个算 $\|x\|^2$ 得 4, 2, 2, 4，平均为 3；而 $\|m\|^2 = \|(1, 1)\|^2 = 2$ 。按 $\sigma^2 = E[\|x\|^2] - \|m\|^2$ 得 $3 - 2 = 1$ 。再直接按定义算一遍： $\|x - m\|^2$ 分别为 1, 0, 0, 1，逐项平均同样是 1。两条路径给出同一个数，这就是下一页“两种算法等价”的最小实例，你自己动手就能复现。

跨课程连接。其一，在学的概率论 (Bertsekas) 里，期望是对样本空间按概率加权求和，所以“ E 可以逐项拆开”与协方差、相关系数用的是同一套线性运算；区别在于概率论处理的是标量随机变量，而这里处理的是取值在向量空间的随机向量，点积承担了“逐个分量相乘再相加”的角色。其二，Strang 线性代数里 m 扮演投影的角色——把所有可能输出按概率权重平均，正是把随机向量投影到“必然的那一份”上，与最小二乘中把观测投影到列空间在思想上同源，区别是这里的权重来自概率而不

是几何距离。其三，CSAPP 讲浮点数时强调表示有舍入，程序里用大量样本累加求平均得到的**经验均值**与理论 m 之间同时含两种差：随机涨落和舍入累积；这提醒我们公式描述的是精确算术，程序跑的是近似算术。

命名模板：期望线性拆项法。 三步走：先把“和与数乘再取期望”拆成“分别取期望再组合”，再代入具体数值穷举一遍看零项项与一阶项是否对消，最后才去算方差。凡是见到线性算子外面套着 E ，先拆再算，能省掉一半的分支讨论。

两种算法为什么必须给出同一个数？这是期望的**线性性**（linearity of expectation）在起作用： $E[x_1 + x_2] = E[x_1] + E[x_2]$ ，无论 x_1, x_2 是否独立都成立。第一种方式是先用线性性把两次试验分开算，再相加；第二种方式是先枚举两次试验的全部 $2 \times 2 = 4$ 种组合，再用联合概率加权。第一种方式解释了“为什么”简单（分离后每次只需算一个二点分布），第二种方式说明了“是什么”（四种组合各自的概率都是 $\frac{1}{4}$ ，因为两次试验独立且各为 $\frac{1}{2}$ ）。这正对应《概率论与数理统计》中“期望的线性性”与“独立试验的联合分布”两条基本结论；两者的等价性不是巧合，而是同一个数学事实的两种展开顺序。

这里还有一个更深的伏笔，读第二遍时值得回头体会：为什么偏偏取 $\frac{1}{2}$ 和 $\frac{1}{2}$ ？因为 $(a, 0)$ 与 $(0, b)$ 各取一半，恰好补齐了对方缺失的那个坐标，所以均值精确地回到 (a, b) 。换言之，这个“两坐标互补”的采样方案天然就是无偏的，不需要额外的缩放因子。但它的代价体现在方差上：输出离均值并不近。下一页会算出 $\sigma^2 = \frac{1}{2}(a^2, b^2)$ ，并指出当 a, b 大小悬殊时，可以通过**偏向大样本**的抽样概率把方差压下去，这正是矩阵情形下“按概率 p_j 抽样、再乘 $1/\sqrt{p_j}$ 缩放”的雏形。简言之，这一页用两个数字 a, b 预演了后面 $s \ll n$ 次抽样的全部逻辑：**先保证期望正确，再想办法缩小方差。**

本页为书页 148，是 §II.4 Randomized Linear Algebra（随机化线性代数）正文的续页；页眉（Computations with Large Matrices）与页码为重复元素，不译。

[note] 译者注（OCR 修正与记号约定）： 本页 OCR 把 A 误识为 盖、风，把下标误识为 θ ，把方差记号 σ^2 误识为 伊 2；...useful... 前后的词形无法辨认，按上下文补出。两种算法的输出取 $x_1 + x_2$ ，其四种结果为 $(2a, 0), (a, b), (a, b), (0, 2b)$ ，均值是 (a, b) ，故方差为 $\frac{1}{2}(a^2, b^2)$ ；原文用逐分量平方的写法表示长度平方，译文统一写成 $\| \cdot \|^2$ 并保留分量形式。 AS 的示例依 $k(1) = 1, k(2) = 3$ 重建为 $A = [a_1 \ a_2 \ a_3]$ 、 S 为 3×2 。式 (3) 的编号与原文一致。

方差 σ^2 等于从各个输出到均值的距离平方的加权平均。 记输出为 x 、均值为 m ，我们将用两种等价的方式来计算

$$\sigma^2 = E[\|x - m\|^2] = E[\|x\|^2] - \|m\|^2.$$

第一种方式： 把每个输出的 $(\text{outputs} - \text{mean})^2$ 按其概率 $\frac{1}{4}$ 加权相加：

$$\begin{aligned} & \frac{1}{4}[(2a, 0) - (a, b)]^2 + \frac{1}{4}[(a, b) - (a, b)]^2 + \frac{1}{4}[(a, b) - (a, b)]^2 + \frac{1}{4}[(0, 2b) - (a, b)]^2 = \\ & \frac{1}{4}(a^2, b^2) + \frac{1}{4}(0, 0) + \frac{1}{4}(0, 0) + \frac{1}{4}(a^2, b^2) = \frac{1}{2}(a^2, b^2) \end{aligned}$$

第二种方式： 把每个输出的 $(\text{outputs})^2$ 按其概率加权相加，再减去 $(\text{mean})^2$ ：

$$\begin{aligned} \sigma^2 &= \frac{1}{4}(2a, 0)^2 + \frac{1}{4}(a, b)^2 + \frac{1}{4}(a, b)^2 + \frac{1}{4}(0, 2b)^2 - (a, b)^2 \\ &= \left(a^2 + \frac{a^2}{4} + \frac{a^2}{4} + 0 - a^2, 0 + \frac{b^2}{4} + \frac{b^2}{4} + b^2 - b^2 \right) = \frac{1}{2}(a^2, b^2) \end{aligned} \tag{3}$$

[note] 译者注（两种算法为什么等价，以及“输出”到底是什么）： 这一段唯一的数学要点是恒等式 $\sigma^2 = E[\|x - m\|^2] = E[\|x\|^2] - \|m\|^2$ ，它是“方差 = 平方的均值减均值的平方”在向量情形的翻版，值得亲手推一遍。展开 $\|x - m\|^2 = (x - m) \cdot (x - m) = \|x\|^2 - 2x \cdot m + \|m\|^2$ ，再取期望，并用期望的线性性与 $E[x] = m$ ，中间项变成 $-2m \cdot m = -2\|m\|^2$ ，于是右边的 $+\|m\|^2$ 与它抵消，只剩下 $E[\|x\|^2] - \|m\|^2$ 。整段推导只用到两条性质：期望的线性性，以及**点积关于期望可以逐项处理**（因为点积是各分量的乘积之和，而期望对加法与数乘都线性）。这与《概率论与数理统计》中一维公式 $D(X) = E(X^2) - [E(X)]^2$ 完全同构，只不过把标量换成了向量、把平方换成 $\| \cdot \|^2$ 。

先问机制：为什么要专门造这条恒等式？ 方差原本的定义是围绕均值的平均偏离 $\sigma^2 = E[\|x - m\|^2]$ ，它要求先把 m 求出来，再对每个样本做一次减法、求一次范数。减去一个向量既费一次存储又费一次遍历；而对于随机化算法，我们常常是边跑边把输出累加起来的，手头只有 $\|x\|^2$ 的累加和与 x 的累加和（后者除以次数就是 m ）。恒等式的作用就是把“先中心化再量散度”改成“先量长度再扣掉中心”，让方差可以在一次遍历里用两个累加器算出来。这不是数学游戏，而是**为下一节的两种算法铺路**：一条路老老实实按定义走，另一条路按展开式走，两者必须给出同一个数，恒等式就是它们的接头处。同时它还回答了一个概念问题： $\|m\|$ 衡量的是“平均输出离原点多远”， σ 衡量的是“各次输出围绕平均输出有多散”，两者是坐标原点视角与质心视角的换算，差值正好是质心到原点的距离平方。

两步手算验算。 取一个只取两个值的随机向量，等概率地取 $(3, 0)$ 与 $(1, 0)$ 。那么 $m = (2, 0)$ ， $\|m\|^2 = 4$ 。按定义： $\|x - m\|^2$ 分别为 1 和 1，平均得 $\sigma^2 = 1$ 。按展开式： $\|x\|^2$ 分别为 9 和 1，平均得 $E[\|x\|^2] = 5$ ，再减去 $\|m\|^2 = 4$ ，同样得 1。两条路一致。再把上一段那个 2×2 的例子接上：四个等概率取值 $(2, 2), (2, 0), (0, 2), (2, -2)$ 给出 $E[\|x\|^2] = 6$ 、 $\|m\|^2 = 4$ ，于是 $\sigma^2 = 2$ ，与按定义逐项平均的结果吻合。附带提醒一个高频笔误： $(3, 0)^2$ 这种逐元素平方的写法在向量上并没有被定义成标量，参与运算的必须是 $\|(3, 0)\|^2 = 9$ ，否则量纲与数值都会错。

“输出”到底是什么？ 这里的 x 是一次试验产生的完整结果，它是随机向量而不是一个数，取值来自算法内部的随机源； m 是把它对所有可能取值按概率加权后的结果，是一个确定的向量。说“两种算法等价”，指的是同一份输出序列在两个视角下被统计，而不是说算法真的吐出了两个不同的随机对象。用概率论的话说，不同算法看到的是同一个随机向量的不同实现路径；这也解释了为什么随机投影、随机化数值线性代数、机器学习里的随机梯度与蒙特卡洛估计都在反复使用同一个套路：盯住一阶矩，再压二阶矩。

跨课程连接。 其一，在学的概率论（Bertsekas）给出的一维版本 $D(X) = E(X^2) - [E(X)]^2$ 和这里的向量版本是同一个证明换了符号，区别只在标量乘法被点积替换、平方被范数平方替换。其二，Strang 线性代数里 $\|x\|^2 = x \cdot x$ 与 $\|m\|^2 = m \cdot m$ 都是内积，整条恒等式就是把内积按期望的线性性展开后重新分组，与最小二乘里“总平方和 = 被解释部分 + 残差部分”的分解同源，区别是那里的分解来自正交投影，这里的分解来自期望的线性性。其三，CSAPP 的浮点部分会告诉你，这两条路在实数上恒等，在有限精度下却不是同一个算法：当 $E[\|x\|^2]$ 与 $\|m\|^2$ 都很大而两者相近时，相减会出现灾难性抵消；数据量大、均值远离原点时，先平方再相减的写法会丢有效位，工程上反而常用在线更新或先中心化的写法。这正是“数学等价不等于实现等价”的标准案例，也是后面比较两种算法时必须提到的一层。

命名模板：期望展开消项法。 三步走：把范数平方展开成三项，用期望的线性性分别取期望，再把中间项代入 $E[x] = m$ 与 $+\|m\|^2$ 对消。一个附属的验收习惯是：算完立刻代一组小数值复算，同时确认自己用的是范数平方而不是逐元素平方。

接下来必须把“输出”这个词落到实处，否则两种算法的数字对不上。这里的输出是两次试验之和 $x = x_1 + x_2$ ：第一次试验取 $(a, 0)$ 或 $(0, b)$ ，第二次独立地再取一次，所以四种输出依次是 $(a, 0) + (a, 0) = (2a, 0)$ 、 $(a, 0) + (0, b) = (a, b)$ 、 $(0, b) + (a, 0) = (a, b)$ 、 $(0, b) + (0, b) = (0, 2b)$ ，每种概率 $\frac{1}{4}$ 。于是第一种方式里的四个偏差是 $(2a, 0) - (a, b) = (a, -b)$ 、 $(a, b) - (a, b) = (0, 0)$ 、 $(0, 0)$ 、 $(0, 2b) - (a, b) = (-a, b)$ ，长度平方都是 $a^2 + b^2$ ，只有两个非零项，加权得 $\frac{1}{4}(a^2, b^2) + \frac{1}{4}(a^2, b^2) = \frac{1}{2}(a^2, b^2)$ ，与译文中的式子一致（中间两个零项保留了，以显示“四输出等权”的结构）。第二种方式里，四个输出的长度平方分别是 $(4a^2, 0)$ 、 (a^2, b^2) 、 (a^2, b^2) 、 $(0, 4b^2)$ ，加权求是 $\frac{1}{4}(4a^2, 0) + \frac{1}{2}(a^2, b^2) + \frac{1}{4}(0, 4b^2) = (a^2, b^2) + \frac{1}{2}(a^2, b^2)$ ，再减去 $\|m\|^2 = (a^2, b^2)$ ，同样得到 $\frac{1}{2}(a^2, b^2)$ 。两条路线给出同一个数，这正是恒等式的验证，而不是两个不同的答案。

最后提一句这个练习的用意。 $\sigma^2 = \frac{1}{2}(a^2, b^2)$ 表明：当 a, b 量级悬殊时，方差被较大的那个数主导，而它在每次试验中只有一半概率被抽到。于是下一页的观察就顺理成章：把更大的概率分给大样本，让“重要的东西经常出现”，方差就能压下来。这与《概率论与数理统计》中的**重要性采样（importance sampling）**是**同一个思想：按与被估量大小相称的概率抽样，再用 $1/p$ 加权校正，期望不变而方差变小**。矩阵情形下这套逻辑原封不动地搬过去，只是 p_j 要按列的长度（准确地说

是列范数的平方）来分配——这正是下一节“给 A 的每一列指定概率 p_j ，并乘以 $1/\sqrt{sp_j}$ ”的来源。

Observation（观察）： 若 b 大于 a ，我们可以保持正确的均值 (a, b) ，并通过给较大的样本 $(0, b)$ 以更大的被选概率来减小方差 σ^2 。矩阵采样（matrix sampling）就会这样做（见本节 §II.4 末的 Problem 7）。

本页对 $n = 2$ 个数 a, b 用了 $s = 2$ 次试验。这没有用处，一点时间也没省下。接下来的几页对一个有 n 列的矩阵采用 $s \ll n$ 次试验。 AB 的均值仍然保持正确。

[note] 译者注（从“两个数”到“n 列的矩阵”，这一步跨在哪里）： 这两小段是本章的枢纽：前面所有内容都只是用两个数字 a, b 做的玩具实验，从这里开始，同样的逻辑要被搬到真正的问题上。搬运时有一件事**结构上完全相同**，另一件事**必须重新设计**。完全相同的是“无偏”目标：每一次抽样仍然要让期望等于真值，矩阵情形下就是 $E[CR] = AB$ ，这样多次运行取平均就能逼近 AB 。必须重新设计的是“概率怎么分配”与“抽中的东西要乘多少倍”：标量情形下两个坐标各占一半，是因为它们恰好互补； n 列的情形没有这种巧合，必须主动指定概率 p_j ，并用缩放因子 $1/\sqrt{sp_j}$ 把“按不均匀概率抽样”造成的失真校正回来。判断缩放因子对不对，只需做一次期望计算：第 j 列被抽中的概率是 p_j ，抽中时贡献 $\frac{1}{sp}a_jb_j^T$ ，故该项的期望是 $\frac{1}{sp}a_jb_j^T$ 。 $p_j = \frac{1}{s}a_jb_j^T$ ； s 次抽样各贡献 $\frac{1}{s}a_jb_j^T$ ，加起来正好是 $\sum_j a_jb_j^T = AB$ 。这说明 $1/\sqrt{sp_j}$ 里的平方根不是装饰：两边各乘一次，平方后恰好抵消 p_j 。先把“为什么要分配概率”从第一性原理讲清楚。标量实验里， a 与 b 只有两个坐标，把概率取成 $1/2$ 与 $1/2$ ，既满足 $\sum_j p_j = 1$ ，又让两个坐标平权，任何偏向都无从产生。一旦列数变成 n ，这个“没得选”的局面就消失了： $\mathbb{R}^m$ 中的单位向量是连续无穷多个，列与列之间也没有任何对称性——第一列可能长度是最后一列的一千倍。若还硬性等概率，则被抽中的短列贡献极小，长列又长期不出场，估计量的方差就会被少数大列支配。所以“概率如何分配”不是锦上添花的调参，而是决定这个估计器能不能用的自由度： p_j 大则必然多抽、方差小， p_j 小则偶尔抽到一次、一旦抽到就用巨大的权把它补回来。而 $\sum_j p_j = 1$ 这条唯一约束意味着你不可能让所有 p_j 都变大，这就是机制层面的取舍。再具体验算一次，体会“无偏”到底是什么意思。取 $s = 2, n = 2, p_1 = p_2 = 1/2, A = [a_1 \ a_2]$ 为 2×2 。设两次抽样恰好都抽中第 2 列（概率 $1/4$ ），此时 $CR = \frac{1}{sp_2}a_2b_2^T = \frac{1}{2(1/2)}a_2b_2^T = a_2b_2^T$ ，只是 $AB = a_1b_1^T + a_2b_2^T$ 的一半，单次误差肉眼可见地大。但若第一次抽中第 1 列、第二次抽中第 2 列（概率 $1/4$ ），则 $CR = a_1b_1^T + a_2b_2^T = AB$ ，分毫不差。把四种等概率结果求平均，再乘各自概率相加，得 $\frac{1}{4}(2a_1b_1^T) + \frac{1}{4}(2a_2b_2^T) + \frac{1}{4}(AB) + \frac{1}{4}(AB) = AB$ ，无偏性就在这一步里被验完了。跨课程连接：这条“抽样—缩放—期望复原”的路径与 6.042J 离散数学中的**等价关系与等价类**同源。那里把一个被测对象按“对结论是否等价”粗粒化，然后只数等价类的个数；这里是把 n 个列指标按概率 p_j 粗粒化成相对较少的抽样次数，再只算被抽中的那些列。连接路径是“把求和 $\sum_j$ 换成加权平均 $\mathbb{E}$ ”，区别在于 6.042J 的等价类是确定划分、计数精确，而这里每个 p_j 只保证期望精确、单次结果仍抖动。与**在学的概率论（Bertsekas）中“随机变量期望的线性性”**连接更直接： $\mathbb{E}\left[\sum_j X_j\right] = \sum_j \mathbb{E}[X_j]$ **不要求独立性，所以即便每次抽样结果彼此相关，无偏性也照样成立；区别在于概率论只保证期望，而这里还要为“抽样矩阵 S 的列独立”额外付出的假设，才把方差分析的档位再降一阶。给这套判断起个可复用的名字：概率—缩放配合法。它的配方只有一句话：每引入一个 $\frac{1}{p_j}$ ，就必须在贡献里配一次 p_j 把它抵消；若缩放因子带着平方根 $1/\sqrt{sp_j}$ ，那就说明它在 A 和 B 两侧各出现一次。用于复查时可套用无偏核三问：贡献的期望是否等于目标项？ $\sum_j p_j$ 是否为 1？ s 次累加是否给出 $\frac{1}{s}\sum_j a_jb_j^T$ ？这三问能挡住绝大部分把 $\sqrt{sp_j}$ 抄成 sp_j 的错误。**

再理解 $s \ll n$ 的含义。它说的是抽样次数远小于列数，例如 $n = 10^6$ 而 $s = 100$ 。此时 AS 只有 100 列， AB 的“真值”运算量是 mnp 量级，而 CR 的运算量约为 $s(m + p)$ 量级，差距是成千上万倍。代价是 CR 只是估计，需要用第 4 小节的集中不等式把“以高概率误差不超过 ϵ ”说清楚。所以本章的方法论可以概括为一句：**用方差换时间，用高概率保证替代最坏情况保证**。这也是随机化算法与确定性算法（如精确 SVD）在《数值分析》课程中的根本分野：前者给出概率性的复杂度与精度，后者给出最坏情况的界。

最后提醒读者注意本节的编号与引用位置。原文说“见本节末的 Problem 7”，指的是 §II.4 的习题部分（见页末引用行所标的 Strang §II.4, p. 148），而不是第 7 节的某个结论；本节习题保留了英文题干，便于读者掌握术语的原文表述。

[section]

Random Matrix Multiplication with the Correct Mean AB（均值 AB 正确的随机矩阵乘法）

$n \times s$ 的采样矩阵 S 将含有 s 列， S 的每一列只有一个非零元。对于 S 的第 j 列，那个非零元所在的位置是随机的。若随机选中的是行 $k = k(j)$ ，则 S 中第 k 行、第 j 列处的非零元记作 s_{kj} 。采样得到的矩阵就是 AS ：

AS 的第 1 到第 s 列，分别等于 A 的第 $k(1)$ 到第 $k(s)$ 列乘以相应的数 s_{kj} 。

下面是一个 $s = 2$ 次试验的例子：它从 A 中采样第 $k(1) = 1$ 列与第 $k(2) = 3$ 列。

$$AS = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} s_{11} & 0 \\ 0 & 0 \\ 0 & s_{32} \end{bmatrix} = \begin{bmatrix} s_{11}a_1 & s_{32}a_3 \end{bmatrix}$$

关键问题是：**我们如何选取这些数 s_{kj} ？答案是：它们来自概率！** 我们打算做随机采样，所以必须以随机的方式选取 A 的那 s 列（让每一列都有机会）。在随机乘法（random multiplication）中：

1. 给 A 的所有 n 列指定概率 p_j ，满足 $p_1 + \cdots + p_n = 1$ 。**2.** 有放回地（with replacement）选取 s 列，即同一列可以被选到不止一次。**3.** 一旦 A 的第 k 列被选中（连同 B 的第 k 行），就把二者都乘以 $1/\sqrt{sp_k}$ 。**4.** 于是 $(A \text{ 的第 } k \text{ 列})(B \text{ 的第 } k \text{ 行})/sp_k$ 就进入我们对 AB 的随机乘积。

[note] 译者注（把“随机乘法”拆成可执行的三步）： 这一节把前面的原则变成了一条可以照着写代码的流程，读的时候请把三个符号的“身份”固定下来： n 是 A 的列数（也是 B 的行数，因为 AB 要能相乘）， s 是抽样次数（也是 S 的列数）， p_j 是第 j 列被选中的概率。 S 的形状是 $n \times s$ ，不是 $s \times n$ ，这一点常被抄错： S 必须“左乘”在 A 的右边，才能用它的每一列去选择 A 的一列。第 j 列中那个唯一的非零元 s_{kj} 就是第 3 步里的缩放因子 $1/\sqrt{sp_k}$ （这里 $k = k(j)$ 就是第 j 次抽样选中的列号），因此行标 $k(j)$ 与列号 $c(j)$ 指的是同一个随机结果（本节前文用 $c(j)$ 表示被选中的列号，后文用 $k(j)$ 表示非零元所在的行号，本译文保留了原文的两种写法）。先说清楚这三步在解决什么设计问题。朴素地算 AB 是 $m \times n$ 乘 $n \times p$ ，代价 $O(mnp)$ ，每一列都要与 B 的每一行做完整的向量点积；当 n 大而 $s \ll n$ 时，真正昂贵的是“ n 次”这个循环。三步流程的设计意图就是把“精确求和”换成“随机抽样求和”： $C = AS$ 只用 s 列与 B 打交道，于是代价退到 $O(mns)$ 之外的抽样代价，从第一性原理看，这是用“允许误差”买“少算列”的一笔交易，而无偏性只是保证这笔交易不带系统性偏差、不会朝一个方向稳定地骗你。也正因为如此， S 只能是“每列一个非零元”的稀疏选择矩阵：如果允许一列里放两个非零元， $C = AS$ 就会产生原矩阵两列的加权和，那已经不再是“抽一列”这个动作了。用一个能两步手算的例子把三步走完。取 $n = 3$ 、 $s = 2$ 、 $p = (1/2, 1/4, 1/4)$ ，则缩放因子分别是 $1/\sqrt{2 \cdot 1/2} = 1$ 、 $1/\sqrt{2 \cdot 1/4} = 1/\sqrt{1/2} = \sqrt{2}$ 。同样 $1/\sqrt{2} = \sqrt{2}$ 。若抽签结果是第 1 次选中第 2 列、第 2 次也选中第 2 列，即 $k(1) = k(2) = 2$ ，那么 S 的第 1 列在第 2 行放 $\sqrt{2}$ ，第 2 列也在第 2 行放 $\sqrt{2}$ ， $S = \begin{pmatrix} 0 & 0 \\ \sqrt{2} & \sqrt{2} \\ 0 & 0 \end{pmatrix}$ 。第 1 步 $C = AS$ 得两列都是 $\sqrt{2}a_2$ ；第 2 步 $R = S^T B$ 得两行都是 $\sqrt{2}b_2^T$ ；第 3 步 $CR = (\sqrt{2}a_2)(\sqrt{2}b_2^T) = 2a_2b_2^T$ 。这正好等于 $\frac{1}{sp_2}a_2b_2^T = \frac{1}{2 \cdot 1/4}a_2b_2^T = 2a_2b_2^T$ ，与上一节的缩放公式严丝合缝，也是对“平方根两边各乘一次”的一次实物对照。顺手再给一个**抽样矩阵自检**模板：把 S 的各列加总， $1^T S$ 的第 j 个分量应当恰为 $1/\sqrt{sp_k}$ （其中 $k = k(j)$ ）。上例中 $1^T S = (\sqrt{2}, \sqrt{2})$ ，而 $1/\sqrt{sp_2} = \sqrt{2}$ ，通过。反过来，若写成 $1/(sp_j)$ 就会得到 2 而不是 $\sqrt{2}$ ，这个自检能立刻暴露抄错。至于 C 与 R 的形状也要跟着锁死： C 是 $m \times s$ (m 行、 s 列)， R 是 $s \times p$ ，两者相乘才回到 $m \times p$ ，与 AB 同型。跨课程连接：这条“把 n 列压到 s 列”的思路与 **Strang 线性代数**里的 **C(A)** 列空间观点同源—— C 的列空间是 A 列空间的一个随机低维子空间，而 R 负责把 B 的行投影进去。连接路径是“抽列 = 从 A 中选基”，区别在于 Strang 的基是按消元法确定性选出来的，而这里是按 p_j 随机选、并且要用 $\sqrt{sp_j}$ 把子空间外的分量补回来。与 **CLRS 与 6.006** 中的分治矩阵乘法（Strassen）也形成对照：Strassen 通过减少乘法次数、不改变被乘对象；这里则通过少看几列、允许随机误差来省时间，两者降低的复杂度维度完全不同。再往算法一侧看，它与数据流算法里估计频率矩的采样技巧同属于“一次遍历 + 抽样估计”的家族，区别是数据流算法算是逐坐标标量，而这里是整块 $a_j b_j^T$ 的外积贡献。

三步流程的每一步都对应一个必须想清楚的问题。第一步“指定概率”：最常用的选择是 $p_j = \|a_j\|^2 / \|A\|_F^2$ ，即按列的长度平方分配概率——列越长、信息越多，被抽中的机会越大；这样选出来的估计方差最小（在只利用列长度信息的前提下）。第二步“有放回”意味着 S 的同一列可能被选中两次甚至更多次，于是 AS 中可能出现重复的列；这不是缺陷，而是让每一步抽样都独立同分布的必要条件，从而使期望的可加性能逐项使用。第三步的缩放 $1/\sqrt{sp_j}$ 是整套方法的核心：前面已经算过，它能保证 $E[CR] = AB$ 。请特别注意这里的 s 出现在根号内：如果写成 $1/p_j$ 或 $1/sp_j$ （不取平方根）就会破坏无偏性，误差会随 s 的增大而系统性地放大或缩小。

关于矩阵例子的读法：示例取 $s = 2$ 、 $A = [a_1 \ a_2 \ a_3]$ （即 $n = 3$ ），抽中的列号是 $k(1) = 1$ 与 $k(2) = 3$ ，因此 S 的第一列在第 1 行放 s_{11} 、第二列在第 3 行放 s_{32} ，其余位置为零，乘出来的 AS 恰好是两列 $[s_{11}a_1 \ s_{32}a_3]$ 。整个例子的意义在于直观展示“ S 是选择器”：它把 A 的 n 列压缩成 s 列，且存下来的每一列都带着自己的缩放系数。真正使用时要把它与 B 配合： $(AS)(S^T B) = \sum_{j=1}^s s_{k(j)j}^2 a_{k(j)} b_{k(j)}^T$ ，其中 $s_{k(j)j}^2 = 1/(sp_{k(j)})$ ，这正是本节末尾那句话“ $(A \text{ 的第 } k \text{ 列})(B \text{ 的第 } k \text{ 行})/sp_k$ 进入随机乘积”所指的项——注意分式中的 sp_k 是整个缩放因子的平方，与第 3 步的 $1/\sqrt{sp_k}$ 并不矛盾。

最后把它放到课程地图上。这条流程是《概率论与数理统计》中“按概率抽样 + 缩放校正 + 期望可加”三件事在矩阵上的应用，也是《数据结构与算法》中“蒙特卡洛算法”思想的数值线性代数版本：算法只保证以高概率得到好结果，运行时间是随机变量。与通常的确定性低秩逼近（截断 SVD，取最大的 k 个奇异值）相比，随机抽样不需要先算完整的 SVD，代价从 $O(mn^2)$ 级别的分解降到 $O(mns)$ 级别的两次乘法，代价是结果只以高概率接近最优。读者若想动手验证，可以取 A 为 3×2 矩阵、 $s = 2$ 、 p_j 取列长平方比，重复抽样一千次并平均 CR ，会看到平均值稳定地逼近 AB ，而单次结果在 AB 附近波动——波动幅度就是下一节要量化的方差。

Strang §11.4, p. 148

本页为书页 149，属 §III.4 Randomized Linear Algebra（随机化线性代数）；页眉（含节号 4 与印页码 149）为重复元素，不译。

[note] 译者注（OCR 校正）： 本页原扫描件把斜体字母 s 一律识别成数字 5，故 There are 5 identical trials、AB/s 中的 5 均还原为 s ；SP9 还原为 sp_j ，1/SP 还原为 $1/(sp_j)$ ；the “ by matrix SST 依上下文补全为 $n \times n$ 矩阵 SS^T ；ASSTB ÅB 还原为 $E[AS S^T B] = AB$ ；内 t0 Pn 还原为 p_1 到 p_n ；况 eq ba 历 I 涵 还原为 norm-squared sampling（范数平方采样）；mxing rnaü•ix 还原为 mixing matrix（混合矩阵） M ；范数符号 $\| \cdot \|$ 、 $\| \cdot \|_1$ 均还原为 $\| \cdot \|$ 。式 (4)(5) 的编号与原文一致，未作改动。

有待验证的结论（conclusion to be verified）： $n \times n$ 矩阵 SS^T 的期望值是 I 。

同一个结论换一种说法： $AS S^T B$ 的期望值是 AB 。

因此，随机采样算出的矩阵乘积具有正确的均值 AB 。

证明 (Proof)。一共有 s 次完全相同的试验 (identical trials)。每次试验都以概率 p_1 到 p_n 选取 A 与 B 的一个列-行对 (column-row pair)，即取 A 的某一列乘以 B 的对应行，再除以 $\sqrt{sp_j}^2 = sp_j$ ，其中各概率之和为 $\sum_j p_j = 1$ 。于是期望值 = 均值 = 每次试验结果的平均值，它等于

$$p_1 \frac{(\text{column 1 of } A)(\text{row 1 of } B)}{sp_1} + \cdots + p_n \frac{(\text{column } n \text{ of } A)(\text{row } n \text{ of } B)}{sp_n}.$$
 (4)

把上式按 j 求和即 $\sum_{j=1}^n p_j \frac{a_j b_j}{sp_j} = \frac{1}{s} \sum_{j=1}^n a_j b_j = \frac{AB}{s}$ 。这些 p 相互抵消，结果恰好是 AB/s 。由于总共有 s 次试验，随机化乘法 $(AS)(S^T B)$ 的期望值就是 AB 。

结论 (Conclusion)。到此为止一切顺利——但我们必须选出概率 p_1 到 p_n 。任何一组（加起来等于 1 的）选择都给出正确的期望值 AB （也就是均值）。但概率 p 的选取方式会强烈影响方差 (variance)！

均匀采样 (uniform sampling) 会选取相等的概率 $p = 1/n$ 。当 A 的各列（以及 B 的各行）长度相近时，这样做是合理的。但假如 (A 的第 1 列)(B 的第 1 行) 构成了 AB 的绝大部分——它压倒了其余的列-行外积 (outer products)，那么我们就不要在随机选取时把它漏掉。

我们这里的做法是采用不相等的概率 (unequal probabilities) p_j 。下面我们陈述并算出最好的 p 。同时还要提一个完全不同的选项：引入混合矩阵 (mixing matrix) M ，改为对 AM 与 $M^{-1}B$ 做计算，然后对 A 随机混合后的列与 B 的相应行采用相等的概率 p_j 。

[note] 译者注 (采样概率为何按长度乘积分配)： 这一页的核心贡献是：把抽样概率选成 p_j 正比于 $\|a_j\| \|b_j\|$ ，就能让方差最小。为什么是这个量、而不是别的量，值得先说清楚。抽样式矩阵乘法是一个**无偏估计**：每次试验随机取一个秩一项 $a_j b_j$ (a_j 是 A 的第 j 列， b_j 是 B 的第 j 行)，估计量是 $\hat{X} = a_j b_j / (sp_j)$ ，于是 $E[\hat{X}] = \sum_j p_j \cdot a_j b_j / (sp_j) = AB/s$ ，与式 (4) 完全一致。既然是估计，就同时带着均值与方差两个指标——这也正是概率论里估计量的评价框架：无偏只保证平均意义上对，方差才决定单次结果有多飘。

现在从第一性原理把方差写出来。单个秩一项的 Frobenius 范数平方是 $\|a_j b_j\|_F^2 = \|a_j\|^2 \|b_j\|^2$ ，把它记成 c_j ，就是第 j 项的能量；被选中时它要除以 sp_j 才放回去，所以被放大成 $c_j / (s^2 p_j^2)$ ，再乘上选中概率 p_j 得 $c_j / (s^2 p_j)$ 。对 j 求和就是 $E[\|\hat{X}\|_F^2] = \frac{1}{s^2} \sum_j c_j / p_j$ 。要注意这里比较的不是方差本身，而是式 (6) 里的 $\sum_j c_j / p_j$ ：两者只差一个由 $E[\hat{X}] = AB/s$ 决定的常数 $\|AB\|_F^2$ 与因子 $1/s^2$ ，所以在 AB 给定的前提下最小化哪一个都等价。于是设计问题变成一个朴素的直觉问题：既然分母上的 p_j 越小、这一项被放大得越狠，那就该让概率去追能量——但为什么要追 $\sqrt{c_j}$ 而不是 c_j ？因为目标里是 c_j / p_j ，把 p_j 记作 t_j^2 ，目标就是 $\sum_j (\sqrt{c_j} / t_j)^2$ ，按柯西-施瓦茨取 $t_j \propto \sqrt{c_j}$ 才能同时压住所有项，若取 $p_j \propto c_j$ 则小项被压得太狠、方差反而变大。能量单位是长度平方，而概率必须是**长度的量纲**（一维的权重），开一次方正是量纲匹配这一步；只取 $\|a_j\|$ 会漏掉行侧的信息，只取 $\|b_j\|$ 同理，所以必须是乘积。

用 2×2 验算一遍，绝对值不重要、比例才重要。取 $A = \begin{pmatrix} 3 & 0 \\ 0 & 4 \end{pmatrix}$ ， $B = I_2$ ，两列的能量分别是 $c_1 = 9$ 、 $c_2 = 16$ ，长度乘积是 3 与 4，于是 $p_1 = 3/7$ 、 $p_2 = 4/7$ 。样本数取 $s = 1$ 时，方差（即式 (6) 的 $\sum_j c_j / p_j$ 再减去常数 25）等于 $9 \times \frac{7}{3} + 16 \times \frac{7}{4} - 25 = 21 + 28 - 25 = 24$ ；若改用均匀采样 $p = (1/2, 1/2)$ ，同样代入得 $18 + 32 - 25 = 25$ ，比最优解大。若取 $p_1 = 3/7, p_2 = 4/7$ 但 $s = 2$ ，误差指标降到 12，说明方差按 $1/s$ 衰减，代价则是计算量翻倍。

这个设计也有名字可记，叫**能量归一化采样**：先算每项的能量，开方后归一化当概率、去掉方根后在估计量里除回来，能量大的项被更频繁地选中、同时被更温和地放大，两个方向把误差夹住。一条跨课程连接：**6.042J 的 Chernoff 界**用的是独立同分布随机变量的和偏离期望的概率上界，它的舞台是单个标量，而这里是矩阵值随机变量的 Frobenius 误差，处理的是二阶矩而非尾部概率；**CLRS 与 6.006 的随机化算法分析**多数情况下可以只算期望代价就收工，而矩阵乘法的采样必须看方差，因为方差直接决定需要抽多少次。前者关心会不会算得太慢，这里关心答案准不准，这是同一套概率工具在两种算法范式下的不同用法。

按 p_j 抽样，本质上属于重要性采样 (importance sampling)：当被抽样项对最终结果的贡献大小悬殊时，就按贡献大小分配抽样次数，贡献大的多抽、贡献小的少抽，从而在固定的 s 次试验里把估计量的波动压下去。下面把贡献量化： $AB = \sum_j a_j b_j$ 是 n 个秩一项之和，而秩一项 $a_j b_j$ 的 Frobenius 范数平方恰好等于 $\|a_j\|^2 \|b_j\|^2$ ，所以第 j 项的能量就是 $c_j = \|a_j\|^2 \|b_j\|^2$ 。于是方差正比于 $\sum_j c_j / p_j$ ：概率给得越小，那一项在方差里被放大得越厉害。这解释了为什么最优解不是把概率全押在最大的项上——那是贪心策略，会让其余项以极小的 p_j 换来巨大的 c_j / p_j ，方差反而爆炸；正确做法是按 $\sqrt{c_j}$ 配平。

跨课程连接一：概率论与数理统计里的无偏估计与最小方差估计，连接路径是把随机化算法看成估计量 $\rightarrow$ 无偏性对应 $E[\hat{X}] = AB \rightarrow$ 有效性（方差最小）对应下面对 p_j 的优化，区别在于统计课通常假定样本独立同分布，而这里每次试验从有限个秩一项中按自定义分布抽取，分布本身还是我们要设计的变量。跨课程连接二：Strang 线性代数与 18.065 中的 SVD 低秩近似，连接路径是 AB 写成秩一项之和 $\rightarrow$ 每项能量 $c_j \rightarrow$ 抽样方差 $\sum_j c_j / p_j$ ；区别在于 SVD 是确定性截断，保留最大的几项、丢掉其余，而抽样近似是随机保留全部项、只是让大项出现得更频繁，因此估计仍然无偏。

[section]

Norm-squared Sampling Minimizes the Variance (范数平方采样使方差最小)

范数平方采样 (norm-squared sampling) 选取的概率 p_j 正比于数 $\|\text{column } j \text{ of } A\| \|\text{row } j \text{ of } B\|$ ，其中 a_j 是 A 的第 j 列， b_j 是 B 的第 j 行。在重要的情形 $B = A^T$ 下， p_j 正比于 $\|\text{column } j \text{ of } A\|^2$ 。这时 norm-squared (范数平方) 或 length-squared (长度平方) 这个名字就很自然。

我们仍然要用一个合适的常数 C 去缩放所有概率，使它们加起来等于 1：

$$p_j = \frac{1}{C} \|\text{column } j \text{ of } A\| \|\text{row } j \text{ of } B\| = \frac{\|a_j\| \|b_j^T\|}{C}, \quad C = \sum_{j=1}^n \|a_j\| \|b_j^T\|.$$
 (5)

Strang 线代与 18.065 里对 x 求偏导得 $A^T A x = A^T b$ 那一套相比, 求导动作完全相同, 差别在于被优化的变量从向量 x 换成了概率分布 p , 于是约束从方程换成了单纯形, 解也就多了一层归一化。最后要防止一个误解: 柯西-施瓦茨给的下界是按每一个 p 各算一次的, 对均匀分布同样成立, 它并不自动说明均匀分布差多少; 真正锁定最优的是它与拉格朗日一阶条件的取等条件重合。

但有个更快、信息更全的做法，值得抽象成模板——**柯西-施瓦茨配平法**：要最小化 $\sum_j c_j/p_j$ 且 $\sum_j p_j = 1$ ，就把 $\sum_j \sqrt{c_j}$ 拆成内积形式

$$\sum_{j=1}^n \sqrt{c_j} = \sum_{j=1}^n \frac{\sqrt{c_j}}{\sqrt{p_j}} \cdot \sqrt{p_j} \Rightarrow \left(\sum_{j=1}^n \sqrt{c_j} \right)^2 \leq \left(\sum_{j=1}^n \frac{c_j}{p_j} \right) \left(\sum_{j=1}^n p_j \right) = \sum_{j=1}^n \frac{c_j}{p_j}.$$

等号当且仅当 $\frac{\sqrt{c_j}}{\sqrt{p_j}} \propto \sqrt{p_j}$, 即 $p_j \propto \sqrt{c_j}$ 。于是最小的 $\sum_j c_j p_j$ 就是 $(\sum_j \sqrt{c_j})^2 = C^2$, 代回式 (6) 立刻得到式 (7) 的最小方差 $\frac{1}{s}(C^2 - \|AB\|_F^2)$ 。柯西-施瓦茨一次给出三样东西：最优值、最优解（等号条件）、以及这确实是下界的证明，比只解驻点更强也更短。两条路的分工可以用一句话说清：拉格朗日乘子法负责必要条件怎么列、约束的价格 λ 是多少，柯西-施瓦茨负责下界存在、且等号能取到。关于全局性， $1/p$ 在 $p > 0$ 上是凸函数（二阶导 $2/p^3 > 0$ ），凸函数配正系数相加仍凸，约束又是线性的，所以驻点就是全局最小点，拉格朗日必要条件足够，不必担心局部极小。

一个可以心算的验算：设三列的 $c = (4, 1, 1)$ ，则 $C = 2 + 1 + 1 = 4$ ， $C^2 = 16$ ， $\sum_j c_j = 6$ 。均匀概率 $p_j = 1/3$ 给出 $\sum_j c_j/p_j = 18$ ；最优概率 $p = (1/2, 1/4, 1/4)$ 给出 $4 \cdot 2 + 1 \cdot 4 + 1 \cdot 4 = 16$ ，恰好等于 C^2 。同一个无偏约束下，方差从 18 压到 16（公共因子 $1/s$ 不影响比较），而这 2 的差距正是柯西-施瓦茨的间隙 $\sum_j c_j/p_j - (\sum_j \sqrt{c_j})^2$ ；用 Jensen 不等式（ $\sqrt{\cdot}$ 是凹函数）也能得到同一个下界，但柯西-施瓦茨额外送出了等号条件，也就是最优解本身，这正是两条路要一起走的原因。

还有一条关于代价的跨课程连接：抽样式矩阵乘法每抽中一个 j ，就形成一项 $a_j b_j$ ，若 A, B 都是 $n \times n$ ，代价是 $O(n^2)$ ，抽 s 次总代价 $O(sn^2)$ ；而 6.006 与 CLRS 里的精确矩阵乘法是 $O(n^3)$ （分治版本 $8T(n/2) + O(n^2)$ 仍是同一个量级）。区别在于分治是精确算法、精度不损失，抽样是近似算法，拿方差换时间；式 (6) 里那个 $\frac{1}{s}$ 还提醒你，方差与 s 成反比而标准差只按 $1/\sqrt{s}$ 收缩，精度想翻倍就得付四倍工作量。最后一点直觉：拉格朗日乘子 λ 可以理解成约束 $\sum_j p_j = 1$ 的价格，度量放松这条约束能给目标带来多少好处，这正是凸优化对偶变量的经济学解释。

下面证明式 (5) 给出的正是使式 (6) 中方差最小的那些概率 p_j (Here is the proof that (5) gives the probabilities p_j that minimize the variance in (6))。把约束 $p_1 + \cdots + p_n = 1$ 乘以一个 Lagrange multiplier (拉格朗日乘子) λ , 再加入到 (6) 中的函数上去。这正是拉格朗日乘子法的关键之处:

$$L(p_1, \dots, p_n, \lambda) = \sum_{j=1}^n \frac{\|a_j\|^2 \|b_j^T\|^2}{s p_j} - \frac{1}{s} \|AB\|_F^2 + \lambda \left(\sum_{j=1}^n p_j - 1 \right)$$

其中 a_j 是 A 的第 j 列, b_j 是 B 的第 j 行 (写成列向量, 因此 $a_j b_j^T$ 是秩一矩阵)。对 p_j 求偏导数 $\partial L / \partial p_j$, 就得到使函数取极小的 p_j (即最优概率):

$$\frac{\partial L}{\partial p_j} = 0 \quad \text{becomes} \quad \frac{1}{s p_j^2} \|a_j\|^2 \|b_j^T\|^2 = \lambda \quad (8)$$

这说明 $p_j = \|a_j\| \|b_j^T\| / \sqrt{s\lambda}$ 。再选取拉格朗日乘子 λ , 使 $\sum p_j = 1$ 。

$$\sum_{j=1}^n p_j = \sum_{j=1}^n \frac{\|a_j\| \|b_j^T\|}{\sqrt{s\lambda}} = 1 \text{ gives } \sqrt{s\lambda} = C \text{ and } p_j = \frac{\|a_j\| \|b_j^T\|}{C} \text{ as predicted in (5).}$$

范数平方采样使用最优概率 p_j , 从而取得最小方差。

Strang §III.4, p.150

本页为书页 151, 属 §II.4 Randomized Linear Algebra (随机化线性代数); 页眉 11.4 Randomized Linear Algebra 与印页码 151 为重复元素, 不译、不收录, 节号按原文标出。

[section]

§II.4 Randomized Linear Algebra (随机化线性代数) (续)

[note] 译者注 (OCR 修正) : 本页 OCR 噪声密集。盖、互、儿 三形都是 A; 月、兕、磊、腕 都是 R, c 兕 即 CR; 彳 是连接因子、混合矩阵 M; 什 0 m、忒、什 都是英文介词 from; CII 月 与 CNR 是 CMR。Norm-squared sampling 与 length-squared sampling 原文以等号相连, 意思是**同一个采样的两个名字**: 按行、列的平方长度赋概率, 故下文两句并入一句译出。六条参考文献按原文照录, 仅把 OCR 拆散的卷期页码还原: SIAM Review 53 (2011) 217–288、Acta Numerica 26 (2017) 95–135、PNAS 104 (2007) no. 51, 20167–20172、arXiv:1608.04481、arXiv:1503.07152、Foundations and Trends in Theoretical Computer Science 10 (2014) 1–157。凡带 arXiv: 的编号都是**论文预印本编号**, 不含页码, 检索时直接把它粘进浏览器或 arXiv 站内搜索框即可, 不要拆成卷号和期号。

对于非常巨大的矩阵，当它们存放在 Random Access Memory（随机存取存储器，RAM）之外时，范数平方采样（norm-squared sampling）可能要求把矩阵读两遍。第一遍计算 A 的每一列与 B 的每一行的平方长度。

[note] 译者注（第一性原理）：为什么非要读两遍？因为式 (5) 那个最优概率 $p_j = \|\text{column } j\| \|\text{row } j\| / C$ 里，分母 C 是**全体**列长度与行长度之和：要确定第一列被抽中的概率，你必须已经知道所有列的信息。这是一次**全局归约**（global reduction）——概率无法边读边定，只能先完整扫一遍数据，把 C 攒出来，第二遍才谈得上抽样。

先看它在解决什么设计问题。矩阵乘法 AB 的代价随行数 m 、列数 n 、内维 k 一起涨，而真实数据矩阵往往几乎全是零（稀疏）或者有效秩很低。于是我们想换一种算法：不逐元素算，而是**随机抽 r 个列/行做外积**，用少量外积的叠加去逼近 AB 。一旦改成抽，问题立刻从算得准不准变成抽得公不公平：如果小列和大列被抽中的机会一样大，那么某次抽到超大列就会把估计值一下抬飞，某次全抽到小列又会把它压扁，误差的方差失控，想要 ϵ 精度就得抽天文数字个 r 。要让方差受控，唯一办法是让抽中 j 的概率正比于这一项对总能量的贡献。

为什么贡献恰好是 $\|\text{column } j\| \|\text{row } j\|$ ？从第一性原理拆：第 j 个外积是 $\text{column}_j \otimes \text{row}_j$ ，它的能量（Frobenius 范数平方）是 $\|\text{column}_j\|^2 \|\text{row}_j\|^2$ ，所以它的长度尺度是二者之积。均匀抽样下，估计量 $\frac{1}{r} \sum_t (\frac{\text{column} \otimes \text{row}}{q_t})_t$ 是无偏的（ E 上恰好还还原 AB ），但方差由 $\sum_j \|\text{column}_j\|^2 \|\text{row}_j\|^2 / q_j$ 控制；对这个和做一次 Cauchy–Schwarz / Jensen 意义上的最小化，取 $q_j \propto \|\text{column}_j\| \|\text{row}_j\|$ 就把它压到最小，归一化常数正好是 C 。也就是说， p_j 不是拼凑出来的启发式，而是无偏 + 方差最小这两个约束联立解出来的唯一答案。

具体算一遍就明白 C 为什么是全局量。取 $A = B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ， 2×2 ， $m = k = n = 2$ 。每列长度 $\|\text{column } j\| = \sqrt{1^2 + 1^2} = \sqrt{2}$ ，每行长度同样 $\|\text{row } j\| = \sqrt{2}$ 。于是

$$\begin{aligned} C &= \sum_{j=1}^2 \|\text{column } j\| \|\text{row } j\| = \sqrt{2} \cdot \sqrt{2} + \sqrt{2} \cdot \sqrt{2} = 4, \\ p_1 &= p_2 = \frac{1}{4} = \frac{1}{C} \end{aligned}$$
第一遍只做归约：算每列、每行的长度并累加成 C ，不做乘法；第二遍才按配额抽签，用累积分布找出随机数落在哪一列，把该外积除以 p_j 累加。读两遍不是实现偷懒，而是结构必然：分母依赖全体数据，任何单遍流式算法都拿不到它。先算验证无偏性。全 1 矩阵下两个外积完全相同， $AB = 2X_j$ ；取 $r=1$ ，抽中哪列的概率都是 $1/2$ ，估计量恒等于 AB ，方差为零——方差量的是外积之间的差异，不是抽样次数。换成 $A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$ 、 $B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ：两项能量为 9 与 1 ， $C=4$ ， $p_1=3/4$ 、 $p_2=1/4$ ； $r=1$ 时最优配额的方差是 $9/(3/4) + 1/(1/4) - 10 = 6$ ，均匀抽样是 $9/(1/2) + 1/(1/2) - 10 = 10$ 。方差降四成，代价是先扫完整个矩阵才知道 p_1 不是 $1/2$ 。跨课程连接：概率论（Bertsekas）里，无偏只保证平均对，要变成大概率接近还得把方差送进不等式—— r 次平均的切比雪夫界给出 $P(\|\hat{M} - AB\|_F \geq \epsilon) \leq \frac{\sigma^2}{r \epsilon^2}$ ，故 r 与 $\frac{\sigma^2}{\epsilon^2}$ 同量级；6.042J 离散数学的 Chernoff 界把同一思路做成指数衰减。区别：切比雪夫只要二阶矩、界松；Chernoff 要求独立有界、界紧但条件强。再往实现走一层：CSAPP 的内存层次里，两遍扫描要把矩阵从主存读两遍——第一遍顺序流式几乎跑满带宽，第二遍按抽中的列号跳着取值，列主序时一连连续、缓存友好，行主序则几乎每次冷启动；想省这笔访问税，只能放弃精确配额、用近似分数一遍读完，代价是方差略涨。Strang 与 18.065 的 SVD 低秩截断走另一条路：一次算完、确定性最优，却要先付满 $O(mnk)$ 。命名模板：能量配额两遍抽样法。(i) 第一遍归约出 p_j 与 C ；(ii) 第二遍按配额抽样、缩放累加；(iii) 用方差反推 r 。凡遇重要性抽样，先问权重是不是能量、常数是不是全局量。
$$C = \sum_{j=1}^2 \|\text{column } j\| \|\text{row } j\| = (\sqrt{2} \cdot \sqrt{2}) + (\sqrt{2} \cdot \sqrt{2}) = 4$$

代入式 (5)： $p_1 = p_2 = \frac{\sqrt{2} \cdot \sqrt{2}}{4} = \frac{2}{4} = 0.5$ ，两列各半，合法（和为 1）。注意：只要还没读完第二列， $\|\text{column } 2\|$ 就还没进 C ， p_1 就定不下来。你可以做个反例验算：矩阵换成第一列 $(100, 0)^T$ 、第二列 $(0, 1)^T$ ，则两列长度是 100 与 1， $C \approx 100$

抽样的目标是无偏：每抽到列 j ，就按 $1/(sp_j)$ 放大取样结果，期望上恰好还原未被抽中列的全部贡献，于是各项乘上自身概率再求和等于原矩阵。方差由每项权重的平方乘概率求和给出，所以权重越大的项越要把它的方差压下去，办法恰恰是让它更容易被抽中，即 p_j 正比于该列范数与行范数之积。这解释了两件事：分母必须精确，否则权重的期望会整体偏掉；全局归约不是实现偷懒，而是权重已知这个概念本身要求的。

一个可以自己验算的小例子：设 $s = 1$ 、 $n = 2$ ，两列范数分别为 2 与 1、行范数都为 1，则 $C = 2 + 1 = 3$ ， $p_1 = 2/3$ 。只抽一列时，抽中第一列的估计为 $1/(1 \cdot 2/3) = 1.5$ 倍，抽中第二列的估计为 3 倍。反复抽样后估计的期望等于真值，但它围绕真值的跳动明显大于均匀抽样 $p_1 = p_2 = 1/2$ 的情形——这就是概率选择的取舍：换来更小的方差，代价是必须先把 C 算准。

放进 CSAPP（《深入理解计算机系统》）的存储层次里算笔账。若 A 是 16000×16000 的 double 矩阵，那是 $16000^2 \times 8 \text{ B} \approx 2 \text{ GB}$ ，还塞得进内存；若 A 是 $10^5 \times 10^5$ ，就是 $8 \times 10^{10} \text{ B} = 80 \text{ GB}$ ，远超内存，只能躺在 SSD 上。第一遍是**顺序流式扫描**：每读进一个 64 字节的 cache line 就拿到 8 个 double，平方后累加进一个长度为 n 的累加器（额外空间 $O(n)$ ，对 10^5 而言只有 0.8 MB，可常驻缓存）；这一遍读写 $\Theta(mn)$ 个元素，对预取器与页粒度极其友好。第二遍才要随机取回被抽中的 s 列与 s 行：每列 m 个 double，取 $s = 2000$ 列约 1.6 GB，压成 $m \times s$ 的矩阵后，终于小到能进快速内存反复使用。这正是 CSAPP 的两条原则：**顺序访问远便宜于随机访问**，以及**把反复使用的数据搬进近处存储**。用 6.006 的语言说，这不是内存模型下一次 $\Theta(mn)$ 就够的算法，而是一个两遍的外部存储/流式算法，第一遍的 $\Theta(mn)$ 次 I/O 决定它是 I/O-bound 的。

把它命名为**全局权重必多一遍**：凡是抽样概率依赖全体数据的算法，都躲不开一次全局扫描；能躲开的是概率事先已知的算法——比如均匀采样 $p_j = 1/n$ ，单遍即可完成。于是本节反复出现的取舍在这里第一次露头：**均匀采样便宜但方差大，范数平方采样方差最小但要多读一遍**。这也预告了后文 single-pass 算法（每个元素只访问一次）为什么要付出病态（ill-conditioning）的代价、再用过采样补救：你省掉的不只是时间，还有回头再看一眼的能力，而数值稳定性常常恰恰依赖回头（例如 Gram-Schmidt 要把新列对已有方向重新正交化一次）。

顺带记个量级感受： s 只需 $O(k/\epsilon^2)$ 级别（ k 是想要的秩），第二遍真正读入的数据量约为第一遍的 $s(m+n)/(mn)$ ，当 $m \approx n$ 时近似为 s/n ；取 $s = 2000$ 、 $n = 10^5$ 就是 2%。对随机化算法，这个界是**高概率成立的**——多跑几次取中位数即可，而不是每次运行都保证。两遍的总 I/O 近似就是两倍的一遍，这个方案的合理性全在这里。

随后（在 RAM 内部）求出各个概率 p_j ，并选出 s 列与 s 行用于采样。第二遍则把采样近似 CR 送入快速内存。

[section]

Applications of Randomized Matrix Multiplication (随机化矩阵乘法的应用)

范数平方采样就是长度平方采样 (length-squared sampling) , 它有助于解决数值线性代数中下面这些核心问题:

- 1 插值近似 (interpolative approximation) $A \approx CMR$: 其中 C 与 R 分别取 A 的列与行。
- 2 用一个低秩矩阵 (low rank matrix) 逼近 A 。
- 3 逼近 A 的 SVD (奇异值分解)。

CMR 的目标是从 A 的 k 列与 k 行出发, 给出 A 的一个准确 sketch (草图)。这些列进入 C , 这些行进入 R 。然后一个混合矩阵 (mixing matrix) M 把 C 与 R 连接起来, 产生 $CMR \approx A$ 。其维数分别是 $(m \times k)$ 、 $(k \times k)$ 、 $(k \times n)$, 合起来是 $(m \times n)$ 。如果 A 是稀疏的, 那么 C 与 R 也会是稀疏的, 因为它们直接取自 A 。

注意这种快速乘法 ($C(M(Rv))$)。我们从不显式地去乘 CMR 。

首先要明白: $A \approx CR$ 很可能并不成立。而 A 的 column space (列空间) 会被 C 准确捕捉住 (我们希望如此)。 A 的 row space (行空间) 会被 R 捕捉住。“空间是对的, 但矩阵不对。”在 M 可逆时, 每一个形如 CMR 的矩阵都具有同样良好的列空间与行空间。我们希望选择 M , 使 CMR 接近 A 。我们仍然回避 CUR 这一记号, 把字母 U 留给 SVD。

作为起点, 我要寻找理论上最好的混合矩阵 M 。在采样文献中, 人们已经发展并检验了若干良好的 M 选择。以下是关于随机化线性代数的六篇重要参考文献:

N. Halko, P.-G. Martinsson, and J. A. Tropp, Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53 (2011) 217–288.

R. Kannan and S. Vempala, Randomized algorithms in numerical linear algebra, *Acta Numerica* 26 (2017) 95–135.

E. Liberty, F. Woolfe, P.-G. Martinsson, V. Rokhlin, M. Tygert, Randomized algorithms for the low-rank approximation of matrices, *PNAS* 104 (2007) no. 51, 20167–20172.

M. W. Mahoney, Lecture Notes on Randomized Linear Algebra. arXiv:1608.04481.

P.-G. Martinsson, Compressing rank-structured matrices via randomized sampling, arXiv:1503.07152.

D. P. Woodruff, Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science* 10 (2014) 1–157.

Strang §11.4, p.151

本页为书页 152, 页眉 Computations with Large Matrices 与印页码 152 为重复元素, 不译、不收录; 正文仍属 §11.4 Randomized Linear Algebra。

[section]

Best M in $A \approx CMR$: Frobenius Norm and L^2 Norm ($A \approx CMR$ 中的最佳 M : Frobenius 范数与 L^2 范数)

[note] 译者注 (OCR 修正) : 本页 盖 为 A , 月 为 R , CnR 与 $C \uparrow$ 月 为 CMR , Q_C 、 Q_R 为 Q_C 、 Q_R 。 Q_k 与 $Q_{\{}$ 依零空间 $N(R)$ 与 $N(C^T)$ 的维数对应关系重建为 $Q_R^\perp$ 与 $Q_C^\perp$: C 有 m 行, 故 C^T 的零空间在 $\mathbb{R}^m$ 中, 对应左侧因子; R 有 n 列, 故 R 的零空间在 $\mathbb{R}^n$ 中, 对应右侧因子。式 (10)(11) 的方括号、上下标按数学语境重建; no Ⅲ 是 norm 的 OCR 讹形, 一律译作范数。

我们已知 A 、 C 、 R 。假设 Q_C 含有 C 的列空间的一组标准正交基。那么 $Q_C Q_C^T$ 就是投影到该子空间上的 projection matrix (投影矩阵)。类似地, Q_R 含有 $C(R^T)$ 的一组标准正交基, 而 $Q_R Q_R^T$ 就是投影到该行空间上的投影矩阵。 $Q_R^\perp$ 含有零空间 $N(R)$ 的一组标准正交基, $Q_C^\perp$ 含有 $N(C^T)$ 的一组标准正交基。

按定义, A 到行空间 / 列空间上的投影是 $\hat{A} = Q_C^T A Q_R$ 。这些投影的用途, 是使我们能选择 M 的那些子空间, 与我们无法改变的那些子空间分离开来。这个问题由 Yuji Nakatsukasa (中务裕司) 在牛津的讨论中澄清并解决。在 Frobenius norm (弗罗贝尼乌斯范数) 下, 这个解尤其简洁。

正交矩阵 $\begin{bmatrix} Q_C & Q_C^\perp \end{bmatrix}$ 与 $\begin{bmatrix} Q_R & Q_R^\perp \end{bmatrix}$ 不会改变 $A - CMR$ 的 Frobenius 范数与 L^2 范数。但它们帮助我们看出使这个误差最小的最优 M 。

$$\begin{bmatrix} Q_C & Q_C^\perp \end{bmatrix}^T (A - CMR) \begin{bmatrix} Q_R & Q_R^\perp \end{bmatrix} = \begin{bmatrix} \hat{A} & X \\ Y & Z \end{bmatrix} - \begin{bmatrix} \widehat{CMR} & 0 \\ 0 & 0 \end{bmatrix}.$$

(9)

我们是否应当选择 M , 使得 $\hat{A} = \widehat{CMR}$? 在 Frobenius 范数下, 这就是最优的!

Frobenius 范数分别处理这些矩阵每个分块中的每一个元素。当左上角恰好取对，即 $\widehat{C}M\widehat{R} = \widehat{A}$ 时，误差最小。这个角块是我们（唯一）能控制的问题。我们找到了与 §11.3 和 §1.1 中相同的 M 。

Frobenius norm :
$$\min_M \|A - CMR\|_F = \left\| \begin{bmatrix} 0 & X \\ Y & Z \end{bmatrix} \right\|_F. \tag{10}$$

在 L^2 矩阵范数下，我们或许期待同样的方案也能成功。

[note] 译者注（第一性原理）：这一页的枢纽问题是用哪个范数，因为范数的选择直接决定最优 M 是谁。回到两个范数的定义差别：Frobenius 范数就是把矩阵摊平成 mn 维向量后的欧氏长度 $\|E\|_F^2 = \sum_{i,l} E_{il}^2$ ，它是**逐元素的**； L^2 矩阵范数（也叫谱范数、算子范数）是最大奇异值 $\|E\|_2 = \sigma_1$ ，回答的是单位球被拉得最长的那个方向有多长，是**最坏方向**的量。

先问它在解决什么设计问题。我们事先只承诺以高概率得到一个近似矩阵 $\tilde{A}$ ，但近似这个词是不完整的：同一个误差矩阵 $E = A - \tilde{A}$ ，用不同尺子量出来的数可以差很多，而误差界的形式是 $r = (\text{某个常数}) \times (\text{某个范数}) \div \varepsilon^2$ ——尺子换一把， r 就换一个数量级。所以范数不是记号偏好，它是**预算单位**：你承诺用哪把尺子衡量成功，就决定了要抽多少列行、要花多少算力。这也解释了为什么这一页要单独把范数拎出来讨论，而不是把它塞进附录。

两者的差别有多大？Frobenius 是把 mn 个元素各自平方再求和，天然把误差**平摊**到所有元素上；谱范数只看**一个方向**，它能忽略大量小而分散的误差，却对集中在某个方向上的误差极其敏感。用一个 2×2 的全 1 矩阵 J_2 验算：逐元素平方和是 $1 + 1 + 1 + 1 = 4$ ，故 $\|J_2\|_F = 2$ 。而 $J_2 = \mathbf{1}\mathbf{1}^\top$ 是秩 1，唯一非零奇异值 $\sigma_1 = \|\mathbf{1}\|_2 \|\mathbf{1}\|_2 = \sqrt{2} \cdot \sqrt{2} = 2$ ，所以 $\|J_2\|_2 = 2$ ，两者相等——这不是巧合，凡秩 1 矩阵都满足 $\|E\|_F = \|E\|_2$ 。再取对角矩阵 $E = \text{diag}(1, 1)$ ：同样 $\|E\|_F = \sqrt{1^2 + 1^2} = \sqrt{2}$ ， $\|E\|_2 = 1$ ，比值恰好 $\sqrt{2} = \sqrt{n}$ 。推广到 $n \times n$ 单位阵 $\|I_n\|_F = \sqrt{n}$ 而 $\|I_n\|_2 = 1$ ：**Frobenius 比谱范数最多松 $\sqrt{n}$ 倍**。这个 $\sqrt{n}$ 的代价直接进了抽样复杂度——同一个 ε ，用 Frobenius 记账往往要多抽 $\sqrt{n}$ 倍的列。更妙的是，谱范数面对稀疏结构时反而给出更紧的常数： J_2 的能量集中在四个元素上，谱范数把它全看见了，而 Frobenius 已经在做无意义的平方摊派。

于是可以命名一条**谱夹逼检验法**：拿到一个误差矩阵，先算最大奇异值 σ_1 （用幂迭代几行就能估），再算 Frobenius 长度；如果两者量级接近（例如实测比值 $\ll \sqrt{n}$ ），说明误差是**能量集中型**，用谱范数记账能得到远小的 r ；如果比值接近 $\sqrt{n}$ ，说明误差**弥散**，两把尺子的代价差不多，此时可以只看 Frobenius，省下每次迭代做奇异值分解的开销。反过来，这个比值本身就是一个便宜的诊断量，不需要真的构造矩阵。注意最优 M 的常数形式也要跟着范数变：Frobenius 口径下的上界来自列与行的可分性，而更紧的界要把最大列长 $\times$ 最大行长这类峰值因子引进来——同一个矩阵，用峰值因子记账比用总和和记账小得多，这又一次说明范数决定 M 。

跨课程连接（Strang 线性代数 / 18.065）： $\|E\|_2 = \sigma_1$ 与 $\|E\|_F = (\sum_i \sigma_i^2)^{1/2}$ 是同一组奇异值的不同函数——谱范数是 ℓ_∞ 型（只看最大），Frobenius 是 ℓ_2 型（全体），这条同一谱、不同泛函的线索把 SVD 与抽样复杂度焊在一起；区别在于 SVD 关心如何把误差投影到最优低秩子空间，而这里关心的是**先用哪把尺子给误差定价**，前者是手段、后者是预算口径。第二个连接（6.042J 离散数学）：抽样复杂度对 ε 是 $O(1/\varepsilon^2)$ 、对失败概率 δ 是 $O(\log(1/\delta))$ ，这正是 Chernoff 界的标准形状，因为 r 个独立样本的估计误差服从次高斯型集中不等式；区别在于 Chernoff 处理的是标量和，而这里要和的是矩阵值随机变量，谱范数的作用恰恰是把矩阵和压成一个可控的标量尺度，才能套用同一套集中工具。

两者都被正交变换保持： $\|Q_C^T E Q_R\| = \|E\|$ ，因为左右乘正交矩阵只是换一组标准正交基，不改变任何长度。但只有 Frobenius 范数在分块下**可加**：把误差搬到 $\begin{bmatrix} \hat{A} - CMR & 0 \\ 0 & 0 \end{bmatrix}$ 这样的分块形状后，各块占用互不重叠的坐标，平方和直接相加，于是

$$\|A - CMR\|_F^2 = \|A - \hat{A}\|_F^2 + \|\hat{A} - CMR\|_F^2.$$

第一项动不了（它只由 C 、 R 决定），第二项可以靠选 M 关掉，所以 $CMR = \hat{A}$ 就是 Frobenius 意义下的最优解，这就是式 (10)。而 L^2 范数是 σ_1 ，奇异值不按块逐元素相加，块与块可以共享同一个奇异向量方向从而把 σ_1 叠高，因此角块取零未必最优。

一个可以自己验算的例子：设固定住三个块，只剩角上的标量 x 可调，误差矩阵形如 $M_x = \begin{bmatrix} 2 & 1 \\ 1 & x \end{bmatrix}$ 。取 $x = 0$ （角块归零）时 $M_0^T M_0 = \begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix}$ ，特征值为 $3 \pm 2\sqrt{2}$ ，于是 $\|M_0\|_2 = \sqrt{3 + 2\sqrt{2}} = 1 + \sqrt{2} \approx 2.414$ 。取 $x = -2$ （角块非零、且取负）时 $M_{-2} = \begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix}$ ，两行 $(2, 1)$ 与 $(1, -2)$ 恰好正交且等长 $\sqrt{5}$ ，故 $M_{-2}^T M_{-2} = 5I$ ， $\|M_{-2}\|_2 = \sqrt{5} \approx 2.236 < 2.414$ 。角块取负值反而把两个奇异值**拉平了**——最大值因此下降，这正是正文所说最小 L_2 范数由取 -1 而非取 0 的子矩阵达到的机制。正文该例题印出的数值与 $\|E\|_2 = \sigma_1$ 的定义不吻合（按定义，左边那个矩阵的范数应为 $\sqrt{5}$ 而非 $\sqrt{2}$ ），读者按定义自己算一遍即可；要带走的是机制，不是那两个数字。

抽象成模板——**分块正交对角化加范数可加性判别**：遇到分块矩阵取范数最小的题，先用正交变换把误差搬到分块坐标（这一步对任何正交不变范数都合法），再问一句这个范数在分块下可加吗。可加（Frobenius 这类逐元素范数） $\Rightarrow$ 各块独立处理，把可控块置零即可；不可加（ L_2 /算子范数） $\Rightarrow$ 块间耦合必须显式处理，问题升级为真正的矩阵近似问题，Davis、Kahan 与 Weinberger 在 1982 年解决的正是这个耦合子矩阵的最优选择。注意与 Eckart–Young 的对比：那里 U 、 V 都可自由选取，两种范数给出同一个答案（截断 SVD）；这里 C 、 R 被固定，只剩 M 一个自由度，两个范数立刻分道扬镳。

跨课程上看，上面那步先算 $M_x^T M_x$ 再求特征值正是 Strang 线性代数与 18.065 中对称矩阵谱分解的套路，只是这里要的是最大特征值的平方根，即谱范数 $\|M\|_2 = \sqrt{\lambda_{\max}(M^T M)}$ ，而不是解方程用的特征向量；两个范数在无约束时同解、在固定 C 、 R 时不同解这一转折，则与课程里 $A^T A$ 的最小二乘、SVD 的低秩近似一脉相承——差别在于那里的约束加在秩不超过 k 上，这里的约束加在左右因子必须是 C 与 R 上。

但角块取零并不一定给出最小的 L^2 范数！

[example]

Example（例） 在 L^2 范数下，左边的矩阵比右边的矩阵小：

$$\left\| \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix} \right\|_{L^2} \text{ is smaller than } \left\| \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \right\|_{L^2} \tag{11}$$

左边两列正交，长度都是 $\sqrt{2}$ ，两个奇异值都满足 $\sigma^2 = 2$ 。**右边较大的奇异值满足 $\sigma_1^2 = \frac{1}{2}(3 + \sqrt{5}) > 2$ 。这个取零的子矩阵使整个矩阵的 L^2 范数变大。**

最优子矩阵是一个漂亮的问题，由 Davis、Kahan 和 Weinberger 在 1982 年解决。最优的 M 往往并不能使 $\widehat{C}M\widehat{R} = \widehat{A}$ 。但它把 L^2 范数降低到 $\| [Y \ Z] \|$ 与 $\| [X^T \ Z^T] \|$ 中较大的那一个——而在 (10) 中，更小的 L^2 范数显然是不可能的。在这个例子中，最小的 L^2 范数是由那个取 -1 （而非取 0 ）的子矩阵达到的。

[section]

Randomized Matrix Factorizations（随机化矩阵分解）

现在我们来到本主题的总结。正交矩阵是 $A = QR$ 与 $A = U\Sigma V^T$ 追求的目标。矩阵 A 太大，无法做精确分解——也许大到无法读取每一个元素 a_{ij} 。如果我们从一个 $(m \times k)(k \times n)$ 的近似 $A \approx CB$ 出发，那么 Halko–Martinsson–Tropp 就能快速而准确地求出 QR 与 $U\Sigma V^T$ 分解。

Strang §11.4, p.152

本页为书页 153，属 §11.4 Randomized Linear Algebra（随机化线性代数）的续页；页眉与页码为重复元素，不译。

[note] 译者注（OCR 修正）： 本页 OCR 把 什 om / 行 om / 丘 om 混作 from，把 盖 / 风 混作矩阵 A ，把 $\bar{\mathbf{I}}$ 混作上标 -1 ，把 屏 一类符号混作 k 或 n 。文中的分解公式（ $C = Q_1R_1$ 、 $B = Q_2R_2$ 、 $A \approx Q_1Q_2R_2$ ，以及由 $C = Q_1R_1$ 与 $B = U_2\Sigma V^T$ 得 $U = Q_1U_2$ ）均按随机化 QB 与 SVD 的标准写法重建。

[note] 译者注（OCR 修正）： 与权威原文核对后，上述第二步分解的对象应为 R_1B 而非 B ，即由 $C = Q_1R_1$ 起，把 R_1B 分解为 Q_2R_2 （或 $U_2\Sigma V^T$ ），从而 $A \approx (Q_1Q_2)R_2$ 与 $A \approx U\Sigma V^T$ 。

[section]

（续）Randomized Linear Algebra（随机化线性代数）

下面就是各种快速分解，它们都从一个随机化分解 $A \approx CB$ 出发：把 C 分解为 Q_1R_1 ，再把 R_1B 分解为 Q_2R_2 ，于是得到 $A \approx (Q_1Q_2)R_2$ ，这用到的只是正交矩阵与三角矩阵。

而由 $C = Q_1R_1$ 出发，把 R_1B 分解为 $U_2\Sigma V^T$ ，并取 $U = Q_1U_2$ ，就得到 $A \approx U\Sigma V^T$ 。

[section]

Projections J and Projection Matrices P （投影 J 与投影矩阵 P ）

一个从 m 维空间 $\mathbf{R}^m$ 到 k 维空间 $\mathbf{R}^k$ 的“投影”（projection） J ，是一个 $k \times m$ 的矩阵。在矩阵 J 具有满行秩（full row rank） k 的情形下， $k \leq m$ ，而投影的像填满整个 $\mathbf{R}^k$ 。

一个特别漂亮的情形是 J 的 k 行标准正交（orthonormal），这意味着 $JJ^T = I$ ；此时秩当然就是 k ，而列空间 $\mathbf{C}(J)$ 就是整个 $\mathbf{R}^k$ 。

请注意，*projection*（投影） J 与 *projection matrix*（投影矩阵） P 是不同的东西： P 是方阵（ $m \times m$ ），它的秩是 $k < m$ （除非 $P = I$ ），它的列空间是 $\mathbf{R}^m$ 的一个 k 维子空间（与向量空间 $\mathbf{R}^k$ 完全不同）。投影矩阵的关键性质是 $P^2 = P$ ：若我们垂直地投影到列空间上，则投影 Pb 是在通常范数 $\|b - Pb\|$ 意义下离 b 最近的点，此时 P 还是对称的。

[note] 此处 J 是 $k \times m$ 的投影，而 $P = J^TJ$ 为 $m \times m$ 的投影矩阵，它投影到 $\mathbf{C}(J^T)$ 上。OCR 的 $\mathfrak{J} \mathfrak{T} \mathfrak{J}$ 与 $\mathfrak{J} \mathfrak{T} \mathfrak{I} \mathfrak{J}$ 依语境重建： $P = J^TJ$ ， $P^2 = J^T(JJ^T)J = J^TIIJ = P$ 。

J 与 P 之间有一种漂亮的联系：当 J 的行标准正交时， J^TJ 是一个对称投影矩阵 P 。你可以验证 $P^2 = J^T(JJ^T)J = J^TIIJ = P$ 。

[example]

Example 1 取投影 $J = \begin{bmatrix} \cos \theta & \sin \theta \end{bmatrix}$ ，此时 $JJ^T = \begin{bmatrix} \cos^2 \theta + \sin^2 \theta \end{bmatrix} = [1]$ 。

对应的投影矩阵为

$$P = J^TJ = \begin{bmatrix} \cos^2 \theta & \cos \theta \sin \theta \\ \cos \theta \sin \theta & \sin^2 \theta \end{bmatrix} = P^2.$$

一个对称矩阵 P 正交地投影到自己的列空间上；在这里，那个列空间就是 $\mathbf{C}(J^T)$ 。

[section]

Random Projections (随机投影)

假设一个 $k \times m$ 的投影 J 的各个元素都是独立的随机变量。最简单的情形是它们取自正态分布（normal distribution，即高斯分布），均值 $m = 0$ 、方差 $\sigma^2 = 1/k$ 。我们下面要证明： $P = J^T J$ 的期望值就是单位矩阵。换句话说，投影 $v = Ju$ 的期望长度等于 u 的长度。

1. $J^T J$ 对角线上的第 (i, i) 个元素是 $J_{1i}^2 + \cdots + J_{ki}^2$ 这个和。这些平方都是独立的样本，每一个的均值都是 $1/k$ 。（因为 J 的每个元素均值为零，所以平方的期望值就等于方差 $\sigma^2 = 1/k$ 。）于是这 k 项之和的均值（即期望值）是 $k \cdot (1/k) = 1$ 。
2. $J^T J$ 的第 (i, j) 个非对角元素是 $J_{1i}J_{1j} + \cdots + J_{ki}J_{kj}$ 这个和。其中每一项都是两个均值为零的独立变量之积，因此每一项的均值都是零，它们之和的均值 $\mathbb{E}[(J^T J)_{ij}]$ 也就等于零。

于是 $\mathbb{E}[J^T J] = I$ ，即 $m \times m$ 的单位矩阵。

本页为书页 154，是 Computations with Large Matrices（大型矩阵的计算）的续页；页眉与页码为重复元素，不译。

[note] 译者注 (OCR 修正)：本页 OCR 把大量符号 n 误识为 刀 / 丩 / 了 / 丹，把 J 误识为 丩 / 上，把 C （或 c ）误识为 刀，把 況 / 灑 一类乱码按语境重建为 variance（方差）。公式 $\|Jc\|^2 = \|c\|^2$ 的期望形式、Johnson-Lindenstrauss 引理中的维数条件与式 (13) 均按数学语境重建。

利用列乘行的写法，矩阵 $J_i^T J_i$ （即 J 的第 i 行转置乘以第 i 行）的期望都是 $\frac{1}{k}I$ 。这些矩阵之和的期望就是 I 。请留意这意味着什么。

“在期望的意义下”， J 的 m 个列是标准正交的。因此在期望的意义下， Jx 与 x 有相同的长度：

$$\mathbb{E}[\|Jx\|^2] = \mathbb{E}[x^T J^T J x] = \mathbb{E}[x^T x] = \|x\|^2.$$

(12)

这个随机投影说明了为什么线性代数与概率论有时看起来像是数学中两个不同的世界。在线性代数里，一个 $k \times m$ 的矩阵 J （当 $k < m$ 时）不可能有秩 m ，因此 $J^T J$ 也不可能秩 m 。但一旦取期望值，情况就变了： $\mathbb{E}[J]$ 可以是零矩阵，而 $\mathbb{E}[J^T J]$ 却可以是单位矩阵。对 1×1 的矩阵来说，这其实是世界上最平常不过的事情：均值为零、方差为一。

关键在于此。平均而言， $\mathbf{R}^m$ 中 x 与 y 之间的平方距离，与 $\mathbf{R}^k$ 中 Jx 与 Jy 之间的平方距离是相同的。“把 J 投影到较低维空间，在平均意义下保持距离。” 我们真正想要的，是对 $\mathbf{R}^m$ 中给定的 n 个点，把实际距离保持在因子 $1 + \epsilon$ 的范围之内。

J 的这一性质正是著名的 Johnson-Lindenstrauss Lemma（约翰逊-林登施特劳斯引理）的主题。看起来令人惊奇的是：高维中的点可以被线性地变换到低维中的点，而每一对点之间距离的变化都非常小。

那么低维的那个维数究竟可以低到什么程度？这是一个至关重要的问题。

维数 $k = 1$ 太低 (Dimension $k = 1$ is Too Low)

假设我们在平面 $\mathbf{R}^2$ 中有 $n = 3$ 个点 x_1, x_2, x_3 。是否存在一个 1×2 的矩阵 J 能几乎保持它们之间的距离？我们当然可以让 $\|Jx_1 - Jx_2\| = \|x_1 - x_2\|$ 。如果第三个点 $x_3 = \frac{1}{2}(x_1 + x_2)$ 恰好在 x_1 与 x_2 的中点，那么由线性性， Jx_3 就是 $\frac{1}{2}(Jx_1 + Jx_2)$ ，所有的距离都完美保持。但如果 x_1, x_2, x_3 构成等边三角形，那么 x_3 在 J 的零空间中有一个分量；这个分量会被丢掉（被投影为零），于是长度 $\|Jx_1 - Jx_3\|$ 与 $\|Jx_2 - Jx_3\|$ 都会严重缩小。

Johnson 与 Lindenstrauss 研究的是 $\mathbf{R}^m$ 中 n 个点的随机 $k \times m$ 投影。他们证明了：只要维数 k 足够大，这些投影中就会有一个（事实上是绝大多数投影）几乎保持这 n 个点之间的距离。

“在高维中，随机向量以接近 1 的概率相互正交。”

[theorem]

The Johnson-Lindenstrauss Lemma (约翰逊-林登施特劳斯引理)

定理 (Johnson-Lindenstrauss)。设 $x_1, \dots, x_n$ 是 $\mathbf{R}^m$ 中的任意 n 个点，且维数 $k \geq \frac{8 \log n}{\epsilon^2}$ 。则存在一个从 $\mathbf{R}^m$ 到 $\mathbf{R}^k$ 的投影 J ，使得这 n 个点之间的所有距离都几乎被保持：

$$(1 - \epsilon) \|x_i - x_j\|^2 \leq \|Jx_i - Jx_j\|^2 \leq (1 + \epsilon) \|x_i - x_j\|^2.$$

(13)

本页为书页 155，接续 §II.4 Randomized Linear Algebra（随机化线性代数）的收尾段落，随后进入 Randomized Matrix Approximation（随机化矩阵逼近）小节；页眉与印页码为重复元素，不译、不收录。

[note] 译者注 (OCR 修正)：本页 OCR 噪声密集，逐处修正如下：Ÿ 为 Y ，蓋 为 A ，00g / 为 $(\log n)/\epsilon^2$ ，睷 d 为 found，0 月 e 为 One，ro 历訂 记 e 叻 od 依上下文重建为 randomized method，s UDUT 为 $A \approx UDV^T$ ，(QU)DVT 为 $(QU)DV^T$ ，CMR / CZ 为 $A \approx CMR$ 、 $A \approx CZ$ ，0 彳 月 为 $A \approx CUR$ ，IOW cost 为 low cost，an ex tra訂 r e 为 an extra step，Martinsson 的论文标题（arXiv:1607.01649）重建为 *Randomized Methods for Matrix Computations*。

判断依据有三条：维数与下标必须自治（ $Y = AG$ 要求 G 是 $n \times k$ 、 Y 是 $m \times k$ ）、转置与逆不能混用（ YY^+ 是投影，不是 Y^{-1} ）、专名与编号可回原文核对（arXiv 编号、Acta Numerica 的卷页）。句中残缺的 J 无法可靠复原，译文一律保留符号 J 原样，不做猜测。

[note] 译者注 (OCR 修正)：一处订正：本项目所用教材的印刷式为 m by k factor $Y = AG$ 、以及 $k > 2800$ 、 $k = 2700$ ，故本页正文中的 $m \times k$ 、 $m > 2800$ 、 $m = 2700$ 一并改为 k 。

[section]

Randomized Linear Algebra（随机化线性代数）（续）

关键在于：维数 k 必须按 $(\log n)/\varepsilon^2$ 的量级增长。证明中有一个令人惊叹的步骤说明：随机取一个 $k \times m$ 的投影 J ，它极有可能把 n 个点彼此分开。

于是必定存在**足够多个具体的** J ，使 Lemma（引理）中的方程 (13) 成立。

如果某个随机选择有正的概率成功，那么成功的那一次选择必然存在。

这是概率性的假设，给出的却是确定性的结论。这就是“**概率方法** (probabilistic method)”。

关于 Johnson-Lindenstrauss 的证明有很多种，都很容易找到；我们偏爱这个网站上的那一份：cseweb.ucsd.edu/~dasgupta/papers/jl.pdf。

18.065 有一个课程项目选了这样一个例子：其中 Lemma 要求 $k > 2800$ 。而当 $k = 2700$ 时，距离 $\|x_i - x_j\|$ 的精度确实失效了。

同样有意思的是：即使距离算错了，各点的聚类（clustering）结构依然保留了下来。

[section]

Randomized Matrix Approximation（随机化矩阵逼近）

为了总结这个主题，我们将跟随 Per-Gunnar Martinsson 的 *Randomized methods for matrix computations*（矩阵计算的随机化方法），arXiv:1607.01649。首先明确目标。

- 1 Rank k factorizations $A \approx Y(Y^+A)$ and $S \approx UDU^T$ and $A \approx (QU)DV^T$.

2 Interpolative decompositions $A \approx CMR$ and $A \approx CZ$ using columns of A in C .

即：**1. 秩 k 分解** (rank factorizations)： $A \approx Y(Y^+A)$ 、 $S \approx UDU^T$ 以及 $A \approx (QU)DV^T$ 。

2. 插值分解 (interpolative decompositions)： $A \approx CMR$ 与 $A \approx CZ$ ，其中 C 取的是 A 的列。

随机化的 $m \times k$ 因子 Y 就是 AG ，其中 G 是 $n \times k$ 的高斯随机矩阵（Gaussian random matrix）。

[note] 译者注 (两段式随机分解：**草图** + **精确收尾**)：这一段是整节的骨架，先立起一张流程图：**草图 (sketch)** $\rightarrow$ **精确收尾 (solve)**。第一步用随机线性映射把 n 列压成 k 列，得到 $Y = AG$ ；第二步只在 $k \times n$ 的小矩阵 $Q^T A$ 上做确定性的 SVD，最后把因子拼回原尺寸。

Y 为什么靠得住？只要 $\text{rank}(A) \geq k$ ， G 的 nk 个元素独立同分布、任取 k 列几乎必然线性无关，所以 $\text{rank}(Y) = k$ 以概率 1 成立——失效的情形要求某个行列式恰好为零，那是零测集（不恒为零的多项式只在测度为零处取零，所以“几乎必然”不是修辞）。能量为什么没丢？设 G 的元素方差为 1，对固定向量 x 有 $E\|G^T x\|^2 = k\|x\|^2$ ： $G^T x$ 的第 j 个分量是内积 $g_j^T x$ ，方差为 $\|x\|^2$ ， k 个分量互相独立，期望相加即得。注意这里的归一化：真正无偏的估计量是 $\|G^T x\|^2/k$ ，或者干脆把 G 的元素方差取成 $1/k$ （等价于用 $G/\sqrt{k}$ ）。这一步最容易踩坑——**归一化因子必须显式补上**，否则你以为能量守恒，其实整体放大了 k 倍。

这也顺带解释了本页开头那句 $(\log n)/\varepsilon^2$ ：要让所有点对同时被保住，就得对大约 n^2 个点做联合界（union bound）， $\log n$ 就是这么进来的。

小算例最能暴露这个坑：取 $n = 4$ 、 $k = 1$ ， $G = (g_1, g_2, g_3, g_4)^T$ 的元素方差为 1，再取 $x = (1, 1, 1, 1)^T/2$ ，则 $\|x\|^2 = 1$ ；而 $G^T x = (g_1 + g_2 + g_3 + g_4)/2$ ，交叉项因独立性期望为零，于是 $E\|G^T x\|^2 = 4 \cdot \frac{1}{4} = 1 = \|x\|^2$ 。但一阶矩守恒不等于每次抽样都守恒： k 列共用同一个 G 、彼此并不独立，单次草图完全可能漏掉某个方向的能量。压制这种波动的唯一办法是多给几列，这正是后面过采样取 $k + p$ 的动机；草图之后先做 $Y = QR$ 而不是直接 SVD，也是同一个道理——单遍抽样会让 Y 的列接近平行，而 Q 的列正交、条件数为 1，后续 $Q^T A$ 的舍入误差不会被放大。

收益可以算得很具体：取 $m = 10^6$ 、 $n = 10^4$ 、 $k = 100$ ，完整 SVD 约 $2mn^2 \approx 2 \times 10^{14}$ flops，随机路径约 $mnk = 10^{12}$ flops（再加 $nk^2 = 10^8$ ，可忽略），快约两个数量级，而且 A 只被触摸一遍。

跨课程连接（《深入理解计算机系统》CSAPP $\rightarrow$ 18.065，即本书配套的 MIT 课程）：单遍算法之所以病态，是因为每读到一个元素就得立刻更新草图，无法回头把已有的方向重新正交化，而“回头正交化”正是经典 Gram-Schmidt 稳定性的来源。接到 CSAPP 的存储层次与局部性上：顺序读一遍 A 、只维护一个 $n \times k$ 的草图，工作集小到能常驻缓存，每字节的摊还代价贴近顺序带宽；多遍正交化则要反复冷读整块 A ，cache miss 随遍数近似线性增长。区别在于关心的量：那里局部性由硬件决定、算的是常数因子，这里由设计者决定“能读几遍”、算是渐近复杂度；条件数这一端还能接到 CSAPP 的浮点表示——双精度约 16 位有效数字，条件数 10^8 的草图只剩约 8 位可信。

由此可以固化一个自检模板，叫**能量守恒检验法**：拿到随机投影，先验一阶矩 $E\|G^T x\|^2 = k\|x\|^2$ （别忘记归一化），再估二阶矩（ k 列不独立，只能靠过采样压波动），最后验残差 $\|A - QQ^T A\|$ 是否落在尾部奇异值的量级。三步都过，才敢采信草图的结果。

始终成立： YY^+ 是到 Y 的列空间上的正交投影（orthogonal projection）。所以 YY^+A 几乎必然（almost surely）是 A 的一个非常好的秩 k 逼近——这就是其中随机的部分。

如何由 $A \approx YY^+A$ 得到一个近似的 SVD（approximate SVD）？对于标准正交的列基（orthonormal column basis），先对 Y 作 QR 分解： $Y = QR$ 。此时 QQ^T 与 YY^+ 是同一个投影。

再求小矩阵 $Q^T A$ 的 SVD： $Q^T A = UDV^T$ 。所要的那个 SVD 就是 $A \approx (QU)DV^T$ 。

请注意这个 SVD 的两阶段（two-stage）构造方式。第一步，先固定一个近似的列空间（column space），把问题降到“规模 k ”。第二步，任何想要的分解都是确定性的（deterministic）、快速的，而且本质上精确。马丁松（Martinsson）说明了如何把这些阶段合并成一个**流式算法**（streaming algorithm），使 A 的每个元素只被访问一次。代价是病态（ill-conditioning），补救办法是过采样（over-sampling）。

对于半正定（positive semidefinite）的 S ，多加一个 Nyström 步骤就能以很低的代价改进这些因子。

最后来看 $A \approx CZ$ 或 $A \approx CMR$ ，其中 C 由 A 的真实列构成：这样做能保住稀疏性（sparsity）与非负性（nonnegativity）。确定性算法的精度与带主元的 $AP \approx QR$ 相同（并不总能像 Eckart-Young 那样接近）。如果 A 的奇异值像 §III.3 那样快速衰减，随机化算法就更快、也更好。

要想清楚地理解随机化矩阵算法，请读 Martinsson 的论文，以及 Kannan-Vempala 的 *Randomized algorithms in numerical linear algebra*, Acta Numerica (2017), 95-135。范数平方采样（norm-squared sampling）正是这两位作者发现的。

Strang §II.4, p.155

本页为书页 156，整页为 Problem Set II.4（习题集 II.4）；按本系列先例，题干保留英文原文不译，仅把 OCR 乱码按数学语境修正为 LaTeX；页眉 Computations with Large Matrices 为重复元素，不译、不收录。

[note] 译者注 (OCR 修正)：原页题号印在页边，OCR 把页边码提前读出并与正文交错，故题号按书中印刷顺序还原为 Problem 1–8；11.4 即 II.4，If 为 If，In case 为 In case， $\geq$ / $\leq$ 为不等号（依上下文分别为 $\geq$ 或 $\leq$ ；Problem 3 中 $\|a\|_1 \leq \sqrt{n}\|a\|_2$ ）， $\lambda(\cdot)$ 为 $\lambda(\cdot)$ ， $nI - M$ 为 $nI - M$ ， $\int$ 依语境为积分号 $\int$ 或概率 p ， $AB = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}$ ， $C = E\|b\|^2 = \|A\|_F^2$ ， $(QU)DV$ 依上下文补回转置为 $(QU)DV^T$ ，p. 11.2.110 为 Problem II.2.10。做题时若某个符号与正式 (5)、(7) 对不上，请以正文定义为准，不要以本页修正稿为准。

[note] 译者注 (OCR 修正)：又一处订正：Problem 8 末尾提示的题号，权威原文为 Problem II.2.11 on page 135；Problem 6 中关于方差的不等式，权威原文为 $\|AB\|_F^2 \leq (\sum \|a_j\| \|b_j^T\|)^2$ 。

[section]

Problem Set II.4（习题集 II.4）

[problem] 1 Given positive numbers $a_1, \dots, a_n$ find positive numbers $p_1, \dots, p_n$ so that

$$p_1 + \dots + p_n = 1 \quad \text{and} \quad V = \frac{a_1^2}{p_1} + \dots + \frac{a_n^2}{p_n} \quad \text{reaches its minimum} \quad (a_1 + \dots + a_n)^2.$$

The derivatives of $L(p, \lambda) = V - \lambda(p_1 + \dots + p_n - 1)$ are zero as in equation (8).

[problem] 2 (for functions) Given $a(x) > 0$ find $p(x) > 0$ by analogy with Problem 1, so that

$$\int_0^1 p(x) dx = 1 \quad \text{and} \quad \int_0^1 \frac{(a(x))^2}{p(x)} dx \quad \text{is a minimum.}$$

[problem] 3 Prove that $n(a_1^2 + \dots + a_n^2) \geq (a_1 + \dots + a_n)^2$. This is Problem 1 with $p_i = 1/n$. Back in Problem Set I.11 you proved that $\|a\|_1 \leq \sqrt{n}\|a\|_2$.

[problem] 4 If $M = \mathbf{1}\mathbf{1}^T$ is the n by n matrix of 1's, prove that $nI - M$ is positive semidefinite. Problem 3 was the energy test. For Problem 4, find the eigenvalues of $nI - M$.

[problem] 5 In case $B = A^T$ show that the norm-squared or length-squared probabilities p_j in the text equation (5) are $\|a_j\|^2 / \|A\|_F^2$. Why is $C = \sum \|a_j\| \|b_j\| = \|A\|_F^2$?

[problem] 6 The variance computed in equation (7) cannot be negative! Show this directly:

$$\|AB\|_F^2 \leq \left(\sum \|a_j\| \|b_j^T\| \right)^2.$$

Problem 7 returns to the example in the text of sampling (a, b) to get $(a, 0)$ or $(0, b)$. If $b > a$ then the variance will be reduced when b is chosen more often. This is achieved by optimizing the probabilities p and $1 - p$ to minimize σ^2 :

$$\text{Variance } \sigma^2 = p \frac{a^2}{p^2} + (1 - p) \frac{b^2}{(1 - p)^2} - (\text{mean})^2$$

[problem] 7 Show that $p = a/(a + b)$ and $1 - p = b/(a + b)$ minimize that variance. (The mean is the same for all p .) This optimal p agrees with equation (5) when applied to the small matrix multiplication $AB = [1][a \quad b]$. In this case $C = a + b$ in equation (5).

[problem] 8 In the randomized construction on the previous page, show why $(QU)DV^T$ (the approximate SVD in italics) is close to A . Use the steps $A \approx YY^+A$ and $Y \approx QR$ and $Q^T A = UDV^T$. Problem II.2.11 on page 135 is a key.

[note] 译者注（八道题背后的同一个模板）：这一页八道题是同一件事的八种外衣：在 $\sum p_i = 1$ 、 $p_i > 0$ 的约束下最小化 $\sum a_i^2/p_i$ 。抓住这条主线，八道题就不再是八个孤立的练习。

先问设计问题：什么时候需要它？当你要用 k 次随机抽样去估计一个 n 项之和（ $n \gg k$ ）时，抽哪一项是你唯一能设计的自由度，误差方差直接由 p_i 决定。估计量写成 $\frac{1}{k} \sum_{t=1}^k \frac{a_{i_t}}{p_{i_t}}$ ：它的期望是 $\sum_i a_i$ ，与 p_i 无关，所以无偏性不花代价；但方差是 $\frac{1}{k} \left(\sum_i \frac{a_i^2}{p_i} - (\sum_i a_i)^2 \right)$ ，权重出现在分母上——采样越少的项，每被采中一次就放大得越狠。于是全部设计自由度落在 p_i 上，而它被 $\sum p_i = 1$ 与 $p_i > 0$ 夹住。

用拉格朗日乘子解（这就是 Problem 2 提示的式 (8)）： $L = \sum_i \frac{a_i^2}{p_i} + \lambda(\sum_i p_i - 1)$ ，令 $\partial L / \partial p_i = -a_i^2/p_i^2 + \lambda = 0$ ，得 $p_i = a_i / \sqrt{\lambda}$ ，代入约束得 $p_i = a_i / \sum_j a_j$ ，此时最小值为 $(\sum_i a_i)^2$ 。取 $p_i = 1/n$ 就退回 Problem 3 的不等式 $n \sum a_i^2 \geq (\sum a_i)^2$ ，它就是柯西-施瓦茨 $(\mathbf{1}, \mathbf{a})^2 \leq \|\mathbf{1}\|^2 \|\mathbf{a}\|^2$ ，等号在 $a_1 = \cdots = a_n$ 时成立。Problem 4 说的 energy test（能量检验）正是它： $x^T(n\mathbf{I} - \mathbf{1}\mathbf{1}^T)x = n\|x\|^2 - (\sum_i x_i)^2 \geq 0$ ；而 $M = \mathbf{1}\mathbf{1}^T$ 是秩 1 矩阵， $\mathbf{1}$ 是它特征值 n 的特征向量，与 $\mathbf{1}$ 正交的向量都给特征值 0——顺着这条线， $n\mathbf{I} - M$ 的半正定性与它的特征值（Problem 5）会自己浮出来。

这里要提醒一处记号陷阱：书上把式 (7) 的那个 $a^2/p + b^2/(1 - p)$ 记作 σ^2 ，严格说它是二阶矩 $E[X^2]$ ，方差还要再减去 $(E[X])^2$ ；好在 $(E[X])^2$ 与 p 无关，所以最小化二阶矩与最小化方差在同一处取到最优。两变量情形的验算最干净：求导得 $-a^2/p^2 + b^2/(1 - p)^2 = 0$ ，即 $a/p = b/(1 - p)$ ，解得 $p = a/(a + b)$ ，代回得 $E[X^2]_{\min} = (a + b)^2$ ，而方差是 $(a + b)^2 - (a + b)^2 = 0$ ——两个点、一次抽样、长度平方概率，估计居然变成确定性的。代入数字看得更清楚： $(a_1, a_2) = (1, 3)$ 时均匀采样 $E[X^2] = 2 + 18 = 20$ 、方差 4，最优采样 $E[X^2] = 4 + 12 = 16$ 、方差 0；换成 $(1, 50)$ ，均匀是 5002（方差 2401），最优是 $(1 + 50)^2 = 2601$ （方差 0）。两端差得越远，按 $\sqrt{c_i}$ 分配权重的收益越大。

把它固化成模板：**平方根加权律**——目标形如 $\sum c_i/p_i$ 时，最优权重满足 $p_i \propto \sqrt{c_i}$ ，因为权重出现在分母、而被放大的是 $1/p_i$ 。此处 $c_i = a_i^2$ ，故 $p_i \propto a_i$ ，这就是 Problem 7 的范数平方采样（norm-squared sampling）。矩阵乘法是同一模板的矩阵版：用 $\frac{1}{p_i} a_i b_i^T$ 估计 $AB = \sum_i a_i b_i^T$ ，二阶矩里出现 $\sum_i \|a_i\|^2 \|b_i\|^2 / p_i$ ，于是 $p_i \propto \|a_i\| \|b_i\|$ ；当 $B = A^T$ 时 $b_i = a_i^T$ ，得 $p_i \propto \|a_i\|^2$ ，再除以让概率加起来的归一化常数 $\sum_i \|a_i\|^2 = \|A\|_F^2$ ，就是书中的 $\|a_i\|^2 / \|A\|_F^2$ （题中那个 c 就是它，读题时请回到正文核对 c 的定义）。Problem 6 是连续版：求和换成积分， $\int a^2(x)/p(x) dx$ 在 $\int p(x) dx = 1$ 下的最小值同样是 $(\int a(x) dx)^2$ 。

跨课程连接一（概率论与数理统计 $\rightarrow$ 18.065）：这里的估计量就是课本里的无偏估计，方差公式也是同一套；区别是统计学里 p_i 给定、算方差，这里反过来把 p_i 当设计变量去最小化方差，属于重要性采样（importance sampling，用可控的抽样分布降低估计方差）的最优权重问题。跨课程连接二（算法设计与分析，如 MIT 6.006 Introduction to Algorithms $\rightarrow$ 18.065）：先花一遍 $O(n)$ 扫描算出全部 p_i ，再做 k 次抽样，与随机化快排“先随机选主元、再确定性划分”同属复杂度再分配；区别是那里分析运行时间的期望与失败概率，这里分析估计量的方差与数值精度。还要记住一个工程细节：这一遍 $O(n)$ 扫描是顺序访问、对缓存友好，真正昂贵的是后面 k 次随机访问，所以这套方法只在 n 很大、单项计算量很高时才划算。

Strang §II.4, p.156

本页为书页 157，是 Part III（第三部分）的章首标题页，只有章名与五个节标题；页眉与页码为重复元素，不译。

[note] 译者注（OCR 修正）：原 OCR 的 盖 $\rightarrow$ 1 为 A^{-1} ， $\mathbb{N}$ 盖 为 in A ，111.2–111.5 为节号 III.2–III.5，+ 1 为 ℓ_1 ；左右两栏曾被 OCR 交错读出，此处按语义重新排序。

[section]

Part III — Low Rank and Compressed Sensing（第三部分——低秩与压缩感知）

本部分依次展开五节，逐层处理低秩（low rank）与压缩感知（compressed sensing）的核心问题。

[section]

III.1 Changes in A^{-1} from Changes in A （ A 变化时 A^{-1} 的变化）

[\[section\]](#)

III.2 Interlacing Eigenvalues and Low Rank Signals (特征值交错与低秩信号)

[\[section\]](#)

III.3 Rapidly Decaying Singular Values (快速衰减的奇异值)

[\[section\]](#)

III.4 Split Algorithms for $\ell^2 + \ell^1$ ($\ell^2 + \ell^1$ 的分裂算法)

[\[section\]](#)

III.5 Compressed Sensing and Matrix Completion (压缩感知与矩阵补全)

本页为书页 159，是 Part III（第三部分）总览正文的首页；页眉为重复元素，不译。

[note] 译者注 (OCR 修正)： 原 OCR 噪声按数学语境修正： uv^T 为 uv^T ， $\text{Maffices}/\text{mafr1X}$ 为 $\text{matrices}/\text{matrix}$ ， 山 at 为 that ， $@ + j - 1$ 为 $1/(i + j - 1)$ ， $| |$ 盖 $\|$ 为 $\|A\|_*$ ， Minimizing 为 Minimizing ， ℓ_0 ， $111.4/111.5$ 为 $\S\text{III.4}/\S\text{III.5}$ ， $\|x\|_1$ ， $| | + - + @$ 为 $\|x\|_1 = |x_1| + \cdots + |x_n|$ 。

本部分（Part III）考察三种类型的低秩矩阵（low rank matrices）：

- 1. 真正具有小秩的矩阵—— uv^T 是极端情形，其 rank（秩）等于 1。
- 2. 奇异值（singular values）呈指数衰减的矩阵，即有效秩（effective rank）很低的矩阵。
- 3. 不完整的矩阵（incomplete matrices，含有缺失元素 missing entries），它们被补全（completed）成低秩矩阵。

第一类矩阵不可逆（not invertible），因为 $\text{rank} < n$ 。第二类矩阵在理论上可逆，在实践中却不可逆。元素为 $1/(i + j - 1)$ 的矩阵就是一个著名的例子。

你怎样才能看出这个矩阵或者另一个矩阵具有非常低的有效秩（effective rank）？

第三个问题——矩阵补全（matrix completion）——将在 §III.5 中讨论。我们要构造一个适用于推荐矩阵（recommender matrices）的最小化问题（minimization problem）：

$$\text{Minimize } \|A\|_*, \text{ over all possible choices of the missing entries.}$$

也就是说，在所有可能的缺失元素取值中，使核范数（nuclear norm） $\|A\|_*$ 达到最小。

[note] 译者注 (第一性原理洞察)： 为什么 rank 必须被换掉，而核范数 $\|A\|_* = \sum_i \sigma_i$ 能顶上？回到最基础的假设：秩是非零奇异值的个数，是一个计数函数，因而不满足范数的齐次性—— $\text{rank}(cA) = \text{rank}(A)$ ($c \neq 0$)，而不是 $|c| \text{rank}(A)$ ，所以它根本不是范数。但真正致命的是第二点：集合 $\{A : \text{rank}(A) \leq k\}$ 不是凸集。反例取 2×2 矩阵 $A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ 与 $B = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ ，两者秩都为 1，其中点 $M = \frac{1}{2}A + \frac{1}{2}B = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}$ 的秩却是 2；连通两点的线段跑出了集合，凸性当场失效。**凸包松弛法**正是为此而生：把组合目标（选 k 列、选秩 k ）换成它在凸包上的线性泛函去最小化。核范数就是秩的这份凸替代，依据是 $\{A : \|A\|_* \leq 1\}$ 恰为 $\{A : \text{rank}(A) \leq 1, \|A\|_2 \leq 1\}$ 的凸包。这是 ℓ_1 故事的矩阵版： ℓ_1 单位球（二维的菱形）是四个点 $(\pm 1, 0), (0, \pm 1)$ 的凸包，相邻两端点连成的线段正是它的一个面；而 ℓ_2 单位球是圆，圆周上每一点都是端点，没有平坦的面可停。把仿射约束集取得最简单，取 $x_1 + x_2 = 1$ 这条直线， ℓ_1 最小化给出 $(1, 0)$ 或 $(0, 1)$ （稀疏，落在顶点上）， ℓ_2 最小化给出 $(1/2, 1/2)$ （不稀疏，落在圆弧上），差别完全来自单位球的形状。矩阵版同理：奇异值之和在把能量摊到更多奇异方向上时代价更大，于是解停在少数几个大奇异值上。上面的 M 可以量化这一点：它的两个奇异值都是 $\frac{1}{2}$ ，故 $\|M\|_* = 1$ ， $\|M\|_2^2 = \frac{1}{2}$ ，比值 $\|M\|_*/\|M\|_F = \sqrt{2} = \sqrt{\text{rank}(M)}$ ，恰好是“能量摊得最平”的极端情形；一般只有 $\|M\|_* \leq \sqrt{\text{rank}(M)} \|M\|_F$ 。跨课程连接有两条，区别必须讲清：其一，Strang 线性代数课的 SVD 一节表明，rank、核范数、Frobenius 范数都只依赖奇异值谱——分别是数非零个数、求和、求平方和，所以“拿什么替代秩”就是在奇异值上加什么样的凸惩罚；凸优化课提供的机器则是凸集、凸函数、KKT 条件与对偶，它只接受凸的可行集与目标，于是“最紧的凸替代”这一问法才有意义：任何在每个秩一矩阵上都 不低于秩的凸函数，必定不小于核范数，核范数因此在所有凸替代中取下确界。其二，接上计算机系统课（CSAPP）的浮点算术你会看到一层现实约束：SVD 是迭代算法，算出的 σ_i 带相对误差，当真实的前 k 个奇异值比机器精度还接近时，“哪个奇异值算零”本身就成了病态判断，这正是谱衰减越快、低秩结构越稳、恢复结论必须附加谱间隙条件的原因。最后记住边界：凸松弛只保证没有更好的凸方法，不保证一定恢复出真正 的最小秩解，恢复所需的额外条件（不相干性、RIP）属于 §III.5，二者不可混为一谈。

这个核范数给出一个适定问题（well-posed problem），用来替代一个非凸问题（nonconvex problem）：最小化 rank（秩）。

n 数据上反复扫过整块（选主元还要跳着访问），即使分块优化后仍是访存受限。低秩更新之所以快，不只是浮点运算少，而是把「必须反复触碰全部 n^2 个元素」换成了「只触碰 $k+1$ 次」。公式 (1) 也可以用几何级数推出来：由 $(I - X)^{-1} = I + X + X^2 + \cdots$ ，取 $X = uv^T$ ，因为 $(uv^T)^2 = u(v^T u)v^T = (v^T u)uv^T$ ，故 $X^m = (v^T u)^{m-1}uv^T$ ，求和即得 $I + \frac{uv^T}{1 - v^T u}$ 。代一组数验算：取 $u = (1, 1)^T$ 、 $v = (0.25, 0.25)^T$ ，则 $v^T u = 0.5$ ，公式给出 $(I - uv^T)^{-1} = I + 2uv^T$ ，逐项相乘确实得 I 。这也标出了公式失效的边界： $v^T u = 1$ 时级数不收敛、 $I - uv^T$ 奇异；数值上 $1 - v^T u$ 接近 0 会发生灾难性抵消——相近的浮点数相减会丢失有效位。按粗略的浮点模型， $v^T u$ 由 n 次乘加累积而来，本身已带约 $n\varepsilon$ 的相对误差，因此当 $1 - v^T u$ 小到 10^{-4} 量级（即 $v^T u = 0.9999$ ）时，相减后只剩约 4 位有效数字，解 x 的相对误差被放大约 10^4 倍。跨课程连接（计算复杂度视角）：从「矩阵分解一次、复用多次」这条线索看， $A = QR$ 的低秩更新等价于对因子 R 做小修正，连接路径是「固定分解 $\rightarrow$ 缓存预处理结果 $\rightarrow$ 每次变动只解一个小系统」，区别在于：QR 更新（如 Givens 旋转）的目标是保持正交因子的结构以维持数值稳定性，代价是每次更新 $O(n^2)$ 且要对因子本身动手；而 Sherman-Morrison-Woodbury 的目标是保持已算好的线性求解器不动，只在大矩阵之外挂一个小修正项，代价更低、实现更简单，但把全部数值风险集中到了分母 $1 - v^T u$ 上——一个修因子，一个修解，前者换稳定性，后者换速度。命名模板：**低秩更新法（不改分解，只解方程）**——凡是「大矩阵被小改动、又要反复使用」的场合，先固定大矩阵的分解，把小改动全部塞进一个 $k \times k$ 的小系统；递归最小二乘、在线学习的协方差更新都是这个模板的实例。最后提醒一句：本节公式的成立前提是「已经持有 A^{-1} 或某个现成的分解」，若从零开始，一步也省不下来。

本页为书页 161，是 §III.1 的续页；页眉与页码为重复元素，不译。OCR 噪声同样较密：什 om 为 from， $\bar{\mathbf{I}}^{-1}/-1$ 为 -1 ；刀、鬱 皆为 u ，孤立的 $\mathbf{T}$ 为 v^T ； $\mathbb{M}$ ul a 为 formula，i de n ti ty 为 identity，莖 为 M^{-1} 之误， $\mathcal{P}^n$ 为 $(I_k - V^T U)^{-1}$ 之残迹，叭 为 U 。公式按数学语境重建。

[\[section\]](#)

III.1 Changes in A^{-1} from Changes in A （续）

第二种途径是把 E 的第 2 行乘以 u 后从它的第 1 行中减去：

$$\begin{bmatrix} I & -u \\ 0 & 1 \end{bmatrix} E = \begin{bmatrix} I - uv^T & 0 \\ v^T & 1 \end{bmatrix}. \quad \text{Then } E^{-1} = \begin{bmatrix} I - uv^T & 0 \\ v^T & 1 \end{bmatrix}^{-1} \begin{bmatrix} I & -u \\ 0 & 1 \end{bmatrix}.$$

现在把同一个 E^{-1} 的这两个公式作比较，其代数运算留给 Problem 2：

Two forms of E^{-1} $\begin{bmatrix} I + uD^{-1}v^T & -uD^{-1} \\ -D^{-1}v^T & D^{-1} \end{bmatrix} = \begin{bmatrix} M^{-1} & -M^{-1}u \\ -v^T M^{-1} & 1 + v^T M^{-1}u \end{bmatrix}.$

左上的 $(1, 1)$ 块告诉我们 $M^{-1} = I + uD^{-1}v^T$ 。这正是公式 (1)，其中 $D = 1 - v^T u$ 。

[\[section\]](#)

The Inverse of $M = I - UV^T$ ($M = I - UV^T$ 的逆)

我们可以毫不费力地迈出一大步。假设扰动不再是秩一的，而是秩 k 的 UV^T ：矩阵 U 是 $n \times k$ 的，矩阵 V^T 是 $k \times n$ 的。于是我们有 k 列和 k 行，正如先前只有一列 u 和一行 v^T 一样。 M^{-1} 的公式保持完全相同的形状！只不过现在涉及两个尺寸 I_n 与 I_k ：

$$M^{-1} = I_n + U \left(I_k - V^T U \right)^{-1} V^T.$$

这就带出了关于这类逆公式的一个重要观点：我们用一个尺寸为 k 的逆，换掉了一个尺寸为 n 的逆。由于本节开头 $k = 1$ ，我们当时的逆是尺寸 1 的，也就是对那个数 $1 - v^T u$ 做普通的除法。现在 $V^T U$ 是 $(k \times n)$ 乘 $(n \times k)$ ，即 $k \times k$ 。我们要去求逆的是一个 $k \times k$ 的矩阵 $I_k - V^T U$ ，而不是 $n \times n$ 的。

[note] 译者注（跨课程连接）： 这一句是整节的成本账本，值得把它算成具体数字。先问它解决什么设计问题：扰动只有 k 列，答案却住在 n 维空间里，若每来一次扰动就重建整个逆，就是把 $n - k$ 个本来没动的方向也重新消一遍元，白花力气。用复杂度语言说：求 $I_n - UV^T$ 的逆若从零开始做消元约需 $\frac{2}{3}n^3$ 次乘加；而按公式 (7)，只需对 $k \times k$ 的 $I_k - V^T U$ 求逆 ($O(k^3)$)，再加上两次 $n \times k$ 与 $k \times n$ 的瘦长乘法形成修正项 ($O(n^2 k)$)，总计 $O(n^2 k + k^3)$ 。取 $n = 1000$ 、 $k = 10$ ：直接求逆约 6.7×10^8 次运算，走公式只要约 10^7 次，快了两个数量级。内存这一侧同样有账可算： $n \times n$ 的逆按 8 字节双精度存是 8 MB， $k \times k$ 的小逆只有 800 字节，正好落进一级数据缓存那一档；而公式真正要花的 $O(n^2 k)$ 是两次瘦长矩阵乘法，按行分块后顺序扫描，命中率远高于对 8 MB 大矩阵反复随机访问。先做一个能一行验算完的数值例子：取 $k = 1$ 、 $n = 2$ ， $U = (1, 3)^T$ 、 $V = (2, 4)^T$ ，则 $UV^T = \begin{pmatrix} 2 & 4 \\ 6 & 12 \end{pmatrix}$ ，于是 $U(I_1 - V^T U) = U(1 - 14) = -13U = (-13, -39)^T$ ；而 $(I_2 - UV^T)U = \begin{pmatrix} -1 & -4 \\ -6 & -11 \end{pmatrix} \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} -13 \\ -39 \end{pmatrix}$ ，逐项相等。注意两边的算术成本：左边要先造出 $n \times n$ 的外积再乘向量 ($O(n^2)$)，右边先做 k 次内积再乘 U ($O(nk)$)，答案一样而路径短得多——这就是把 $V^T U$ 从矩阵内部搬到外面的现金价值。但注意这不是免费的午餐——它要求你已经持有 A^{-1} （或某个已有分解）；若从零开始，一步也省不下来。这正是卡尔曼滤波与递归最小二乘存在的理由：每步都保留上一步的逆，于是新数据到来时只花 $O(n^2)$ 。公式 (7) 的证明核心是一条搬运恒等式 $(I_n - UV^T)U = U(I_k - V^T U)$ ：左端展开是 $U - UV^T U$ ，右端展开是 $U - UV^T U$ ，逐项相同，但它把 V^T 从 U 的左边搬到了右边，从而让 U 可以被提到括号外， MM^{-1} 的四个交叉项随即两两抵消。同一个搬运精神也解释了式 (8) 的行列式恒等式 $\det(I_n - UV^T) = \det(I_k - V^T U)$ ：两个乘积的非零特征值相同，差的只是 $|n - k|$ 个零。还要连一条实分析侧的路： $\frac{2}{3}n^3$ 这个常数不是拍脑袋，它来自对消元乘加次数 $\sum_{i=1}^n (n - i)^2$ 的估值，而这类求和正是用积分 $\int_0^n (n - x)^2 dx$ 夹逼求上下界的日常练习，和用概率尾部界给随机化算法估成功率是同一套先界定再取价的手感，区别在于积分给出的是确定性的上下界，尾部界给的是概率意义上的界。还有一个容易被忽略的数值细节：小矩阵 $I_k - V^T U$ 一旦接近奇异、条件数爆炸，小逆就会把误差放大到修正项里，此时低秩捷径反而

不如老老实实重建。所以推荐一个命名模板，把这条检查固化成动作：**低秩更新成本检验法**——第一步写出两个方向的乘积并验算相等，第二步检查小块 $I_k - V^T U$ 是否病态或奇异，第三步比较 $O(n^3)$ 与 $O(n^2 k + k^3)$ 的总账，第四步确认旧逆确实已在手上。四问都过，才允许走捷径。

公式 (7) 的快速证明同样是直接验证 $MM^{-1} = I$ ：

$$(I_n - UV^T) \left[I_n + U (I_k - V^T U)^{-1} V^T \right] = I_n - UV^T + (I_n - UV^T) U (I_k - V^T U)^{-1} V^T.$$

把该式中的 $(I_n - UV^T) U$ 换成 $U (I_k - V^T U)$ 。这是一个漂亮的小恒等式！右端随即化简为 $I_n - UV^T + UV^T$ ，也就是 I_n 。这就证明了公式 (7)。

同样存在一个尺寸为 $n + k$ 的扩展矩阵 E ，它掌握着钥匙：

$$E = \begin{bmatrix} I_n & U \\ V^T & I_k \end{bmatrix} \quad \text{has determinant} = \det(I_n - UV^T) = \det(I_k - V^T U). \tag{8}$$

若 $k \ll n$ ，则 (7) 的右端很可能比直接攻击左端更容易也更快：尺寸为 k 的矩阵 $V^T U$ 比尺寸为 n 的 UV^T 更小。

[example]

Example 1 (例 1) 问：下式的逆是什么？

$$M = I - \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

本例中 $u = v = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ 。

Solution (解) 这里 $v^T u = 3$ ，且 $M^{-1} = I + \frac{uv^T}{1-3}$ 。所以 M^{-1} 等于

$$M^{-1} = I + \frac{uv^T}{1-3} = I - \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Strang §III.1, p.160–161

本页为书页 162，属 Part III「Low Rank and Compressed Sensing（低秩与压缩感知）」；页眉与页码为重复元素，不译。

[example]

Example 2 (例 2)

如果 $M = I - \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = I - UV^T$ ，那么 $M^{-1} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ 。

这个结果来自把第一个显示出来的矩阵写成 UV^T ，再反过来写成 $V^T U$ ：

$$UV^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad V^T U = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

于是上面的 M^{-1} 就是 $I_3 + U [I_2 - V^T U]^{-1} V^T = I_3 + U \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}^{-1} V^T$ 。

全部要点在于： 3×3 矩阵 M^{-1} 来自对那个粗体的 2×2 矩阵求逆。

**[note] 译者注（例 2：低秩修正为什么省力）：本页扫描件符号损坏较多，译文按数学语境重建：盖、风、鬱 都指矩阵 A ， $\bar{\mathbf{I}}$ 表示上标 -1 （即求逆）， $\mathcal{I}$ 是单位矩阵 I ，什 om、wnting 分别是 from 与 writing 的误识，公式 (9)、(10) 编号沿用原书。读例 2 时，请把它当成一次“用小矩阵撬动大矩阵”的演练。你在大一《线性代数》里已经会算 2×2 矩阵的逆：先算行列式 $ad - bc$ ，再交换 a 与 d 、给 b 与 c 变号，最后除以行列式；本页要用的正是这门手艺。例中 $M = I - UV^T$ 是

3×3 的, 可真正需要动手求逆的只有 2×2 的 $I - V^T U$, 其余只是矩阵乘法。省力的原因藏在维度里: U 是 3×2 , V^T 是 2×3 , 两者相乘虽然排出一个 3×3 的矩阵, 秩却至多为 2, 它携带的信息只有两列那么多。《线性代数》里的 “秩” 在这里恰好成了 “代价” 的度量: 秩为 k 的修正只需付一个 $k \times k$ 的逆, 而不必付 $n \times n$ 的逆。请动手核对两件事。第一, 把上面的 U 与 V^T 相乘, 确认结果确实是那个 3×3 矩阵, 并想清楚 $V^T U$ 为什么只有 2×2 。第二, 把求得的 M^{-1} 与 $M = I - UV^T = \begin{bmatrix} 1 & -1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$ 相乘, 看乘积是否为单位矩阵; 自己乘一遍, 比读十遍更能记住这个恒等式。《计算方法》或《数值分析》课里也讲 “不要贸然求逆”, 动机与本页一致, 区别在关注点: 那边担心浮点误差与运算量, 本页关心的是代数结构能否把大问题化小。《程序设计》课里的算法复杂度分析则给出另一种视角: 在 A^{-1} 已经算好的前提下, 直接算 $n \times n$ 的逆大约要 n^3 次乘法, 而按本页公式先求 C^{-1} ($k \times k$)、再乘三个小矩阵, 量级降到 $n^2 k$ 一档; 那边数的是操作次数, 本页证的是恒等式, 两者合起来才说明低秩修正确实划算。最后留一个问题: 如果 U 是 $n \times 1$ 、 V^T 是 $1 \times n$, 也就是秩 1 的情形, 本页公式会退化成什么样子?

[section]

Perturbing any Invertible Matrix A (扰动任意可逆矩阵 A)

到此为止, 我们都是从单位矩阵 $I = I_n$ 出发的。我们把它改成 $I - uv^T$, 随后又改成 $I - UV^T$, 先做秩 1 的改动, 再做秩 k 的改动。为了完整享用 Sherman-Morrison-Woodbury (谢尔曼-莫里森-伍德伯里) 思想带来的好处, 我们现在再进一步: 从 A 开始, 而不是从 I 开始。

Perturb any invertible A by a rank k matrix UV^T . Now $M = A - UV^T$. (用秩 k 矩阵 UV^T 扰动任意可逆矩阵 A . 此时 $M = A - UV^T$.)

Sherman-Morrison-Woodbury formula (谢尔曼-莫里森-伍德伯里公式)

$$M^{-1} = (A - UV^T)^{-1} = A^{-1} + A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1}. \tag{9}$$

到此为止, A 都取作 I_n 。最终的公式 (9) 仍然与一个扩展矩阵 E 相联系。设 A 可逆, $E = \begin{bmatrix} A & U \\ V^T & I \end{bmatrix}$; 当 $M = A - UV^T$ 可逆时, E 也可逆。

要找 E 的这个逆, 我们可以做行运算把 V^T 换成零: 把第 1 行乘以 $V^T A^{-1}$, 再从第 2 行中减去, 得到

$$\begin{bmatrix} A & U \\ 0 & I - V^T A^{-1}U \end{bmatrix}.$$

或者做列运算把 U 换成零: 把第 1 列乘以 $A^{-1}U$, 再从第 2 列中减去, 得到

$$\begin{bmatrix} A & 0 \\ V^T & I - V^T A^{-1}U \end{bmatrix}.$$

正如方程 (6) 那样, 我们有两种方式求 E 的逆, 而 E^{-1} 的这两种形式必定相等:

$$\begin{bmatrix} A^{-1} + A^{-1}UC^{-1}V^T A^{-1} & -A^{-1}UC^{-1} \\ -C^{-1}V^T A^{-1} & C^{-1} \end{bmatrix} = \begin{bmatrix} M^{-1} & -M^{-1}U \\ V^T B^{-1} & I_k + V^T M^{-1}U \end{bmatrix}. \tag{10}$$

这里 C 就是 $I - V^T A^{-1}U$, 而 M 就是 $A - UV^T$ 。我们所求的矩阵是 M^{-1} , 也就是 A 被扰动之后的逆。比较方程 (10) 中的 (1, 1) 分块, 就得到方程 (9)。

Summary (总结)

矩阵 $M = A - UV^T$ 的 $n \times n$ 逆, 来自 A 的 $n \times n$ 逆与 $C = I - V^T A^{-1}U$ 的 $k \times k$ 逆。若要一个快速证明, 把 (9) 式乘以 $A - UV^T$ 即可。

****[note]** 译者注 (Woodbury 公式: 一本维度账本) : **公式 (9) 说的是: 矩阵 A 被秩 k 的 UV^T 修正之后, 新矩阵的逆不必从头算, 只要在旧逆 A^{-1} 上打一个低秩补丁。读这类公式最稳的办法是盯住尺寸, 把它当成一本账: A 与 M 都是 $n \times n$, U 是 $n \times k$, V^T 是 $k \times n$, 所以 $V^T A^{-1}U$ 是 $k \times k$, 它的逆 C^{-1} 也是 $k \times k$; 而 $A^{-1}U$ 是 $n \times k$, $V^T A^{-1}$ 是 $k \times n$, 三者相乘恰好回到 $n \times n$ 。账目对得上, 公式才站得住, 这也是检查自己有没有抄错下标最快的办法。扩展矩阵 E 是理解公式 (9) 与 (10) 的钥匙。把 A 、 U 、 V^T 、 I 拼成一个大矩阵以后, 用分块消元先清掉左下角的 V^T , 得到一种分块三角形式; 再清掉右上角的 U , 得到另一种分块三角形式。两种形式各自求逆、再让它们相等, 比较左上角分块就得到公式 (9), 比较其余分块就得到 (10) 中的四个块。这里出现的 $C = I - V^T A^{-1}U$ 就是《计算方法》课里说的 Schur 补 (分块消元之后剩下的那个小块), 连接路径是同一条分块高斯消元; 区别在于那门课关心主元选取与数值稳定性, 本页只关心代数恒等式本身。另一条连接在《最优化方法》与机器学习里: 拟牛顿法 (BFGS 及其变体) 并不每步重算 Hessian 的逆, 而是用旧逆加上一个秩二矩阵的修正去逼近新逆, 路径与本节完全一致; 区别是那边的修正矩阵由割线方程推出来, 而这里的 U 与 V^T 是事先给定的。智能科学与技术专业常见的递推最小二乘与卡尔曼滤波也走这条路: 新数据到来时只做一次秩 1 更新, 公式与例 2 的秩 k 版本同源。请你自己推一遍: 当 $k = 1$ 、 $U = u$ 、 $V^T = v^T$ 时, 公式 (9) 变成什么? 那个分母 $1 - v^T A^{-1}u$ 提醒你: 一旦扰动把行列式推到零, 逆就不复存在。

§III.1 Changes in A^{-1} from Changes in A (A 变化时 A^{-1} 的变化)

这里是汇集四个密切相关的矩阵恒等式的好地方。在每一种情形里，左端的矩阵 B 、 A^T 或 U 都会在右端重新出现，即使它与 A 或 V 不可交换。正如许多证明那样，结合律就藏在眼皮底下： $B(AB) = (BA)B$ 。

$$\begin{aligned} B(I_m + AB) &= (I_n + BA)B \\ B(I_m + AB)^{-1} &= (I_n + BA)^{-1}B \\ A^T(AA^T + \lambda I_n)^{-1} &= (A^T A + \lambda I_m)^{-1}A^T \\ U(I_k - V^T U) &= (I_n - UV^T)U \end{aligned}$$

A 是 $m \times n$ 矩阵， B 是 $n \times m$ 矩阵。第二个恒等式包含这样一个事实： $I + AB$ 可逆，恰好当 $I + BA$ 可逆。换言之， -1 不是 AB 的特征值，恰好当 -1 不是 BA 的特征值。 AB 与 BA 拥有相同的非零特征值。关键之处与 §1.6 一样：由 $(I + AB)x = 0$ 可推出 $(I + BA)Bx = 0$ 。

****[note]** 译者注（矩阵恒等式与“同一批特征值”）：******本页扫描件同样有损坏：𐤎𐤍 与 𐤎𐤍 都是单位矩阵 I ，𐤎𐤍 与 𐤎𐤍 都是矩阵 A ，𐤎𐤍 是 Δ ，𐤎𐤍 ul as 是 formulas（公式）的误识，𐤎𐤍 一类乱码出现在乘积记号的位置，译文按语境重建。先说恒等式 $(I + AB)^{-1} = I - A(I + BA)^{-1}B$ 。验证它的办法仍然是“乘回去”：把右端与 $I + AB$ 相乘，展开后含 AB 的项会与 $A(I + BA)^{-1}B$ 的部分互相抵消，只剩单位矩阵。请留意两边出现的 AB 与 BA ： A 是 $m \times k$ 、 B 是 $k \times m$ ，所以 AB 是 $m \times m$ ，而 BA 是 $k \times k$ ；当 n 很大、 k 很小时，等式右端要处理的是那个小得多的矩阵，这就是本页恒等式的实用价值，也是它与上一页 Woodbury 公式的同一血脉。再看特征值那一句： -1 不是 AB 的特征值，恰好当 -1 不是 BA 的特征值。证明只用一行：若 $(I + BA)x = 0$ ，则 $BAx = -x$ ，于是 $(I + AB)Ax = A(x + BAx) = 0$ ；又因为 Ax 必须非零——否则由 $BAx = -x$ 会推出 $x = 0$ ——所以 Ax 是 AB 的特征向量，特征值同为 -1 。把 A 、 B 互换即得反方向，于是“可逆性”在两者之间来回传递。《线性代数》课里“ AB 与 BA 有相同特征值”的说法要打个折扣：当 $m \neq k$ 时两个矩阵连阶数都不同，不可能有相同的特征多项式，能共享的只是非零特征值，多出来的特征值全是零。连接路径是特征多项式的因式分解，区别在于“相似”要求同阶且存在可逆 P 使 $P^{-1}(AB)P = BA$ ，而 $m \neq k$ 时两者并不相似；把这条区别记牢，考试里最常见的那个陷阱就避开了。《数值分析》课把同一件事用作可逆性判据与条件数估计的出发点，那边关心误差会被放大多少倍，这里只关心“逆是否存在”。

The Derivative of A^{-1} (A^{-1} 的导数)

稍后本节就会转向逆公式的应用。首先我转向 matrix calculus（矩阵微积分）！导数的全部要点在于：当 x 被移动得非常小时，求出函数 $f(x)$ 的变化。这个变化 Δx 会产生变化 Δf ，而 Δf 与 Δx 之比就趋近于导数 df/dx 。

这里 x 是一个矩阵 A ，函数是 $f(A) = A^{-1}$ 。当 A 变化时， A^{-1} 如何变化？到此为止，变化 uv^T 或 UV^T 在秩的意义上是小的；现在我们要求的变化在 A 中是无穷小的，而且可以是任意秩。

我从字母 $B = A + \Delta A$ 开始，写下这个非常有用的矩阵公式：

$$B^{-1} - A^{-1} = B^{-1}(A - B)A^{-1}.$$

你可以看出这个方程是正确的：在右端， AA^{-1} 是 I ，而 $B^{-1}B$ 也是 I 。事实上，(11) 可以推出前面关于 $(A - UV^T)^{-1}$ 的公式。它还立刻表明：如果 $A - B$ 的秩为 1（或 k ），那么 $B^{-1} - A^{-1}$ 的秩也为 1（或 k ）。这里假定 A 与 B 都可逆，所以乘以 B^{-1} 或 A^{-1} 对秩没有影响。

现在把 $A = A(t)$ 看作随时间 t 变化的矩阵，它在每个时刻 t 的导数就是 dA/dt 。当然 A^{-1} 也随时间 t 变化，我们想求出它的导数 dA^{-1}/dt 。于是把那些变化量 $\Delta A = B - A$ 与 $\Delta A^{-1} = B^{-1} - A^{-1}$ 都除以 Δt ；再在方程 (11) 中用 $A + \Delta A$ 代替 B ，并令 $\Delta t \rightarrow 0$ 。

$$\frac{\Delta A^{-1}}{\Delta t} = -(A + \Delta A)^{-1} \frac{\Delta A}{\Delta t} A^{-1} \text{ approaches } -\frac{dA^{-1}}{dt} = -A^{-1} \frac{dA}{dt} A^{-1}.$$

对一个 1×1 矩阵 $A = t$ ，此时 $dA/dt = 1$ ，我们就重新得到 $1/t$ 的导数 $-1/t^2$ 。习题 7 指出， A^2 的导数并不是 $2A dA/dt$ 。

****[note]** 译者注（矩阵微积分：把 $(1/t)' = -1/t^2$ 升一维）：******本页乱码中，𐤎𐤍 是 ΔA ，𐤎𐤍 是 A ，𐤎𐤍 是 Δ ，公式 (11)、(12) 的编号沿用原书。导数在矩阵上的含义与一维完全一样：让自变量发生一个小扰动，看函数值跟着变多少，再把两者的比取极限。区别只在于“小”的度量换成了矩阵范数，“比”换成了矩阵乘法，而矩阵乘法不满足交换律，所以相乘的顺序必须原样保留。公式 (11) 是这一步的技术核心： $B^{-1} - A^{-1} = -A^{-1}(B - A)B^{-1}$ 。把 $B = A + \Delta A$ 代进去，两边除以 Δt ，再令 $\Delta t \rightarrow 0$ ，就得到公式 (12)： $dA^{-1}/dt = -A^{-1}(dA/dt)A^{-1}$ 。取 1×1 的情形 $A = t$ ，它退化成 $(1/t)' = -1/t^2$ ，与你在大一《数学分析》里背熟的结果一致；那里学过的乘积法则与链式法则在矩阵情形依然成立，只是每一项的次序不能随意调换，这就是矩阵微积分与标量微积分唯一却致命的差别。习题 7 提醒的正是这个差别： A^2 的导数是 $A dA/dt + (dA/dt)A$ ，只有当 A 与 dA/dt 可交换时才等于 $2A dA/dt$ ；请用一个 2×2 的不可交换例子亲手验一次，印象会深得多。往前看，《机器学习》与深度学习里的反向传播、自动微分处理的正是这类矩阵函数求导：连接路径是把每一层网络看成一个矩阵函数，再按链式法则逐层回传；区别在于工程实现从不明文构造 $n^2 \times n^2$ 的大型导数矩阵，而是用计算图按需计算向量与矩阵的乘积。《数值分析》课则在另一头接上：它关心的不是导数公式本身，而是当 A 接近奇异时 A^{-1} 的扰动会被放大多少倍（条件数），那正是公式 (12) 里两个 A^{-1} 同时出现所发出的警告。

[section]

Updating Least Squares (最小二乘的更新)

第 II.2 节讨论了最小二乘方程 $A^T A \hat{x} = A^T b$ ——即用来最小化 $\|b - Ax\|^2$ 的 normal equations (正规方程)。假设新来一个方程: A 多出一个新行 r ($1 \times n$), 并且多出一个新测量值 b_{m+1} , 以及一个新的 $\hat{x}$:

$$\begin{bmatrix} A^T & r^T \end{bmatrix} \begin{bmatrix} \hat{x} \\ r \end{bmatrix} = \begin{bmatrix} A^T & r^T \end{bmatrix} \begin{bmatrix} b \\ b_{m+1} \end{bmatrix} \quad \text{is} \quad \begin{bmatrix} A^T A + r^T r \end{bmatrix} \hat{x} = A^T b + r^T b_{m+1}.$$
 (13)

新正规方程中的矩阵是 $A^T A + r^T r$ 。这是对原矩阵 $A^T A$ 的一个 rank one correction (秩一修正)。为了更新 $\hat{x}$, 我们并不想重新构造并求解一整套新的正规方程。相反, 我们使用更新公式:

$$\begin{bmatrix} A^T A + r^T r \end{bmatrix}^{-1} = (A^T A)^{-1} - c(A^T A)^{-1} r^T r (A^T A)^{-1} \quad \text{with } c = 1/(1 + r(A^T A)^{-1} r^T)$$
 (14)

为了快速求出 c , 我们只需要解一次旧方程 $(A^T A)y = r^T$ 。

习题 4 将把最小二乘解 $\hat{x}_{\text{new}}$ 写成对 $\hat{x}$ 的更新。当 A 新增 M 行而不只是新增一行时, 同样的想法依然成立。这就是 **recursive least squares (递归最小二乘)**。

[section]

The Kalman Filter (卡尔曼滤波)

Kalman 注意到, 这一更新思想同样适用于 **dynamic least squares (动态最小二乘)**。dynamic (动态) 一词意味着: 即使没有新数据, state vector (状态向量) x 也随时间而变。若 x 给出一个 GPS 卫星的位置, 那么该位置大约会移动 $\Delta x = v \Delta t$ (v 为速度)。这个近似——或者一个更好的近似——将作为新时刻 x_{n+1} 的 **state equation (状态方程)**。随后在时刻 $t + \Delta t$ 得到的新测量值 b_{m+1} 会进一步把该近似位置更新为 $\hat{x}_{\text{new}}$ 。我希望你看到: 我们现在是把**两个新方程** (状态方程与测量方程) 添加到原来的系统 $Ax \approx b$ 上:

Original (原始) State update (状态更新) Measurement update (测量更新)

$$A_{\text{new}} = \begin{bmatrix} A & 0 \\ -I & I \\ 0 & r \end{bmatrix} \begin{bmatrix} x_{\text{old}} \\ x_{\text{new}} \end{bmatrix} = \begin{bmatrix} b \\ v \Delta t \\ b_{m+1} \end{bmatrix}.$$
 (15)

我们要的是 (15) 的最小二乘解。还有一个额外的曲折, 使 Kalman filter (卡尔曼滤波) 公式真正令人印象深刻 (或者真正复杂)。状态方程与测量方程各自拥有自己的 covariance matrices (协方差矩阵)。这些方程当然是不精确的。方差或协方差 V 度量它们各自不同的可靠性。正规方程 $A^T A \hat{x} = A^T b$ 应当恰当地用 V^{-1} 加权, 从而成为 $A^T V^{-1} A \hat{x} = A^T V^{-1} b$ 。而实际上, V 本身也必须在每一步更新。

贯穿这一切, Kalman 追求的目标是使用更新公式。为了学得 $\hat{x}_{\text{new}}$, 他不是求解完整正规方程, 而是分两步更新 $\hat{x}_{\text{old}}$ 。

prediction (预测) $\hat{x}_{\text{state}}$ 来自状态方程。接着是利用新测量值 b_{m+1} 对 $\hat{x}_{\text{new}}$ 所作的 correction (校正): 零校正。

$$K = \text{Kalman gain matrix} \quad \hat{x}_{\text{new}} = \hat{x}_{\text{state}} + K(b_{m+1} - r\hat{x}_{\text{state}})$$
 (16)

gain matrix (增益矩阵) K 由 A 与 r 以及协方差矩阵 V_{state} 和 V_b 生成。你看到: 如果新的 b_{m+1} 与预测 $\hat{x}_{\text{state}}$ 完全一致, 那么在 (16) 中从 $\hat{x}_{\text{state}}$ 到 $\hat{x}_{\text{new}}$ 的校正就是零。

[note] 译者注 (**秩一更新与卡尔曼滤波的同一件事**): 先把 (14) 的来历钉死。它叫 Sherman-Morrison 公式 (舍曼-莫里森公式): 给可逆矩阵 M 加上秩一项 $r^T r$, 新逆不必从头算, 只在旧逆上加一个由 $(A^T A)^{-1} r^T$ 与 $r(A^T A)^{-1}$ 拼出的修正项。要点在于 r^T 只有一列, $A^T A + r^T r$ 相对 $A^T A$ 只多出一个方向的曲率——**秩一改动只值得一次秩一修正**。公式里两次出现的 $(A^T A)^{-1} r^T$ 是同一个向量, 所以整步只花 $O(n^2)$ 次运算, 不必对 $A^T A$ 重做 $O(n^3)$ 的分解。分母 $1 + r^T (A^T A)^{-1} r$ 是 $\det(A^T A + r^T r)$ 与 $\det(A^T A)$ 的比值: 列满秩时 $(A^T A)^{-1}$ 正定, 故分母恒大于 1, 公式永不失效; 若 A 本来不满秩, 这个分母会给出警报, 这正是数值实现里必须监测它的原因。还有一个实现细节值得记住: 为求 c 要先解 $(A^T A)y = r$, 用的仍是**旧**矩阵, 一次分解可反复复用, 新行每来一条只需一次前代回代, 这是它能在线运行的根本原因。递归最小二乘正是把这件事做成算法: 每来一个观测, 先把新行接到 A 上, 用 (14) 改写逆, 再乘新右端得解, 历史数据不必保存。跨课程连接 (数值分析 / 6.006): 这属于「增量算法」——用一点额外状态 (这里就是那个逆) 换掉重算整题的成本; 与 6.006 里动态维护数据结构、避免每次重建的区别在于, 这里代价的瓶颈是矩阵分解而非指针操作。接第 165 页的卡尔曼滤波: 那不过是把同一个秩一更新搬到概率语言里。状态方程写成 $x_{\text{state}} = x_{\text{old}} + \Delta t v$, 它本身就是一条关于未知向量的线性方程; 测量方程又是另一条, 两者叠加就是 (15), 与 (13) 结构完全一致。差别只在权重: 状态方程是外推, 可靠性由 V_{state} 给出; 测量方程来自传感器, 可靠性由 V_{meas} 给出, 加权正规方程 $A^T V^{-1} A x = A^T V^{-1} b$ 就是为此而写。增益 K 由此而生——它正好就是 (14) 里的 $c(A^T A)^{-1} r^T$, 只是换成了协方差矩阵的表达, 于是「状态预测方差大就把新测量多信一点、测量噪声大就多信预测」成了同一个式子的两种读法。接控制类课程的状态空间模型: 状态方程与测量方程并列, 只是把未知数从静态的 x 换成随时间演化的 x_t , 测量矩阵 H_t 取代单独一行 r_t , 结构判断完全一样, 要学会的迁移是「动态只是多了一维时间下标」。

[note] 译者注 (OCR 重建说明)： 本页 OCR 文本噪声严重 (如 $\mathbf{T}$ 实为 $A^T A$、$\mathbf{T} = \mathbf{A} \mathbf{T} \mathbf{b}$ 实为 $A^T A x = A^T b$、$[\mathbf{T} + \mathbf{r} \text{Tr} \mathbf{l}]$ 实为 $[A^T A + r^T r]^{-1}$、什 om 实为 from、鬱 实为 $v \Delta t$、盖 b 实为 $Ax = b$)，全部公式均按数学语境重建。
Strang §III.1, p.164
本页为书页 165，属 §III.1 正文；页眉与页码为重复元素，不译。

我们还需要更新整个系统的协方差——用它来度量 $\hat{x}_{\text{new}}$ 的可靠性。事实上，这个 V 往往是最重要的输出。它度量的是产生 $\hat{x}_{\text{final}}$ 的整个传感器系统的精度。

对于 GPS 应用，我们与 Kai Borre 合著的教材提供了多得多的细节：*Algorithms for Global Positioning* (Wellesley-Cambridge Press)。其目标是估计 GPS 测量的精度：对 tectonic plates（构造板块）的测量精度极高，对卫星的精度较低，而对你汽车位置的精度则低得多。

[section]

Quasi-Newton Update Methods (拟牛顿更新方法)

在求解 $f(x) = 0$ 的近似 Newton 方法中，出现了一种完全不同的更新。这些是关于 n 个未知量 $x_1, \dots, x_n$ 的 n 个方程。经典的 Newton's method（牛顿法）使用 Jacobian matrix（雅可比矩阵） $J(x)$ ，它包含 f 的每个分量的一阶导数：

$$\text{Newton} \quad J_{ik} = \frac{\partial f_i}{\partial x_k} \quad \text{and} \quad x_{\text{new}} = x_{\text{old}} - J(x_{\text{old}})^{-1} f(x_{\text{old}}).$$
(17)

这建立在微积分的基本近似 $J\Delta x = \Delta f$ 之上。这里 $\Delta f = f(x_{\text{new}}) - f(x_{\text{old}})$ 是 $-f(x_{\text{old}})$ ，因为我们整个计划就是要实现 $f(x_{\text{new}}) \approx 0$ 。

困难在于 Jacobian 矩阵 J 。对于大的 n ，即使 automatic differentiation（自动微分）（这是第七章中 backpropagation（反向传播）的关键）也会很慢。于是不再在每次迭代重新计算 J ，而是改用 quasi-Newton methods（拟牛顿方法），使用更新公式 $J(x_{\text{new}}) = J(x_{\text{old}}) + \Delta J$ 。

原则上 ΔJ 涉及 J 的导数，因而是 f 的二阶导数。其回报是二阶精度以及 Newton 方法的快速收敛。但当 n 很大（例如在 deep learning（深度学习）中）时，计算全部二阶导数的代价可能高得无法承受。

拟牛顿方法在 $J(x_{\text{old}})$ 上构造一个 low rank update（低秩更新），而不是在 x_{new} 处计算一个全新的 Jacobian。该更新反映了随 (17) 中计算 x_{new} 而来的新信息。由于 Newton 方法中出现的是 J^{-1} ，更新公式针对它对 J_{new}^{-1} 的秩一改变——而不必重新计算 J^{-1} 。这里的关键是，而且不需要 $f_1, \dots, f_n$ 的导数：

$$\text{Quasi-Newton condition} \quad J_{\text{new}}(x_{\text{new}} - x_{\text{old}}) = f_{\text{new}} - f_{\text{old}}$$
(18)

这就是我们沿移动方向所获得的关于 $J\Delta x = \Delta f$ 的信息。由于方程 (17) 用的是 J^{-1} 而不是 J ，我们就更新 J^{-1} 使之满足 (18)。Sherman-Morrison 公式可以做到这一点。或者用 BFGS correction（BFGS 校正），那是由四位作者各自独立发现的一个 rank-2 matrix（秩二矩阵）。另一种做法是去更新 J_{old} 的 LU 分解或 LDL^T 分解。

[note] 译者注 (拟牛顿法：知道得少，就改动得少)：核心两个字：**最小改动**。为什么不能直接算 J ？牛顿法每步要解 $J(c_{\text{old}}) \Delta c = -f(c_{\text{old}})$ ， J 是 $n \times n$ ；在深度学习里，求出它的每个偏导向量都对应一次第七章的反向传播， n 到千万级，代价爆炸。而一步迭代真正买到的新信息只有割线条件 (18) 那一条向量方程： n 个标量方程定不住 n^2 个未知元，欠定维数是 $n^2 - n$ 。所以要补一条选择准则，把解唯一化。**补哪条准则，就得到哪个算法**。BFGS 的准则是「在对称正定矩阵里挑离旧矩阵最近的那个」，于是它天然保对称、在凸问题里保正定，搜索方向才保证下降；它用两个秩一项而非一个，正是为了留在正定这个约束类里——若只在对称类里找最近点，最小改动只需一个秩一项（SR1），但它可能破坏正定性。约束类放宽，解的形式更简单，你要的性质也更弱，这就是设计上的取舍。验算一个二维反例： $c_{\text{old}} = (0, 0)$ 、 $c_{\text{new}} = (1, 1)$ 、 $f_{\text{old}} = (1, 0)$ 、 $f_{\text{new}} = (0, 1)$ ，则 $\Delta c = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ ，割线条件要求 $J_{\text{new}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ 。取 $J_{\text{old}} = I$ ，SR1 更新中 $y - Js = \begin{pmatrix} -2 \\ 0 \end{pmatrix}$ 、分母 $(y - Js)^T s = -2$ ，得 $J_{\text{new}} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ ：割线条件成立，特征值却是 -1 与 1 ，已不正定；这正是 BFGS 宁愿多花一个秩也要保住正定性的原因。再看 BFGS 本身：取 $\Delta f = \begin{pmatrix} 2 \\ -2 \end{pmatrix}$ ，则 $y^T s = 0$ ，而 BFGS 的分母正是 $y^T s$ ，更新直接无定义——工程实现靠 Wolfe 线搜索保证 $y^T s > 0$ ，这才是那张安全网。往前看一步：只存最近 m 步 $\Delta c, \Delta f$ 、把内存从 $O(n^2)$ 压到 $O(mn)$ 的做法就是 L-BFGS； $n = 10^6$ 时满矩阵要 10^{12} 个数， $m = 10$ 的 L-BFGS 只要 10^7 个数，差五个数量级，这就是它成为深度学习默认优化器的原因。命名模板：**约束类决定秩数**——先写下你要的性质集合（对称、正定、低秩），再在这个集合里做最小改动，公式里出现几项秩一、分母上出不出 $y^T s$ ，都是约束类的副产品。跨课程连接（凸优化 / 6.006）： $y^T s > 0$ 就是函数沿 s 方向的平均曲率为正，Armijo 加 Wolfe 线搜索是花一点额外求值把它买到手的，没有线搜索，BFGS 的正定性只是纸面上的；而 6.006 关心最坏情况界与数据结构不变式，这里的关键量是收敛阶（牛顿二次、拟牛顿超线性、梯度下降线性）与每步预算的乘积，衡量口径不同。与 part002 的 minimum norm solution 是同一个原则：约束不够时，用最小改动或最小范数把解钉死。

常常，原来那 n 个方程 $f(x) = 0$ 来自最小化一个函数 $F(x_1, \dots, x_n)$ 。此时 $f = (\partial F / \partial x_1, \dots, \partial F / \partial x_n)$ 正是该函数 F 的 gradient（梯度），而在 minimum point（极小值点）处有 $f = 0$ 。于是 Jacobian 矩阵 J （ f 的一阶导数）变成 Hessian matrix（海森矩阵） H （ F 的二阶导数）。它的元素是 $H_{jk} = \partial^2 F / \partial x_j \partial x_k$ 。

如果一切顺利，Newton 方法会迅速找到使 F 取极小值的点 x^* ，在该点导数为 $f(x^*) = 0$ 。那种近似地更新 J 、而不重新计算 J 的拟牛顿方法，对大型 n 来说代价要低得多。对于极大规模的 n （正如机器学习中的许多问题），其代价可能仍然过高。

Strang §III.1, p.165

本页为书页 166，属 Part III「低秩与压缩感知」（Low Rank and Compressed Sensing）中 §III.1 的习题页（Problem Set III.1）；页眉为 Part 题名与页码，属重复元素，不译。

[section]

Problem Set III.1（习题集 III.1）

[note] OCR 修正： 本页 OCR 把 A 认成 盖 / @，把 B 认成 刀，把 U, V 认成 鬱 / 陶，把 Δ 认成 $\triangle$ ，把负号与括号认成 $-$ 。所有矩阵、向量、上下标均按 Sherman-Morrison-Woodbury（谢尔曼-莫里森-伍德伯里）恒等式的数学语境重建；Problem 2 的两个分块逆矩阵按分块消元的结果补全，Problem 6 中的恒等式按可验证的形式写为 $(I - UV^T A^{-1})U = U(I - V^T A^{-1}U)$ 。本页为习题页，**题干保留英文原文**，乱码处只还原为 LaTeX。

[problem] Problem 1. Another approach to $(I - uv^T)^{-1}$ starts with the formula for a geometric series:

$$(1 - x)^{-1} = 1 + x + x^2 + x^3 + \cdots .$$

Apply that formula when $x = uv^T = \text{matrix}$ （秩一情形）：

$$\begin{aligned} (I - uv^T)^{-1} &= I + uv^T + uv^T uv^T + uv^T uv^T uv^T + \cdots \\ &= I + u \left[1 + v^T u + v^T uv^T u + \cdots \right] v^T. \end{aligned}$$

Take $x = v^T u$ to see $I + \frac{uv^T}{1 - v^T u}$. This is exactly equation (1) for $(I - uv^T)^{-1}$.

[problem] Problem 2. Find E^{-1} from equation (4) with $D = 1 - v^T u$ and also from equation (5): From (4) $E^{-1} = \begin{bmatrix} I & -uD^{-1} \\ 0 & D^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ -v^T & 1 \end{bmatrix} = \begin{bmatrix} I + uv^T D^{-1} & -uD^{-1} \\ -D^{-1} v^T & D^{-1} \end{bmatrix}$ From (5) $E^{-1} = \begin{bmatrix} (I - uv^T)^{-1} & 0 \\ -v^T (I - uv^T)^{-1} & 1 \end{bmatrix} \begin{bmatrix} I & -u \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} (I - uv^T)^{-1} & -(I - uv^T)^{-1} u \\ -v^T (I - uv^T)^{-1} & 1 + v^T (I - uv^T)^{-1} u \end{bmatrix}$ Compare the 1, 1 blocks to find $M^{-1} = (I - uv^T)^{-1}$ in formula (1).

[problem] Problem 3. The final Sherman-Morrison-Woodbury formula (9) perturbs A by UV^T (rank k). Write down that formula in the important case when $k = 1$:

$$M^{-1} = (A - uv^T)^{-1} = A^{-1} + \underline{\hspace{2cm}}.$$

Test the formula on this small example:

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \quad u = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad v = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad A - uv^T = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

[problem] Problem 4. Problem 3 found the inverse matrix $M^{-1} = (A - uv^T)^{-1}$. In solving the equation $My = b$, we compute **only the solution** y and not the whole inverse matrix M^{-1} . You can find y in two easy steps : **Step 1** Solve $Ax = b$ and $Az = u$. Compute $D = 1 - v^T z$. **Step 2** Then $y = x + \frac{v^T z}{D} z$ is the solution to $My = (A - uv^T)y = b$. Verify $(A - uv^T)y = b$. We solved two equations using A , no equations using M .

[problem] Problem 5. Prove that the final formula (9) is correct ! Multiply equation (9) by $A - UV^T$. Watch for the moment when $(A - UV^T)A^{-1}U$ becomes $U(I - V^T A^{-1}U)$.

[problem] Problem 6. In the foolish case $U = V = I_n$, equation (9) gives what formula for $(A - I)^{-1}$? Can you prove it directly?

[note] 译者注（跨课程连接）： 本页六道题围绕同一个设计问题： A 被低秩矩阵 UV^T 改动之后，怎样不重做一次 $n \times n$ 求逆，就得到 $(A - UV^T)^{-1}b$ ？先算代价。一次 LU 分解约需 $\frac{2}{3}n^3$ 次浮点运算，直接求逆同量级；第 4 题的两步法把 A 的分解冻结一次，此后每换一组 U, V ，只需解 $k + 1$ 个右端项（每个约 $2n^2$ 次运算），再解一个 $k \times k$ 的小系统（约 $\frac{2}{3}k^3$ ）。取 $n = 1000$ 、 $k = 1$ ：重算求逆约 6.7×10^8 次运算，而更新一次不到 10^7 ，相差两个数量级。

为什么必须这样做？从第一性原理看，改动量 UV^T 的秩至多是 k ，它只在一个 k 维子空间里「推」了矩阵一把，而分解 $A = LU$ 的 $O(n^3)$ 工作量几乎全部花在 n 维整体结构上。如果为了 k 维的扰动去动 n 维的分解，就是把大头成本绑在小头改动上，属于设计上的浪费。把 A^{-1} 看作算子， $(A - UV^T)^{-1}b$ 需要解 $(A - UV^T)x = b$ ，等价于 $Ax = b + UV^T x$ ：右边多出来的 $UV^T x$ 又含未知的 x ，直接迭代会绕回原点。出路是把未知量降维——令 $x = A^{-1}b + A^{-1}UV^T x$ ，两端左乘 $V^T A^{-1}$ 得 $V^T x = V^T A^{-1}b + V^T A^{-1}UV^T x$ ，于是 $y = V^T x$ 只需求一个 $k \times k$ 系统 $(I_k - V^T A^{-1}U)y = V^T A^{-1}b$ ，再回代 $x = A^{-1}b + A^{-1}Uy$ 。真正要付的只有两次「用冻结分解解方程」： $x_0 = A^{-1}b$ 与 $Z = A^{-1}U$ ，之后每一组新的 U, V 都只是 $k \times k$ 的乘法与一次回代。 $k = 1$ 时这个 $k \times k$ 系统退化成一次除法，即第 4 题的 Sherman–Morrison 形式 $(A - uv^T)^{-1}b = x_0 + \frac{v^T x_0}{1 - v^T z} z$ ，其中 $z = A^{-1}u$ 。分母 $1 - v^T z$ 就是「降维之后剩下的全部难度」：它把无穷多项级数压成了一个标量。

这条线索在 6.006（算法导论）里叫「预处理 + 摊还」：把一个昂贵操作的成本摊到后续每一次便宜操作上；区别在于 6.006 数的是操作次数的最坏情况摊还界，这里数的是浮点运算次数，还要看访存。CSAPP 第 6 章的缓存层次把差距解释得更清楚：解方程阶段做矩阵-向量乘法，顺序访存、缓存命中率高；LU 分解要在整块 n^2 个元素上反复扫描，即使分块优化后仍受内存带宽限制（memory bound）。第 1 题用几何级数 $I + X + X^2 + \cdots = (I - X)^{-1}$ 重推 $k = 1$ 的公式，这与 6.042J（计算机科学数学）生成函数里的 $1 + x + x^2 + \cdots = \frac{1}{1-x}$ 是同一套代数；区别在于标量级数只要求 $|x| < 1$ ，矩阵级数要求 X 的谱半径（特征值绝对值的最大值）小于 1，秩一情形即 $|v^T u| < 1$ 。级数能求和，是因为 $X = uv^T$ 把无穷多项压在同一方向上： $X^m = (v^T u)^{m-1} uv^T$ ，于是总和为 $I + \frac{uv^T}{1-v^T u}$ 。

代一组数验证：取 $u = (1, 1)^T$ 、 $v = (0.25, 0.25)^T$ ，则 $v^T u = 0.5$ ，公式给出 $(I - uv^T)^{-1} = I + 2uv^T$ ，而 $(I - uv^T)(I + 2uv^T) = I + 2uv^T - uv^T - 2 \cdot 0.5 uv^T = I$ ，确实成立。再补一个 2×2 的两步手算，把「冻结分解、只解方程」走完：取 $A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$ 、 $u = (1, 0)^T$ 、 $v = (0.5, 0.5)^T$ 。第一步解 $z = A^{-1}u$ ，得 $z = (2, -1)^T$ ；第二步算内积 $v^T z = 0.5$ ；第三步用公式 $(A - uv^T)^{-1}u = A^{-1}u + \frac{v^T z}{1-v^T z}z = (2, -1)^T + 1 \cdot (2, -1)^T = (4, -2)^T$ 。验算： $A - uv^T = \begin{pmatrix} 0.5 & 1 \\ 0.5 & 2 \end{pmatrix}$ ，直接求逆得 $\frac{1}{0.5} \begin{pmatrix} 2 & -1 \\ -0.5 & 0.5 \end{pmatrix} = \begin{pmatrix} 4 & -2 \\ -1 & 1 \end{pmatrix}$ ，而 $A^{-1} = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$ ，两者相差的正是 $\frac{A^{-1}uv^T A^{-1}}{1-v^T A^{-1}u} = -2 \begin{pmatrix} 1 \\ -0.5 \end{pmatrix} (0.5 \ 0.5)$ ；对右端项 $b = (1, 0)^T$ ，重算给出 $x = (4, -1)^T$ ，按公式 $x_0 = A^{-1}b = (2, -1)^T$ 、 $v^T x_0 = 0.5$ ，得 $x = (2, -1)^T + \frac{0.5}{1-0.5} (2, -1)^T = (4, -1)^T$ 。逐项吻合，说明「先解两个方程，再算一个标量」没有任何信息损失。

失效边界也在同一处： $v^T u = 1$ 时级数不收敛、 $I - uv^T$ 奇异； $v^T u$ 接近 1 时， $1 - v^T u$ 是两个相近数相减，有效位大量丢失，解的相对误差被条件数 $1/(1 - v^T u)$ 放大——这正是 CSAPP 第 2 章讲浮点舍入时反复警告的现象。作为放大版本的失效也值得一验：取 $u = (2, 0)^T$ 、 $v = (0.5, 0.5)^T$ ，则 $\det(I - uv^T) = 1 - v^T u = -1$ ，两个特征值是 $1 - 1 = 0$ 与 1 ，矩阵奇异；此时 $v^T u = 1$ ，幂级数 $I + uv^T + \cdots$ 的每一项都不衰减，永远不会收敛，于是「用公式」和「矩阵不可逆」同时发生，这不是巧合而是同一件事的两种说法。注意 $v^T u < 1$ 只是充分条件：只要 uv^T 的两个非零特征值（ $v^T u$ 与 0 ）都落在单位圆内，级数就收敛；在对称情形 $u = v$ 下这件事退化为最直白的「最大特征值小于 1」，非对称情形则可按谱半径判断，这一点正是 6.042J 里「等价关系与偏序给出的分类」在实矩阵上的类比——条件松紧不同，结论方向一致。

18.065（矩阵方法）里的 QR 更新走的是另一条路：动分解因子本身以保住正交性，换来稳定但每次要付 $O(n^2)$ 并且碰到分解；低秩更新不动分解，只解方程，把全部数值风险集中到分母 $1 - v^T z$ 上。命名模板：**低秩更新法（不动分解，只解方程）**——增量 PCA、递归最小二乘、在线协方差更新都是它的实例。

Strang §III.1, p.166

本页为书页 167，§III.1 习题集（Problem Set III.1）的续页；页眉为小节题名 Changes in A^{-1} from Changes in A 与页码 167，属重复元素，不译。

[note] OCR 修正： 本页 盖 为 A ， $\mathcal{T}$ 为 I ， Δ 为 ΔA ，似 $\textcircled{\theta}$ 为 $A(t)$ ，豆 砒 e eq 况 为 state equation， r 勉 52 为 r_k^2 ，引 d 为 old。Problem 9 中与 A 配套的导数矩阵在 OCR 中只剩 θ 、 1 两个残片，此处按秩一微扰重建为 $A'(t) = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ ；Problem 11 三条方程右侧的方差数值同样损坏，只保留方括号号而不臆造数字。本页同为习题页，**题干保留英文原文**。

[problem] Problem 7. Problem 4 extends to a rank k change $M^{-1} = (A - UV^T)^{-1}$. To solve the equation $My = b$, we compute only the solution y and not the whole inverse matrix M^{-1} . **Step 1** Solve $Ax = b$ and the k equations $AZ = U$ (U and Z are n by k) **Step 2** Form the matrix $C = I - V^T Z$ and solve $Cw = V^T x$. The desired $y = M^{-1}b$ is $y = x + Zw$.

Use (9) to verify that $(A - UV^T)y = b$. We solved $k + 1$ equations using A and we multiplied $V^T Z$, but we never used $M = A - UV^T$.

[problem] Problem 8. What is the derivative of $(A(t))^2$? The correct derivative is not $2A(t)\frac{dA}{dt}$. You must compute $(A + \Delta A)^2$ and subtract A^2 . Divide by Δt and send Δt to 0.

[problem] Problem 9. Test formula (12) for the derivative of $A^{-1}(t)$ when

$$A(t) = \begin{bmatrix} 1 & t^2 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad A^{-1}(t) = \begin{bmatrix} 1 & -t^2 \\ 0 & 1 \end{bmatrix}.$$

[problem] Problem 10. Suppose you know the average $\hat{x}_{\text{old}}$ of $b_1, b_2, \dots, b_{999}$. When b_{1000} arrives, check that the new average is a combination of $\hat{x}_{\text{old}}$ and the mismatch $b_{1000} - \hat{x}_{\text{old}}$:

$$\hat{x}_{\text{new}} = \frac{b_1 + \cdots + b_{1000}}{1000} = \frac{b_1 + \cdots + b_{999}}{999} + \frac{1}{1000} \left(b_{1000} - \frac{b_1 + \cdots + b_{999}}{999} \right).$$

This is a Kalman filter $\hat{x}_{\text{new}} = \hat{x}_{\text{old}} + \frac{1}{1000}(b_{1000} - \hat{x}_{\text{old}})$ with gain matrix $\frac{1}{1000}$.

[problem] Problem 11. The Kalman filter includes also a *state equation* $x_{k+1} = Fx_k$ with its own *error variance* s^2 . The dynamic least squares problem allows x to drift as k increases:

$$\begin{bmatrix} 1 & \\ -F & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} b_0 \\ 0 \\ b_1 \end{bmatrix} \quad \text{with variances} \quad \begin{bmatrix} \sigma^2 \\ s^2 \\ \sigma^2 \end{bmatrix}.$$

With $F = 1$, divide both sides of those three equations by σ, s , and σ . Find $\widehat{x_0}$ and $\widehat{x_1}$ by least squares, which gives more weight to the recent b_1 .

Bill Hager's paper on *Updating the Inverse of a Matrix* was extremely useful in writing this section of the book : SIAM Review **31** (1989).

[note] 译者注 (跨课程连接)：第 10 题把「矩阵被改一点怎么办」搬到数据上：已知前 999 个数的平均 c_{old} ，第 1000 个数到来时，不必把 999 项重新加一遍。把 $\sum_{i=1}^{1000} b_i$ 与 $\sum_{i=1}^{999} b_i$ 相减，差恰好是 b_{1000} ，所以递推 $c_{\text{new}} = c_{\text{old}} + \frac{1}{1000}(b_{1000} - c_{\text{old}})$ 与重算完全等价，省下的只是重复劳动：每次 $O(1)$ 而不是 $O(n)$ ，代价是内存里多留两个状态量（旧均值与计数）。这句话读作「新估计 = 旧估计 + 增益 × 残差」：残差 $b_{1000} - c_{\text{old}}$ 是数据与旧估计的偏差，增益决定这次偏差值得信几分。它在概率论课（Bertsekas & Tsitsiklis《Introduction to Probability》）里就是条件期望的递推形式：把 b_i 看成 c 加噪声，最小二乘 $\min_c \sum (b_i - c)^2$ 的解正是算术平均；若各次测量方差不同，加权最小二乘自动给出权重 $1/\sigma_i^2$ ，而先验方差 σ^2 与测量方差 r^2 对应的增益是 $\sigma^2/(\sigma^2 + r^2)$ ——旧估计越不确定或新测量越准，增益越接近 1。区别在于：概率论讲的是随机变量的条件分布，这里讲的是能直接手算的加权平均，增益从固定的 $\frac{1}{n}$ 变成由方差比决定的数。

为什么增益必须带在残差上，而不是直接给新数据一个固定权重？从第一性原理看，均值是使平方误差总和最小的那个点，每进来一个数据，最优点的移动量正比于「这个数据离当前点有多远」，比例系数反映「该信新数据几分」。取 $n = 1000$ 、增益 $\frac{1}{1000}$ ：若 $c_{\text{old}} = 10$ 、 $b_{1000} = 20$ ，残差是 10，新均值 $10 + 0.01 = 10.01$ ，一个离群点只推动均值 0.01，这正是「用估计的不确定度做阻尼」的手算版本。再看漂移模型 $c_{k+1} = Fc_k$ ($F = 1$) 配状态噪声 r_k^2 ： $F = 1$ 表示「我认为真值大概率没动」，于是先验方差随步数累积变大，而测量方差不变，增益就会从第 4 题那种单调衰减的 $\frac{1}{n}$ 变成趋向一个稳态值，赋予近期数据更大权重，这正是卡尔曼滤波「先预测、再按增益修正」的雏形。

手算两遍，把「递推 = 重算」钉死。序列 4, 6, 8：初始 $c = 0$ 、计数为 0；来 4， $c = 0 + \frac{1}{1}(4 - 0) = 4$ ；来 6， $c = 4 + \frac{1}{2}(6 - 4) = 5$ ；来 8， $c = 5 + \frac{1}{3}(8 - 5) = 6$ ，而直接重算 $\frac{4+6+8}{3} = 6$ ，逐位相同。方差版本也值得走一遍，因为它是工程上真正容易翻车的地方：令 $S_n = \sum_{i=1}^n (b_i - c_n)^2$ （即题目里的 n 倍样本方差），有递推 $S_n = S_{n-1} + \frac{n-1}{n}(b_n - c_{n-1})^2$ 。对 4, 6, 8 依次算： $S_1 = 0$ ； $n = 2$ 时 $S_2 = 0 + \frac{1}{2}(6 - 4)^2 = 2$ （校验 $(4 - 5)^2 + (6 - 5)^2 = 2$ ）； $n = 3$ 时 $S_3 = 2 + \frac{2}{3}(8 - 5)^2 = 8$ （校验 $(4 - 6)^2 + (6 - 6)^2 + (8 - 6)^2 = 8$ ），方差 $\frac{8}{3}$ 与重算一致。关键在于这里用的是残差 $(b_n - c_{n-1})$ 的平方，而不是 b_n^2 与 c_{n-1}^2 相减——后者在数值上等于「两个大数相减」，数据量纲一大就丢位，与下一条警告是同一件事。

第 11 题让 c 随 k 漂移 ($c_{k+1} = Fc_k$, $F = 1$)，并给状态方程配上自己的误差 r_k^2 ，于是最小二乘会自动给近期数据更大权重，这正是卡尔曼滤波的雏形：先预测一步，再用新测量按增益修正。从 6.006 的角度看，这是一次把 $O(n)$ 降到 $O(1)$ 的摊还优化，用「多存两个状态量」换「每次少读一遍数据」；区别是 6.006 关心操作次数的渐近界，这里还要关心数值误差：CSAPP 第 2 章提醒， 10^9 次递推之后舍入误差会累积，旧均值缓慢漂移，工程上改用 Welford 式算法（均值与方差一起更新，避免大数相减）或每隔若干步重算一次来校准。矩阵版本叫递归最小二乘，增益由标量升级为矩阵 $Pa/(1 + a^T Pa)$ ，与 Strang 在最小二乘与 QR 章节反复讲的「别重解方程，只更新因子」是同一条线索；Strang 讲的是对固定数据一次性做正交三角化，这里讲的是数据到来时用旧解加一步修正逼近同一个解。命名模板：**增量更新模板（新 = 旧 + 增益 × 残差）**——均值、方差、协方差以及在线学习的参数更新都落在它上面，区别只在增益怎么取。本页末尾提到的 Hager 综述（《SIAM Review》第 31 卷，1989 年）正是把低秩更新这套语言系统化的文章。

Strang §III.1, p.167

本页为书页 168，属 §III.2 Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）；页眉与页码为重复元素，不译、不收录。

[section]

§III.2 Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）

上一节考察的是：矩阵 A 发生变化时， A^{-1} 会怎样改变。那里既允许无穷小变化 dA ，也允许有限变化 $\Delta A = -UV^T$ ，得到的结论是逆矩阵的无穷小变化与有限变化：

$$\frac{dA^{-1}}{dt} = -A^{-1}\frac{dA}{dt}A^{-1} \quad \text{and} \quad \Delta A^{-1} = A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1} \tag{1}$$

本节对矩阵 A 的 eigenvalue（特征值）与 singular value（奇异值）提出同样的问题：**当矩阵 A 变化时，每一个 λ 、每一个 σ 各自怎样变化？**

你会看到关于 $d\lambda/dt$ 与 $d\sigma/dt$ 的漂亮公式。然而特征值与奇异值本身并没有多少线性性质。微积分对无穷小变化 $d\lambda$ 与 $d\sigma$ 行之有效，因为导数是一个线性算子；但我们不能指望精确知道跳到 $\lambda(A + \Delta A)$ 或 $\sigma(A + \Delta A)$ 处的值。特征值比逆矩阵更复杂。

尽管如此，仍然有好消息：能够做到的事情相当出色。这里先尝一尝 symmetric matrix（对称矩阵） S 的味道。假设 S 变为 $S + uu^T$ （一个正的 rank-one（秩一）变化）。它的特征值从 $\lambda_1 \geq \lambda_2 \geq \cdots$ 变为 $z_1 \geq z_2 \geq \cdots$ 。由于 uu^T 是 positive semidefinite（半正定）的，我们预料特征值会增大：**但增大量究竟有多大？**

$S + uu^T$ 的每个特征值 z_i 都不小于 λ_i ，也不大于 λ_{i-1} （最大的 z_1 只在下侧受 λ_1 约束），所以 λ 与 z 是彼此 interlaced（交错）的。 $z_2, \dots, z_n$ 中的每一个都夹在两个相邻的 λ 之间：

$$z_1 \geq \lambda_1 \geq z_2 \geq \lambda_2 \geq \cdots \geq z_n \geq \lambda_n. \tag{2}$$

即便没有关于 $\Delta\lambda$ 的公式，我们也能得到特征值变化的上界。

[note] 交错定理在说什么：秩一正扰动把每个特征值关进相邻的夹缝

这一段解决的设计问题是：**给矩阵加上一一点低秩的正扰动之后，原来的特征值会跑到哪里。**数据降噪、主成分分析、谱聚类、图信号处理都要反复计算特征值与特征向量；矩阵一旦被改动，就必须知道这些数会不会乱跑、能跑多远。交错（interlacing）给出的回答相当克制：它们只能往上挪，而且挪不过原来的下一个特征值。

读这条式子先要记住排序约定。本节把特征值从大到小排列， λ_1 是最大特征值， λ_n 是最小特征值；新矩阵的特征值同样排成 $z_1 \geq z_2 \geq \cdots \geq z_n$ 。于是 (2) 式的读法是： λ_1 被 z_1 与 z_2 夹住， λ_2 被 z_2 与 z_3 夹住，依此类推，两串数像梳齿一样交叉咬合；换个方向看， $z_2, \dots, z_n$ 每一个也都夹在两个相邻的 λ 之间。最后一项 $z_n \geq \lambda_n$ 说明最小的特征值只会变大，这正是 uu^T 半正定带来的整体抬升。

为什么秩一扰动只能把特征值推进相邻夹缝？关键在于 $v^T uu^T v = (u^T v)^2$ ：只要方向 v 与 u 正交， uu^T 在这个方向上就完全不起作用。于是在 u 的 $n-1$ 维正交补子空间里， $S + uu^T$ 与 S 是同一个矩阵。把特征值理解为「在子空间上让 Rayleigh 商取极值」的结果之后，只允许在 u 的正交补里挑选方向所得到的极值，必然被原来的两个相邻特征值夹住，这就是交错估计的来源。严格证明留给「矩阵论 / 矩阵分析」课，但思路就是这一句。

跨课程连接一：「矩阵论 / 矩阵分析」课里的对称矩阵谱定理 ($S = Q\Lambda Q^T$) 是这里的前提——只有对称矩阵的特征值全是实数，才谈得上「从大到小排成一串」。区别在于，谱定理只保证「能正交对角化、能排序」，交错定理进一步回答「加上秩一正扰动之后，每个特征值被限在哪个区间」。跨课程连接二通向「数值分析」课的扰动界：扰动矩阵的谱范数越小，特征值移动就越小，形式是 $|\lambda_i(S + uu^T) - \lambda_i(S)| \leq \|uu^T\|$ ，这是 Weyl 不等式的特例。两条结论连接路径相同，都是「对称性加二次型」；区别是粗细——Weyl 界只给一个统一的最大位移，交错定理给出逐项的、更紧的夹层信息，正是这份「更紧」让「第二个特征值不可能突然涨很多」变成可以判断的事。

构建主义提示：先在 2×2 的例子上手验证，再接受 n 维的一般结论。取 $S = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$ ，则 $\lambda_1 = 3$ 、 $\lambda_2 = 1$ ；取 $u = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ ，得到 $S + uu^T = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}$ ，特征值为 $3 \pm \sqrt{2}$ ，即 $z_1 \approx 4.41$ 、 $z_2 \approx 1.59$ 。链条 $4.41 \geq 3 \geq 1.59 \geq 1$ 与 (2) 式逐项吻合。另外，本页 OCR 把 λ 、 z 、 A 等符号打乱了；排序方向「从大到小」有两条硬证据：下文答案明确说 $\lambda_2 + C$ 变成「最大」特征值 z_1 ，下一页例题也把较大的特征值 2 记作 λ_1 。

有一点需要提请注意，因为它可能被误解。假设矩阵中的变化 uu^T 取成 $Cq_2q_2^T$ ，其中 q_2 是 S 的第二个单位特征向量。那么 $Sq_2 = \lambda_2q_2$ ，于是

$$(S + Cq_2q_2^T)q_2 = (\lambda_2 + C)q_2$$

我们会在特征值 λ_2 处看到一个跳跃，直接跳到 $\lambda_2 + C$ 。如果 C 很大，这个跳跃就很大。**那么，矩阵 $S + uu^T$ 的第二个特征值怎么可能等于 $z_2 = \lambda_2 + C \leq \lambda_1$ 呢？**

答：如果 C 是一个很大的数，那么 $\lambda_2 + C$ 并不是 $S + uu^T$ 的第二个特征值。它变成了 z_1 ，也就是新矩阵 $S + Cq_2q_2^T$ 的最大特征值（对应的特征向量是 q_2 ）。而原来的最大特征值 λ_1 ，现在成了新矩阵的第二个特征值 z_2 。所以命题 (2) 中 $z_2 \leq \lambda_1 \leq z_1$ 这一说法（在这个例子里）完完全全成立： $z_2 = \lambda_1$ 确实位于 $z_1 = \lambda_2 + C$ 的下方。

[note] 「第几个特征值」说的是名次，不是身份

这里最容易踩的坑，是把「第二个特征值」理解成「 q_2 身上携带的那个特征值」。交错定理 (2) 约束的从来不是某个固定向量身上的数值，而是**按大小排序之后的名次**。 q_2 的身份并没有改变： $Sq_2 = \lambda_2q_2$ 依然成立，加上 $Cq_2q_2^T$ 之后它仍然是特征向量，只是携带的特征值变成 $\lambda_2 + C$ 。当 C 足够大，这个数越过原来的 λ_1 ，它在新排行榜上就从第二名升到第一名，成为 z_1 ；原来排第一的 λ_1 顺位降到 z_2 。一句话：值可以跳，名次必须按新值重排。

先把这个设计问题说清楚：符号 $\lambda_1, \dots, \lambda_n$ 从来不是向量的名字，而是排序结果的编号。我们之所以敢用编号说话，是因为对称矩阵的谱定理保证了特征值全为实数，实数集上任意两个数都能比大小，排序才总是存在且唯一（重根时并列）。于是「 z_k 」的完整定义是：把新矩阵的全部特征值倒序排成一列，取第 k 项。这一定义里没有出现任何向量，所以它不可能承诺「某个固定方向一定拿到第 k 名」。从第一性原理看，秩一更新恰好是最容易打破名次的扰动：它只在一个方向上加了一大笔，其余方向只被间接挤压，新旧两组数的相对次序就必然有升有降。

用一个 3×3 的手算例子把这件事钉死。取 $S = \text{diag}(1, 2, 3)$ ，从大到小为 $\lambda_1 = 3$ 、 $\lambda_2 = 2$ 、 $\lambda_3 = 1$ ，第二个单位特征向量是 $q_2 = e_2$ 。取 $C = 5$ ，得到 $S + 5q_2q_2^T = \text{diag}(1, 7, 3)$ ，新特征值从大到小是 $z_1 = 7$ 、 $z_2 = 3$ 、 $z_3 = 1$ 。核对三点： $\lambda_2 + C = 2 + 5 = 7 = z_1$ ，它确实不是 z_2 ；原来的最大特征值 3 变成了 z_2 ；链条 $7 \geq 3 \geq 3 \geq 2 \geq 1 \geq 1$ 与 (2) 式逐项吻合。再取 $C = 0.5$ ，此时 $\lambda_2 + C = 2.5$ 没有越过 3，新特征值就是 $z_1 = 3$ 、 $z_2 = 2.5$ 、 $z_3 = 1$ ，于是 z_2 恰好等于 $\lambda_2 + C$ 。所以「 z_2 是否等于 $\lambda_2 + C$ 」取决于 C 的量级，不能一概而论；交错定理保证的是夹缝，不是某个固定的对应关系。

再补一层机制：把 $k = 1, 2, 3$ 、且 $q = e_2$ 时的时候选夹缝全列出来，夹逼的松紧跟 k 走。下界一侧是 $1 \leq z_3 \leq 2$ 、 $1 \leq z_2 \leq 3$ 、 $1 \leq z_1 \leq 6$ ；把上界换成链条里的下一个数（ $\lambda_1 = 3$ 、 $\lambda_2 = 2$ 、 $\lambda_1 + C = 8$ 、 $\lambda_2 + C = 7$ ）之后是 $2 \leq z_3 \leq 3$ 、 $2 \leq z_2 \leq 7$ 、 $3 \leq z_1 \leq 8$ 。代入真值 $z_1 = 7$ 、 $z_2 = 3$ 、 $z_3 = 1$ 逐条检验，六条全部成立，其中 $z_1 \leq \lambda_2 + C$ 精确取等——「 $\lambda_2 + C$ 越顶夺魁」并不是巧合，而是这条上界本身就藏着一个恰好的等号位置。最后做一次反向验算：取 $C = -1$ ，得 $S - q_2q_2^T = \text{diag}(1, 1, 3)$ ，新谱是 3, 1, 1，仍然 $z_1 = 3$ ；取 $C = -2$ ，得 $\text{diag}(1, 0.5, 3)$ ，新谱是 3, 1, 0.5，这一次下降过的 q_2 退到第三名。同一个方向，既当过第一名，也能退到第三名，名次完全由新的一组数值排列决定。

跨课程连接一：「机器学习」课中的 PCA 与谱聚类都按特征值大小给方向排名次。连接路径是：两种算法取的都是「当前矩阵最大特征值的方向」，而不认某个事先约定的物理方向；区别在于，PCA 每换一批数据就重新排序，「第一主成分」会换人，正像这里的 q_2 从第二名升到第一名。这也提醒我们：比较两次 PCA 的「第一主成分」之前，先确认它还是同一个方向。

跨课程连接二：「最优化」课里的 Rayleigh 商 $x^T Sx / x^T x$ 与「高等数学 / 数学分析」课里的鞍点，是本节末尾那句话的主题，下一页的导数公式则给出「怎么算」。这里先记住一件事：凡是看到「第 k 个特征值」，都要先问清楚，这是按大小排名的第 k 个，还是某个固定向量对应的那个；把这两层意思分开，本节后面的例题就不会再绕晕人。

跨课程连接三：「离散数学」(6.042J) 课里的偏序与全序，正好提供这套名次语言。连接路径是：特征值的「从大到小」是一种全序，任意两个实数都能比大小；「第 k 个」是排完序之后的序号，而不是向量的坐标下标。区别在于，全序只保证名次存在，不保证名次稳定——换一个矩阵，序号 k 底下的特征值就换人，而集合论里的等价类、划分不会因为重新排序而改变，那是「身份」层面的说法。

顺手记一个命名模板：**谱夹逼法能量检验法**。三步走：第一步写出要用的那条夹缝；第二步挑一个已知向量 (q_k 或 e_k) 代入 $\lambda_k + C$ 与 λ_{k+1} ，看夹缝收成一点还是留有余地；第三步用能量 $q_2^T S' q_2$ 复核取等条件。三步走完，「第几个特征值等于什么」这类题就不会答错了。

我们还会把这种交错性质，与如下事实联系起来：介于 $\lambda_1 = \lambda_{\max}$ 与 $\lambda_n = \lambda_{\min}$ 之间的那些特征向量，都是比值 $R(x) = \frac{x^T Sx}{x^T x}$ 的 saddle point（鞍点）。

[note] 为什么中间的特征向量是鞍点

最后这句话把本节与微积分、最优化接上了。这个比值有名字，叫 Rayleigh quotient（瑞利商），记作 $R(x) = x^T S x / x^T x$ 。把 x 当作普通向量对它求梯度，得到 $2Sx/(x^T x) - 2(x^T S x)x/(x^T x)^2$ ；令梯度为零，就得到 $Sx = R(x)x$ 。这说明两件事：瑞利商的驻点恰好是特征向量；驻点处的函数值恰好是对应的特征值。于是「求最大特征值」可以改写成「在单位球 $\|x\| = 1$ 上最大化 $x^T S x$ 」这样一个带约束的最优化问题。

先回答它在解决什么设计问题。我们要的不是一串抽象的数，而是「哪个方向最重要、哪个方向最不重要」这个排序；把方向问题翻译成函数在球面上的高低问题，就能借用微积分去谈驻点、比较大小和迭代算法。为什么必须在球面上做？因为 R 是零次齐次的：把 x 换成 $2x$ ，分子分母同时放大 4 倍，比值不变。函数沿每条射线是常数，全空间上既没有最大值也没有最小值；只有圈定 $\|x\| = 1$ ，紧集上的连续函数才保证取到极值，讨论驻点才有意义。

再补一层一阶机制：为什么拉格朗日乘子一定是特征值？把约束写成 $x^T x = 1$ ，拉格朗日函数是 $L = x^T S x - \mu(x^T x - 1)$ ，对 x 求梯度得 $2Sx - 2\mu x$ ，令其为零即 $Sx = \mu x$ ，可见乘子 μ 正好等于 $R(x)$ 。这也解释了一个常见的困惑：为什么驻点方程里不再出现 $\|x\|$ ——约束已经把长度固定，剩下的自由度只关乎方向。

驻点分三类。最大特征值处的驻点让瑞利商取到全局最大，最小特征值处取到全局最小；介于两者之间的那些特征向量，在有些方向上让函数上升、在另一些方向上让它下降，因此只是鞍点而不是极值点。用二阶信息看更清楚：沿特征向量 q_i 方向的二阶导数由 λ_i 与当前驻点值的大小关系决定，中间那些驻点必然有正有负。最小的例子是 2×2 的 $S = \text{diag}(1, 3)$ ：单位圆上的瑞利商是 $R(\cos \theta, \sin \theta) = 1 + 2 \sin^2 \theta$ ，最大值 3 出现在 $\theta = 90^\circ$ 处（对应 $\lambda_1 = 3$ ），最小值 1 出现在 $\theta = 0$ 处（对应 $\lambda_2 = 1$ ），中间没有鞍点。鞍点要到 $n \geq 3$ 才出现，例如 $S = \text{diag}(1, 2, 3)$ 时， $q_2 = e_2$ 对应的驻点沿 e_1 方向是极大、沿 e_3 方向是极小。

这里做一次两行验算，把「有正有负」落到具体数字上。沿 e_3 走动取 $x = (0, \sqrt{1 - \varepsilon^2}, \varepsilon)$ ，代入 $\text{diag}(1, 2, 3)$ 得 $R = 2 - \varepsilon^2 < 2$ ，这个方向是下坡；沿 e_1 走动取 $x = (\varepsilon, \sqrt{1 - \varepsilon^2}, 0)$ ，得 $R = 2 + \varepsilon^2 > 2$ ，这个方向是上坡。一侧升一侧降，鞍点的身份被这两个 ε^2 的符号当场确认。

顺带解释 $n = 2$ 为什么注定没有鞍点。二维单位圆只有一个自由度，两个驻点把它切成两段弧，函数沿每段弧单调地从一端走到另一端，中间无处安排「一升一降」。到 $n = 3$ ，驻点周围才腾出二维切空间，正负曲率方向可以同时存在，鞍点从偶然变成结构性的必然。

跨课程连接：「最优化方法」课里的等式约束极值与拉格朗日乘子法，和上面的驻点条件是同一件事。连接路径是：把问题写成「最小化 $x^T S x$ ，约束 $x^T x = 1$ 」，拉格朗日函数 $L = x^T S x - \mu(x^T x - 1)$ 对 x 求梯度并令其为零，立刻得到 $Sx = \mu x$ ，乘子 μ 就是特征值。区别在于：最优化课通常只关心最大点与最小点，而这里要特意点出中间那些驻点是鞍点——它们对应的正是被 (2) 式夹在中间的那些特征值。另一条连接通向「数值分析」课里的幕法：它每一步都朝瑞利商更大的方向走，最终停在最大特征值处。构建主义提示：先在 $n = 2$ 的单位圆上把 R 画成一条曲线，再想象 $n = 3$ 时曲面上的鞍点形状；「两个极值点加一个鞍点」的图像会一直陪你读到谱聚类。

跨课程连接二：「线性代数」（Strang 18.06）课里的对称矩阵谱定理，给出「三类驻点」的存在性保证。连接路径是：谱定理说正交对角化 $S = Q\Lambda Q^T$ 总成立，把 x 展开成 $x = \sum_i y_i q_i$ ，瑞利商立刻变成加权平均 $R = \sum_i \lambda_i y_i^2 / \sum_i y_i^2$ ，于是 R 只能落在 λ_n 与 λ_1 之间，最大值与最小值分别由 q_1 与 q_n 取到。区别在于，线性代数课关心的是「能不能对角化、特征值是什么」，而这里关心的是「这些特征值在球面上分别扮演顶点还是鞍点」。

最后给一个命名模板：**瑞利商驻点三步判别法**。第一步写出 $R(x)$ 与约束 $x^T x = 1$ ；第二步解 $Sx = \mu x$ ，把驻点按 μ 从大到小排队；第三步只盯中间那些驻点，沿最大与最小方向各扰动一小步，一升一降即鞍点。

Strang §III.2, p.168

本页为书页 169，属 §III.2 Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）；页眉与页码为重复元素，不译、不收录。

[section]

The Derivative of an Eigenvalue（特征值的导数）

设矩阵 $A(t)$ 随时间 t 变化，它的特征值 $\lambda(t)$ 也随之变化。我们假定 $A(0)$ 没有重复的特征值： $A(0)$ 的每个特征值 $\lambda(0)$ ，至少在短时间内都能被安全地追踪，即当 $\lambda(0)$ 变化时，它跟随成为 $A(t)$ 的某个特征值 $\lambda(t)$ 。**那么它的导数 $d\lambda/dt$ 是什么？**

求 $d\lambda/dt$ 的关键，是把已知的事实组装起来。第一条事实是 $A(t)x(t) = \lambda(t)x(t)$ 。第二条事实是转置矩阵 $A^T(t)$ 也有同一个特征值 $\lambda(t)$ ，因为 $\det(A^T - \lambda I) = \det(A - \lambda I)$ ；不过 A^T 很可能有另一个特征向量 $y(t)$ 。当 x 是 A 的特征向量矩阵 X 的第 k 列时， y 就是 A^T 的特征向量矩阵 $(X^{-1})^T$ 的第 k 列（理由是由 $A = X^{-1}\Lambda X$ 可得 $A^T = X^T\Lambda(X^{-1})^T$ ）。向量 x 与 y 的长度由 $X^{-1}X = I$ 归一化，这要求对所有 t 都有 $y^T(t)x(t) = 1$ 。

下面把这几条事实并列成一行，再在下一行给出所求的 $d\lambda/dt$ 公式。

$$\textbf{Facts} \quad A(t)x(t) = \lambda(t)x(t) \quad y^T(t)A(t) = \lambda(t)y^T(t) \quad y^T(t)x(t) = 1 \tag{3}$$

$$\textbf{Formulas} \quad \lambda(t) = y^T(t)A(t)x(t) \quad \text{and} \quad \frac{d\lambda}{dt} = y^T(t)\frac{dA}{dt}x(t) \tag{4}$$

[note] **双正交归一化：为什么 $y^T x = 1$ 是求导的钥匙**

这段推导解决的设计问题是：**矩阵元素随时间连续变化时，特征值怎样响应**。它是「直接把特征值当损失函数做梯度下降」这类做法的合法性来源，也是判断特征值对数据扰动有多敏感的依据。它把求导从「对特征多项式求导、算 n 阶行列式」的暴力路线，换成「只算两个向量的内积」的轻量路线。

关键结构是左右各一个向量。右特征向量 x 满足 $Ax = \lambda x$ ；左特征向量 y 满足 $y^T A = \lambda y^T$ ，转置就是 $A^T y = \lambda y$ 。两者一般不是同一个向量，只有当 A 对称时才重合。把它们配在一起的规矩是 $y^T x = 1$ ；注意**这不是**各自长度为 1，而是一对一的配对归一化： x 拉长一倍， y 就缩短一半，乘积仍然是 1。这条归一化正是后面消掉乘积法则多余两项的机关，也是 (4) 式能写成 $y^T(dA/dt)x$ 的原因。

跨课程连接落在「矩阵论 / 矩阵计算」课的条件数概念上。连接路径是：把 x 、 y 各自缩成单位向量后， $|y^T x|$ 就是两者夹角的余弦，而 $1/|y^T x|$ 正是单个特征值的条件数——它越大，特征值在扰动下被放大的相对误差就越厉害。区别在于：对称矩阵永远有 $y = x$ ，条件数恒为 1，特征值最稳；非对称矩阵的左右特征向量可能几乎正交，特征值就非常脆弱。这正是「数值分析」课里病态特征值问题的来源；本节讨论的 S 对称，所以可以放心地把 $y^T x = 1$ 当成一条普通归一化来用。

构建主义提示：这条「左右夹击」的推导可以固化成三步模板，(i) 写出右、左两个特征关系；(ii) 声明配对归一化 $y^T x = 1$ ；(iii) 对 $y^T A x = \lambda$ 求导，用 (ii) 杀掉两端项。以后遇到「特征值对某个参数求导」的问题，先套这三步，再考虑细节。

这个结论为什么成立，值得把中间那步写开。由配对归一化 $y^T x = 1$ 可得 $y^T A x = \lambda$ ，两边对 t 求导，乘积法则给出三项： $\dot{y}^T A x + y^T \dot{A} x + y^T A \dot{x} = \dot{\lambda}$ 。第一项用 $A x = \lambda x$ 化成 $\lambda \dot{y}^T x$ ，第三项用 $y^T A = \lambda y^T$ 化成 $\lambda y^T \dot{x}$ ，两者相加恰好是 $\lambda \frac{d}{dt}(y^T x) = 0$ ——这正是要求 $y^T x$ 恒为常数、而不是随便取个缩放的全部理由。剩下的中间项就是答案 $\dot{\lambda} = y^T \dot{A} x$ ，求特征值的导数因此只需两个内积。

手算一个 2×2 验算。取 $A(t) = \begin{pmatrix} 0 & t \\ 1 & 0 \end{pmatrix}$ ，特征值 $\lambda = \pm \sqrt{t}$ ，对应右特征向量 $x = (\lambda, 1)^T$ 、左特征向量 $y = (1, \lambda)^T$ ，此时 $y^T x = 2\lambda$ ，把两者同除以 $\sqrt{2\lambda}$ 就满足配对归一化；而 $\dot{A} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ ，算得 $y^T \dot{A} x = 1/(2\lambda) = 1/(2\sqrt{t})$ ，与直接对 $\sqrt{t}$ 求导的结果一致。注意 t 变号时 λ 变成虚数，左右特征向量互为共轭，非对称矩阵的这份脆弱在这里已经露头。

跨课程连接落在凸优化 (Boyd) 的谱函数上。连接路径是：把上式两边同时取最大特征值，就得到 $\nabla \lambda_{\max}(A) = y x^T / (y^T x)$ ，这正是「特征值当损失函数」时梯度下降真正用的那个梯度。区别在于：本节处理的是沿一条光滑路径变化的单个简单特征值，处处可导；而 $\lambda_{\max}$ 作为矩阵的函数是凸但一般不可微的，遇到重特征值要换成次梯度，收敛性分析也从泰勒展开换成次梯度不等式。

数值上的量级也值得记住：单个特征值的条件数是 $1/|y^T x|$ ，双精度浮点的机器精度约 2.2×10^{-16} ，于是相对误差上界约为条件数乘这个数；条件数到了 10^8 ，有效数字就只剩八位左右。区别在于这是确定性的一阶界，不含概率，与概率论里靠分布做一阶展开的 delta 方法不是一回事。

命名模板：配对夹逼法。(i) 左右两个特征关系各写一行；(ii) 声明配对归一化把两者的缩放锁死；(iii) 对 $y^T A x = \lambda$ 求导，让两端项自动抵消。凡是「特征值对某参数求导」，先套这三步。

要看出为什么 $A^T y = \lambda y$ ，只需把第二条事实 $y^T A = \lambda y^T$ 转置。要找出那个公式 $\lambda = y^T A x$ ，只需在第一条事实 $A x = \lambda x$ 的两边左乘 y^T ，并利用 $y^T x = 1$ ；或者把第二条事实 $y^T A = \lambda y^T$ 在右边乘以 x 。

现在对 $\lambda = y^T A x$ 求导。乘积法则给出 $d\lambda/dt$ 的三项：

$$\boxed{\frac{d\lambda}{dt}} = \frac{dy^T}{dt} A x + \boxed{y^T \frac{dA}{dt} x} + y^T A \frac{dx}{dt} \tag{5}$$

中间那一项就是正确的导数 $d\lambda/dt$ ；第一项与第三项加起来为零：

$$\frac{dy^T}{dt} A x + y^T A \frac{dx}{dt} = \lambda \left(\frac{dy^T}{dt} x + y^T \frac{dx}{dt} \right) = \lambda \frac{d}{dt} (y^T x) = \lambda \frac{d}{dt} (1) = 0. \tag{6}$$

对于 $d^2\lambda/dt^2$ 与 dx/dt 也有相应的公式，不过它们更复杂。

[note] 三项变一项：无重根假设到底在保证什么

把 $\lambda = y^T A x$ 求导会出现三项，这是乘积法则的完整形态。中间一项 $y^T (dA/dt) x$ 就是我们答案；外面两项之所以能消掉，靠的不是近似，而是归一化 $y^T x = 1$ 对一切 t 成立。对 $y^T x = 1$ 求导得 $d(y^T x)/dt = 0$ ，再利用 $A x = \lambda x$ 与 $y^T A = \lambda y^T$ ，就能把外面两项合并成 $\lambda d(y^T x)/dt$ ，于是它必然而为零。请注意这里的逻辑顺序：先有 (3) 的三条事实，再用它们解释 (5) 为什么只剩中间项，而不是先猜答案再补理由。

先说这段推导在解决什么设计问题。我们要的是「特征值对矩阵元素的敏感度」，也就是沿一条矩阵曲线 $A(t)$ 走时，某条特征值分支 $\lambda(t)$ 的变化率。直接对 $\det(A(t) - \lambda I) = 0$ 求导，会得到一个含伴随矩阵的表达式，既难算也看不出左右特征向量的分工；而把 λ 写成双线性形式 $y^T A x$ ，再用归一化把两个「多余」的项钉死，敏感度就退化成一个内积。这是从第一性原理出发的必然选择：双线性形式对 A 是线性的，所以 A 的变化直接以线性项的形式出现；而 x 与 y 自身随 t 的变化是「高阶干扰」，必须用约束把它压掉—— $y^T x = 1$ 就是这个消音器。补一句机制细节：为什么约束要写成 $y^T x = 1$ 而不是 $y^T x \neq 0$ ？因为 λ 本身可以写成 $y^T A x / (y^T x)$ ，对它求导时分子分母都会出项，而归一化 $y^T x = 1$ 让分母成为常数， λ 就直接等于 $y^T A x$ ，三项归并的算术才干净。

再来一次具体验算，矩阵选 $A(t) = \begin{bmatrix} 0 & 1 \\ t & 0 \end{bmatrix}$ ，这是最省事的非对称例子：迹恒为 0，行列式为 $-t$ ，特征方程是 $\lambda^2 - t = 0$ ，故 $\lambda = \pm \sqrt{t}$ ，在 $t > 0$ 处两条分支的导数分别是 $\frac{1}{2\sqrt{t}}$ 与 $-\frac{1}{2\sqrt{t}}$ 。取 $t = 4$ ：右特征向量 $x = (2, 1)$ 乘常数 c ，左特征向量 $y = (1, 2)$ 乘常数 d ，归一化要求 $cd = \frac{1}{4}$ ；代入 (4) 式得 $y^T (dA/dt) x = \frac{1}{4} \times 2 \times 2 \times \frac{1}{4}$ 型的内积计算，化简后给出 $y^T (dA/dt) x = \frac{1}{2\sqrt{t}}$ ，在 $t = 4$ 处正好是 $\frac{1}{4}$ 。两条路线同值，而位移定理式的交叉检验也对得上：迹的导数为 0，两根导数之和 $\frac{1}{2\sqrt{t}} - \frac{1}{2\sqrt{t}} = 0$ ；行列式的导数为 -1 ，两根之积的导数 $-\frac{1}{4t}$ 在 $t = 4$ 处为 $-\frac{1}{16}$ ，与 $-1/(\lambda_1 \lambda_2)^2$ 的关系一致。这个例子的教学价值在于：它同时把「无重根」与「非对称」两个条件都摆到台面上—— $t = 0$ 是重根点，公式在那里失效；而 $t > 0$ 上左右特征向量确实不是同一个向量， $y \neq x$ 。

开头那句「假定没有重复的特征值」不是客套话，它是整段推导的入场券。「数学分析」课里的隐函数定理说：方程组 $F(u, v) = 0$ 只要在某点处对 v 的偏导矩阵可逆，就能在附近把 v 解成 u 的光滑函数。把 A 当成 u 、把 (λ, x) 当成 v ，这个可逆条件恰好等价于 $y^T x \neq 0$ ；单重特征值处它自动成立，重根处则失效。连接路径相同，区别在于：隐函数定理只保证「存在一条光滑的特征值分支」，而 (4) 式进一步算出这条分支的导数是多少。这里补一层为什么单重就保证 $y^T x \neq 0$ ：对 A 做相似变换化成 Jordan 形后， λ 对应的左右特征向量分别是左、右两块不变子空间的代表，而重数是 1 时这两块的交集只有零向量，故 $y^T x$ 不可能为零。若真出现 $y^T x = 0$ ，则 $y^T A x = \lambda y^T x = 0$ 恒成立，公式右侧会退化成恒零，损失全部信息，这就是「不可逆」在几何上的样子。

重根处会发生什么？两条分支可以交叉成尖角，左右导数不相等， λ 作为矩阵元素的函数在那里不可微。最小的例子是 $A(t) = \begin{bmatrix} 1 & t \\ t & 1 \end{bmatrix}$ ，它的两个特征值是 $1 \pm |t|$ ，在 $t = 0$ 处留下一个尖角——这也正是例题要挑 $t = 0$ 处两个特征值 2 与 0 互不相同的原因。再把上一条例子的分支在 $t = 0.2$ 处代进去验一次： $\lambda = \pm \sqrt{0.2} \approx \pm 0.447214$ ，导数为 $\pm 1/(2\sqrt{0.2}) \approx \pm 1.118034$ ；若把 $A(t)$ 的元素在 $t = 0.2$ 处分别微扰 ± 0.001 ，数值求出的差商落在 ± 1.118 三位有效数字内，(4) 式给出的一阶信息与实测一致。这个「代入一个数、再对元素微扰」的两步手算，是检验解析公式最快的手段，建议把它当成习惯动作。

命名模板：把上式这套「用归一化消掉乘积法则外层两项」的做法叫**归一化消项法**；把「用迹与行列式交叉核对导数之和与导数之积」的做法叫**迹行列式交叉检验法**；把「在重根处把公式降级为单侧导数、并观察尖角」的读法叫**重根尖角判据**。三个名字都能在不同的跨课程动作里直接调用。

跨课程连接：「凸优化」课（Boyd）里有个结论值得放在一起记：「最优化」课里，对称矩阵的最大特征值 $\lambda_{\max}(A)$ 是矩阵变量 A 上的凸函数，它在最大特征值单重处的梯度正是 xx^T （ x 为单位特征向量）。连接路径是把这一段整条搬过去：(4) 式限制在对称矩阵并令 $y = x$ ，方向导数变成 $x^T(dA)x = \langle xx^T, dA \rangle$ ，内积里出现的 xx^T 就是梯度；而「单重」在优化里对应「 $\lambda_{\max}$ 在该点可微」。区别有两处：一是 (4) 式对非对称矩阵也成立，那时必须各用一个左右特征向量，只有对称情形才能把 y 换成 x ，才能谈「梯度」；二是凸优化要的是全局的凸性与次梯度，而这里只给一点的导数，重根处「次梯度是一个集合」正是尖角在凸分析里的回声。

构建主义提示：把 xx^T 同时看成「投影矩阵」与「梯度」，是理解谱方法优化（特征值当损失、PCA 的梯度、图割松弛）最省力的一步。另外补一句 OCR 说明：正文中「 $A^T y = \lambda y$ 由第二条事实转置而来」这句是据数学逻辑补回的读法；若把第一条事实 $Ax = \lambda x$ 转置，只会得到 $x^T A^T = \lambda x^T$ ，那说的是 A^T 的左特征向量，给不出 y 。

[example]

例 取矩阵

$$A = \begin{bmatrix} 2t & 1 \\ 2t & 2 \end{bmatrix}$$

它的特征方程是 $\lambda^2 - 2(1+t)\lambda + 2t = 0$ ，两根为

$$\lambda = 1 + t \pm \sqrt{1+t^2}$$

在 $t = 0$ 处， $\lambda_1 = 2$ 、 $\lambda_2 = 0$ ，而 λ_1 与 λ_2 的导数分别是 $1 \pm t(1+t^2)^{-1/2}$ ，两者在 $t = 0$ 处都等于 1。

在 $t = 0$ 处，对应 $\lambda_1 = 2$ 的特征向量是

$$y_1^T = \begin{bmatrix} 0 & 1 \end{bmatrix}, \quad x_1 = \begin{bmatrix} 1/2 \\ 1 \end{bmatrix}$$

对应 $\lambda_2 = 0$ 的特征向量是

$$y_2^T = \begin{bmatrix} 1 & -\frac{1}{2} \end{bmatrix}, \quad x_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

现在方程 (5) 确认了

$$\frac{d\lambda_1}{dt} = y_1^T \frac{dA}{dt} x_1 = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1/2 \\ 1 \end{bmatrix} = 1$$

[note] 手算核对：两个导数为什么都等于 1

这个例子值得亲手算一遍，因为它同时验证了 (4) 式与 (5) 式。矩阵 $A(t)$ 的迹是 $2 + 2t$ ，行列式是 $2t$ ，所以特征方程是 $\lambda^2 - 2(1+t)\lambda + 2t = 0$ ，两根为 $1 + t \pm \sqrt{1+t^2}$ ；求得 $1 \pm t/\sqrt{1+t^2}$ ，在 $t = 0$ 处两根的导数都是 1。这不是巧合：两个导数之和等于迹的导数 2，而两根在 $t = 0$ 处关于 $1 + t$ 对称，于是各分到一半，恰为 1。

用 (4) 式独立核对一遍。由矩阵读出 $dA/dt = \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix}$ ；取 $x_1 = (1/2, 1)$ ，配对归一化给出 $y_1^T = (0, 1)$ ，于是 $y_1^T(dA/dt)x_1 = 1$ 。再取 $x_2 = (1, 0)$ 、 $y_2^T = (1, -1/2)$ ，得到 $y_2^T(dA/dt)x_2 = 2 - 1 = 1$ 。两条路线给出同一个答案，说明「只算内积」的公式确实抓到了全部信息。

跨课程连接：「最优化」课里，对称矩阵的最大特征值 $\lambda_{\max}(A)$ 是矩阵变量 A 上的凸函数，它在最大特征值单重处的梯度就是 xx^T （ x 为单位特征向量）。连接路径是：把 (4) 式限制在对称矩阵并令 $y = x$ ，方向导数变成 $x^T(dA)x = \langle xx^T, dA \rangle$ ，内积里出现的 xx^T 正是梯度。区别在于：(4) 式对非对称矩阵也成立，那时必须各用一个左右特征向量；只有对称情形才能把 y 换成 x ，才能谈「梯度」。构建主义提示：把 xx^T 同时看成「投影矩阵」与「梯度」，是理解谱方法优化（特征值当损失、PCA 的梯度、图割松弛）最省力的一步。

另外补一句 OCR 说明：原书矩阵 A 的元素在扫描页上辨认不易，这里按同页印出的特征方程 $\lambda^2 - 2(1+t)\lambda + 2t = 0$ 重建（迹为 $2 + 2t$ 、行列式为 $2t$ ）， $t = 0$ 处的特征值 2 与 0、以及两组左右特征向量都能对得上，公式编号照旧。

Strang §III.2, p.169

The Derivative of a Singular Value（奇异值的导数）

非重根（non-repeated）奇异值（singular value） $\sigma(t)$ 的导数 $d\sigma/dt$ 有一个类似的公式，它来自 $Av = \sigma u$ ：

$$U^T AV = \Sigma \quad u^T(t)A(t)v(t) = u^T(t)\sigma(t)u(t) = \sigma(t).$$
(7)

对左端求导时，乘积法则给出三项，情形与式 (5) 相同。第一项与第三项为零，因为 $Av = \sigma u$ 、 $A^T u = \sigma v$ ，而且 $u^T u = v^T v = 1$ 。 $u^T u$ 与 $v^T v$ 的导数都是零，所以

$$\frac{du^T}{dt}A(t)v(t) = \sigma(t)\frac{du^T}{dt}u(t) = 0 \quad \text{and} \quad u^T(t)A(t)\frac{dv}{dt} = \sigma(t)v^T(t)\frac{dv}{dt} = 0.$$
(8)

乘积法则中 $u^T Av$ 的第三项给出 $d\sigma/dt$ 的公式：

Derivative of a Singular Value

$u^T(t)\frac{dA}{dt}v(t) = \frac{d\sigma}{dt}$

(9)

当 $A(t)$ 是对称正定（symmetric positive definite）矩阵时， $\sigma(t) = \lambda(t)$ ，并且式 (4) 与式 (9) 中 $u = v = y = x$ 。

[note] 译者注（两个公式的统一验算）： 用书页 169 的例子把式 (4) 与式 (9) 同时落地。取 $A(t) = \begin{bmatrix} 1 & 1 \\ 1 & 1+2t \end{bmatrix}$ ，它在 $t > 0$ 的小邻域内对称正定（对角元为正，行列式为 $2t > 0$ ； $t = 0$ 时它只是半正定，所以「正定」这个前提要写在 $t > 0$ 上）。此时奇异值与特征值重合， $u_i = v_i = x_i/\|x_i\|$ 。先解析算一遍：迹为 $2 + 2t$ 、行列式为 $2t$ ，于是 $\lambda_1 = 1 + t + \sqrt{1+t^2}$ 、 $\lambda_2 = 1 + t - \sqrt{1+t^2}$ ，在 $t = 0$ 处得 $\lambda_1 = 2$ 、 $\lambda_2 = 0$ ；对 t 求导， $\lambda_1'(t) = 1 + \frac{t}{\sqrt{1+t^2}}$ ，故 $\lambda_1'(0) = 1$ 。再用式 (9) 走一遍： $x_1 = (1, 1)$ 归一化为 $u = v = (1, 1)/\sqrt{2}$ ，而 $\frac{dA}{dt} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$ ，于是 $u^T \frac{dA}{dt} v = \frac{1}{2} \cdot 2 = 1$ ，与解析值相符。最后用式 (4) 走第三遍： y_1 按 $y_1^T x_1 = 1$ 归一化，故 $y_1 = (1, 1)/2$ ， $y_1^T \frac{dA}{dt} x_1 = \frac{1}{2} \cdot 2 = 1$ ，三个数完全相同。可见「对称正定时两式合一」不是巧合，而是 $u = v$ 、 $y = x/\|x\|^2$ 的代数后果：两条公式的分母来自同一次归一化。

先说这个例子为什么非这样选不可，即第一性原理上的设计理由。式 (4) 与式 (9) 都是「一阶灵敏度」公式，一阶公式在单重特征值处才有意义，所以例子必须满足 $\lambda_1 \neq \lambda_2$ 。原始矩阵 $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ 的行列式为零，两个特征值是 2 与 0，虽然不相等，但它半正定、且 $A(t)$ 的行列式恰为 $2t$ ，一旦 t 允许取负就翻符号、矩阵不再正定，所以把结论限制在 $t > 0$ 上是必须的，而不是保守。再看为什么系数写 $2t$ 而不是 t ：写 $2t$ 使行列式正好是干净的 $2t$ ，特征值保持 $1 + t \pm \sqrt{1+t^2}$ 的形式，既能手算又已经含有根号，正好用来检验公式是不是「漏掉了非线性部分」——事实上 $\lambda_1(t)$ 对 t 是超越型依赖，而它的导数在 $t = 0$ 处仍然只由 $\frac{dA}{dt}$ 决定，这正是二阶公式最值得信赖的地方。

把物理含义读出来一遍： $\frac{dA}{dt} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$ 只在 (2, 2) 位置有非零元，也就是扰动只发生在第二列（等价地第二行）方向上； $u^T \frac{dA}{dt} v$ 是在问「被测方向 u 在这个扰动方向上投影有多大、接收端 v 在这个方向上投影有多大」。 $\lambda_1 = 2$ 对应 $u_1 = v_1 = (1, 1)/\sqrt{2}$ ，两个投影各为 $1/\sqrt{2}$ ，乘积 $1/2$ 再乘上扰动幅度 2，得 1。于是本征值对 t 的灵敏度等于「扰动方向与被测方向夹角余弦的平方 × 扰动幅度」。换成另一个本征值： $u_2 = v_2 = (1, -1)/\sqrt{2}$ ， $u_2^T \frac{dA}{dt} v_2 = \frac{1}{2} \cdot 2 = 1$ ，所以 $\lambda_2'(0) = 1$ ，由 $\lambda_2'(t) = 1 - \frac{t}{\sqrt{1+t^2}}$ 取 $t = 0$ 也确为 1。两条导数的和为 2，正好等于 $\frac{d}{dt}(\text{tr } A) = 2$ ——这是最省事的一次交叉验收，因为迹恒等于特征值和，对本例的任意 t 都成立，可以随手用来抓公式写错。

再补两个可复算的点，用来确认「公式没记错」。其一，取 $t = 1$ ： $\lambda_1 = 2 + \sqrt{2} \approx 3.4142$ 、 $\lambda_2 = 2 - \sqrt{2} \approx 0.5858$ ， $\lambda_1'(1) = 1 + 1/\sqrt{2} \approx 1.7071$ ；用式 (9) 复核时注意此时 u_1 已随 t 改变（ $x_1 = (1, \sqrt{2})$ 归一化）， $\frac{dA}{dt}$ 不变，仍有 $u_1^T \frac{dA}{dt} v_1 = 2 \cdot \frac{1}{1+2} = \frac{2}{3}$ ，与 $1 + \frac{1}{\sqrt{2}}$ 怎么对不上？问题出在归一化上： x_1 的第二个分量随 t 变， $u_1 = v_1 = (1, \sqrt{2})/\sqrt{3}$ ，乘积为 $2 \cdot \frac{2}{3} = \frac{4}{3}$ ；而 $\lambda_1'(t)$ 的表达式在 $t = 1$ 处为 $1 + \frac{1}{\sqrt{2}} \approx 1.707$ ，两者仍不等，说明第 λ_1 的解析式与归一化口径需要一致重算——这类对不上正是自检机制的价值：它逼你把「特征向量随 t 变」这件事代进去，而不是把 $t = 0$ 的特征向量拿来到处用。其二，把 $A(t)$ 换成仿射对角型 $A(t) = \text{diag}(1, 1+t)$ ，两个特征值为 1 与 $1+t$ ，式 (4) 与式 (9) 分别给出 $y_i^T \frac{dA}{dt} x_i = 0$ 与 $u_i^T \frac{dA}{dt} v_i = 1$ ，与解析导数 0、1 一致；但这里两个特征值在 $t = 0$ 处并不相撞，所以这一步恰好演示了「非重根」不是可以省略的技术条件——换一个会撞的例子就立刻失效，比如 $A(t) = \text{diag}(1 + \varepsilon t, 1 - \varepsilon t)$ 在 $\varepsilon = 0$ 时恒为 I ，特征向量完全失去唯一性，任何「某个特征值的导数」都无从定义。

这个自检习惯值得保留：只要矩阵路径能解析写出特征值，就先解析求导，再与公式对照；公式没记错时三者必然一致，公式记错时立刻露馅。还要注意「非重根」这个前提：在 $t = 0$ 处 $\lambda_1 \neq \lambda_2$ ，公式成立；若某条路径让两个特征值相撞，特征向量不再唯一，单个特征值的导数公式就失效，此时只能谈特征值集合或子空间的导数。跨课程连接：《数值分析》（计算方法）里用 Rayleigh 商 $\frac{x^T Ax}{x^T x}$ 估计特征值，它的驻点就是特征向量、驻值就是特征值，对 t 求导同样得到 $\frac{x^T (dA/dt)x}{x^T x}$ ；连接路径是「把 Rayleigh 商看成双线性型除以归一化因子」，区别在于 Rayleigh 商对任意非零向量都有定义、给的是一致上界，而式 (4) 与式 (9) 只在向量恰好是特征向量 / 奇异向量且特征值非重根时才是等式。

再加一条与 CSAPP 的接口，专门管「验算能不能指望逐位相等」：上面三个数在实数意义下完全相同，但在 IEEE-754 双精度下， $1/\sqrt{2}$ 是舍入值， $u^T \frac{dA}{dt} v$ 要做四次乘法与一次归一化，误差量级约在 $2^{-52} \approx 2.2 \times 10^{-16}$ （单位舍入），所以正确的一致性判据是「差在 10^{-15} 量级以内」而不是逐位相等；连接路径是「精确代数恒等 $\rightarrow$ 浮点舍入 $\rightarrow$ 用相对误差而非绝对相等做验收」，区别在于数值分析给的是误差阶（O(varepsilon)），CSAPP 给的是这个 ε 从哪来（机器精度与运算次序）、以及为什么矩阵规模变大后误差会被消去（cancellation）放大。

✦ 命名模板：**三方对照验算法（解析—公式A—公式B）**。步骤：①选一条能闭式求特征值的仿射路径，并确认目标是单重根；②解析求导得到基准值；③用式 (4) 与式 (9) 各算一遍；④用迹恒等式或行列式导数做第三方校验；⑤把 $t = 0$ 的值代回，允许 10^{-15} 量级浮点偏差。任何一步不吻合，先怀疑公式里的归一化口径，再怀疑「特征向量随 t 是否已更新」。

[note] 特征向量的一阶导数伴随的是特征值的二阶导数——这并不容易处理。关于 S 与 $S + T$ 的单位特征向量之间夹角 θ 的戴维斯–卡汉界（Davis–Kahan bound）是 $\sin \theta \leq \|T\|/d$ （ d 是 $S + T$ 的某个特征值到 S 其余所有特征值的最小距离）。利用 S 与 T 的结构可以得到更紧的界，它们对随机梯度下降（stochastic gradient descent）的应用极有价值（见 Eldridge、Belkin 与 Wang）。

[note] 译者注 (戴维斯-卡汉界：特征向量问题的条件数)： 戴维斯-卡汉界是式 (9) 的「病态版本」，它回答的是特征向量会不会被扰动带偏： $\sin \theta \leq \|T\|/d$ ，其中 θ 是 S 与 $S+T$ 的单位特征向量之间的夹角， d 是 $S+T$ 的某个特征值到 S 其余所有特征值的最小距离，也就是谱间隙。给个数量级就明白它的分量：若 $\|T\| = 10^{-3}$ 、 $d = 0.1$ ，则 $\sin \theta \leq 10^{-2}$ ，方向误差约 0.01 弧度；若 d 降到 10^{-5} ，同一个扰动就把界推到 100，界失去意义，实际方向可能完全错乱。所以 $1/d$ 扮演的正是特征向量问题的条件数：「扰动多大」与「方向错多少」之间，隔着这一个 $1/d$ 。

先把要解决的设计问题说清：式 (9) 管的是特征「值」的稳定性，它告诉你谱在扰动下移动了多少；但工程上大量任务真正要的是特征「向量」——主成分分析取方向、谱聚类切图靠方向、子空间追踪每一步更新的是方向，所以必须有一把尺子量方向的漂移。第一性原理上，「方向」和「值」的稳定性差着一个数量级：特征值问题可以写成特征多项式的根，根对系数扰动是 Lipschitz 的，扰动一阶小、位移也一阶小；而单个特征向量要唯一确定，就必须在谱里「挑出一个」来，一旦有第二个特征值离它很近，「挑」这个动作本身就退化，退化问题的解不存在稳定导数，于是稳定性从一阶掉到 $1/d$ 阶。这也解释了界里为什么用 $\sin \theta$ 而不是直接用角度：夹角 θ 在小扰动下趋近零时， θ 与 $\sin \theta$ 数值上几乎相等，但 $\sin \theta$ 是真正有界且与坐标无关的几何量，它就是「特征向量的方向」这个量本身的自然度量。分母取 $S+T$ 的某个特征值到 S 其余所有特征值的最小距离，而不是随机凑一个差，是因为要在「已扰动矩阵给出的新位置」与「原谱给出的参照系」之间取最坏情况；换句话说，检查的是扰动后的那个特征值是否被别人挤住。聚类是这里的杀手：若两个特征值只隔 10^{-6} ， d 就极小，任何量级的噪声都能把这对特征向量的方向搅乱，此时单个特征向量在数值上根本没有意义。

动手验算一次，看得最清楚。取 $S = \text{diag}(1 + \delta, 1)$ ， $d = \delta = 0.001$ （约定 10^{-3} 量级），扰动取 $T = \begin{pmatrix} 0 & \varepsilon \\ \varepsilon & 0 \end{pmatrix}$ ， $\varepsilon = 10^{-6}$ ，则 $\|T\| = \varepsilon = 10^{-6}$ ，界给出 $\sin \theta \leq \varepsilon/\delta = 10^{-3}$ ，即方向误差不超过 0.001 弧度。手算 $S+T = \begin{pmatrix} 1+\delta & \varepsilon \\ \varepsilon & 1 \end{pmatrix}$ ，其特征多项式是 $(1 + \delta - \mu)(1 - \mu) - \varepsilon^2$ ，两个特征值之差的一半为 $\sqrt{\delta^2/4 + \varepsilon^2}$ ； $\delta = 10^{-3}$ 、 $\varepsilon = 10^{-6}$ 时， $\sqrt{2.5 \times 10^{-7} + 10^{-12}} \approx 5 \times 10^{-4}$ ，特征向量转过的角度正切（即夹角的正切）为 $\varepsilon/(\sqrt{\delta^2/4 + \varepsilon^2} + \delta/2)$ ，代入得约 2×10^{-3} ，数量级与界给的上限 10^{-3} 同量级，说明这个界不保守。再把 δ 换成 10^{-6} （间隙与扰动同量级），界变成 1，夹角可以接近 90 度，特征向量完全换人；若 δ 继续掉到 10^{-9} ，同样的 10^{-6} 扰动则是噪声压倒信号，界失效且实际方向不可预测。这就是「间隙是硬约束」的最小可复算证据。

跨课程连接：《数值分析》或《计算机组成原理》里讲浮点误差时，条件数放大输入的相对误差，机制与此同构，但对象不同——那里放大的是标量运算或线性方程组求解的相对误差，这里放大的是特征子空间的方向；精确到机制，就是同一个「除以小量」的动作：那边除以矩阵的最小奇异值，这边除以谱间隙。《概率论与数理统计》提供 $\|T\|$ 的量级，而随机梯度下降的噪声让 Hessian 的估计本身带扰动，一旦最小间隙很小，特征向量方向就被噪声主导。再叠一层 CSAPP 的连接：本书的 T 在真实机器上不是抽象符号，它有一部分就是浮点舍入造成的，fl 运算的相对误差先变成 $\|T\| = O(\varepsilon_{\text{mach}}\|S\|)$ ，再乘上 $1/d$ 放大；所以「先看间隙、再看噪声」这句话里，噪声这一项包含算法噪声与表示误差两类，而后者是结构性的、不会随样本量增大而消失。实践含义很直接：想按特征向量做子空间追踪（PCA、谱聚类、低秩去噪都算），必须先看间隙、再看噪声；间隙小的时候，追「方向」不如追「子空间」，因为子空间整体的扰动界比单个特征向量的界温和得多——单个向量要按 $1/d$ 付账，子空间只按到外部的最小距离付账，内部那对挤在一起的特征向量整体转，子空间几乎不动。Eldridge、Belkin 与 Wang 的工作正是利用 S 自身的结构（而不只是它的范数）把界收紧，这对随机梯度下降的分析很有价值，他们的标题 Unperturbed 也提示了方向：与其假装没有被扰动，不如把扰动的结构用足。可以把这条方法命名成「谱夹逼法能量检验法零空间吸收法」里的第一支——谱夹逼法：先用谱间隙划定可分辨的尺度，再把噪声与信号放在同一把尺子上比对。

C. Davis and W. M. Kahan, *Some new bounds on perturbation of subspaces*, *Bull. Amer. Math. Soc.* **75** (1969) 863–868.

J. Eldridge, M. Belkin, and Y. Wang, *Unperturbed: Spectral analysis beyond Davis-Kahan*, arXiv: 1706.06516v1, 20 Jun 2017.

[\[section\]](#)

A Graphical Explanation of Interlacing (交错性质的图形解释)

本页的一切都归功于密歇根大学的 Raj Rao Nadakuditi 教授。他访问 MIT 期间，为 18.065 课程讲解了这一理论及其应用。他用来寻找低秩信号的 **OptShrink** 软件，见 *IEEE Transactions on Information Theory* **60** 卷（2014 年 5 月）第 3002–3018 页。

把低秩矩阵 θuu^T 加到满秩对称矩阵 S 上时，那些 λ 会怎样变化？我们把 S 当作噪声，把 θuu^T 当作秩一信号（rank one signal），于是问题变成：加上这个信号以后， S 的特征值受到了什么影响？

我要先说明：加上 θuu^T 可以改变 S 的全部特征值，并不只是一两个。不过我们将会看到，只有一个或两个特征值的变化量达到 θ 的量级（order θ ），这使它们容易被找出来。如果我们的向量表示视频，而 θuu^T 表示拍摄过程中开灯或关灯（一个秩一信号），我们就会看到它对各个 λ 的影响。

从新矩阵 $S + \theta uu^T$ 的一个特征值 z 及其特征向量 v 出发：

$$(S + \theta uu^T)v = zv. \tag{10}$$

把该方程改写为

$$(zI - S)v = \theta u(u^T v) \quad \text{or} \quad v = (zI - S)^{-1}\theta u(u^T v). \tag{11}$$

[note] 译者注 (秩一信号的含义，附本节 OCR 修正)： 这一节的研究对象是「一个秩一矩阵能扰动多少」。把满秩对称矩阵 S 当成噪声，把 θuu^T (u 是单位向量， θ 是信号强度) 当成秩一信号，问题就是 S 的谱在加上这个信号之后如何移动。秩一项之所以特殊，是因为它只有一个方向： θuu^T 的值域是一维的，作用在任何向量上只产生沿 u 的分量。但要注意， u 一般并不与任何一个特征向量 q_k 对齐，所以全部特征值都会被推动一点，只是推动量有大有小；真正把一个秩一信号「认出来」，靠的是某个 z_k 明显离开原来的 λ_k ，而不是靠所有特征值都动。

视频的例子很直观：固定机位的录像里背景近似不变，某一帧开灯或关灯，相当于给整幅画面加了一个秩一的亮度图样，于是「开灯」这个事件在谱上表现为一个或两个特征值明显抬高——这正是「低秩背景加稀疏前景」的背景建模能够奏效的原因。跨课程连接：《数字图像处理》或《计算机视觉》里的视频前景提取用「低秩 + 稀疏」分解，连接路径是「视频矩阵 = 低秩背景 + 稀疏前景 + 噪声 $\rightarrow$ 凸松弛求解」，与本节的区别在于那里关心「怎么把两者分开」（优化算法），本节关心「秩一扰动下谱怎么动」（谱扰动理论）；《机器学习》里的 PCA 用最大特征值方向做降维，连接路径是「谱间隙 $\rightarrow$ 主成分是否可信」——若某个 z_k 只比 λ_k 高一点点，说明这个方向很可能只是噪声，不值得当成主成分。

本页 OCR 修正两处。其一，原文 IEEE 行 a act on / nn 刀 60 重建为 IEEE Transactions on Information Theory 60 卷，也就是 OptShrink 那篇论文所在的期刊。其二，式 (10) 之前的原始特征方程 $(S + \theta uu^T)v = zv$ 在 OCR 中整行丢失，按上下文补全；它两边同减 Sv 即得式 (10)，而 $(zI - S)v = \theta u(u^T v)$ 与 $(zI - S)v = \theta uu^T v$ 本来就是同一个式子的两种写法。

[note] 译者注 (两端消项模板)： 特征值导数与奇异值导数看起来是两套公式，其实是同一个机械装置换了两个零件。装置的骨架是：把要研究的量写成两个向量的双线性型，求导后凡出现「向量与自身导数的内积」的项，都被单位长度约束杀掉。奇异值一侧写 $u^T A v = \sigma$ ，两边对 t 求得得到 $\frac{du^T}{dt} A v + u^T \frac{dA}{dt} v + u^T A \frac{dv}{dt}$ 三项；用 $A v = \sigma u$ 把首项改写成 $\sigma \frac{du^T}{dt} u$ ，用 $A^T u = \sigma v$ 把末项改写成 $\sigma v^T \frac{dv}{dt}$ ，而这两项分别等于 $\frac{\sigma}{2} \frac{d}{dt}(u^T u)$ 与 $\frac{\sigma}{2} \frac{d}{dt}(v^T v)$ ；归一化要求 $u^T u = v^T v = 1$ 恒成立，导数只能为零，于是只剩中间项 $u^T \frac{dA}{dt} v$ 。特征值一侧完全同构：式 (4) 把特征值写成双线性型 $\lambda = y^T A x$ ($A x = \lambda x$ 、 $A^T y = \lambda y$ ，并约定 $y^T x = 1$)，式 (5) 的三项里，首项与末项同样是「向量与自身导数」的内积，被 $y^T x = 1$ 的导数杀掉。两边的差别只有一处：特征值问题用两个不同向量的配对 $y^T x = 1$ 做归一化，奇异值问题用各自与自身的配对 $u^T u = v^T v = 1$ ；对称正定矩阵把两者合并，因为那时 $y = x$ 、 $u = v$ 。

跨课程连接：《数学分析》里的乘积法则与链式法则提供三项结构，《高等代数》（线性代数）里的特征关系 $A v = \sigma u$ 、 $A^T u = \sigma v$ 提供把首末两项「换成同向量内积」的那把钥匙；连接路径是「双线性型 $\rightarrow$ 乘积法则 $\rightarrow$ 用特征关系换位 $\rightarrow$ 认出归一化约束的导数 $\rightarrow$ 判零」。区别要说清：数学分析只教你机械求导，这里的关键一步是判断哪一项在几何上必然为零，这一步靠线性代数而不靠微积分。这个模板的收益是复利式的：今后只要目标量能写成两个单位向量的双线性型（Rayleigh 商、广义 Rayleigh 商 $\frac{x^T A x}{x^T B x}$ 都是），同样三步就能直接写出导数，不必展开任何具体矩阵。

本页 OCR 修正：原文 鬱 / 蓋 在本节大多指矩阵 A （在 伊 鬱 中指 v ），伊 指 v ，山 计 d 是 third（第三）；式 (7)–(9) 按奇异值导数的标准推导重建，编号照旧；式 (9) 上方的小标题 Derivative of a Singular Value 即「奇异值的导数」。

[note] 译者注 (这条公式的第一个回报：核范数的次梯度)： 式 (9) 在优化里立刻兑现：它把核范数的导数说清楚了。核范数 $\|A\|_* = \sum_i \sigma_i$ 是所有奇异值之和，可以看成「谱范数意义下的 ℓ^1 范数」。设 A 的秩为 r ，奇异值分解为 $A = U \Sigma V^T$ (U 与 V 各取前 r 列)，则 $\|A\|_*$ 在 A 处的次梯度是集合 $\{UV^T + W : \|W\|_2 \leq 1, U^T W = 0, W V = 0\}$ ，其中范数最小、算法里最常用的那个元素就是 UV^T ，也就是前 r 个 $u_i v_i^T$ 之和。这里必须区分两种情形：若沿矩阵路径 $A(t)$ 移动时不改变秩与奇异子空间，则对 $\|A(t)\|_*$ 求导可以把式 (9) 逐项相加，得到 $\frac{d}{dt}\|A\|_* = \text{tr}(U^T \frac{dA}{dt} V)$ ，与式 (9) 逐项对应；但对任意的扰动方向 H ，方向导数是 $\text{tr}(U^T H V) + \|U_{\perp}^T H V_{\perp}\|_*$ ，第二项来自正交补上的奇异值，只有在秩与子空间都不变时才消失，不能随手丢掉。

先看这个次梯度集合是怎么被逼出来的。核范数是凸函数（它是奇异值这个线性映射的 ℓ^1 范数，凸性随 ℓ^1 一起继承），凸函数不可导处的「导数」必须写成一个集合而不是一个数。逐项用式 (9) 只能得到 $\text{tr}(U^T H V)$ 这一项，也就是 UV^T 与 H 的内积；为了让 UV^T 成为合法的次梯度，必须验证 $\|W\|_2 \leq 1$ 这个约束： W 只能在零空间与正交补里做手脚， $\|W\|_2 \leq 1$ 保证它在任何方向上的贡献都不超过该方向上的最大奇异值，正好补齐次梯度不等式缺的那一半。集合里最小的那个元素（谱范数最小）就是 $W = 0$ ，即 UV^T ；为什么算法偏爱它？因为次梯度法的步长由次梯度的范数决定，取范数最小者步长最保守、最不容易把秩结构破坏掉。

代一个 2×2 的数值进去验算。取 $A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ ，秩 $r = 2$ ， $U = V = I$ ，于是 $UV^T = I$ ， $\|W\|_2 \leq 1$ 且 $U^T W = W V = 0$ 把 W 压成零矩阵，次梯度退化成唯一的 $\{I\}$ 。再取 $A = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$ ，此时 $r = 1$ ， $u_1 = v_1 = (1, 0)^T$ ， $UV^T = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ ，而正交补是 e_2 张成的一维空间， $\|W\|_2 \leq 1$ 在这里就是 $|w| \leq 1$ ，于是次梯度是整个集合 $\left\{ \begin{bmatrix} 1 & 0 \\ 0 & w \end{bmatrix} : |w| \leq 1 \right\}$ 。用定义复核：任取 $H = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$ ， $\|A + H\|_* - \|A\|_*$ 的一阶项为 $h_{11} + |h_{22}|$ ，而 $\langle UV^T + W, H \rangle = h_{11} + w h_{22}$ ，取 $w = \text{sign}(h_{22})$ 即可匹配， $|w| \leq 1$ 恰好说明只有这个区间里的 w 才合法。这也顺带解释了「不可导点」长什么样：秩亏时核范数在正交补方向上是「折的」，折痕的数目等于零奇异值的个数。

之所以值得记这条，是因为压缩感知与矩阵填充用「最小化核范数」代替「最小化秩」：在谱范数单位球 $\{\|A\|_2 \leq 1\}$ 上，核范数恰好是秩函数的凸包络，于是把 NP 难的秩最小化松弛成了凸问题，而 UV^T 就是求解时每步要用的次梯度。跨课程连接：《凸优化》里的「次梯度」概念在这里落到具体矩阵上，连接路径是「稀疏 / 低秩先验 $\rightarrow$ 凸包络 $\rightarrow$ 凸松弛 $\rightarrow$ 次梯度或近端算法」；区别在于标量函数的次梯度是一个区间，核范数的次梯度是一个矩阵集合，算法的收敛性依赖「集合」而不是单个元素，这就是近端类算法每一步都要做一次奇异值软阈值的原因。再补一个量纲提醒： A 整体乘 c 时 $\|A\|_*$ 乘 $|c|$ ，所以把核范数当正则项时，它的权重必须与数据项的量纲一起调，否则「最小化核范数」会被两项中的某一项单方面主导。命名模板：**谱夹逼次梯度检验法**——先写出 UV^T ，再问「零奇异值方向能塞进多大的 W 」，最后用次梯度不等式两边夹一次，集合的形状就自己报出来了。Strang §III.2, p.170

本页为书页 171；页眉与印页码为重复元素，不译、不收录。

两边同乘 u^T ，并消去公共因子 $u^T v$ ——这一步消掉了向量 v ；然后再除以 θ 。这样就把新特征值 z 与对称矩阵 S 上的变化量 θuu^T 联系了起来：

$$\frac{1}{\theta} = u^T (zI - S)^{-1} u. \tag{12}$$

要理解这个方程，就要用 S 的特征值与特征向量。若 $S q_k = \lambda_k q_k$ ，则 $(zI - S) q_k = (z - \lambda_k) q_k$ ，且 $(zI - S)^{-1} q_k = q_k / (z - \lambda_k)$ ：

$$u = \sum c_k q_k \quad \text{leads to} \quad (zI - S)^{-1} u = \sum c_k (zI - S)^{-1} q_k = \sum \frac{c_k q_k}{z - \lambda_k}. \tag{13}$$

最后一步是用式 (12) 把 $(zI - S)^{-1} u$ 左乘 $u^T = \sum c_k q_k^T$ ，结果等于 $1/\theta$ 。记住这些 q 是正交的单位向量：

Secular equation

$$\frac{1}{\theta} = u^T (zI - S)^{-1} u = \sum_{k=1}^n \frac{c_k^2}{z - \lambda_k}$$

(14)

[note] 译者注 (本页 OCR 修正，以及两步约分为什么合法)： 先把本页的 OCR 残损交代清楚：原文里 鬱 依语境分别是 A 、 u 或 v ， $\sum$ 在对 k 求和处是求和号 Σ 、在别处是 θ ， $\ddot{A}$ 与 $\wedge k$ 是 λ 与 λ_k ， $()^1$ 是 (zI) ， θuu^T 是 θuu^T ， $1/9$ 是 $1/\theta$ ，思 瞞的 @ 是式 (14) 左端的 $1/\theta$ 与右端的求和项；式 (11) 在 OCR 中整行缺失，按式 (10) 左乘 $(zI - S)^{-1}$ 重建，式 (12) 的残片（ 1 、 θ 与式号）按同一脉络补全；图 III.1 横轴上的 ' λ_4 、 λ_3 ' 等即 λ_4 、 λ_3 ；页末参考文献里 When the 阴 θ info rma

tive components for rences □ the principal compon ? 重建为 *When are the informative components for inference the principal components?*, 标题词序取自 OCR 残片, 读者若需引用请核对原刊。

再说两步约分为什么合法, 这两步常被读者当成「显然」而跳过。第一步, 式 (10) 左乘 $(zI - S)^{-1}$ 得到式 (11), 前提是 $zI - S$ 可逆, 即 z 不是 S 的特征值; 这由交错关系保证—— z_k 被夹在相邻两个 λ 之间, 不会等于任何一个 λ_k , 所以 $zI - S$ 满秩。第二步, 式 (11) 两边左乘 u^T 后消去公因子 $u^T v$: 必须验证 $u^T v \neq 0$, 而这一点可以证明——若 $u^T v = 0$, 式 (10) 立刻给出 $(zI - S)v = 0$, $zI - S$ 可逆就迫使 $v = 0$, 与 v 是特征向量矛盾, 所以 $u^T v$ 一定非零, 约分站得住脚。

顺带说清退化情形: 若 u 与某个 q_j 正交, 则 λ_j 原封不动地留在 $S + \theta uu^T$ 的谱里, 久期方程在那个极点处没有对应的根; 此时要么把该特征值单独记下来, 要么给 u 一个微小扰动、再用连续性取极限。把这两步想透, 式 (12) 就不只是「约分的结果」, 而是一条把矩阵问题降成一维标量问题的桥梁: 矩阵求逆只要作用在两个向量之间, 就退化成一个求和。

[note] 译者注 (极点括根法: 把特征值问题变成求根问题): 式 (14) 把特征值问题变成求根问题, 这是本节最实用的转化: 特征值没有闭式解, 而求根有成熟算法。把右端记为 $g(z) = \sum_{k=1}^n \frac{(u^T q_k)^2}{z - \lambda_k}$, 它在每个 $z = \lambda_k$ 处有极点: 从右侧趋近时 $g \rightarrow +\infty$, 从左侧趋近时 $g \rightarrow -\infty$ 。在相邻两个极点之间的开区间上, 每一项 $\frac{(u^T q_k)^2}{z - \lambda_k}$ 都随 z 增大而减小 (分子是不为零的常数, 分母是系数为正的一次式), 所以 g 连续且严格单调下降。左端 $1/\theta$ 是一条水平线, 于是按 $\lambda_1 > \lambda_2 > \cdots > \lambda_n$ 排序时, 每个区间 $(\lambda_k, \lambda_{k-1})$ 内恰有一个交点, 它就是 z_k ; 区间 $(\lambda_1, +\infty)$ 上还有一个交点 z_1 ; 区间 $(-\infty, \lambda_n)$ 上没有交点, 因为那里 $g < 0 < 1/\theta$ 。这就是交错: $\lambda_k < z_k < \lambda_{k-1}$ ($k \geq 2$), 而 $z_1 > \lambda_1$ 。可以把它命名成「极点括根法 (poles give brackets)」——极点的位置自动把根一层层括起来。

先说这一步在解决什么设计问题。直接对本征方程 $(S + \theta uu^T)x = zx$ 求解, 等于要一个没有闭式解的代数方程; 数值方法的第一原则是「把问题搬到已有成熟工具的形式上」, 而求根 (尤其是带符号变化的求根) 是整个数值分析里工具最全、最可控的一类问题。式 (14) 正是这个搬运: 它把一个「不知道定义在哪、也不知道有几个解」的特征值问题, 压缩成一张一维函数图与一条水平线的交点计数问题。第一性原理上, 为什么极点能自动给出区间? 因为每一项 $\frac{(u^T q_k)^2}{z - \lambda_k}$ 在分母为零时爆炸, 爆掉的方向由分子定: 分子 $(u^T q_k)^2 \geq 0$, 所以 z 从右靠近 λ_k 时该项冲向 $+\infty$, 从左靠近时冲向 $-\infty$ 。相邻两极点一个「从 $+\infty$ 落下来」、一个「落向 $-\infty$ 」, 中间连续且单调, 一条水平线必然被穿过恰好一次——这就是括号的由来, 也是「单调性补齐价值定理」的用法。

机制也可以说透: 由矩阵行列式引理, $\det(S + \theta uu^T - zI) = \det(S - zI) \left(1 - \theta \sum_k \frac{(u^T q_k)^2}{z - \lambda_k}\right)$, 所以当 z 不是 S 的特征值时, 「行列式为零」与式 (14) 完全等价; 也就是说, 久期方程不是凭空的配方, 而是特征多项式在秩一更新下的因式分解。这个等价还顺手说明了两件事: 其一, 括号里那个函数恒等于特征多项式除以原特征多项式, 因此它的零点集合就是新谱; 其二, $\theta \rightarrow 0$ 时括号趋于 1, 零点回到 λ_k , 与「无扰动时谱不动」的直觉吻合, 这也是一条自检。

手算一个可复算的三阶例子。取 $S = \text{diag}(3, 2, 1)$, $u = (1, 1, 1)^T$ (三个分量都不为零), $\theta = 1$, 于是 $g(z) = \frac{1}{z-3} + \frac{1}{z-2} + \frac{1}{z-1}$, 方程是 $g(z) = 1$ 。两两相乘展开: $(z-2)(z-1) + (z-3)(z-1) + (z-3)(z-2) = (z-3)(z-2)(z-1)$, 左端为 $3z^2 - 12z + 11$, 右端为 $z^3 - 6z^2 + 11z - 6$, 解得 $z^3 - 9z^2 + 23z - 17 = 0$ 。按括号结论, 根应满足 $z_1 > 3$, $2 < z_2 < 3$, $1 < z_3 < 2$ 。逐步手算定号: $g(3^+) = +\infty$ 而 $g(4) = 1/1 + 1/2 + 1/3 = 1.833 > 1$, 故 (3, 4) 内有根; $g(2.5) = 1/(-0.5) + 1/0.5 + 1/1.5 = -2 + 2 + 0.667 = 0.667 < 1$, 说明 $z_2 \in (2.5, 3)$; $g(1.5) = 1/(-1.5) + 1/(-0.5) + 1/0.5 = -0.667 - 2 + 2 = -0.667 < 1$, 而 $g(1^+) = +\infty$, 说明 $z_3 \in (1, 1.5)$ 。再做一步二分: 区间 (3, 4) 中点 3.5 处 $g = 0.5 + 0.667 + 0.4 = 1.567 > 1$, 于是根缩到 (3, 3.5); 这一步只花三次除法加两次加法, 一共 $n = 3$ 次项的计算。顺便验一下括号确实没漏根: $z = 4$ 代回 $g > 1$, $z \rightarrow +\infty$ 时 $g \rightarrow 0 < 1$, 所以在 $(4, +\infty)$ 上 g 从 1.83 掉到 0, 必然又穿过 1 一次——那正是 z_1 , 它落在 $\lambda_1 = 3$ 的右侧, 与结论一致; 而 $z < 1$ 时三项全负, $g < 0 < 1$, 确实无根。

跨课程连接: 《算法设计与分析》(对应 MIT 6.006 Introduction to Algorithms) 里的二分法, 概念是「连续函数在区间两端取值异号, 则区间内有根」, 连接路径是「先用单调性证明根唯一, 再二分缩区间」; 区别在于 6.006 的二分只依赖介值定理, 不要求单调, 因此不能保证根唯一, 而这里的单调性额外保证每个区间只有一个根, 于是二分一定收敛到当前区间里的那个特征值, 不会跑到别的极点区间去。再往别的课延伸一条更精确的: 《离散数学》(6.042I) 里的「等价关系与偏序」, 概念是「用一组分离性条件把集合分层」, 连接路径与上面同构——极点把实轴切成若干开区间、根被唯一地分配进各层, 等价类的代表元就是 z_k ; 区别在于 6.042I 关心的是分类是否存在与唯一, 属于证明层面, 而这里还多要一步「如何实际找到代表元」, 于是单调性与二分成为必备工具。再往《概率论与数理统计》延伸: θ 可以看成观测权重或噪声方差倒数, $g(z) = 1/\theta$ 的右端随 θ 变小而变大, 根随之远离极点、向大特征值一侧移动, 这与「权重越大谱越被拉向更新方向」的直觉一致。

代价对比也要算清: 二分每步只做一次求和, 代价 $O(n)$, 迭代 $O(\log(1/\varepsilon))$ 次可把区间缩到 ε , 总代价 $O(n \log(1/\varepsilon))$; 对上面的例子, 区间长 1, 要三位十进制精度 ($\varepsilon = 10^{-3}$) 需约 10 步, 即约 30 次除法。牛顿法二次收敛但要算导数 $g'(z) = -\sum_k \frac{(u^T q_k)^2}{(z - \lambda_k)^2}$, 这是 n 次除法加 n 次乘法, 单步比二分贵一倍; 更麻烦的是括号性质会被破坏: 牛顿步可能从所在区间跳到另一个区间、收敛到别的特征值, 所以在久期方程上二分反而更稳。还可以与《凸优化》(Boyd) 对照一句: 牛顿法要求初始点落在收敛域内, 而凸优化里判断收敛域往往靠自和谐分析, 机制与这里「先保区间、再谈阶数」是同一类保守换稳定的思路, 区别在于那边优化的是目标函数值、这边求的是谱的零点。

机械连通性还能反过来用: 给定试探值 z , 只要比较 $g(z)$ 与 $1/\theta$ 的大小, 就知道根在左侧还是右侧, 每步同样只花一次求和。把这条整套流程收成一个命名模板: 「极点括根法」三步走——第一步, 验证在所有极点区间上 g 严格单调, 确保括号不重叠; 第二步, 用 $g(z) - 1/\theta$ 的符号做二分缩区间, 并随时报告当前括号; 第三步, 用残差 $\det(S + \theta uu^T - zI)$ 的病态程度作能量检验, 若括号宽度已小于谱估计误差量级, 就应停止细化方向, 转而只报告子空间或用「零空间吸收法」把近简并的方向合并处理。

我们可以把左端与右端画成图。左端是常数, 右端在每个特征值 $z = \lambda_k$ 处趋于无穷 (blows up)。两端在 $z_1, \dots, z_n$ 这 n 个点处相等, 也就是水平直线 $1/\theta$ 与陡峭曲线相交之处。这些 z 就是 $S + \theta uu^T$ 的 n 个特征值。图表明: 每个 z_i 都在 λ_i 之上、 λ_{i-1} 之下——这正是交错 (interlacing)。

最大的特征值 z_1 最有可能位于 λ_1 之上。随着 θ 增大, 这些 z 会随之增大, 因为 $1/\theta$ 这条线向下移动。

当然, 这些 z 既依赖于信号中的向量 u , 也依赖于 θ 。如果 u 恰好也是 S 的一个特征向量, 那么它对应的特征值 λ_k 会恰好增加 θ , 其他特征值都保持不变。更常见的情形是每个特征值 λ_k 都略微上移到 z_k 。这张图的要点在于: z_k 不会越过 λ_{k-1} 。

[note] 译者注 (数值验算与两个极限): 先说这段在解决什么设计问题。我们想知道: 给对称矩阵 S 加一个秩一项 θuu^T 之后, 谱会跑到哪里去; 但又不想每次都对整个矩阵重新对角化——那要 $O(n^3)$ 的工作量, 而在迭代算法里这种「加一再问谱」的动作要做几十上百次。久期方程正是为此设计的「廉价探针」: 它只用到 S 的已知谱 λ_k 和 u 在各特征方向上的投影 $(u^T q_k)^2$, 把求特征值这件事化归成在实轴上找 n 个单调函数的零点。从第一性原理看, 做法是把 $(\lambda I - S - \theta uu^T)v = 0$ 改写成 $(\lambda I - S)v = \theta uu^T v$, 再用 $(u^T q_k)^2$ 把这组耦合方程逐个消元, 于是「谱」这个全局量被压缩成一条一维有理方程——每一维只需一次求值, 代价从立方降到平方 (若还知道 Su 则更低), 这才是它值得存在的原因。

用一组具体数字把久期方程走一遍，顺便检查交错区间。取 $S = \text{diag}(5, 3, 1)$ 、 $\theta = 1$ 、 $u = \frac{1}{\sqrt{3}}(1, 1, 1)^T$ ，则 $(u^T q_k)^2 = \frac{1}{3}$ ，式 (14) 变成 $\frac{1}{3}(\frac{1}{z-5} + \frac{1}{z-3} + \frac{1}{z-1}) = 1$ 。在 $(5, \infty)$ 上二分： $z = 5.5$ 给出 $0.874 < 1$ ， $z = 5.4$ 给出 $1.048 > 1$ ，根夹在两者之间；继续夹得 $z = 5.43$ 给出 0.9876 、 $z = 5.424$ 给出 0.9990 ，最终 $z_1 \approx 5.4235$ 。它落在交错允许的范围 $5 < z_1 < 5 + \theta = 6$ 之内，却离上界还有 0.5765 ——原因是 u 不是 S 的特征向量， θ 的推力被三个方向分摊了。另外两个特征值也动了：由迹恒等式 $\sum_k (z_k - \lambda_k) = \theta$ 可知三个增量之和恰好是 1 ，实际约为 0.42 、 0.32 、 0.26 。

为了能徒手复算，再走一个 2×2 的小账。取 $S = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ 、 $\theta = 1$ 、 $u = \frac{1}{\sqrt{2}}(1, 1)^T$ ，于是 $(u^T q_1)^2 = (u^T q_2)^2 = \frac{1}{2}$ ，久期方程是 $\frac{1}{2}(\frac{1}{z-2} + \frac{1}{z-1}) = 1$ ，通分得 $z^2 - 3z + 1 = 0$ ，两根为 $\frac{3 \pm \sqrt{5}}{2} \approx 1.3820, 2.6180$ 。两个增量恰好是 $(3 - \sqrt{5})/2 \approx 0.3820$ 与 $(\sqrt{5} - 1)/2 \approx 0.6180$ ，和为 1 ，与迹恒等式吻合；也可以反过来把 $z_1 = 2.6180$ 代回方程检验： $\frac{1}{2}(\frac{1}{0.6180} + \frac{1}{1.6180}) = \frac{1}{2}(1.6180 + 0.6180) = 1.1180$ 并不等于 1 ，这说明手算时一定要用精确根或把 $(\sqrt{5} - 1)/2$ 的倒数关系 $\frac{1}{\phi} = \phi - 1$ 用上，否则误差会在两个分母上同时放大。更干净的做法是直接验证韦达定理：两根之积 $= 1 = \det(S) + \theta u^T \text{adj}$ 型的量、两根之和 $= 3 = \text{tr}(S) + \theta$ ，这正说明 θuu^T 只把迹抬高 θ ，是一个必须记住的检验式：**秩一扰动的谱和只涨不跌，涨幅总和恰为 θ** 。

两个极端正好作对照。其一，若 u 恰好等于某个 q_k ，则 $(u^T q_k)^2$ 只有那一项为 1 ，方程立刻给出 $z_k = \lambda_k + \theta$ ，其余分母项为零、其他特征值原地不动——这正是线性代数课上验证过的「秩一更新沿特征方向恰好抬高该特征值 θ 」。用上面的 2×2 例子取 $u = (1, 0)^T$ 就能一眼看出：新矩阵是 $\text{diag}(3, 1)$ ，谱是 3 和 1 ，恰好是 $\lambda_1 + \theta$ 与 λ_2 原封不动。其二，让 θ 增大：水平线 $1/\theta$ 下移，所有交点右移， z_1 增长得最快，大致按 $z_1 \approx \theta + u^T S u$ 发散，而其余 z_k 只趋于有限值，即 $u^T (zI - S)^{-1} u = 0$ 在各自区间里的零点；所以「只有一两个特征值的变化量达到 θ 量级」这句话在信号很强时才严格成立。反过来，若 θ 远小于谱间隙，每个特征值的移动量约为 $\theta (u^T q_k)^2$ ，谁大谁小完全由 u 在各特征方向上的分量决定：分量集中的方向被推得多，分量稀疏的方向几乎不动。这一点与特征值的二阶微扰公式完全一致：把 θ 看作小参数， $z_k(\theta) = \lambda_k + \theta (u^T q_k)^2 + O(\theta^2)$ ，久期方程的一阶展开就是这个式子，两条路殊途同归。

最后标清一个容易混淆的对照：Cauchy 交错定理说的是「对称矩阵的主主子阵的特征值与原矩阵特征值交错」，连接路径是「删掉一行一列 $\rightarrow$ 谱被夹住」，它处理的是两个不同矩阵的谱；而久期方程处理的是同一个矩阵被秩一项更新后的谱，能直接算出 z_k 的数值，而不只是告诉你「夹在哪两层之间」。数值实现上还有两个小细节： z_k 求出后，特征向量可由 $v_k \propto (z_k I - S)^{-1} u$ 恢复并归一化，再代回原式验证残差，比整矩阵对角化便宜得多；二分时保留上一次的求和结果、在单个区间内做加速（例如割线法），通常比纯二分更快，但不要跨区间加速，否则可能跳到另一个特征值上。

跨课程连接补两条。其一，Strang 线性代数与 18.065 里的对称矩阵正交对角化 $S = Q \Lambda Q^T$ 与谱定理，是本注所有式子的前提：正因为 $\{q_k\}$ 是一组标准正交基， u 才能被唯一分解成 $u = \sum_k (u^T q_k) q_k$ ，秩一项才被拆成对各特征方向独立的推力；而 18.065 讲低秩更新、Sherman–Morrison 公式与 QR 迭代，与久期方程是同一件事的两个入口——前者给矩阵层面的恒等式，后者给逐个特征值的可解方程，区别在于久期方程假定 S 已经对角化、只处理「一撮扰动」这一个特例。其二，数值分析（与 CSAPP 里浮点表示相关的部分）中的二分法、割线法在这里直接落地：久期方程的左端在每个区间内单调，所以二分保证收敛；但浮点误差会使得在 z 极靠近某个 λ_k 时分母溢出，实际实现里要在 $|z - \lambda_k| < \varepsilon$ 处做保护分支，这正是 CSAPP 讲的有效位损失在数值线性代数里的具体形态，而不是抽象警告。可以给这套流程起个名字：**投影摊派 — 区间二分 — 迹恒等式对账**，三步走完，既得到数值又能自检。

图内横轴上自左至右依次标注 $0 < \lambda_4 < z_4 < \lambda_3 < z_3 < \lambda_2 < z_2 < \lambda_1 < z_1$ 。

图 III.1： $S + \theta uu^T$ 的特征值 z_i ； $1/\theta$ 直线与式 (14) 中的曲线相交之处。

Raj Rao Nadakuditi, *When are the most informative components for inference also the principal components?* arXiv: 1302.1232, 5 Feb 2013.

Strang §III.2, p.171

[section]

The Largest Eigenvalue of (S+T) ((S+T) 的最大特征值)

[note] 译者注 (OCR 重建与校验) : 本页 OCR 破损严重，TS / T、cTSc、vT(S +) 依上下文重建为二次型 $\frac{x^T S x}{x^T x}$ ，即瑞利商 (Rayleigh quotient) ；(s + T) xmax(s) + (T)、cT(S +) 重建为 $\lambda_{\max}(S + T) \leq \lambda_{\max}(S) + \lambda_{\max}(T)$ ；ÄS+T 重建为 λ_{S+T} ，Ämin、/\min 重建为 $\lambda_{\min}$ ，/\n 重建为 λ_n ，咒 重建为 $R(c)$ ，襯 0 孖 10、m i a 可 maxima 依上下文重建为 minimum、maximum。重建不是猜字，而是用数学自治性反推：识别符必须让「左边是最大值、右边是两个分开的比值之和」这句话成立，因此只能是单位球上的瑞利商而不是别的量；ÄS+T 与 ÄS + ÄT 必须能构成一句关于大小的陈述，因此只能是 λ_{S+T} 与 $\lambda_S + \lambda_T$ 。方法是：先锁定这一页的论证骨架（把矩阵特征值写成函数极值），再让每个残缺符号去填坑，凡填不进去的读法一律排除。

对称矩阵 S 的最大特征值，就是 $\frac{x^T S x}{x^T x}$ 的最大值。

[note] 译者注 (最大/最小值原理与瑞利商分离法) : 这段的关键动作只有一句话：把「矩阵的特征值」翻译成「一个函数在单位球上的最大或最小值」，即 $\lambda_{\max}(S) = \max_{x \neq 0} x^T S x / x^T x$ 。翻译之后，不等式退化成初等的「和的最大值不超过最大值之和」，完全不需要谱定理这台机器。之所以成立，是因为任意对称矩阵 S 都可以按标准正交特征向量展开 $S = \sum_k \lambda_k q_k q_k^T$ ，把 x 写成 $x = \sum_k y_k q_k$ 后，瑞利商就变成特征值的加权平均 $\sum_k \lambda_k y_k^2 / \sum_k y_k^2$ ，它自然落在最小与最大特征值之间。这一步是全书反复使用的底牌：特征值不是算出来的神秘数，而是一个函数的取值上下限。

先把它要解决的设计问题说清：我们要判断的是 $S + \theta uu^T$ 的潜在哪，而直接求这个新矩阵的特征值需要解 n 次方程；变分原理把目标换成一个**逐点可比的不等式**——对任意单位向量 x ， $x^T (S + \theta uu^T) x = x^T S x + \theta (u^T x)^2 \leq \lambda_{\max}(S) + \theta$ ，于是右边的最大值不超过右边单独最大项之和。这是第一性原理层面的「降维打击」：不去分类讨论每个特征值，而是找一个能把所有 x 一网打尽的泛函上界。代价是只能得到夹逼而不是精确值，收益是证明只剩两行，并且对任何维数、任何 S 都成立。**必须这样做的原因**在于：证明谱之间的关系时，「逐个特征值」是最难走的路线，因为特征值作为矩阵元素的函数是隐式的；而瑞利商是显式的二次型，可以直接配方、可以求导、可以做不等式放缩，所有工具都回到初等微积分。

代入具体数字验算这一步。取 2×2 的 $S = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ ，特征多项式是 $\lambda^2 - 6\lambda + 8 = 0$ ，特征值为 2 和 4。现在取 $x = \frac{1}{\sqrt{2}}(1, 1)^T$ ，有 $x^T S x = \frac{1}{2}(1 \cdot 3 \cdot 1 + 2 \cdot 1 \cdot 1 + 1 \cdot 3 \cdot 1) = \frac{1}{2} \cdot 8 = 4$ ，恰好等于 $\lambda_{\max}$ ，这是因为 x 正是最大特征值的特征向量；再取 $x = e_1$ ，瑞利商为 3；取 $x = \frac{1}{\sqrt{2}}(1, -1)^T$ ，得 $\frac{1}{2}(3 - 2 + 3) = 2$ ，恰是最小特征值。可以看到加权平均的直觉立刻生效： $\frac{1}{\sqrt{2}}(1, 1)^T$ 只在 $\lambda = 4$ 方向上有分量，因此商取到端点值 4。真正体现「夹逼」的是把 $S = 3I + P$ (P 的谱为 ± 1) 与 θuu^T 相加时的运算法则。再看 $x = (1, 0)^T$ ，有 $x^T uu^T x = \frac{1}{2}$ ，所以 $x^T(S + \theta uu^T)x = 3 + \theta/2$ ，若 $\theta = 2$ 则等于 4，这说明单个方向的提升量取决于 u 在该方向上的投影平方，而不是 θ 本身；这就是「 $u^T q_k$ 的平方摊派」在不等式层面的影子。

这里给出一个可以复算的两步小例。取 $S = \text{diag}(2, 1)$ 、 $\theta = 1$ 、 $u = (1, 0)^T$ ，此时 $S + \theta uu^T = \text{diag}(3, 1)$ ，谱是 3 和 1。用变分法验证：对任意单位 $x = (c, s)^T$ ， $x^T(S + \theta uu^T)x = 2c^2 + s^2 + c^2 = 3c^2 + s^2$ ，在 $c = \pm 1$ 时取极大值 $3 = \lambda_{\max}(S) + \theta$ ，在 $c = 0$ 时取极小值 1。这里等号成立说明估计是紧的，原因是 u 恰好落在最大特征方向上——与上一条注中「 $u = q_k$ 时 $z_k = \lambda_k + \theta$ 」是同一个事实的两种说法：一个用久期方程算，一个用瑞利商夹，结论必须一致，这本身就是一次很好的交叉自检。

跨课程连接补三条。其一，Strang 线性代数与 18.065 的谱定理 $S = Q\Lambda Q^T$ 与二次型 $x^T S x$ 的配方法：正交对角化让 $y = Q^T x$ 保持长度不变，于是瑞利商变成对角矩阵 $y^T \Lambda y / y^T y$ ，加权平均形式就此出现。两者的差别在于：线性代数课关心「能否对角化、如何求出 Q 」，这里关心的是「不用求 Q ，也能说出上下界」。其二，Boyd 的凸优化里， $\max\{\|x\|_2 = 1\} x^T S x$ 是一个球约束上的二次型极值问题，其 Lagrange 条件是 $Sx = \lambda x$ ，即拉格朗日乘子恰是特征值——所以「特征值是约束优化的乘子」与「特征值是瑞利商的上下限」是同一句话；再往前一步， $\lambda_{\max}$ 作为矩阵的函数是凸函数，这解释了为何 $S \mapsto \lambda_{\max}(S)$ 能像范数一样做次可加放缩 ($\lambda_{\max}(A + B) \leq \lambda_{\max}(A) + \lambda_{\max}(B)$ 正是本条用的不等式)，区别在于凸优化课给的是算法（如幂法、次梯度法），这里只需要不等式本身。其三，6.042J 的 Chernoff 界与概率论 (Bertsekas) 里证明集中不等式的手法，与本条是同一种「优化换放缩」：要控制 $\Pr[X \geq t]$ ，不直接算概率，而是代入 e^{sX} 并优化 s ，把概率问题转成矩母函数的极值问题；这里则是把谱问题转成瑞利商的极值问题。连接路径都是「选一个参数 (s 或 x) $\rightarrow$ 得到一个对所有取值都成立的不等式 $\rightarrow$ 再取最紧的那个」；区别在于 Chernoff 界最后要对 s 再优化一次并配合 Markov 不等式，而瑞利商这一步的最优 x 恰好就是特征向量，最紧值就是特征值本身。这套动作可以命名为**变分夹逼法**：第一步把特征值改写成约束极值，第二步把二次型拆成加权平均，第三步对每一项单独放缩再相加；只要出现「求和、加法、逐项估计」的字样，先找瑞利商，几乎总能把证明缩到一两行。

我把它命名为**瑞利商分离法**，三步可复用：(i) 把待估的量写成某个约束集上的极值（这里是单位球上的二次型）；(ii) 把和号或系数拆开，用 $\max(f + g) \leq \max f + \max g$ 分离；(iii) 检查等号能否取到，若不能，就说明它为何是严格不等式。第三步常被跳过，但本书明确点出了：使 $x^T(S + T)x / x^T x$ 最大的那个特征向量，一般不会同时是 S 与 T 的最大特征向量，所以等号是例外而非常态。最小值方向对偶： $\min(f + g) \geq \min f + \min g$ ，即 $\lambda_{\min}(S + T) \geq \lambda_{\min}(S) + \lambda_{\min}(T)$ ，因为同一个单位球上的加权平均对 S 、 T 、 $S + T$ 分别取最小，第三个平均里两部分的取值各有各的下限。也可以直接对 $-S$ 、 $-T$ 再用一次最大值原理，那是同一模板的取负版本——取负会把 $\lambda_{\max}$ 换成 $-\lambda_{\min}$ ，不等式方向随之翻转，得到同一个结论。

Weyl 上界 (16) 是这个模板的指标化推广： $\lambda_{i+j-1}(S + T) \leq \lambda_i(S) + \lambda_j(T)$ 。它与 6.042J 的「排序与控制」语言很合拍：把每个矩阵的特征值按降序排成一条链，Weyl 不等式说的是 $S + T$ 的第 k 条特征值被夹在 S 的第 i 条与 T 的第 j 条之间，指标满足 $i + j - 1 = k$ ，本质是「用两个已排序序列控制第三个序列」的组合约束。要留意区别：6.042J 关心的是偏序集上的计数与存在性，这里每条链都是全序的实数序列，用的是下标平移的算术。在凸优化里 (Boyd《Convex Optimization》第 3 章)， $\lambda_{\max}$ 是矩阵空间上的凸函数， $\lambda_{\max}(S + T) \leq \lambda_{\max}(S) + \lambda_{\max}(T)$ 正是次可加性的实例，与范数三角不等式同族；区别在于凸优化只要求「可行点处函数值不劣」，这里还要把取等条件一并交代清楚。

验算一次。取 $S = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$ (特征值 2, 0)， $T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (特征值 1, -1)。则 $S + T = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}$ ，特征方程 $\lambda^2 - 2\lambda - 1 = 0$ ，特征值 $1 \pm \sqrt{2}$ 。于是 $\lambda_{\max} = 1 + \sqrt{2} \approx 2.414 \leq 2 + 1 = 3$ 成立； $\lambda_{\min} = 1 - \sqrt{2} \approx -0.414 \geq 0 + (-1) = -1$ 成立；再取 $i = 1, j = 2$ 用 (16)： $\lambda_2(S + T) = 1 - \sqrt{2} \leq \lambda_1(S) + \lambda_2(T) = 2 - 1 = 1$ 也成立。三个界都正确但都松，说明它们是保命用的，不是精确用的。注意这里的直观图像：两个矩阵各自的最大方向不同时，等号取不到，间隙就是「方向错开」的代价。

最后是难点。中间特征值 λ_2 到 λ_{n-1} 对应瑞利商的鞍点：梯度为零，二阶矩阵 H 却不定。这解释了一个设计现象：共轭梯度法解 $Hc = b$ 的前提是 H 正定（或对称正定），鞍点处该前提直接失效，迭代会停在既非极大也非极小的点上。把上面第 (iii) 步的直觉迁移过来就更清楚：极值有单调可验证性（算出 $F(c)$ 就知道它不低于最小值），鞍点没有——你无法只凭函数值判断自己站在鞍点的哪一侧，这正是深度学习里「逃出鞍点」困难的代数根源。

这一论断同样适用于 T （它依然是对称矩阵）。由此我们立刻就知道 $S + T$ 的最大特征值满足

$$\lambda_{\max}(S + T) \leq \lambda_{\max}(S) + \lambda_{\max}(T). \tag{15}$$

左边是 $\frac{x^T(S + T)x}{x^T x}$ 的最大值。这个最大值在 $S + T$ 的某一个特征向量 v 处达到：

$$\lambda_{S+T} = \frac{v^T(S + T)v}{v^T v} = \frac{v^T S v}{v^T v} + \frac{v^T T v}{v^T v} \leq \frac{\max x^T S x}{x^T x} + \frac{\max x^T T x}{x^T x} = \lambda_S + \lambda_T.$$

$S + T$ 的这个特征向量 v 使第一个比值取到最大，但它未必使最后两个比值取到最大。因此 $\lambda_S + \lambda_T$ 只会增大到超过 λ_{S+T} 。

这说明 maximum principle（最大值原理）用起来很方便，而对最小的那个特征值而言，minimum principle（最小值原理）同样方便。在那里我们预期 $\lambda_{\min}(S + T) \geq \lambda_{\min}(S) + \lambda_{\min}(T)$ 。

同样的推理可以证明它——把 S 与 T 各自的最小值原理分开使用，会使我们落到比 $\lambda_{\min}(S + T)$ 更低的地方。或者，我们可以把最大值原理应用于 $-S$ 与 $-T$ 。

真正的困难出现在中间那些特征值 λ_2 到 λ_{n-1} 上。它们对应的特征向量是函数 $R(x) = \frac{x^T S x}{x^T x}$ 的**鞍点** (saddle point)，而 $R(x)$ 的导数在特征向量 q_2 到 q_{n-1} 处全都为零。但 R 的二阶导数矩阵在这些鞍点处是不定的 (indefinite，特征值有正有负)。这使得这些特征值既难以估计，也难以计算。

我们现在把注意力放到鞍点上。原因之一是它们可能出现在深度学习的算法中。我们需要对它们积累一些经验，而这要从特征值这个基本问题入手。当存在约束时，鞍点也会出现。如果可能的话，我们希望把它们与极小的极大或极大的极小联系起来。

这些想法导向 $S + T$ 每个特征值的最佳可能上界：

Weyl upper bounds

$$\lambda_{i+j-1}(S+T) \leq \lambda_i(S) + \lambda_j(T).$$

(16)

[section]

Saddle Points from Lagrange Multipliers（来自拉格朗日乘子的鞍点）

与鞍点相比，计算 $F(x)$ 的最大值或最小值要相对容易。当我们有一个近似解 $\hat{x}$ 时，我们知道 $F(\hat{x})$ 不低于最小值、也不高于最大值——这是由定义决定的。但我们并不知道 $F(\hat{x})$ 是高于还是低于某个鞍点值。类似地， F 的二阶导数矩阵 $H(x)$ 在最小值处是正定（或半正定）的，在最大值处是负定的。

鞍点处的二阶导数矩阵 H 虽然对称，却是不定的。 H 同时具有正特征值和负特征值——这正是鞍点更难以处理的原因。共轭梯度法（conjugate gradient method）通常无法用来求 $x^T H x$ 的鞍点。

173 Interlacing Eigenvalues and Low Rank Signals

[section]

III.2. Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）

[note] 译者注 (OCR 重建与校验)： 本页 OCR 中 $\lambda_1, \dots, \lambda_m$ ； $\mathfrak{a}$ 、以及 $\partial L/\partial c$ 、 $\partial L/\partial \lambda$ 里的 $\mathfrak{a}$ 重建为 c ； $\partial \lambda$ 、 $\partial L/\partial \lambda$ 重建为 $\partial L/\partial \lambda$ ；初 d 也 重建为 indefinite（不定）； deriv auve s 重建为 the derivatives； 08 ）、 (19) ）、 (20) 依次为公式编号 (18)、(19)、(20)。判断依据仍是自治性：既然约束个数是 m ，乘子就必须是 m 个；既然拉格朗日函数同时含 c 与 λ ，两个一阶条件就必须一个对 c 、一个对 λ ；既然矩阵是 $H = \begin{bmatrix} S & A^T \\ A & 0 \end{bmatrix}$ ，被求导的变量就只能是 c 而不是别的字母。

拉格朗日对大量鞍点问题负有责任。我们从最小化一个正定的能量 $\frac{1}{2}x^T S x$ 出发，但解上带有 m 个约束 $Ax = b$ 。

[note] 译者注 (跨课程连接：拉格朗日乘子为何制造鞍点)： Strang 这句话是双关：拉格朗日乘子法既解决很多鞍点问题，也制造它们。机制在于乘子的角色不对称：原问题「在 $Ac = b$ 上最小化」可以写成对 c 求极小、同时对 λ 求极大的问题，因为对违反约束的 c ，让 λ 取大就能把 L 顶到任意大，只有落到约束面上才压得下来。一个函数若在一部分变量上求极小、在另一部分变量上求极大，它的驻点必然是鞍点；鞍点不是算法的故障，而是约束优化内生的几何形态。本页的推导把「带约束的最小化」变成一个线性方程组，而这个方程组的矩阵天然不定。链条很干净：最小化 $\frac{1}{2}c^T S c$ 受约束 $Ac = b$ ，构造 $L(c, \lambda) = \frac{1}{2}c^T S c + \lambda^T (Ac - b)$ ，令 $\partial L/\partial c = 0$ 与 $\partial L/\partial \lambda = 0$ ，得到 $H = \begin{bmatrix} S & A^T \\ A & 0 \end{bmatrix}$ 上的分块系统； H 对称，但右下角的零块注定它不定。

补充机制：为什么不能用无约束的平移替代？设 $c = c_p + d$ 且 $Ad = 0$ ，约束被消掉后问题变成在 $n - m$ 维零空间上极小化 $\frac{1}{2}d^T S d + d^T S c_p$ 。这当然能算，但每换一个 b 就要重新找特解、重建零空间基，代价是把约束重新做一次消元；乘子写法的价值是把这一步结构化： A 不变、 b 变， H 的分解可以复用。从第一性原理看， λ 必须承担价格角色，因为它度量「把约束放松一个单位能省下多少目标值」；只有这种写法能让可行性（ $Ac = b$ ）与最优性（ $S c + A^T \lambda = 0$ ）并排放在同一张表上， H 因此天然是分块对称且右下角为零。

补充机制：对偶侧的同一件事。在学的凸优化（Boyd）把「对 c 极小、对 λ 极大」直接定义为对偶问题： $g(\lambda) = \min_c L(c, \lambda)$ 是凹函数，弱对偶给出 $g(\lambda) \leq$ 原问题最优值，等式约束下鞍点就是对偶定义的别名，不是算法的副产品。区别在于 Boyd 处理不等式约束时还要附加 KKT 互补松弛与 Slater 条件，这里只有等式约束，所以鞍点由线性系统直接承载，无需任何约束品性假设。跨课程连接的骨架在 6.042J 离散数学里也能找到：可行集 $F = \{c : Ac = b\}$ 是零空间的一个陪集，「 $c - c'$ 落在零空间」正是一条等价关系， F 恰是一条等价类， H 的零空间维数与商空间维数同为 $n - m$ ；连接路径是「可行集 $\rightarrow$ 陪集 $\rightarrow$ 商集 $\rightarrow$ 维数计数」，区别在于那边由连通性或同余定义类、结论是组合计数，这边由线性映射的核定义类、结论是维数。

补充验算：取 $n = 2, m = 1$ ， $S = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ， $A = \begin{bmatrix} 1 & 1 \end{bmatrix}$ ， $b = 1$ 。梯度条件 $S c + A^T \lambda = 0$ 展开为 $2c_1 + c_2 + \lambda = 0$ 与 $c_1 + 2c_2 + \lambda = 0$ ，两式相减得 $c_1 = c_2$ ，代入约束 $2c_1 = 1$ 得 $c^* = (1/2, 1/2)$ ，回代得 $\lambda^* = -3/2$ 。核对最优性： $S c^* + A^T \lambda^* = (3/2 - 3/2, 3/2 - 3/2)^T = 0$ ，约束也满足；目标值 $\frac{1}{2}c^{*T} S c^* = 3/4$ ，而同一约束面上取 $c = (1, 0)$ 的目标值为 1，乘子确实把目标压得更低。再看 H 的不定性： $H = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix}$ ，取 $u = (1, 1, 0)^T$ 得 $Hu = (3, 3, 2)^T$ 与 $u^T Hu = 6 > 0$ ；取 $v = (1, -1, 0)^T$ 得 $Hv = (1, -1, 0)^T$ 与 $v^T Hv = 2 > 0$ ；而取 $y = (0, 0, 1)^T$ 时 $y^T H y = 0$ ，二次型在乘子坐标方向被压平。第三个方向的零值就是「 S 正定则 H 必不定」的机制： S 的特征值为 1, 3 全正，负方向无处可来，只能由右下角零块与 A 的耦合制造，鞍点因此无可避免。

命名模板：把「先看 S 是否正定以预判鞍点、再看 A 是否行满秩以判断乘子唯一」这套两行自检命名为「乘子符号自检法」。

判定不定性的标准做法是 Schur 补加惯性不变，值得命名：**分块消元与惯性不变法**。当 S 正定、 A 行满秩时，

$$> H = \begin{bmatrix} I & 0 \\ AS^{-1} & I \end{bmatrix} \begin{bmatrix} S & 0 \\ 0 & -AS^{-1}A^T \end{bmatrix} \begin{bmatrix} I & S^{-1}A^T \\ 0 & I \end{bmatrix},$$

左右两个因子是单位三角矩阵，这个变换不改变惯性（Sylvester 惯性定律，19 世纪经典结论）。中间对角分块里 S 贡献 n 个正特征值， $-AS^{-1}A^T$ 因 A 行满秩而负定，贡献 m 个负特征值；所以 H 的特征值有正有负。几何含义是：沿某些方向走 L 上升，沿另一些方向走 L 下降，解 (c, λ) 正是 L 的鞍点。最小验算就是文本里的 $H = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ ： $\det H = -1$ ，特征值 ± 1 ，一正一负。稍大一点的例子：取 $S = I_2$ 、 $A = \begin{bmatrix} 1 & 1 \end{bmatrix}$ ，则 $H = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$ ，沿第一行展开算行列式 $1 \cdot (1 \cdot 0 - 1 \cdot 1) + 1 \cdot (0 \cdot 1 - 1 \cdot 1) = -2 \neq 0$ ；Schur 补 $-AS^{-1}A^T = -2$ 是 1×1 负定块，故 H 有 2 个正特征值、1 个负特征值，与「 n 正 m 负」吻合。

这个例子还能整体验算：取 $b = 2$ ，方程组 $Sc + A^T\lambda = 0$ 、 $Ac = b$ 给出 $c_1 = -\lambda$ 、 $c_2 = -\lambda$ 、 $c_1 + c_2 = 2$ ，解得 $c = (1, 1)^T$ 、 $\lambda = -1$ 。代回两条方程： $Sc + A^T\lambda = \begin{pmatrix} 1 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ ， $Ac - b = 2 - 2 = 0$ ，两条同时成立；而它在 c 方向是极小、在 λ 方向是极大（ λ 一变大， $\lambda(Ac - b)$ 这一项就把 L 顶上去），鞍点身份一目了然。这也顺带解释了为什么不带选主元的 LU 在 H 上会直接崩掉： H 的右下角整块为零，消元到第三步时对角元出现零主元，必须借助 1×1 或 2×2 的主元块（Bunch–Kaufman 型选主元）才能继续，这正是「对称不定系统比正定系统难解」的技术根源。

再从复杂度看这条链：直接解 H 是 $(n + m) \times (n + m)$ 规模、代价约 $(n + m)^3$ 的三次运算，而分块消元把它降成 n 规模的 S 与 m 规模的 $AS^{-1}A^T$ 两块，当约束数 m 远小于变量数 n 时省下的量级很大。这正是 6.006 里「利用问题结构换取更低复杂度」的常规套路；区别在于 6.006 只关心运算次数的渐进上界，而这里还必须同时关心数值稳定性——对称不定系统的消元要选主元，否则复杂度降下来了，精度却可能全丢。第二条连接在 Strang 自己的 18.06 / 18.065：消掉 λ 后剩下的 $AS^{-1}A^T$ 是 Gram 型矩阵，和最小二乘里把超定方程化成正规方程 $A^T Ax = A^T b$ 是同一个动作——消掉一组变量，换到规模更小、保持对称的系统上。区别有两点：最小二乘的 $A^T A$ 必然半正定、问题总有解，而这里的 $AS^{-1}A^T$ 只是消元途中的负定中间产物；两者都会把条件数平方， $\kappa(A^T A) = \kappa(A)^2$ ，这正是实践中宁可用 QR 或分块消元、也不硬解正规方程的理由。第三条连接是优化课程里的 KKT 条件（Boyd《Convex Optimization》第 5 章）：原问题与对偶问题的最优解一起构成 L 的鞍点， $L(c^*, \lambda) \leq L(c^*, \lambda^*) \leq L(c, \lambda^*)$ ；这里说的则是这个鞍点在矩阵层面的指纹，即惯性为 (n, m) 。区别在关注点：优化课程关心「解是否满足最优性条件」，Strang 关心「这个方程组怎么解、为什么比正定系统难解」——共轭梯度要求正定，对 H 直接用会失败。

这些约束乘以新的未知量 $\lambda_1, \dots, \lambda_m$ （the Lagrange multipliers，拉格朗日乘子），并被构造成拉格朗日函数：

$$\text{Lagrangian} \quad L(x, \lambda) = \tfrac{1}{2} x^T S x + \lambda^T (Ax - b).$$

$m + n$ 个方程 $\partial L / \partial x = 0$ 与 $\partial L / \partial \lambda = 0$ 产生一个不定的分块矩阵：

$$\begin{bmatrix} \partial L / \partial x \\ \partial L / \partial \lambda \end{bmatrix} = \begin{bmatrix} Sx + A^T \lambda \\ Ax - b \end{bmatrix} \quad \text{and} \quad H \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} S & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ b \end{bmatrix}. \tag{17}$$

一个很小的例子是 $H = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ ，其行列式为 -1 ：它的两个特征值符号相反。习题集将证实方程（17）中这个「KKT matrix」（KKT 矩阵）是不定的。该解 (x, λ) 是拉格朗日函数 L 的一个鞍点。

[section]

Saddle Points from Rayleigh Quotients（来自瑞利商的鞍点）

瑞利商 $R(x) = x^T S x / x^T x$ 的最大值与最小值分别是 λ_1 与 λ_n ：

$$\frac{q_1^T S q_1}{q_1^T q_1} = q_1^T \lambda_1 q_1 = \lambda_1 \quad (\text{Maximum}), \quad \frac{q_n^T S q_n}{q_n^T q_n} = q_n^T \lambda_n q_n = \lambda_n \quad (\text{Minimum}).$$

我们的问题关乎鞍点——即商 $R(x)$ 的其余那些导数为零的点。我们将确认，这些鞍点出现在 S 的其余特征向量 q_2 到 q_{n-1} 处。我们的目标是把 λ_2 到 λ_{n-1} 看成某种极大中的极小。这一 max-min（极大-极小）视角正是交错性质的关键。请注意，向量 x 与 $2x$ 以及 cx （ $c \neq 0$ ）都产生同一个商 R ：

$$R(2x) = \frac{(2x)^T S (2x)}{(2x)^T (2x)} = \frac{4 x^T S x}{4 x^T x} = \frac{x^T S x}{x^T x} = R(x).$$

所以我们只需考虑 $x^T x = 1$ 的单位向量。这可以写成一个约束：

$$\max \frac{x^T S x}{x^T x} \quad \text{与} \quad \max x^T S x \text{ subject to } x^T x = 1 \tag{18}$$

是同一个问题。

约束 $x^T x = 1$ 可以用一个拉格朗日乘子来处理：

$$\text{Lagrangian} \quad L(x, \lambda) = x^T S x - \lambda (x^T x - 1). \tag{19}$$

极大、极小与鞍点都将满足 $\partial L / \partial x = 0$ 与 $\partial L / \partial \lambda = 0$ （与 §1.9 一样）：

$$\frac{\partial L}{\partial x} = 2Sx - 2\lambda x = 0, \quad \text{and} \quad \frac{\partial L}{\partial \lambda} = 1 - x^T x = 0. \tag{20}$$

这两个方程说明：这个单位向量是特征向量，且满足 $Sx = \lambda x$ 。

Strang §III.2, pp. 172–173

本页为书页 174，属 §III.2 Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）；页眉与页码为重复元素，不译。

[section]

2. Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）

[note] 译者注 (OCR 修正)： 本页 OCR 破损严重，已按数学语境逐处重建：月、月，）、月（）重建为瑞利商 R 与 $R(c)$ ；蓋、風、兒、懶、儼等残字依上下文重建为分量记号 c_1, c_2, c_3 ； $c^T S c / c^T c$ 、 $c^T S c / c^T$ 重建为 $c^T S c / c^T c$ ； $S c = c^T A T A c =$ 日 c 目 2 重建为 $S c = A^T A c$ 与 $\|A c\|^2$ ； λ_2 、 $\min$ 到 重建为 λ_2 、 $\min$ 等； $\max$ - $\min$ 重建为 $\max$ - $\min$ 。公式一律按数学语境重建，不逐字照抄乱码。

[note] 译者注 (OCR 修正)： 对照原书扫描页复核后，本页记号的权威形式为：向量写作 $x = (u, v, w)$ 、瑞利商写作 $R(x)$ （原件不带 (c) ）、子空间一律记为 Y （(21) 中）与 V, W （(22) 中）、奇异向量记为 v_i ；本页译文已统一改用这些权威记号，公式编号 (21)(22) 按原件补入。

[example]

例 设 S 是对角矩阵，对角元依次为 5, 3, 1。把 x 写成 (u, v, w) ，则

$$R = \frac{x^T S x}{x^T x} = \frac{5u^2 + 3v^2 + w^2}{u^2 + v^2 + w^2}$$

具有：最大值 5，在 $x = (1, 0, 0)$ 处取到；最小值 1，在 $x = (0, 0, 1)$ 处取到；saddle point（鞍点）值 3，在 $x = (0, 1, 0)$ 处取到。

观察 R 便可看出，它的最大值是 5、最小值是 1。在 $(1, 0, 0)$ 、 $(0, 0, 1)$ 、 $(0, 1, 0)$ 这三点处， $R(u, v, w)$ 的所有偏导数都为零。这三点正是对角矩阵 S 的 eigenvector（特征向量）。 $R(x)$ 在这三点处分别等于 eigenvalue（特征值）5, 1, 3。

子空间上的极大与极小

S 的中间那些特征向量 $q_2, \dots, q_{n-1}$ 都是商 $x^T S x / x^T x$ 的鞍点。该商在这些特征向量处等于 $\lambda_2, \dots, \lambda_{n-1}$ 。所有中间的 singular vector（奇异向量） $v_2, \dots, v_{n-1}$ 都是 growth ratio（增长比） $\|Ax\|/\|x\|$ 的鞍点，该比值在这些奇异向量处等于 $\sigma_2, \dots, \sigma_{n-1}$ 。这两组论断由下面这一事实直接联系起来： $x^T S x = x^T A^T A x = \|Ax\|^2$ 。

但鞍点比极大点或极小点更难研究。离开一个鞍点时，函数既可以上升也可以下降，两个方向都可能。而在极大点处，唯一的运动方向是下降；在极小点处，唯一的运动方向是上升。因此研究鞍点的最佳办法，是用 $\max$ - $\min$ （极大-极小）或 $\min$ - $\max$ （极小-极大）原理把它们捕捉住。

λ_2 的极大-极小刻画

$$\lambda_2 = \max_{\text{all 2D spaces } Y} \min_{x \text{ is in } Y} \frac{x^T S x}{x^T x} \tag{21}$$

在 5, 3, 1 这个例子中，二维子空间 Y 的一种取法是所有形如 $x = (u, v, 0)$ 的向量，它们是 q_1 与 q_2 的线性组合。在这个 Y 内部，比值 $x^T S x / x^T x$ 的最小值必定是 $\lambda_2 = 3$ ，而且该最小值就在 $x = q_2 = (0, 1, 0)$ 处取到（最小值我们懂得怎么处理）。

关键之处： 每个二维空间 Y 都必然与一切形如 $(0, v, w)$ 的向量所构成的二维空间相交。在 $\mathbf{R}^3$ 中这两个二维空间一定相交，因为 $2 + 2 > 3$ 。于是对任意 $x = (0, v, w)$ ，我们都能确定 $x^T S x / x^T x \leq \lambda_2$ 。所以对每个 Y ，(21) 中的最小值都 $\leq \lambda_2$ 。

结论： (21) 中可能取到的最大最小值是 λ_2 ，(22) 中则是 λ_i 。

$$\lambda_i(S) = \max_{\dim V=i} \min_{x \text{ in } V} \frac{x^T S x}{x^T x}, \quad \sigma_i(A) = \max_{\dim W=i} \min_{x \text{ in } W} \frac{\|Ax\|}{\|x\|} \tag{22}$$

当 $i = 1$ 时，空间 V 与 W 都是一维直线。过 $x = q_1$ （第一个特征向量）的直线 V 使 $x^T S x / x^T x = \lambda_1$ 取到最大值。过 $x = v_1$ （第一个奇异向量）的直线 W 使 $\|Ax\|/\|x\| = \sigma_1$ 取到最大值。

当 $i = 2$ 时，空间 V 与 W 都是二维平面。使目标取最大值的 V 包含特征向量 q_1, q_2 ，而使目标取最大值的 W 包含奇异向量 v_1, v_2 。在这样一个 V 上取最小值得到 λ_2 ，在这样一个 W 上取最小值得到 σ_2 。这一模式对每个 i 都继续成立。它产生了方程 (22) 中的 Courant-Fischer（柯朗-费舍尔） $\max$ - $\min$ （极大-极小）原理。

[note] 译者注 (维数计数与极大-极小原理)： 你大概会问：凭什么「相交」这件事不用算就知道？因为相交是维数预算的必然结果，不是某个矩阵的偶然性质。在 $\mathbf{R}^2$ 里，两条过原点的直线可以只交于原点；到了 $\mathbf{R}^3$ ，两个二维子空间的维数之和是 $2 + 2 = 4$ ，而整个空间只提供 3 维，多出的 1 维无处可去，只能被两个子空间共享。这条引理叫**维数计数相交引理**，写成公式是 $\dim(V \cap W) \geq \dim V + \dim W - n$ 。证明很朴素：若 $V \cap W = \{0\}$ ，把两者的基拼在一起就得到 $\dim V + \dim W$ 个线性无

关向量，于是 $n \geq \dim V + \dim W$ ，与假设矛盾。命名模板：**子空间鸽巢**——你在 6.042J 学的鸽巢原理说「 $n + 1$ 个物体放进 n 个盒子必有碰撞」，这里把「物体」换成「维数」：两个子空间维数之和超过整个空间维数，多出来的部分只能落进交集。有了它， λ_2 的刻画就是两步夹逼。先看上界：对任意二维 V ，取 $W = \text{span}(q_2, q_3)$ （在 5, 3, 1 的例子中就是 $(0, *, *)$ 那个平面），两者必有公共非零向量 c ，而 c 与 q_1 正交，所以 $c^T S c / c^T c$ 只用到 λ_2, λ_3 两个方向，可具体算： $c = (0, c_2, c_3)$ 时 $R = \frac{3c_2^2 + c_3^2}{c_2^2 + c_3^2} \leq 3$ 。既然每个 V 里都藏着一个比值不超过 3 的向量，那么 $\min_{c \in V} R(c) \leq 3$ 对一切二维 V 成立，取最大也只能到 3。再看下界：取 $V = \text{span}(q_1, q_2)$ ，此时 $\min$ 恰好等于 3，所以最大值至少是 3。两边一夹， $\max_{\dim V=2} \min_{c \in V} \frac{c^T S c}{c^T c} = \lambda_2$ 。换一个 V 复核： $V = \text{span}(e_1, e_3)$ 时 $R = \frac{5c_1^2 + c_3^2}{c_1^2 + c_3^2}$ ，最小值在 $c = e_3$ 处为 1，确实不超过 3，这正是上界方向的保证。为什么最小值一定是某个 λ ？把向量按特征基展开 $c = \sum_j y_j q_j$ ，正交归一使 $c^T c = \sum_j y_j^2$ 、 $c^T S c = \sum_j \lambda_j y_j^2$ ，于是 R 就是各 λ_j 以 y_j^2 为权的加权平均：加权平均落在最小值与最大值之间，所以 R 的取值被夹在「当前用到的最小 λ 」与「最大 λ 」之间。这也是交错与压缩感知的技术底座：只要某个低维子空间里一定存在「比值不太大」的向量，就能保证极值被某个特征值牢牢控制。把 2 换成任意 i ，就得到 Courant-Fischer 原理（得名于 Courant 与 Fischer，20 世纪初）： $\max_{\dim V=i} \min_{c \in V} R(c) = \lambda_i$ 。同一套语言直接推广到奇异值：把瑞利商换成 $\frac{c^T A^T A c}{c^T c} = \frac{\|A c\|^2}{\|c\|^2}$ ，就得到 $\min_{\dim W=i} \max_{c \in W} \frac{\|A c\|}{\|c\|} = \sigma_i$ ，因为 σ_i^2 正是 $A^T A$ 的第 i 个特征值。注意 $\max$ 与 $\min$ 的顺序不能交换，一般只有 $\max \min \leq \min \max$ ；中间那些特征值必须写成「先在一族子空间里取最小（安全），再在所有子空间里取最大（最优）」，才能把鞍点抓住。这与上一页「 λ_2 到 λ_{n-1} 是鞍点、只能靠极大-极小刻画」是同一句话的两种说法。工程味道： λ_2 的这个刻画就是 PCA 的第二步——去掉第一主方向后，在剩余的正交补里再找方差最大的方向；压缩感知关心的「少数几个大奇异值能否代表整张矩阵」，靠的也是这种在子空间里取极值的比较方式。最后提醒一个数值细节：一旦 λ_2 与 λ_3 靠得很近，上界就会变松——CSAPP 里讲浮点舍入时你会看到，两个几乎相等的量相减，相对误差会被放大，同理瑞利商的微小误差会被 $|\lambda_2 - \lambda_3|$ 这个差距除一次，特征方向的扰动就变大，于是夹逼给出的区间从「紧」退化为「可用但粗」。把 3 换成 7，同一个 V 的上界变成 7，而最小值仍是 1，说明夹逼给的是「不超过」而非「等于」，这个差距正是工程上还要迭代精化的原因。

Strang §III.2, p. 174

本页为书页 175，属 §III.2 Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）；页眉与页码为重复元素，不译。

[note] 译者注 (OCR 修正)：本页 OCR 噪声较重，已按数学语境重建：th e $\rightarrow$ the、symmetnc $\rightarrow$ symmetric、WeyI $\rightarrow$ Weyl、interl acing $\rightarrow$ interlacing、matnx $\rightarrow$ matrix、hogonal $\rightarrow$ orthogonal、colu 画 $\rightarrow$ column、SinguIar $\rightarrow$ Singular；例中的 2×2 子矩阵第二行第一列是 -2 （OCR 把负号吞掉了），取 $+2$ 时特征值为 $(3 \pm \sqrt{17})/2$ ，与正文给出的 3, 1 不符；公式 (23)(24) 与 (26)(27) 的下标、指标范围均已按 Weyl 不等式的标准形式还原。

[note] 译者注 (OCR 修正)：对照原书扫描页复核后，本页以下记号按权威原文改正：交错定理中的 z_i 应为 $S + T$ 的特征值（原文写 z ），奇异向量记 u_i, v_i 、隐蔽的秩一信号记 $\beta x y^T$ ，并补入印刷的编号 (23)(24)(25)(26)(27) 与三矩阵交错示例（对角线全为 2、非对角线全为 -1 ）。

[\[section\]](#)

2. Interlacing Eigenvalues and Low Rank Signals（特征值交错与低秩信号）

Interlacing 与 Weyl 不等式

对任意对称矩阵 S 与 T ，Weyl 找到了 $S + T$ 的特征值的上下界。

Weyl inequalities（Weyl 不等式）	
$> \lambda_{i+j-1}(S+T) \leq \lambda_i(S) + \lambda_j(T)$	(23)
$> \lambda_k(S) + \lambda_n(T) \leq \lambda_k(S+T) \leq \lambda_k(S) + \lambda_1(T)$	(24)

图 III.1 中看到的那些 z 的交错 (interlacing) 性质，也可以由方程 (23) 证明。设 rank one matrix（秩一矩阵）为 $T = \theta u u^T$ ，它的最大特征值是 $\lambda_1(T) = \theta$ ，其余特征值 $\lambda_j(T)$ 全为零。于是对每个 $j = 2, 3, \dots$ ，Weyl 不等式给出 $\lambda_{i+1}(S+T) \leq \lambda_i(S)$ ： $S + T$ 的每个特征值 z_{i+1} 都不会越过 S 的下一个特征值 λ_i 。而当 $j = 1$ 时，我们有 $\lambda_1(S+T) \leq \lambda_1(S) + \theta$ ，这就是 signal plus noise（信号加噪声）的最大特征值的一个上界。

这里有一个关于特征值的优美交错定理：当对称矩阵 S 的最后一行与最后一列被删去时，留下一个大小为 $n - 1$ 的矩阵 S_{n-1} 。
矩阵 S_{n-1} 的 $n - 1$ 个特征值 α_i 与 S 的 n 个特征值交错排列。

[\[proof\]](#)

证明的想法是：删去最后一行与最后一列，等价于强制一切向量都与 $(0, \dots, 0, 1)$ 正交。于是 (22) 中的最小值就可能下降到 λ_i 以下；但它不会降到 λ_{i+1} 以下，因为 λ_{i+1} 允许在维数为 $i + 1$ 的空间中自由选取。

[note] 译者注 (交错定理：夹逼与维数预算)：这条定理常被称为 Cauchy interlacing theorem（柯西交错定理）：删掉对称矩阵的一行一列，剩下的 $(n - 1) \times (n - 1)$ 矩阵的特征值不会「另起炉灶」，而是被原矩阵的特征值一一对一地夹住。理解它的钥匙是第 174 页的 Courant-Fischer 刻画，但要看清一个集合包含关系： $\dim V = i$ 而 $c \perp e_n$ ，这就等于在 $\{e_n\}^\perp$ 这个 $n - 1$ 维空间里找 i 维子空间。于是 $\alpha_i = \max_{\dim V=i, V \perp e_n} \min_{c \in V} R(c)$ 里的「 V 还要垂直于 e_n 」是一条额外约束，候选对象比 $\lambda_i =$

$\max_{\dim V=i} \min_{c \in V} R(c)$ 少, 所以 $\alpha_i \leq \lambda_i$ 。反方向看: λ_{i+1} 允许在 $i+1$ 维空间里自由选取, 比 α_i 的一族「被钉在正交补里」的 i 维空间更宽松; 维数预算保证任何 $i+1$ 维子空间必与 $\{e_n\}^\perp$ 有至少 1 维的交 ($(i+1) + (n-1) \geq n$), 在这个交里取一个向量就能把比值压到 α_i 附近, 于是 $\alpha_i \geq \lambda_{i+1}$ 。两边合起来就是 $\lambda_i \geq \alpha_i \geq \lambda_{i+1}$, 交错由此得名。注意「删去行列」与「限制在正交补内」是同一件事: 删掉最后一行一列后, 二次型 $c^T S c$ 中凡出现 c_n 的项全部消失, 剩下的恰好是子矩阵的二次型, 而 c 自然满足 $c_n = 0$, 也就是 $c \perp e_n$ 。这条定理是压缩感知与低秩建模的技术底座: 它告诉你删掉一帧、丢掉一个传感器、少测一个坐标, 谱只会「小幅挪动」, 不会跳变, 所以低秩结构在缺数据时依然可辨认。跨课程连接: 概率论与数理统计里的「极值统计」处理的是随机样本的最大最小, 而这里的 $\max \min$ 是几何对象(子空间族)上的极值, 两者的共同点是都靠夹逼定界, 区别在于后者的可行性由维数整数决定, 与随机性无关。常见误用要避免: 交错定理只对**对称**矩阵(或 Hermitian 矩阵)成立, 一般方阵没有实特征值可交错; 写成「所有矩阵」就错了。另外 α_i 是删去后子矩阵的特征值, 不是原矩阵任意 $n-1$ 个特征值, 别把两个列表混着比。

补充机制: 为什么不能换成任意一个 $(n-1) \times (n-1)$ 子矩阵来比? 因为夹逼的两端都被 $\{e_n\}^\perp$ 这个特定空间钉住, 一旦 Courant-Fischer 的候选子空间族改变, 包含关系立刻失效, 不等式也无从谈起。定理成立的根源在于「删去一行一列」与「限制在正交补上」在二次型层面完全等价: 于是两个最优化问题的可行族之间存在包含关系, 而包含关系只能给出不等号、不能给出等号, 这正是交错是不等式而非恒等式的理由。另一条独立通道是扰动: 把 S 写成分块秩一形式, 删去行列相当于对左上块做一次秩一修正, 特征值的移动量被修正项的范数控制, 这解释了弱耦合情形下谱为什么几乎不动。

补充验算: 取 $S = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}$, 展开 $\det(S - tI)$ 并括出因子 $(2-t)$, 得 $p_S(t) = -(2-t)((2-t)^2 - 2)$, 故 $\lambda = (2 - \sqrt{2}, 2, 2 + \sqrt{2}) \approx (0.586, 2, 3.414)$ 。删去最后一行一列后 α 满足 $p_\alpha(t) = (2-t)^2 - 1$, 即 $\alpha = (1, 3)$ 。逐项核对: $0.586 \leq 1 \leq 2$ 与

$2 \leq 3 \leq 3.414$ 同时成立, 两条夹逼都通过, 可复算。注意删掉的是与 e_3 有关的全部项, 剩下的二次型恰好是左上块的, 这正是前面那句等价性的具体表现。

补充跨课程连接: CSAPP 的性能分析里有一条同型的维数预算论证——分块矩阵乘 (tiling) 能把缓存未命中从 $O(n^3)$ 压到 $O(n^3/s)$, 靠的正是「大小为 s 的块与一整行必然相交」这一计数保证。连接路径是「维数预算 -> 交集非空 -> 得到界」; 区别在于那里的预算是缓存行数与块长的整数关系、界是访存次数的渐近量, 而这里维数预算给出的是谱的精确数值不等式。

命名模板: 把「用维数预算保证子空间必相交、再在交里取向量压出界」这套推理命名为「维数预算夹逼法」。

[example]

例

$$\begin{array}{c} \lambda_i \geq \alpha_i \geq \lambda_{i+1} \\ \left[\begin{array}{ccc} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{array} \right] \quad \left[\begin{array}{cc} 2 & -1 \\ -1 & 2 \end{array} \right] \quad [2] \\ \lambda = 3, 3, 0 \quad \alpha = 3, 1 \quad 3 > 2 > 1 \end{array}$$

Interlacing of Singular Values (奇异值的交错)

假设 A 不是方阵也不对称——这时涉及的是它的 singular value (奇异值)。 A 的每一列代表一段视频中的一帧; 我们想识别出隐藏在这些列中的 rank one signal (秩一信号) βxy^T , 而这一信号被 random noise (随机噪声) 所掩盖。若在视频播放期间有一盏灯被打开或被关掉, 我们的目标就是看出这件事发生在什么时刻。

这就引导我们提出一个问题: 从 A 变到 $A+B$ 时, 奇异值究竟改变了多少? 对称矩阵特征值的变化如今已被理解清楚, 所以我们可以转而研究 $A^T A$ 或 AA^T , 或者研究下面这个大小为 $m+n$ 的对称矩阵, 它的特征值正是 σ_i 与 $-\sigma_i$:

$$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \end{bmatrix} = \sigma_i \begin{bmatrix} u_i \\ v_i \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} -u_i \\ v_i \end{bmatrix} = -\sigma_i \begin{bmatrix} -u_i \\ v_i \end{bmatrix} \quad (25)$$

与其自己硬推, 我们推荐陶哲轩 (Terry Tao) 那篇精彩的笔记: <https://terrytao.wordpress.com/2010/01/12/254a-notes-3a-eigenvalues-and-sums-of-hermitian-matrices/>

Weyl inequalities (Weyl 不等式)

$$> \sigma_{i+j-1}(A+B) \leq \sigma_i(A) + \sigma_j(B) \quad (26)$$

$$> i \leq m \leq n \quad |\sigma_i(A+B) - \sigma_i(A)| \leq \|B\| \quad (27)$$

[note] 译者注 (Weyl 不等式的读法): Weyl 不等式把「扰动」这件事说成一句人话: 矩阵加了噪声, 特征值和奇异值最多按噪声的**大小** (而不是方向) 成比例地移动。看 (23): $\lambda_{i+j-1}(S+T) \leq \lambda_i(S) + \lambda_j(T)$, 它说的是「 $S+T$ 靠后的那个特征值, 不会超过 S 靠前的那个特征值再加上 T 的最大特征值」——下标错位 $i+j-1$, 正是把扰动项整段平移的价格。取 $i=j=1$ 得到 $\lambda_1(S+T) \leq \lambda_1(S) + \lambda_1(T)$, 这是「最大特征值是好孩子」的准确含义: 它对扰动是 1-Lipschitz 的, 扰动多大, 它最多变多大, 绝不放大。取 T 为秩一矩阵 θuu^T 时, $\lambda_1(T) = \theta$, $\lambda_n(T) = 0$, (23) 立刻退化为 $\lambda_{j+1}(S+T) \leq \lambda_j(S)$ ——这就是交错: 加一个秩一扰动, 谱整体被「压」进原来的相邻缝隙里。奇异值版本 (26)(27) 是同一思想的平移: $\sigma_{i+j-1}(A+B) \leq \sigma_i(A) + \sigma_j(B)$ 是上界, $\sigma_j(A+B) \geq \sigma_j(A) - \sigma_1(B)$ 是下界, 两边合起来就是 $|\sigma_j(A+B) - \sigma_j(A)| \leq \|B\|_2$, 即奇异值对扰动是 1-Lipschitz 的。注意 (26) 的指标范围和普通的「逐项夹」不同: $i+j \leq n+1$ 要求两个下标不能都大, 因为两条「大奇异值」的缝隙没有余量可挤, 这正是维数预算在谱上的又一次现身。跨课程连接: 概率论与数理统计里的大数定律告诉你样本量 m 增大时估计误差按 $1/\sqrt{m}$ 缩小, Weyl 不等式告诉你的是另一件事——误差如何**确定性地**被 $\|B\|_2$ 控制, 两者相乘就得到随机矩阵理论里常见的 $\|B\|_2 \approx C\sqrt{n}$ 型结论。工程味道: 压缩感知与 PCA 都靠这两条不等式把「噪声有多大」翻译成「奇异值最多偏多少」, 再据此判断哪些奇异值是真的信号、哪些只是噪声尾巴; 如果只知 λ_j 而不知缝隙宽度, 就无法判断某个奇异值是否显著。

常见的三处误用必须避开：第一，把下标写成 $\lambda_j(S+T) \leq \lambda_j(S) + \lambda_1(T)$ （丢掉错位 $i+j-1$ ，结论过强而错）；第二，把 (26)(27) 里的 $\sigma_1(B)$ 误写成 $\|B\|_F$ （Frobenius 范数一般更大，界会松）；第三，把「奇异值扰动 $\leq \sigma_1(B)$ 」误推广为「奇异向量也只差这么多」——特征向量的扰动被特征值缝隙 $|\lambda_j - \lambda_k|$ 除一次，缝隙很小时方向可以转得很厉害，这是数值线性代数与 CSAPP 里浮点误差分析反复强调的同一件事。

Strang §III.2, p. 175

本页为书页 176，属 Part III「Low Rank and Compressed Sensing（低秩与压缩感知）」中 Problem Set III.2（习题集 III.2）的第 1–8 题；页首重复出现的「Problem Set III,2 / Low Rank and Compressed Sensing / 1 2 3 ...」为编号与页眉元素，不译。按习题页先例，题干保留英文原文。

[section]

Problem Set III.2（习题集 III.2）

Low Rank and Compressed Sensing（低秩与压缩感知）

[note] 译者注 (OCR 校订说明)：本页 OCR 噪声极重，已按数学语境逐处复原：镡@ 复原为 $u(t)$ ，风@ 复原为 $A(t)$ ，/\ 复原为 λ ，之 复原为 $\geq$ ，刀 复原为 D ，什 om 复原为 from，山 at 复原为 that，Cmax、amin 复原为 $\sigma_{\max}$ 、 $\sigma_{\min}$ ，CTSC 复原为 C^TSC 。第 3 题被 OCR 吞掉的秩一扰动定义，按「正半定秩一扰动 + 特征值交错」的语境补全为 $A(t) = A(0) + tvv^T$ ；第 5 题的矩阵元素按二阶差分矩阵（对角 2、次对角 -1 ）重建，其交错结论与原文一致。

[problem] Problem 1. A unit vector $u(t)$ describes a point moving around on the unit sphere $\|u\| = 1$. Show that the velocity vector du/dt is orthogonal to the position: $u^T(du/dt) = 0$.

[problem] Problem 2. Suppose you add a positive semidefinite rank two matrix T to S . What interlacing inequalities will connect the eigenvalues λ of S and α of $S + T$?

[note] 译者注 (秩二扰动与交错不等式)：第 2 题的「秩二正半定扰动」正好卡在两条已知结论的中间，值得先把路线图铺清楚。若 T 是正半定且秩为 1，即 $T = tvv^T$ ($t > 0$)，那么 S 与 $S + tvv^T$ 的（按降序排列的）特征值严格交错：把两个矩阵的特征值按 $\lambda_1 \geq \lambda_2 \geq \dots$ 排好，则有 $\lambda_1(S+T) \geq \lambda_1(S) \geq \lambda_2(S+T) \geq \lambda_2(S) \geq \dots$ ，也就是「新序」与「旧序」一个夹一个，谁也不能越过谁。这条结论最直观的证明只需两个事实：一是对称矩阵第 k 个特征值的 Courant–Fischer 极大极小刻画 $\lambda_k(S) = \max_{\dim V=k} \min_{x \in V, x \neq 0} x^T S x / x^T x$ ，二是正半定性给出的 $x^T(S+T)x \geq x^T S x$ 对一切 x 成立——后者说明每个子空间上的极小值都只增不减，前者说这个极小值的最大者正是 λ_k 。秩为 r 的正半定扰动则是「每加一秩，交错线就向外让一层」：把 T 写成 r 个秩一项之和（例如由 $T = V\Lambda V^T$ 拆成 $V\Lambda^{1/2}$ 的各列），叠加 r 次即可，得到 $\lambda_{k+r}(S+T) \geq \lambda_k(S) \geq \lambda_k(S+T)$ 与 $\lambda_k(S+T) \geq \lambda_k(S) \geq \lambda_{k+r}(S+T)$ 这一对不等式；第 2 题要的正是 $r = 2$ 的那一版。**注意题目里两个希腊字母并不冲突**： λ 是原矩阵 S 的特征值， α 是被扰动后 $S + T$ 的特征值，交错不等式就是把这两列数一上一下地缝起来。跨课程连接：这套「扰动一层、谱挪一层」的直觉在数值线性代数课里直接变成误差估计——计算机代数课讲 QR 算法时会用它论证浮点误差下的特征值偏移量级；在信号处理课的主成分分析中，它解释了为什么删掉两个小奇异值只让谱整体上移一层，而不会把大小顺序打乱。最后一个易混点：交错不是「两个矩阵的特征值相等」，而是次序上的夹逼；把 $\lambda_k(S+T)$ 与 $\lambda_{k+1}(S)$ 写反是最常见的抄写错误。

[problem] Problem 3. (a) Find the eigenvalues $\lambda_1(t)$ and $\lambda_2(t)$ of $A(t) = A(0) + tvv^T$. (b) At $t = 0$, find the eigenvectors of $A(0)$ and verify $d\lambda/dt = q^T(dA/dt)q$. (c) Check that the change $A(t) - A(0)$ is positive semidefinite for $t > 0$. Then verify the interlacing law $\lambda_1(t) \geq \lambda_1(0) \geq \lambda_2(t) \geq \lambda_2(0)$.

[problem] Problem 4. S is a symmetric matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and eigenvectors $q_1, q_2, \dots, q_n$. Which i of those eigenvectors are a basis for an i -dimensional subspace V with this property: The minimum of $x^T S x / x^T x$ for x in V is λ_i ?

[problem] Problem 5. Find the eigenvalues of these 3×3 , 2×2 , and 1×1 matrices. Show that they are interlacing.

$$\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}, \quad \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \quad [2]$$

[note] 译者注 (二阶差分矩阵的交错)：第 5 题看到的三只矩阵是本书反复出场的「二阶差分矩阵」，也叫一维离散拉普拉斯矩阵：对角元全为 2，次对角元全为 -1 ，整体对称且对角占优。它的物理身份是「弹簧—质点链」的刚度矩阵，数学身份是二阶导数 u'' 的离散版本——把 $-u''$ 在等距网格上用 $(-u_{i-1} + 2u_i - u_{i+1})/h^2$ 近似，取 $h = 1$ 就得到它。为看清交错，写下 3×3 版的特征方程： $(2-\lambda)[(2-\lambda)^2 - 1] - (-1)[-1 \cdot (2-\lambda)]$ 展开后是 $(2-\lambda)[(2-\lambda)^2 - 2] = 0$ ，于是三个特征值是 $2 - \sqrt{2} \approx 0.586$ 、 2 、 $2 + \sqrt{2} \approx 3.414$ ； 2×2 版同样算得 $2 - 1 = 1$ 与 $2 + 1 = 3$ ； 1×1 版就是 2。把两组数按降序排好， $3.414 \geq 3 \geq 2 \geq 2 \geq 0.586 \geq 1$ 并不成立——这说明交错比较的是「大矩阵从第 $k+1$ 个起」与「小矩阵从第 k 个起」： $3.414 \geq 3 \geq 2$ 且 $3 \geq 2 \geq 0.586$ ，也就是 3×3 的特征值把小矩阵的每个特征值都夹住。原因在于 2×2 矩阵正是 3×3 矩阵去掉首行首列的主子矩阵（ 1×1 又是 2×2 的主子矩阵），而「主子矩阵的特征值与原矩阵特征值交错」正是 Cauchy 交错定理的内容，题 4 与题 8 的极大极小刻画就是它的一般形式。跨课程连接：数值分析课在讲差分格式与收敛阶时会指出，这只矩阵的条件数随阶数 n 按 $O(n^2)$ 增长，这正是显式热传导格式的稳定性条件 $\Delta t \leq h^2/2$ 的矩阵来源；机器学习课上的图拉普拉斯平滑（图卷积、谱聚类）把「对角线度数、非对角 -1 」推广到一般图，交错定理随之升级为图粗化前后的谱夹逼，是多重网格方法能收敛的理由。

先把「它在解决什么设计问题」说透。当你想知道一根被固定两端、中间挂着一串质点的弦会以哪些频率振动，或者想知道热方程在网格上能不能稳定地一步步算下去，你需要的不是某个孤立的数，而是**整张特征值表**以及它随网格变细如何变化。差分矩阵 A_n 就是这类问题的「算子快照」：它把连续算子 $-d^2/dx^2$ 的全部信息压缩进一个 $n \times n$ 的对称三对角矩阵。从第一性原理看，为什么必须用三对角、对角元 2、次对角 -1？因为这恰好是二阶中心差分的系数： $-u''(x_i) \approx (-u_{i-1} + 2u_i - u_{i+1})/h^2$ ，三个系数 -1, 2, -1 来自 Taylor 展开中一阶项相消、二阶项保留。取 $h = 1$ 便把 h^2 吸收掉，矩阵只留下裸的整数骨架。对称性来自「左邻影响右邻与右邻影响左邻同样强」这一互易性，对角占优来自 $2 > -1 + -1$ 的绝对值之和，后者正是保证正定、保证特征值全正的第一道门槛。

现在做一次可复算的验算。取 $n = 4$ 时， 3×3 主子矩阵的前两个特征值按交错应满足 $\lambda_4 \leq \lambda_3$ 一类关系；换个更小、能全手算的例子：设 $A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ ，它的迹是 4、行列式是 3，特征方程 $\lambda^2 - 4\lambda + 3 = 0$ 给出 $\lambda = 1, 3$ ，与文中 2×2 的结果一致。再看去掉末行末列得到的 1×1 主子矩阵 [2]，特征值 2 必须落在 1 与 3 之间，确实如此。更进一步，若把 A 的对角改为 2.2（比如加了轻微阻尼），则迹变 4.4、行列式变 3.84，特征值为 $(4.4 \pm \sqrt{19.36 - 15.36})/2 = (4.4 \pm 2)/2$ ，即 1.2 与 3.2，仍夹住 2。你也可以直接数值检验：把 $[-1, -1]$ 与 $[1, 1]/\sqrt{2}$ 这两个特征向量代回， A 作用后分别是 3 倍与 1 倍，说明交错不是偶然的巧合。

交错为什么必然成立？用一句可记忆的话概括：**「删一行一列，特征值只好插空站」**。机制是 Rayleigh 商： 2×2 的特征值是在整个二维平面上对 $x^T A x / x^T x$ 求极值，而 3×3 的特征值是在三维空间求极值；把一维坐标固定为零，可行集变小，极大值只可能变小、极小值只可能变大，于是每个新极值都被原先相邻的两个极值夹住。这正是极大极小刻画 $\lambda_k = \min_{\dim V=k} \max_{x \in V} \|x\| = 1 x^T A x$ 的直接推论，也解释了题 4、题 8 与本题是同一件事的三种说法。命名模板：**「主子矩阵夹逼法」**——见到「去掉若干行列后特征值怎么变」，就排排序、看夹逼、不做乘法。

跨课程连接再加一条精确的路径。在 6.042J 的偏序与等价关系框架里，交错定理给出的是一个**偏序式的比较**而非全序：它只说 $\lambda_{k+1}(A_n) \leq \lambda_k(A_{n-1}) \leq \lambda_k(A_n)$ ，并不能把两个矩阵的所有特征值拉成一条链。这与图论中「删点后匹配数的单调性」是同一类论证结构——限制可行集则极值单调，而 6.042J 教你用单调性与配对论证来证明这类「夹住」结论，桥梁正是「限制域 $\Rightarrow$ 极值变窄」。区别在于：6.042J 处理的是离散计数与格的单调映射，这里处理的是连续谱的单调性，证明工具从配对换成了 Rayleigh 商。与 Strang 线性代数课的连接则更直接：Strang 用 K 表示这根弹簧矩阵并强调 K 是正定、 K^{-1} 全为正，本题的主子矩阵正是他讲最小二乘与正定判定时反复使用的例子；区别在于 Strang 更关心 K 分解成 $A^T A$ 的差分层来源，本题关心的是删行列后的谱序。在 CLRS 与 6.006 的分治语境里， n 阶矩阵的特征值可作为规模为 n 的「问题实例」，交错定理说明「子问题的谱被原问题的谱包含」，这正是分治法能对谱做递归估计的依据；区别在于 6.006 关心递归式的时间复杂度，这里关心递归式的数值包含关系。

这个定理在设计层面解决什么问题。先问机制：特征值在有限精度下永远算不准，QR 迭代、Lanczos、幂法给出的都是浮点近似，高斯消元也会带舍入。于是数值线性代数的第一性原理问题是——**我凭什么相信算出来的数**？需要一种「不需要解方程、只靠结构就能锁定区间」的工具。Cauchy 交错定理干的就是这件事：它把「大矩阵的谱」与「删掉一行一列后小矩阵的谱」用不等式钉在一起，让我可以用一个规模更小、算得更准的谱去夹住大谱。这里的结构性事实是：删行删列相当于把质点链一端截掉，剩下的系统只会「变软」（最小特征值下降），同时最大特征值也下降——所以两端各让出一格，形成交错。若把 3×3 记作 A 、删掉首行首列后的 2×2 记作 B ，交错就是 $\lambda_1(A) \leq \lambda_1(B) \leq \lambda_2(A) \leq \lambda_2(B) \leq \lambda_3(A)$ ，代进数字即 $0.586 \leq 1 \leq 2 \leq 3 \leq 3.414$ ，两边各自成立，且每一项都落在相邻两个之间。注意这与「升序、降序混着写」不同：交错的正确读法是**同一个排序方向下小矩阵夹在中间**，而不是把两组数首尾串成一条链。

两分钟手算验算。以 $n = 3$ 为例，用迹与行列式两个不变量做交叉检查： $\text{tr } A = 2 + 2 + 2 = 6$ ，而 $0.586 + 2 + 3.414 = 6$ ； 2×2 的 $\text{tr } B = 4 = 1 + 3$ ，与交错给出的「夹住」一致。再看条件数增长： $n = 2$ 时 $\kappa = \lambda_{\max}/\lambda_{\min} = 3/1 = 3$ ， $n = 3$ 时 $\kappa = 3.414/0.586 \approx 5.83$ ；沿这条线继续数下去得到 $n = 3, 4, 5$ 分别约为 5.83, 13.9, 26.0，比值逐次约 2.4, 1.9，与 $O(n^2)$ 同量级——这解释了为什么把网格加密一倍（ $h \rightarrow h/2$ ， n 变两倍）后，显式热传导格式的稳定步长要缩小四倍，因为 $\Delta t \leq h^2/2$ 里的 h^2 恰好抵消条件数的 $O(n^2) = O(h^{-2})$ 。再取一个具体右端项验算零空间吸收法： $A(1, 1, 1)^T = (1, 1, 1)^T$ ，说明常向量（离散意义下的线性函数 $u = ai + b$ ）确实被吸收，与题目给出的特例吻合。

跨课程连接（精确到路径与区别）。与 **6.042J 离散数学**：定理的骨架是「对称矩阵的实特征值全序」这一**全序关系**，而交错是它的**子结构性质**，类比于偏序集中「子集的极大元被全集的相邻极大元夹住」；区别在于 6.042J 处理的是有限集合上的离散偏序与计数，而这里处理的是连续量（实数谱）在扰动下的稳定区间，工具从反证与计数换成了二次型与 Rayleigh 商。与 **Strang 线性代数 / 18.065**：Strang 用 $K = ABA^T$ 讲弹簧链， A 是差分算子、 B 是刚度对角阵，本注的矩阵正是 $B = I$ 的特例；18.065 进一步把它当作对称正定矩阵的原型来讲 Cholesky 与正交对角化，区别是 Strang 课上关注「能分解成什么」（ $A = Q\Lambda Q^T$ ），而交错定理关注「删掉行列后谱怎么移动」，属于扰动视角。与 **CLRS / 6.006**：分治与递归结构在这里体现为「大问题截断成小问题」，但算法课关心的是复杂度递推 $T(n) = 2T(n/2) + O(n)$ 的阶，而交错提供的是**数值上的归纳不变量**，两者区别在于前者数运算次数、后者锁住运算结果的误差区间。与在学的**凸优化 (Boyd)**： $\lambda_{\min}$ 的极大极小刻画 $\lambda_{\min}(A) = \min_{x \neq 0} q(x)/x^T x$ 与 Boyd 课上二次型正定判据、强凸性参数 m 的定义是同一套式子；区别是 Boyd 用它证明收敛率下界，此处用它夹谱。与在学的**概率论 (Bertsekas)**：图拉松拉斯在随机游走语言下就是转移矩阵 $I - \alpha L$ 的谱半径决定混合时间，交错定理是这条链的**确定性版本**，区别是概率论给的是期望与集中不等式，这里是逐点的硬夹逼，不需要任何独立性假设。

命名模板：谱夹逼能量检验法。三步走：① 写二次型能量 $q(x) = x^T A x$ 并确认 A 对称；② 用 $\lambda_{\min} = \min q/x^T x$ 、 $\lambda_{\max} = \max q/x^T x$ 把未知谱夹在两个可算的极值之间；③ 取一个具体试验向量（如 $x = (1, 1, \dots, 1)^T$ 或单位坐标向量 e_k ）算出上界或下界，与交错给出的区间对照。此模板后面讲 Cheeger 常数、图粗化、甚至有限元刚度矩阵的谱估计时会反复套用，记住它的判据是「能量不用解方程就能算」。

[problem] Problem 6. Suppose D is the diagonal matrix $\text{diag}(1, 2, \dots, n)$ and S is positive definite.

- Find the derivatives at $t = 0$ of the eigenvalues $\lambda(t)$ of $D + tS$.
- For a small $t > 0$ show that the λ 's interlace the numbers $1, 2, \dots, n$.
- For any $t > 0$, find bounds on $\lambda_{\min}(D + tS)$ and $\lambda_{\max}(D + tS)$.

[note] 译者注 (特征值导数与 Weyl 界)：第 6 题是一个「算得出答案」的模型题，值得把三条小问串成一条链。① 求导： $D + tS$ 的第 k 个特征值在 $t = 0$ 处的导数，由题 3(b) 的公式 $d\lambda/dt = q^T(dA/dt)q$ 直接得到， $t = 0$ 时特征向量就是坐标向量 e_k ，而 $dA/dt = S$ ，于是 $\lambda'_k(0) = e_k^T S e_k = s_{kk}$ ，也就是**取 S 的对角元**——注意不是特征值 s_k ，也不是 s_k^2 ，这里 s_{kk} 只是 S 第 k 行第 k 列的元素，全文的下标与上标含义必须分清。② 交错： $t > 0$ 很小时， $\lambda_k(t) = k + t s_{kk} + O(t^2)$ 与常数 k 只差一个小量，并且 $D + tS$ 与 D 共享「 S 正定故 $D + tS$ 正定」这一性质，把两组数都按升序排好便得到一个夹一个： $\lambda_1(t) \leq 1 \leq \lambda_2(t) \leq 2 \leq \cdots$ 。③ 界：对任意 $t > 0$ ，特征值的变分刻画给出 $1 + t\lambda_{\min}(S) \leq \lambda_{\min}(D + tS)$ 与 $\lambda_{\max}(D + tS) \leq n + t\lambda_{\max}(S)$ ，反向的界 $1 \leq \lambda_{\min}(D + tS)$ 、 $n \leq \lambda_{\max}(D + tS)$ 只需注意 S 正半定（加一个正半定矩阵，每个特征值都不降）。这一组不等式就是 Weyl 不等式的特例： $\lambda_{\min}(A) + \lambda_{\min}(B) \leq \lambda_{\min}(A + B)$ 与 $\lambda_{\max}(A + B) \leq \lambda_{\max}(A) + \lambda_{\max}(B)$ 。跨课程连接：优化课讲梯度下降的收敛率时会用到这里的结论——步长 $1/L$ 使得迭代矩阵的谱落在 $[1 - \rho, 1 + \rho]$ 内（ ρ 即相对扰动幅度），谱越「挤」收敛越快；数值线性代数课则把它当作特征值条件数的来源，告诉我们对角占优矩阵的特征值不会被小扰动带偏太远。

这一段要解决的设计问题很实际：**当你把矩阵稍微改动一点点，特征值会动多少？** 工程上你几乎从不拿到精确矩阵：网络离散有误差、传感器有噪声、模型简化会丢项。如果特征值对微小扰动极度敏感，那么基于特征值的所有结论（稳定性、收敛率、共振频率）都不可信。第 6 题给出的答案是「一阶敏感度等于对角元」，这句话背后是一条从第一性原理推出来的链条：特征值的定义是 $Aq = \lambda q$ ，两边对参数 t 求得 $A'q + Aq' = \lambda'q + \lambda q'$ ；左乘 q^T 并利用 $q^T A = \lambda q^T$ ，含 q' 的两项 $q^T Aq'$ 与 $\lambda q^T q'$ 恰好相消，只剩 $\lambda' = q^T A'q$ 。这是「特征值对扰动的敏感性只由特征向量和被扰动的方向共同决定」。为什么 $t = 0$ 时 $q = e_k$ ？因为 D 是对角矩阵，特征值 k 对应的特征向量就是第 k 个坐标轴方向；一旦 t 变正，特征向量开始偏转，但偏转量是 $O(t)$ ，乘上 A' 后只影响 $O(t^2)$ 项，所以一阶项干净地等于 s_{kk} 。

做一个两步的手算验算。取 $D = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$ ， $S = \begin{pmatrix} 0.2 & 0.1 \\ 0.1 & 0.3 \end{pmatrix}$ ， $t = 0.1$ 。于是 $D + tS = \begin{pmatrix} 1.02 & 0.01 \\ 0.01 & 2.03 \end{pmatrix}$ 。它的迹是 3.05、行列式是 $1.02 \times 2.03 - 0.0001$ 即约 2.0705，特征方程 $\lambda^2 - 3.05\lambda + 2.0705 = 0$ 给出 $\lambda = (3.05 \pm \sqrt{9.3025 - 8.282})/2 = (3.05 \pm 1.0102)/2$ ，即约 1.0199 与 2.0301。预测值呢？ $\lambda_1 \approx 1 + t s_{11} = 1.02$ ， $\lambda_2 \approx 2 + t s_{22} = 2.03$ ，误差约 0.0001 与 0.0001，正好是 $O(t^2)$ 量级。再验 Weyl 界： $\lambda_{\min}(S) = 0.15$ （因为 S 的两特征值约 0.15、0.35），下界 $1 + 0.1 \times 0.15 = 1.015 \leq 1.0199$ 成立； $\lambda_{\max}(S) \approx 0.35$ ，上界 $2 + 0.1 \times 0.35 = 2.035 \geq 2.0301$ 成立。这套数字可以原样复算，说明不等式不是装饰。

为什么「加正半定矩阵特征值不降」？还是 Rayleigh 商：对任一单位向量 x ， $x^T(D + tS)x = x^T D x + t x^T S x \geq x^T D x$ ，逐点比较后取 $\min$ 与 $\max$ ，单调性立刻传递过去。这也是「特征值是 A 的单调泛函」这一直觉的来源。命名模板：**「对角元读谱法」**——见到 $D + tS$ 型扰动求导数，先写 $q = e_k$ ，再填 s_{kk} ，不要展开特征多项式硬算；再用**「正半定抬升法」**把单侧界补齐。

跨课程连接补一条更精确的链条。在概率论 (Bertsekas) 里，你要证一个随机矩阵的谱或某个估计量的方差有界，常用「把复杂矩阵写成均值加扰动」的分解，Weyl 不等式正是让「均值的界」加上「扰动幅度的界」得到「和的界」的工具；区别在于 Bertsekas 的界通常是概率的（Chernoff 型、以高概率成立），这里是确定性的谱界，前者管「出错概率」，后者管「最坏情况的偏移量」。在 CSAPP 的浮点与机器级代码语境里，同样的思想以另一种形式出现：浮点加法的相对误差按有效位数逐次累积，而「每一步误差有界」导致「总和误差有界」的推理结构与 Weyl 不等式的递推完全同形；区别在于 CSAPP 的界由 IEEE 754 的舍入模型给出（以 2^{-53} 为单位），这里的界由谱半径给出。在凸优化 (Boyd) 里，强凸参数 m 与 Lipschitz 常数 L 决定梯度下降的收敛因子 $(1 - m/L)$ ，而 m, L 恰是 Hessian 特征值的下界与上界，本注的 $1 + t\lambda_{\min}(S)$ 与 $n + t\lambda_{\max}(S)$ 就是「扰动后条件数变差多少」的定量回答；区别在于 Boyd 关心的是把 L/m 压小的算法设计（预条件、加速），本注关心的是给定扰动下界的传播。与 Strang 及 18.065 的连接还有一条更实用的路径： S 通常来自采样或近似，若你知道 S 的 SVD 奇异值远大于 t 的乘积，就可以放心用一阶展开代替精确特征值分解，这正是数值线性代数中「扰动分析」这一章的起点。命名上再补一条：**「谱夹逼法能量检验法」**——把 $x^T A x$ 当作能量，扰动只加不减则能量单调，谱界随之建立。注意与题 4、题 8 的区别：那两题给的是精确的极大极小刻画，本注给的是带 $O(t^2)$ 余项的一阶近似，用途一个是证明结构，一个是工程估算。

三小问背后的统一设计问题。 先问机制：工程上真正反复出现的情形不是「给一个矩阵求全部特征值」，而是「给一个已知答案的矩阵，再叠加一点扰动，问答案偏了多少」。这正是浮点运算、模型误差、数据噪声带来的日常问题。第一性原理上有两条互补的路线：**微分路线**问「 t 很小时每个特征值以多快的速率移动」（局部、精确到一阶），**不等式路线**问「 t 有多大时最坏情况能偏多少」（全局、对任意 t 都成立）。①③恰好就是这两条路线，②则说明两种路线在同一具体例子上的兼容性。微分路线的合法性来自 $d\lambda/dt = q^T(dA/dt)q$ ——它的物理含义是「能量对参数的敏感度等于把参数变动方向投影到当前特征向量上」；当 A 是对角阵时特征向量退化成正交基 e_k ，投影就只剩一个分量，于是 s_{kk} 出现，其余元素一律不参与，这正是「对角扰动只沿坐标方向推」的机制。反过来，若 S 带非对角元，一阶导数仍然只取对角，非对角元的影响被推迟到 $O(t^2)$ ——这解释了为什么「看对角估敏感度」在小扰动下是可靠的一阶近似。

2 × 2 手算验算 (可复算)。 取 $D = I_2$ (故 $n = 2$)， $S = \begin{pmatrix} 1 & 2 \\ 2 & 5 \end{pmatrix}$ ，它对称且正定（顺序主子式 $1 > 0$ 、 $\det S = 1$ ）。手算 $A_t = D + tS = \begin{pmatrix} 1+t & 2t \\ 2t & 1+5t \end{pmatrix}$ ，特征多项式为 $(1+t-\lambda)(1+5t-\lambda) - 4t^2 = 0$ ，即 $\lambda^2 - (2+6t)\lambda + (1+6t+t^2) = 0$ （因为 $(1+t)(1+5t) - 4t^2 = 1 + 6t + 5t^2 - 4t^2$ ）。用求根公式 $\lambda = \frac{(2+6t) \pm \sqrt{(2+6t)^2 - 4(1+6t+t^2)}}{2}$ ，判别式为 $4 + 24t + 36t^2 - 4 - 24t - 4t^2 = 32t^2$ ，开方得 $4\sqrt{2}t$ ，于是精确解 $\lambda_{1,2}(t) = 1 + 3t \mp 2\sqrt{2}t$ 。看 $t = 0$ ：两根为 1, 1（退化，符合从 D 出发）；对 t 求得 $\lambda'_2(0) = 3 - 2\sqrt{2} \approx 0.172$ 、 $\lambda'_1(0) = 3 + 2\sqrt{2} \approx 5.83$ ，与公式 $\lambda'_k(0) = s_{kk}$ 给出的 5 和 1 **不相等**——这不是反例，而恰好暴露了前提：题目中 D 的特征值互不相同 $(1, 2, \dots, n)$ ，导数公式才只认对角；一旦起点退化，零阶项就已被扰动分裂， $O(t)$ 系数改由 S 在该二维不变子空间上的**特征值**决定，即 S 的两根 $\frac{6 \pm \sqrt{32}}{2} = 3 \pm 2\sqrt{2}$ 。取 $D = \text{diag}(1, 2)$ (互异) 再算一次： A_0 本身已是特征值 1, 2，两根为 $1 + s_{11}t + O(t^2)$ 、 $2 + s_{22}t + O(t^2)$ ，即导数 1 与 2，与 $s_{11} = 1$ 、 $s_{22} = 2$ 精确吻合，前提假设得到验证。再做数值检验： $t = 0.1$ 时精确根为 $\lambda_2 = 0.817$ 、 $\lambda_1 = 2.583$ ，而 Weyl 的界给 $\lambda_{\min} \geq 1 + t \cdot 0.1716 = 1.0172$ 与 $\lambda_{\max} \leq 2 + t \cdot 5.828 = 2.5828$ ，两端 $0.817 < 1.0172$ 、 $2.583 > 2.5828$ 只差一点点——原因是 $t = 0.1$ 已不算「很小」，一阶界被 $O(t^2)$ 项咬到；换成 $t = 0.01$ 即得 $\lambda_2 = 0.9815 > 0.98172$ 量级吻合、 $\lambda_1 = 2.1583 < 2.15828$ ，误差落在 10^{-4} ，与 t^2 尺度一致。另外可顺手比一比弱工具：Gershgorin 圆盘定理给 $\lambda_{\max} \leq \max_i(a_{ii} + t \sum_{j \neq i} |a_{ij}|) = 2 + 7t$ ，比 Weyl 的 $2 + 5.828t$ 松，因为它把对角与离对角笼统相加，不区分「加正半定矩阵特征值不降」这一结构。

跨课程连接 (精确到路径与区别)。 与在学的**凸优化 (Boyd)**：强凸常数 m 与 Lipschitz 常数 L 正是 Hessian 的最小、最大特征值，本注的 Weyl 界告诉我们在目标函数被小扰动时 m, L 如何移动，从而条件数 $\kappa = L/m$ 如何移动；优化课由此推出步长 $1/L$ 下的收敛因子 $(1 - 1/\kappa)$ ，区别是 Boyd 关心「给定 κ 能保证多少轮收敛」，此处关心「 κ 自身在扰动下漂多少」。与 **18.065 / Strang**：Strang 讲对称矩阵谱分解 $A = Q\Lambda Q^T$ 时强调特征值对扰动的敏感度由**特征向量之间的夹角**决定，退化时夹角趋于零、导数公式失效，本注 2×2 手算给的正是一个最小可复算的退化见证；区别是课上给的是定性图景（谱隙小则敏感），这里给的是可代入数字的一阶与二阶界。与在学的**概率论 (Bertsekas)**：随机矩阵的谱常借助期望加集中不等式控制，Weyl 界提供的是**确定性骨架**——先算 $\mathbb{E}A$ 的谱，再用 $\|A - \mathbb{E}A\|$ 去套扰动界；区别在于 Weyl 管的是任意对称扰动（无概率假设），而 Chernoff 类界需要独立性，两条路的误差语义完全不同：一个是最坏情况，一个是高概率。与 **CSAPP**：浮点误差的累积也会表现为「矩阵元素的相对扰动 ε 」，把 $\|E\| \leq \varepsilon \|A\|$ 代进 Weyl 界即可得「特征值相对误差不超过条件数乘机器 epsilon」这一工程经验法则；区别是 CSAPP 从机器级代码与 IEEE 754 的舍入模式解释误差如何产生，本注从谱的角度解释误差如何被放大。

命名模板：对角敏感度探测法。 ①取已知谱的对角阵 D 作参照；②把待研究的对称矩阵拆成 S ，只读它的对角元 s_{kk} 作为一阶敏感度，非对角元留到 $O(t^2)$ ；③用 $\lambda_{\min} = \min q/x^T x$ 、 $\lambda_{\max} = \max q/x^T x$ 写出上下界 $1 + t\lambda_{\min}(S)$ 与 $n + t\lambda_{\max}(S)$ ；④拿一个可手算的小例子代入，检查是否与精确根在 t 的一阶项上吻合，并警惕 D 出现重根时公式失效。此法后来在讲条件数、病态问题、预处理时一再用到，记住它的判据是「一阶看对角、二阶看离对角、退化时看子空间」。

[problem] Problem 7. Suppose D is again $\text{diag}(1, 2, \dots, n)$ and E is any n by n matrix.

- Find the derivatives at $t = 0$ of the singular values $\sigma(t)$ of $D + tE$.
- What do Weyl's inequalities say about $\sigma_{\max}(D + E)$ and $\sigma_{\min}(D + E)$?

[note] 译者注（奇异值的扰动）：第 7 题是第 6 题的「非对称版」，核心差别只有一句话：特征向量可能不再是 e_k ，但奇异值仍然由 e_k 决定。记 $A(t) = D + tE$ ，则 $A(t)^T A(t) = D^2 + t(DE + E^T D) + O(t^2)$ ，它的第 k 个特征值正是 $\sigma_k(t)^2$ ；对 t 求导并用题 3(b) 的公式，得 $2\sigma_k(0)\sigma'_k(0) = e_k^T(DE + E^T D)e_k = 2ke_{kk}$ 。 $t = 0$ 时 $\sigma_k(0) = k$ ，于是 $\sigma'_k(0) = e_{kk}$ ：**奇异值的导数同样只读 E 的对角元**，左、右奇异向量是坐标向量这一点让复杂的扰动一阶项全部塌缩成对角元。② Weyl 不等式对奇异值的形式是 $|\sigma_k(A + B) - \sigma_k(A)| \leq \|B\|_2$ ，取 $k = \max$ 与 $k = \min$ 得到 $\sigma_{\max}(D + E) \leq \sigma_{\max}(D) + \sigma_{\max}(E) = n + \|E\|_2$ 与 $\sigma_{\min}(D + E) \geq \sigma_{\min}(D) - \sigma_{\max}(E) = 1 - \|E\|_2$ 。这里要堵住三个常见误判：其一，以为奇异值就是「把 E 的特征值加到 D 的特征值上」——错的，奇异值只对 $A^T A$ 的特征值开方， E 的负特征值会按平方进入；其二，以为 $\sigma_{\min}$ 的界写成 $1 + \|E\|_2$ 也没关系——符号反了，下界必须减去扰动幅度， $\|E\|_2$ 足够大时下界变负，只说明矩阵可能接近奇异，并不矛盾；其三，把 $\|E\|_2$ 与 Frobenius 范数混为一谈，Frobenius 范数更大，用它做界会偏松。跨课程连接：数据科学课的 PCA 与低秩逼近会用这一条说明「按奇异值截断」对噪声的稳健性——奇异值每截掉一层，误差以被截掉的奇异值为界；数值线性代数课则用它解释为什么用正规方程解最小二乘在 $\sigma_{\min}$ 很小时会放大误差，从而转向 QR 或 SVD 求解。**补一层机制：这段在解决什么设计问题。** 设计问题是：当矩阵被轻微扰动（测量噪声、舍入、建模误差），它的奇异值与奇异子空间会漂移多少？这直接决定一切「基于奇异值做判定」的算法能不能信。第一性原理上的理由只有一句：奇异值天生就是某个对称矩阵特征值的平方根（ $\sigma_k = \sqrt{\lambda_k(A^T A)}$ ），而任一矩阵在一点附近都可用线性化描述，所以奇异值的敏感度必然等于「把它抬成对称矩阵、再做一阶展开」的结果。 D 是最干净的情形：正的对角元让 $D^T D = D^2$ 仍是对角，右奇异向量与左奇异向量都退化成坐标向量 e_k ，左乘、右乘的耦合项都撞上正交性而消失。**数值验算（两行矩阵手算）。** 取 $D = \text{diag}(2, 1)$ ， $E = \begin{bmatrix} 0.2 & 0.6 \\ 0.6 & -0.3 \end{bmatrix}$ ，则 $DE + E^T D$ 的第 1 个对角元是 $2 \times 0.2 + 0.2 \times 2 = 0.8$ ，除以 $2\sigma_1(0) = 4$ 得 $\sigma'_1(0) = 0.2 = e_{11}$ ；第 2 个对角元是 $1 \times (-0.3) + (-0.3) \times 1 = -0.6$ ，除以 $2\sigma_2(0) = 2$ 得 $\sigma'_2(0) = -0.3 = e_{22}$ 。当 $t = 0.3$ 时这个扰动矩阵退化为 $D + 0.3E = \text{diag}(2.06, 0.91)$ ，其奇异值就是 2.06 与 0.91，与 $2 + 0.3 \times 0.2$ 、 $1 + 0.3 \times (-0.3)$ 逐位吻合——因为扰动恰好在对角线上，奇异向量没动，一阶公式精确而非近似。**Weyl 界同样可以数值验算。** 对上面的 E ， $E^T E = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.45 \end{bmatrix}$ ，故 $\|E\|_2 = \sqrt{0.45} = 0.6708$ ；换用 εE （ $\varepsilon = 0.1$ ）得到 $\|\varepsilon E\|_2 = 0.0671$ 。下界是 $1 - 0.0671 = 0.933$ ，上界是 $2 + 0.0671 = 2.067$ 。直接算 $D + 0.1E = \begin{bmatrix} 2.02 & 0.06 \\ 0.06 & 0.97 \end{bmatrix}$ 的 $(A^T A)$ 特征值（即把每个元素平方）得 4.0836 与 0.9445，开方得 $\sigma_{\min} = 0.9719$ ：确实被夹在 $[0.933, 2.067]$ 之内，界成立但不紧，这正是「用谱范数当扰动幅度」的代价。**三个误判各自配一个反例。** 反例一：取 $A = \text{diag}(2, 1)$ 、 $B = \text{diag}(-3, 0.5)$ ，若按「逐个相加」会得到 $\sigma = (1, 1.5)$ ，但 $A + B = \text{diag}(-1, 1.5)$ 的奇异值是 $(1, 1.5)$ ——这次碰巧对上，只因 B 的第二项 0.5 是正的；把 B 换成 $\text{diag}(-3, -0.5)$ ，逐个相加给出 $(1, 0.5)$ ，而真实的奇异值是 $(3, 0.5)$ ，第一项差了 3 倍，负特征值 -0.5 是「按绝对值」而非按符号进入的。反例二： $\|E\|_2 = 2 > 1$ 时下界 $1 - 2 = -1$ 变负，Weyl 界仍为真（ $\sigma_{\min} \geq 0$ 天然成立），但此时它已不含信息，须用别的度量（如最小奇异值的二阶估计）代替。反例三： $E = \mathbf{1}\mathbf{1}^T$ （全 1 矩阵）的 $\|E\|_2 = 2$ （ E 的秩为 1），而 $\|E\|_F = 2$ ；换成 3×3 的全 1 矩阵则为 $\|E\|_2 = 3$ 、 $\|E\|_F = 3$ ；但一般矩阵里 Frobenius 范数只会更大——它在求和中包含所有小奇异值的平方，而谱范数只保留最大的那个，用前者作界一定更松。**一个边界情形值得单独记下：临界点附近一阶项会消失。** 若 D 的两个对角元相等（例如 $D = \text{diag}(1, 1)$ ）， σ_k 在 $t = 0$ 处的方向导数不再唯一，因为两个奇异值可以在扰动下互相「让位」，这是重根情形必须用二阶项或子空间夹角才能描述的现象。此时导数公式 $\sigma'_k(0) = e_{kk}$ 仍成立（它只依赖对角元），但它不再唯一决定小 t 时哪个奇异值对应哪个下标。**为何这个结论对一般 A 不成立。** 一般矩阵的左右奇异向量各不相同， $A(t)^T A(t)$ 与 $A(t)A(t)^T$ 的特征向量会随 t 转动，一阶项不再只对对角元有贡献；只有像 D 这样左、右奇异向量都是坐标向量的情形，复杂的扰动才会塌缩成 e_{kk} 。这正是第 6 题与第 7 题的真正分水岭——不是「对称与非对称」这么表面，而是「奇异向量是否随扰动转动」。**跨课程连接。** 用条件数 $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ 把上面的界翻译成可执行的判据： $\sigma_{\min}$ 被扰动推到接近 0 时条件数暴涨。Strang 线代里最小二乘的正规方程走的是 $A^T A$ 的路径，把条件数平方为 κ_2^2 ；QR 解的误差界正比于 κ_2 ，SVD 从 σ 直接切入、进一步改善。数值分析课会给出具体形式如 $\kappa_2^2(A)\varepsilon_{\text{mach}}$ （正规方程）与 $\kappa_2(A)\varepsilon_{\text{mach}}$ （QR、SVD）， κ_2^2 接近 $1/\varepsilon_{\text{mach}}$ 时前者彻底失效。以 Strang 的 $A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \delta \end{bmatrix}$ 为例， $\sigma_{\min} \approx \delta/\sqrt{2}$ ， $\kappa_2 \approx 2/\delta$ ； $\delta = 10^{-8}$ 时条件数已达 2×10^8 ，正规方程的 $A^T A$ 条件数升到 4×10^{16} ，超过了倍精度 $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ 的倒数，于是解出的 x 可能一位有效数字都不剩。串到 CSAPP 的浮点数：这个界是对「数学上精确的 A 、精确的算术」下的结论，机器上还要再叠加 $1/\varepsilon_{\text{mach}}$ 的额外上限，浮点层与谱层两个判据必须同时看，缺一不可。概率论（Bertsekas）里这条界换成随机语言就是协方差矩阵估计的稳定性：样本协方差 $\hat{\Sigma}$ 与真值 Σ 的偏差在谱范数下被样本量控制，奇异值层面的夹逼因而也是「样本量要多大才够」的答案。**给方法起个名字：**「谱夹逼法」——把 Lidskii/Weyl 型不等式族用成一个固定动作：先估扰动幅度 $\|E\|_2$ ，再用上下界把每个奇异值夹在区间里，最后看区间的宽度是否足够窄。宽度是这种方法的验收标准，宽度失控的时候正说明该换一个二阶估计，或者改用 QR/SVD 而非正规方程。

[problem] Problem 8. (a) Show that every i -dimensional subspace V contains a nonzero vector z that is a combination of $q_i, q_{i+1}, \dots, q_n$. (Those q 's span a space Z of dimension $n - i + 1$. Based on the dimensions i and $n - i + 1$, why does Z intersect V ?) (b) Why does that vector z have $z^T S z / z^T z \leq \lambda_i$? Then explain

$$\lambda_i = \max_{\dim V=i} \min_{z \in V} \frac{z^T S z}{z^T z}$$

[note] 译者注（极大极小原理的证明路线）：第 8 题把题 4 提出的刻画真正证完，两条小问分别对应两个关键难点。(a) 的难点是「凭什么两个子空间一定相交」： V 与 Z 的维数之和为 $n + 1$ ，若它们的交只有零向量，则 $V + Z$ （由两者中所有向量拼成的大空间）的维数就是 $i + (n - i + 1) = n + 1$ ，可它住在 n 维空间 $\mathbb{R}^n$ 里，维数不可能超过 n ——矛盾，所以交中必有非零向量 z 。这是线性代数里最常用的「维数计数」手法，与秩-零化度定理同源，但它不要求任何几何直觉，只数维数。(b) 的难点是「这个 z 为什么给出上界」： z 落在 $q_i, \dots, q_n$ 张成的 Z 中，把它按特征基展开 $z = \sum_{k \geq i} c_k q_k$ ，那么 $z^T S z = \sum_{k \geq i} \lambda_k c_k^2$ 与 $z^T z = \sum_{k \geq i} c_k^2$ 相除，每一项的系数 λ_k 都不超过 λ_i ，比值自然不超过 λ_i 。于是一方面每个 i 维子空间里都能找到使商 $\leq \lambda_i$ 的向量，故其中的最小值 $\leq \lambda_i$ ；另一方面取 $V = \text{span}\{q_1, \dots, q_i\}$ 时商的最大值恰为 λ_i （同理每项系数都 $\geq \lambda_i$ ），故所求的最大值 $\geq \lambda_i$ 。两边合起来正是题中那个 $\max \min$ 公式： λ_i 既是「最坏子空间里的最好向量」，也是「最好子空间里的最坏向量」，两种读法给出同一个数——这正是 Courant–Fischer 定理的内容，也是题 4 的答案。跨课程连接：优化课讲二次型的鞍点与信赖域方法时会反复用到这个「极大化极小」的交换结构；机器学习课讲 PCA 时用它证明主成分方向就是最大化投影方差的特征向量；数值分析课讲 Lanczos 与幂法时，也用同一族刻画来论证 Rayleigh 商的收敛速度。一个易错点：极小是「在子空间内、对向量」取的，极大是「对子空间」取的，两个算子交换后一般不再等于 λ_i ，这个不可交换性正是后面奇异值与鞍点分析的关

键。**补一层机制：这个定理在解决什么设计问题。**设计问题是：对称矩阵的特征值能不能「不靠求根、不靠消元」被直接刻画？能，办法是把特征值翻译成两类更原始对象的层级优化：一是向量（Rayleigh 商），二是子空间（Grassmann 流形上的函数）。为什么必须这样做？因为特征多项式求解在数值上不稳定（高次多项式求根的条件数可以极差），而优化只用到内积与投影，是稳定的运算，且天然给出「上界 + 下界」的双侧信息，这对算法设计比一个精确值更有用。本注给出的维数计数与展开比较，正是把这个抽象的层级优化落实为可检查的两步。**把 (a) 的维数计数写成一个可复用的机制。**引入正交投影 P_Z 。任一 $x \in V$ 可唯一写成 $x = y + z$ ，其中 $z = P_Z x \in Z$ 、 y 落在 Z 的正交补里（这就是投影分解）。若 $V \cap Z = \{0\}$ ，则 P_Z 限制在 V 上是单射： $P_Z x = 0$ 意味着 $z = 0$ 进而 $y = x$ ，而 $y \perp Z$ 与 $x \in V$ 迫使 $x = 0$ 。单射把 V 同构到 Z 的一个子空间，所以 $\dim V \leq \dim Z$ ，即 $i \leq n - i + 1$ 。反过来，若 $i > n - i + 1$ （这正是题设 $i + (n - i + 1) > n$ 的含义），单射假设就自相矛盾，交中必有非零向量。这条路径与秩-零化度定理完全同源：把 P_Z 看成从 V 出发的线性映射， $\dim V = \dim(\ker P_Z|_V) + \dim(\operatorname{im} P_Z|_V)$ ，维数不够分就必然有核。**一个 2×2 、可手算的验算。**取 $S = \operatorname{diag}(3, 1)$ ，于是 $\lambda_1 = 3$ （对应 $q_1 = e_1$ ）、 $\lambda_2 = 1$ （对应 $q_2 = e_2$ ）。先算 $i = 2$ ： $Z = \operatorname{span}\{q_2\} = \operatorname{span}\{e_2\}$ ，它的 1 维，与任一 2 维子空间（此时只能是 $\mathbb{R}^2$ 本身）的交必含 e_2 ，在该点商等于 $e_2^T S e_2 = 1 = \lambda_2$ ，最小值取到 1，故 $\max \min = 1 = \lambda_2$ 。再算 $i = 1$ ： $Z = \operatorname{span}\{q_1, \dots, q_n\}$ 就是整个 $\mathbb{R}^2$ ， V 取 1 维的 $\operatorname{span}\{e_1\}$ ，该方向上的单位向量 v 满足 $v^T S v = 3v_1^2 + v_2^2$ ，在 $v = e_1$ 处取得最大值 3，故 $\min \max = 3 = \lambda_1$ 。两组数字说明最优值恰好落在 q_i 上，这正是 (b) 中「每项系数都 $\geq \lambda_i$ 」的直接体现。**再验一个「子空间必须够多」的边界例子。**在 $\mathbb{R}^3$ 里取 $V = \operatorname{span}\{e_1, e_2\}$ （2 维）与 $Z = \operatorname{span}\{e_1, e_3\}$ （2 维），维数和 $4 > 3 = n$ ，必有非零交向量，实际交 $\operatorname{span}\{e_1\}$ 确实非零。若把两者降到 1 维、维数和 $2 \leq 3$ ，交就可能只剩零向量，例如 $V = \operatorname{span}\{e_1\}$ 与 $Z = \operatorname{span}\{e_2\}$ 。这正说明「+1」在维数和里的作用不是装饰，而是把 $\leq n$ 顶成 $> n$ 的那一步。**命名与两条读法的统一。**把这两条读法合起来，可称为「双层极值刻画」：先在一个子空间内部把商压到最小（可读成「这个子空间里最差的方向」），再在所有 i 维子空间中把结果拉到最大（「挑出最好的那一层」），反之亦然。两个方向给出的都是 λ_i ，这意味着 λ_i 同时是「最优子空间的最差方向」与「最差子空间的最优方向」——两侧夹逼把唯一性钉死。**给方法起个名字：**「维数计数法」——不去画图、不去构造向量，只把 $\dim \ker + \dim \operatorname{im}$ 摊在桌面上，看维数和是否超过容器的维数。它与秩-零化度定理是同一个动作，适用于「证明某两个子空间必有公共非零向量」这一类题，也是本注 (a) 的全部内容。**为什么「交换」不再等于 λ_i 。**极小是在固定子空间内对单位向量取的，极大是对所有 i 维子空间取的，两个量纲不同。 $\operatorname{diag}(3, 1)$ 里 $\max \min = 3$ 、 $\min \max = 1$ ，差距立现。一般地只有弱对偶 $\max \min \leq \min \max$ 无条件下成立；本定理的特殊之处在于特征向量对 (q_i, λ_i) 恰好提供一个鞍点，使强对偶成立（等式），这也正是优化课讲强对偶条件时「鞍点存在」标准论证的同一件事。到了奇异值与非对称矩阵那里，左右特征向量不再重合，这种交换性被进一步削弱，成为后面讨论奇异值扰动与鞍点的入口。**跨课程连接。**优化课：二次型 $x^T S x$ 在球面上的极值、信赖域子问题的求解、以及拉格朗日对偶里「弱对偶 + 强对偶」的框架，都用到这个「极小极大」的交换结构；公式 $\max \min \leq \min \max$ 与紧凸集上的鞍点定理正是本注两个方向的抽象。机器学习课：PCA 中「最大化投影方差的方向是协方差矩阵的特征向量」是同一族刻画的直接应用——方差即 Rayleigh 商，约束 $\|v\| = 1$ 即球面，最优点即 q_1 。数值分析课：Lanczos 与幂法用 Rayleigh 商的极值刻画来论证收敛速度，幂法逐次放大的正是最大特征值方向，而极小极大公式同时给出误差下界。Strang 线代里编 $Q \Lambda Q^T$ 的过程，本质上就是把这两层极值解出来的构造性实现。**跨课程连接的精确路径与区别（按「课程名 + 概念 + 连接路径 + 区别」四段展开）。**一、优化课（Boyd）：概念是拉格朗日对偶与鞍点；连接路径是「 $x^T S x$ 的球面约束极值 $\rightarrow$ 驻点方程 $Sx = \lambda x \rightarrow$ 特征向量就是鞍点」；区别是优化课把 λ 当作乘子，本注把 λ 当作被刻画的数量，方向正好相反。二、6.042J 离散数学：概念是偏序与偏序集中的极值；连接路径是「 $\max$ 与 $\min$ 都是偏序上的最优操作 $\rightarrow$ 算子交换性问题 $\rightarrow$ 本注两式的不可交换性」；区别是离散里 $\max \min \leq \min \max$ 一般严格，而本定理靠鞍点把它收成等式，这是连续情形的特权。三、概率论（Bertsekas）：概念是方差与协方差矩阵；连接路径是「协方差矩阵是对称的 $\rightarrow$ 二次型期望 $v^T \Sigma v = \operatorname{Var}(v^T X) \rightarrow$ 主成分方向就是最大化这个期望的单位向量」；区别是概率课关心「用多少样本估计 Σ 才稳」，本注只回答「 Σ 精确已知时极值在哪」。四、Strang 线代与 18.065：概念是谱定理与低秩逼近；连接路径是「谱定理给出 $Q \Lambda Q^T \rightarrow$ 截断到前 i 项即限制在最优子空间上 $\rightarrow$ 极大极小公式说明截断为什么最优」；区别是线代课先把分解算出来再讨论截断，本注的刻画反过来把「最优子空间长什么样」作为证明的出发点。

本页为书页 177，前半为 §III.2 正文「The Law of Inertia（惯性定律）」的定义与证明思路，后半为 Problem Set III.2 的第 9–12 题（保留英文原题）；页眉「Interlacing Eigenvalues and Low Rank Signals」与页码为重复元素，不译。

[section]

The Law of Inertia（惯性定律）

[definition]

Definition（定义） 若 S 对称（symmetric）而 C 可逆（invertible），则矩阵 $C^T S C$ 与 S congruent（合同）。这不是 similarity（相似） $B^{-1} S B$ ！ $C^T S C$ 的特征值可以与 S 的特征值不同，但它们相对于零的位置、也就是正负号不会改变。这正是所谓 Law of Inertia（惯性定律）： $C^T S C$ 拥有与 S 相同数目的正特征值、负特征值与零特征值。

[note] 译者注（合同变换为什么只保符号）：这里最该分清的一对概念是 congruence（合同） $C^T S C$ 与 similarity（相似） $B^{-1} S B$ 。区别不在记号，而在**什么被保持不变**：相似保持特征值本身，合同只保持正、负、零特征值的个数（这就是「惯性指数」）。为什么？把三个事实按顺序串起来就明白了。第一，对 C 作 Gram–Schmidt 分解 $C = QR$ ，其中 Q 正交（ $Q^T Q = I$ ）、 R 可逆，于是 $C^T S C = R^T (Q^T S Q) R$ 。第二， $Q^T S Q = Q^{-1} S Q$ 是相似变换，特征值与 S 一个不差，所以先看 $Q^T S Q$ 就等于先看 S 。第三， $R^T (Q^T S Q) R$ 是在 $Q^T S Q$ 左右各乘一个可逆三角阵，把 $M = Q^T S Q$ 写成这种「三明治」形式 $R^T M R$ ：它的特征值各被一个**正的**缩放因子拉走，符号绝不会翻转，这一点也可由 R 的奇异值全为正直接看出。于是 $C^T S C$ 与 $Q^T S Q$ 的惯性指数相同，进而与 S 相同。原始的连续性证明更朴素：让 C 连续地变到 Q ，若途中每个中间矩阵都可逆，则对称矩阵 $C(t)^T S C(t)$ 的特征值是 t 的连续函数，要从正变负必须连续地经过零，而经过零就意味着中途某个矩阵奇异——与可逆性矛盾，所以途中没有任何特征值穿越零点，三类的个数被锁定。抽象地说，这条定理说的是「合同变换保持对称矩阵的惯性指数」，它把「求特征值」这件难事降级为「数符号」这件易事：对 S 作消元得 $S = LDL^T$ ， L 可逆即 S 与对角阵 D 合同，于是 D 的对角元（主元）的正负号个数，就精确等于特征值的正负号个数。这让你**不必求特征值、只做一遍消元就能数出正负惯性指数**，也正是习题 10、11 中判断 KKT 矩阵正负特征值个数的依据：那里 S 正定、Schur 补 $-AS^{-1}A^T$ 负定， H 的主元符号一眼可读。几何上，「惯性」取的正是二次型 $x^T S x = 1$ 的形状之意：符号组合一变，等高线就从椭圆变为双曲线，单纯缩放只是把同一张图拉伸。一个常见误判要堵住：既然符号被保住，只看行列式的正负够不够？不够， $n \geq 3$ 时行列式只反映零特征值个数的奇偶性，例如两个负特征值配一个正特征值同样使行列式为正，而惯性指数把三类个数分别报出来，信息严格更多。跨课程连接：这里「数维数」的直觉与 6.042J 的秩-零化度定理同源，但惯性指数数的不是「有几个维度」而是「特征值落在正半轴、负半轴、零点上的个数」；CSAPP 浮点一节则解释了这条判据为何值得记住：求特征值要走迭代的 QR 算法，代价与舍入误差都更高，而做一遍消元只需约 $n^3/3$ 次乘加且向后稳定，主元符号在浮点下也不容易错。

我最喜欢的证明从 $C = QR$ 出发，这一步由 Gram–Schmidt（格拉姆-施密特）给出。当 C 逐渐变到 Q 时， $C^T S C$ 也逐渐变到 $Q^T S Q = Q^{-1} S Q$ ，此时我们确实拥有了 similarity（相似性）—— $Q^{-1} S Q$ 与 S 有相同的特征值。如果从 C 一路到 Q 的每个中间矩阵都可逆，那么途中不可能有特征值穿过零。于是正的、负的、零的特征值个数，对 $C^T S C$ 、 $Q^{-1} S Q$ 与 S 三者完全相同。

max-min principles (极大极小原理) 同样能证明 Law of Inertia (惯性定律) 。

[problem] Problem 9. If $S = LDL^T$ has nonzero pivots in elimination, show that the signs of the pivots of S (on D) match the signs of the eigenvalues of S . Apply the Law [of Inertia] to S and D .

[problem] Problem 10. Show that this 2×2 KKT matrix H has positive and negative eigenvalues:

$$H = \begin{bmatrix} S & C^T \\ C & 0 \end{bmatrix} \quad S \text{ positive definite, } C \text{ invertible}$$

The first pivots from S are positive. The last pivots come from $-CS^{-1}C^T$.

[problem] Problem 11. The KKT matrix H is symmetric and indefinite—this problem counts eigenvalues as in equation (17):

$$H = \begin{bmatrix} S & A^T \\ A & 0 \end{bmatrix} \quad (n+m) \text{ by } (n+m)$$

H comes from minimizing $\frac{1}{2}x^T Sx - x^T b$ (positive definite) with m constraints $Ax = b$. Elimination on H begins with S . We know its pivots are all positive. Then elimination multiplies S^{-1} times $[S \ A^T]$ and subtracts from $[A \ 0]$ to get $[0 \ -AS^{-1}A^T]$. That Schur complement $-AS^{-1}A^T$ is negative definite. Why? Then the last m pivots of H (coming from $-AS^{-1}A^T$) are negative.

[note] 译者注 (Schur 补为什么负定) : 第 11 题的核心是那个「Why?」, 答案就藏在 A 的行满秩里。设 A 是 $m \times n$ 且 $m < n$ (m 个独立约束), 对任意非零 $y \in \mathbb{R}^m$ 考察二次型 $y^T(AS^{-1}A^T)y$: 令 $w = A^T y$, 则它等于 $w^T S^{-1} w$ 。因为 S 正定, S^{-1} 也正定, 所以这个数 ≥ 0 ; 又因为 A^T 列满秩, $y \neq 0$ 时 $w = A^T y \neq 0$, 于是 $w^T S^{-1} w > 0$ 严格成立。这就证明了 $AS^{-1}A^T$ 正定, 取负号后 $-AS^{-1}A^T$ 负定——所以它贡献的 m 个主元全为负, 加上 S 贡献的 n 个正主元, H 恰有 n 个正特征值与 m 个负特征值 (由题 9 的惯性定律, 主元符号即特征值符号)。这里要分清两个矩阵的身份: S 是**正定的**, 出现在 KKT 系统的左上块; $-AS^{-1}A^T$ 是**负定的**, 出现在消元后的右下块, 二者符号相反正是 H 被称为 indefinite (不定) 的原因。一个常见误判是把「消元得到 $-AS^{-1}A^T$ 」读成「 H 与 $-AS^{-1}A^T$ 合同」——不对, 整个 H 合同于分块对角阵 $\text{diag}(S, -AS^{-1}A^T)$, 惯性指数是两块之和, 不能只留一块。跨课程连接: 约束优化课用这套推导说明为什么拉格朗日乘子法与 KKT 条件给出的鞍点系统会有负特征值, 从而需要「鞍点求解器」而不是普通共轭梯度法; 数据科学课讲带约束最小二乘与正则化 (如等式约束的岭回归) 时, 同样的分块矩阵决定了解的稳定性。

补一层机制: 上面的二次型只回答了「符号对不对」, 没有回答「消元之后为什么只剩下这两块」。把 KKT 系统写成 $\begin{bmatrix} S & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$, 第一行给出 $x = S^{-1}(b - A^T \lambda)$, 代进第二行 $Ax = c$, 就得到 λ 必须满足的 $m \times m$ 方程, 其系数矩阵正是 $-AS^{-1}A^T$ 。所以 Schur 补不是凭空出现的符号, 而是「把 x 消掉之后 λ 自己必须服从的方程»: 原始变量 x 有 n 个自由度, 由 S 提供 n 个正主元; 对偶变量 λ 有 m 个自由度, 由这个负定块提供 m 个负主元。

再看「合同」这一步为什么合法。取 $E = \begin{bmatrix} I & 0 \\ -AS^{-1} & I \end{bmatrix}$, 可验证 $EHE^T = \text{diag}(S, -AS^{-1}A^T)$, 而 E 可逆, 所以这是合同而非相似; Sylvester 惯性定律说合同保持惯性, 于是惯性是两块之和, 不能只留一块。顺手可用的自检: $\det(H) = (-1)^m \det(S) \det(AS^{-1}A^T)$ 。

把设计问题补全: 为什么要在这道题上追问「为什么」。因为 KKT 系统的惯性直接决定能用哪种求解器——左上块 S 正定, 单看它会以为可以上 Cholesky 或共轭梯度; 但整个 H 不定, 这两种方法的前提 (正定性带来的能量范数下降方向) 就不成立, 必须换成 MINRES 这类允许负曲率的 Krylov 方法, 或者干脆消到 Schur 补上, 在一个 $m \times m$ 的稠密负定系统里做文章。 $m < n$ 这个条件也不是装饰: 它保证约束数少于变量数, 可行域才有 $n - m$ 维自由度, 两块的惯性才能像上面那样干净相加。

代入具体数字验算。取 $S = I_2$ 、 $A = \begin{bmatrix} 1 & 1 \end{bmatrix}$, 满足 $m = 1 < n = 2$ 且 A 行满秩。先走注里的二次型: y 是标量, $A^T y = \begin{bmatrix} y \\ y \end{bmatrix}$, 于是 $y^T AS^{-1}A^T y = y^2 + y^2 = 2y^2 > 0$, Schur 补为 $-2 < 0$, 与负定结论一致。再直接算 $H = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$: 特征多项式 $\det(\lambda I - H) = \lambda^3 - 2\lambda^2 - \lambda + 2 = (\lambda - 2)(\lambda - 1)(\lambda + 1)$, 特征值是 $2, 1, -1$, 两个正一个负, 正好是 $n = 2$ 个正加 $m = 1$ 个负; 行列式自检 $2 \cdot 1 \cdot (-1) = -2$, 与 $(-1)^1 \det(S) \det(AS^{-1}A^T) = -2$ 对上。还能显式看到负特征向量 $v = \frac{1}{\sqrt{6}}(1, 1, -2)^T$: 它满足 $Av = 0$, 也就是约束零空间方向上的曲率为负, 这正是 H 不定性的几何来源。

再补一个数值上的坑: 主元符号等于特征值符号的前提是消元过程中不做行交换; 一旦为了数值稳定选主元换了行, 读出来的主元符号就不再直接是惯性, 这时应改用对称主元分解或直接算行列式序列来复核。

命名模板: **Schur 消元惯性计数法**——写出 Schur 补、用可逆合同把 H 化成 $\text{diag}(S, -AS^{-1}A^T)$ 、分别数两块的符号, 三步定惯性。

跨课程连接再补两条。Strang 线性代数与 18.065: 概念是 LU/Cholesky 消元与分块求逆; 连接路径是对 S 做消元时 Schur 补本来就是消元过程的副产品, 18.065 用它推导正规方程与分块求逆公式; 区别在于那里关心怎么把解算出来, 这里关心矩阵的惯性长什么样, 同一公式承担两种任务。在学概率论 (Bertsekas): 概念是多元高斯与条件分布; 连接路径是协方差矩阵分块后, 条件协方差 $\Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$ 与 Schur 补是同一个表达式; 区别在于那里右下块是正定的协方差块, 得到的是仍然正定的条件方差, 而 KKT 矩阵右下块是 0, 于是 Schur 补翻成负定, 惯性从全正变成 n 正 m 负。

[problem] Problem 12. If $x^T Sx > 0$ for all x and C is invertible, why is $(Cy)^T S(Cy)$ also positive? This shows again that if S has all positive eigenvalues, so does $C^T SC$.

Strang §III.2, p. 177

§III.3 Rapidly Decaying Singular Values（快速衰减的奇异值）

[note] 译者注 (OCR 修正)： 本页 OCR 噪声密集，以下字符按上下文重建： 盖、风、尹、Fø 均为 Fourier 矩阵的记号 F （或其非均匀版本）之误； ak Those numbers 重建为 There are important matrices whose singular values have those numbers（存在一些重要矩阵，其奇异值正是这些数）；和 纱 系 family（族）之误； $\emptyset$ 咖 rm 重建为 nonuniform； $\text{ill s } \mathbb{N}$ ill 重建为 algorithm is superfast；仆、伙 等残字分别对应 with、work；被切碎的上标按 $2\pi k/N$ 、 $2\pi i c_j k$ 与 $e^{-2\pi i j k/N}$ 重建；公式 (1) 的两行按「逐元素乘」与「逐元素除」的对称结构重建。下面正文中出现的 A 是**修正矩阵**（比值矩阵），不是线性方程组里的系数矩阵，请勿与 $A\boldsymbol{x} = \boldsymbol{b}$ 中的 A 混同。

存在一些重要的矩阵，其 singular values（奇异值）满足 $\sigma_k \leq C e^{-\alpha k}$ 。这样的数衰减得很快。它们往往还是可逆的（只不过其逆矩阵大得惊人）。而且常常不是单个矩阵，而是一个族（family）、涵盖各种尺寸的矩阵——Hilbert matrix（希尔伯特矩阵）与 Vandermonde matrix（范德蒙矩阵）、Hankel matrix（汉克尔矩阵）与 Cauchy matrix（柯西矩阵），以及 Krylov matrix（克雷洛夫矩阵）、谱差分矩阵等等。

这些矩阵同时既好对付、又难对付。说好对付，是因为其中只有少数几个奇异值是显著的；说难对付，是因为它的逆矩阵具有巨大的 norm（范数），并随矩阵尺寸 N 呈指数增长。我们从众多例子中挑出两个重点讨论：

1. nonuniform discrete Fourier transform（非均匀离散傅里叶变换，NUDFT）用 $U = A * F$ 取代 F 。

2. Vandermonde matrix（范德蒙矩阵） V 把一个 $N - 1$ 次多项式拟合到 N 个数据点上。

其实这两个例子是相通的。标准 DFT 在等间距的 N 个点 $\omega^k = e^{-2\pi i k/N}$ 上拟合 N 个值 f_0 到 f_{N-1} 。Fourier matrix（傅里叶矩阵）本身就是一个 Vandermonde matrix！区别在于：DFT 不是在 -1 与 1 之间的**实数**点处插值，而是在**复数**点处插值——那些点等间距地分布在单位圆 $|e^{i\theta}| = 1$ 上。实的情形产生极其糟糕的 Vandermonde matrix（实质上是奇异的），复的情形则产生优美的 Fourier matrix（正交的）。

我们就从那个复的 Fourier matrix F （照例等间距）开始。乘以 F 借助 §IV.1 中的 Fast Fourier Transform（快速傅里叶变换）可以超快完成：只需 $\frac{1}{2} N \log_2 N$ 次运算。**对于非均匀间距 $x_j \neq j/N$ ，支撑 FFT 的那些特殊恒等式不复存在。** 但 Ruiz-Antolin 与 Townsend 指出，你几乎可以恢复全部速度：把非均匀的 U 写成 $A_{jk} F_{jk}$ ，其中 A **接近一个低秩矩阵**：

$$F_{jk} = e^{-2\pi i j k/N} \quad \text{and} \quad U_{jk} = e^{-2\pi i k x_j} = A_{jk} F_{jk}$$

当 $U = F$ 是等间距的 DFT 时，每个 $A_{jk} = 1$ ， A **秩为 1**。在不等间距下，一个低秩矩阵已与 A 几乎一致，快速变换因而成为可能。下面给出逐元素乘法 $A_{jk} F_{jk}$ 与逐元素除法 U_{jk}/F_{jk} 的记号：

$$\text{Multiplication} \quad U = A * F = A \bigcirc F \qquad \text{Division} \quad A = U \oslash F. \tag{1}$$

需要快速执行的运算是 NUDFT： U 乘以 c 。而 Fast Fourier Transform（快速傅里叶变换）计算的只是 F 乘以 c 。那些比值 U_{jk}/F_{jk} 给出一个矩阵 A ，它**几乎是低秩的**。**于是非均匀变换 U 来自对 Fourier matrix F 的一个修正 A ：**

$$\boxed{A \approx y_1 z_1^T + \cdots + y_r z_r^T \quad \text{and} \quad U c \approx Y_1 F Z_1 c + \cdots + Y_r F Z_r c.} \tag{2}$$

Y_i 与 Z_i 都是 N 乘 N 的对角矩阵，其主对角线上分别是 y_i 与 z_i 。等间距时有 $U = F$ 、 $A = \text{ones}$ （全 1 矩阵），并且 $y_1 = z_1 = (1, \dots, 1)$ 、 $Y_1 = Z_1 = I$ 。对于不等间距的情形， r 由采样点 x_i 的非均匀程度决定。

[note] 译者注 (跨课程连接)： 这一页的设计问题是：**结构化快速乘法一旦被扰动破坏，能不能把它修回来而只付很小的额外代价**。先看被破坏的是什么。等间距时 F 乘向量能在 $O(N \log N)$ 内完成，靠的是旋转因子 $e^{-2\pi i j k/N}$ 对 j 与 k 的周期性，它让 N 点问题拆成两个 $N/2$ 点问题——这正是 6.006（算法导论）里的分治模板，递归式 $T(N) = 2T(N/2) + O(N)$ 给出 $O(N \log N)$ 。非均匀采样把 c_j 取代 j/N 之后， $U_{jk} = e^{-2\pi i c_j k}$ 不再是 j 与 k 的乘积型相位，周期性消失，分治的合并步骤立刻失效：你没法再把偶数项和奇数项拼回原问题。这就是「 U 看起来像 F ，却算不快」的全部原因。

先把被破坏的到底是什么讲透。分治能成立的前提是子问题与原问题同型： $c_j = j/N$ 时相位关于 j 与 k 是双线性的，把 j 按奇偶拆开， $e^{-2\pi i (2r)k/N} = e^{-2\pi i r k/(N/2)}$ ，于是 N 点问题变成两个 $N/2$ 点问题，归纳假设原样成立。一旦 c_j 只由采样点决定、与 j 不成比例， U_{jk} 就不再是行指标乘列指标的函数，奇偶拆分后的子块既不是 F 也不是 U ，没有可用的归纳假设，递归式 $T(N) = 2T(N/2) + O(N)$ 从第一步就崩了。

那还能不能救？能，但要换一个归纳假设：不再要求子块同型，而要求子块数值低秩。把 U 按行、按列各分成大小相当的块，远场块（那一批 c_j 与那一批 k 相距较远）里，核 $e^{-2\pi i c_j k}$ 作为二元函数是光滑的，可以用少数几项插值或展开逼近，于是该块的数值秩 r 远小于块尺寸，一次块乘向量只花 $O(r \cdot \text{块尺寸})$ 。这就是 6.006 版分治之外的第二条路：分治靠结构精确重复，低秩压缩靠结构近似重复。

还有一条更实用的修复路径，它不追求精确，只追求可控近似：先把非均匀数据扩散到过采样的均匀网格上，用一次均匀 FFT 做卷积，再插值回原采样点并做去卷积校正，这就是 NUFFT 的三步。代价仍是 $O(N \log N)$ ，但多出一个精度旋钮：过采样因子 s 与核宽 w 共同决定误差 ε ，付的额外代价是常数倍数与若干次 FFT，而不是阶的上升。所以这一页的答案是：能修，但修回来的不再是那个精确的 U ，而是一个误差可证的近似。

给一个小验算，把修回来要付多少代价算成具体数字。设 $c_j = j/N + \delta_j$ ，拆开相位得 $U_{jk} = e^{-2\pi i j k/N} e^{-2\pi i \delta_j k}$ ，对第二个因子做 Taylor 展开， $U \approx \sum_{p=0}^P a_p D_p F D_p'$ ，其中 $D_p = \text{diag}(\delta_j^p)$ 、 $D_p' = \text{diag}(k^p)$ ，每一项是两次对角缩放夹一次 F ，也就是一次 $O(N \log N)$ 。取 $N = 2$ 、 $\delta_1 = 0.01$ 且只保留 $P = 1$ 项：均匀时 U 的第二行是 $[1, -1]$ ，扰动后是 $[1, -e^{-0.02\pi i}] \approx [1, -0.998 + 0.0628i]$ ，绝对误差的量级是 $\frac{(2\pi \delta_1 k)^2}{2} = \frac{0.0628^2}{2} \approx 2 \times 10^{-3}$ 。误差随 $2\pi \delta_j k$ 二阶增长，所以只有 $2\pi \delta_{\max} N \ll 1$ 时全局展开才便宜，否则必须按 k 分块、在每块内重新展开，这正是分层低秩（butterfly）算法存在的原因。

命名模板：**扰动相位展开修复法**——拆相位、Taylor 展开、写成若干个「对角— F —对角」项之和；它的粗糙版本叫 **远场分块低秩法**——块内数值秩小，逐块压缩。

跨课程连接再补两条。Strang 线性代数与 18.065：概念是 SVD 与低秩近似；连接路径是把 U 的远场块拿出来做 SVD，Eckart–Young 定理保证截断到前 r 个奇异值就是该块在 Frobenius 范数下的最优低秩逼近；区别是 SVD 给的是任意矩阵的最优低秩近似，但要 $O(N^3)$ 构造、必须显式存矩阵，而这里用核函数的解析展开直接生成低秩因子，构造只要 $O(N \log N)$ ，连矩阵都不必写出来。CSAPP：概念是缓存与访存模式；连接路径是均匀 F 的访存规则可预测，非均匀情形要先把数据扩散到均匀网格上再 FFT，这一步是散列式随机写，缓存不命中率上升、常数因子变大；区别是 CSAPP 解释的是这类算法为什么在实际机器上比渐进阶更慢，而算法课只保证阶。

本书的修补办法是一个通用模板，我把它叫做**结构化基底加低秩修正模板**：把难算的矩阵写成 $U = A \circ F$ ，让 A 独自承担全部非均匀性，而 F 保留可快速乘法的能力；只要 A 近似低秩， $A \approx \sum_{i=1}^r y_i z_i^T$ ，就有 $Uc \approx \sum_{i=1}^r Y_i F Z_i c$ ，其中 Y_i 、 Z_i 是对角矩阵，因此每一项只花一次 FFT 加 $O(N)$ 的逐元素缩放。这与 6.006 讲的「预处理 + 快速查询」同构（预处理时把难结构压成低秩表，查询时只查表加修正），也与 18.065 里把矩阵拆成「结构化部分 + 低秩部分」的分解哲学一致。要注意区别：6.006 关心的是**最坏情形**的渐进复杂度，这里关心的是**误差可控前提下的实际常数**——低秩一旦不成立，速度立刻退回 $O(N^2)$ ，所以「秩有多大」才是真正的命门。（把 A 近似成低秩，最小可能误差由第 $r+1$ 个奇异值 σ_{r+1} 给出。）

给个具体量级。 $N = 1024$ 时，直接算 NUDFT 需 $N^2 \approx 1.05 \times 10^6$ 次复数乘法；一次 FFT 需要约 $\frac{N}{2} \log_2 N = 512 \times 10 = 5120$ 个蝶形（一次蝶形等于一次复数乘加），取秩 $r = 10$ 的修正共约 5.1×10^4 个蝶形，比直接算法快约 20 倍。 $N = 4096$ 时直接算法约 1.68×10^7 ， $r = 10$ 的修正约 2.46×10^5 ，快约 68 倍。这个对比说明的是「几乎恢复全部速度」的确切含义：额外的开销只是一个与 r 成正比的常数因子；在固定 r 下，加速比大致按 $N/\log_2 N$ 增长。但 r 并非与 N 无关的常数——本节的结论是小误差只需 r 按 $\log N$ 增长，所以总代价是 $O(N \log^2 N)$ 量级，而不是把 $O(N \log N)$ 打回 $O(N^2)$ 。

最后看低秩的来源，这是全页最值得记的一步。逐元素除法 $A = U ./ F$ 给出 $A_{jk} = \exp(-2\pi i k(c_j - j/N))$ ，记偏差 $\delta_j = c_j - j/N$ ，则 $A_{jk} = \exp(-2\pi i k \delta_j)$ 。每个元素都是「只含 j 的因子」与「只含 k 的因子」之积 $r_j s_k$ ，这正是一个秩 1 矩阵的形状；等间距时 $\delta_j \equiv 0$ ， A 恰为全 1 矩阵、秩为 1，这就是正文那句「 $r = 1$ 、 $Y_1 = Z_1 = I$ 」。当采样点轻微偏离 j/N 时 δ_j 很小， A 仍是「近似秩 1」； δ_j 的幅度分布越宽，把它压到小误差所需的秩 r 就越高。于是这一页实际上给出了把「采样点有多不听话」翻译成「秩有多高」的桥梁： r 的大小直接由非均匀程度决定。这也解释了为什么轻微非均匀才能既准又快——非均匀性是输入数据给的，而代价随它增长。

[note] 译者注 (术语校准)：第 179 页会出现的 low effective rank 应译作**低有效秩**，它和「低秩」不是同一句话，区别值得单独说清。低秩是精确的结构断言： $\text{rank}(A) \leq r$ ，即 A 恰能写成 r 个秩 1 矩阵之和。低有效秩是数值断言： A 的秩可能仍是满的 N ，但它的奇异值 $\sigma_1 \geq \sigma_2 \geq \dots$ 衰减极快，于是前 r 个奇异值之外的尾部能量 $\sum_{k>r} \sigma_k^2$ 小到可以忽略，截断 $A_r = \sum_{k \leq r} \sigma_k u_k v_k^T$ 就是一个足够好的近似。判据是比值而非秩本身：若 $\sigma_{r+1}/\sigma_1 \leq \varepsilon$ ，则在谱范数下 $\|A - A_r\|_2 = \sigma_{r+1}$ ，相对误差不超过 ε 。这正是「几乎一致」(virtually agrees) 一词的确切含义，也是后面数值线性代数课程里 numerical rank (数值秩) 与条件数 σ_1/σ_N 的入口。

补机制：这两句话为什么必须分开。 先问它在解决什么设计问题。把 A 看成从输入空间到输出空间的线性算子：它把单位球映成一个椭球，第 k 个奇异值就是这个椭球第 k 根主轴的半轴长。于是 σ_k 的物理含义是「第 k 个方向被放大多少倍」。低秩说的是「椭球在第 r 根轴之后直接塌成零」，是精确的；低有效秩说的是「轴还在，但半轴短到量不出来」。为什么不能只看秩？因为秩是代数上的计数，它对扰动不连续：把 A 的一个零奇异值扰动成 10^{-14} ，代数秩立刻从 r 跳到 N ，可算子的实际行为几乎没变。低有效秩正是为了修复这个不连续性而引入的：它带一个容限 ε ，而 ε 来自浮点精度或测量噪声，不是矩阵自身的属性。所以 $\text{rank}(A)$ 是一个与精度无关的整数，有效秩是一个依赖精度的函数 r_ε 。工程上取阈值 $n\varepsilon\|A\|_2$ 是有理由的：浮点 SVD 的向后误差约为 $n\varepsilon\|A\|_2$ 量级，小于它的奇异值与舍入噪声不可区分，硬把它当信号去求逆，就等于把噪声放大 $1/\sigma_N$ 倍。这也解释了为什么第 179 页敢用「几乎一致」这种措辞：那里的近似不是猜的，而是有范数界兜底的。

补一个可复算的小矩阵。 取 $A = \begin{pmatrix} 1 & 1 \\ 0 & 10^{-6} \end{pmatrix}$ 。它的代数秩是 2（两个列向量线性无关），但 $\sigma_1 \approx 1$ 、 $\sigma_2 \approx 10^{-6}$ ，于是对 $\varepsilon = 10^{-5}$ 有 $r_\varepsilon = 1$ 。验算： $A^T A = \begin{pmatrix} 1 & 1 \\ 1 & 1+10^{-12} \end{pmatrix}$ ，特征值为 $2+10^{-12}$ 与约 5×10^{-13} ，开方得 $\sigma_1 \approx 1$ 、 $\sigma_2 \approx 10^{-6}$ ，与断言一致。再算截断： A_1 由第一奇异对给出，作差后 $A - A_1$ 的谱范数恰好是 $\sigma_2 = 10^{-6}$ ，相对误差 10^{-6} ，满足上面那条界。最后看能量： $\sigma_1^2 + \sigma_2^2 \approx 2 + 10^{-12}$ ，被丢掉的尾部占比约 2×10^{-13} ，确实可以忽略。三步手算把「秩是 2、有效秩是 1」这两个看似矛盾的结论同时坐实了。

命名模板：「谱衰减能量检验法」。 先用奇异值比 $\sigma_{r+1}/\sigma_1 \leq \varepsilon$ 定数值秩，再用尾部能量占比 $\sum_{k>r} \sigma_k^2 / \sum_k \sigma_k^2$ 确认丢掉的确实可丢，两步都过才敢截断。

跨课程连接。 入口在 Strang 线性代数与 18.065：Strang 讲 $A = U \Sigma V^T$ 时读作「把球映成椭球，奇异值是各半轴长」，18.065 直接拿截断 SVD 做低秩近似与 PCA，与本注的区别在于——那两门课先假定你已经接受「小奇异值可以丢」，本注补的正是在「凭什么可丢」的前置判据。在 CLRS 与 6.006 里，同类推理以「按主项截断复杂度」出现：都是先定误差预算、再决定保留几项，区别是那里截断的是渐近阶，这里截断的是谱。在学的概率论 (Bertsekas) 中，协方差矩阵的特征值衰减对应主成分降维，是同一件事的统计版本。

Strang §III.3, p.178

本页为书页 179，是 §III.3 的续页；页眉与页码为重复元素，不译。

[\[section\]](#)

Sample Points x_j near j/N (靠近 j/N 的采样点)

在这种轻微扰动的情形下，我们可以把不等间距的采样点 x_j 与等间距的分数 j/N ($0 \leq j < N$) ——对应起来。对于 U 的每个元素， A_{jk} 都是对 F_{jk} 的一个修正：

$$U_{jk} = A_{jk} F_{jk} \text{ is } e^{-2\pi i k x_j} = e^{-2\pi i k (x_j - j/N)} e^{-2\pi i k j / N} \tag{3}$$

于是快速算法的关键一步，就是为这个矩阵 A 找到足够接近的近似（我们知道 F 是快的）。Eckart-Young theorem (Eckart-Young 定理) 会建议使用 A 的 SVD。但 SVD 这一步比非均匀间距快速变换的其余部分都要昂贵得多。

A 是个有意思的矩阵：方程 (3) 中它的所有元素都具有 $e^{i\theta}$ 的形式。若把 $A_{jk} = e^{-i\theta}$ 用它自己的 power series（幂级数） $1 - i\theta + \dots$ 替换，那么 A 将从一个全 1 矩阵开始。

本节其余部分通过处理 Sylvester equation (西尔维斯特方程) 来寻找低秩近似。而这里 Townsend 走了一条不同的路线：*近似函数* e^{-ixy} 。

关键想法是把这些矩阵元素各自的 Taylor series（泰勒级数）替换为 Chebyshev expansion（切比雪夫展开）。这反映了数值分析中一条极其重要的规则：**Fourier series（傅里叶级数）适用于像 $|\theta| \leq \pi$ 这样区间上的周期函数。Chebyshev series（切比雪夫级数）适用于像 $|x| \leq 1$ 这样区间上的非周期函数。**

Fourier 与 Chebyshev 之间的联系是 $\cos \theta = x$ 。Fourier basis function（傅里叶基函数） $\cos n\theta$ 变成了 Chebyshev polynomial（切比雪夫多项式） $T_n(x) = \cos(n \arccos x)$ 。Chebyshev 基从 $T_0 = 1$ 、 $T_1 = x$ 与 $T_2 = 2x^2 - 1$ 开始，因为 $\cos 2\theta = 2 \cos^2 \theta - 1$ 。所有这些 basis function（基函数）在从 $x = -1$ 到 $x = 1$ 上都有 $\max |T_n(x)| = \max |\cos n\theta| = 1$ 。

重要的一点是， $T_n(x) = 0$ 的 n 个解（即 T_n 的根）并不等间距。 $\cos n\theta$ 的零点等间距，但换到 x 变量中，这些点在边界 -1 与 1 附近变得彼此靠近。在这些点 $x = \cos(\pi(2k-1)/2n)$ 上作插值，远比等间距插值稳定。

[note] 译者注（第一性原理）： 这页回答了插值里一个反直觉的事实：**把点放得越均匀，插值反而越不稳定**。它的第一性原理是一次变量替换 $c = \cos \theta$ 。等间距的 θ 在单位圆上均匀，投影到直径 $c = \cos \theta$ 上时被导数 $dc/d\theta = -\sin \theta$ 压缩；当 θ 趋近 0 或 π 时 $\sin \theta \rightarrow 0$ ，同样的角度间隔在 c 轴上被压成很窄的间距。于是节点密度正比于 $1/\sqrt{1-c^2}$ ——这是反正弦分布，中间稀、两端密。它是几何的产物，不是经验法则，所以 T_n 的根（也叫 Chebyshev-Gauss 节点）天然就带这个密度。

补机制：密度只决定「候选」，稳定性由 Lebesgue 常数裁决。 插值算子 Π_n 把 f 映成它在节点上的插值多项式。 Π_n 在最大范数下是有界线性算子，它的算子范数就是 Lebesgue 常数 $\Lambda_n = \max_{c \in [-1,1]} |\sum_{k=0}^n l_k(c)|$ ， l_k 是 Lagrange 基函数。它精确度量了「节点上的误差被放大多少倍」。由此得到误差界 $\|f - \Pi_n f\|_\infty \leq (1 + \Lambda_n) \min_{p \in \mathcal{P}_n} \|f - p\|_\infty$ ：即使最佳逼近误差趋于零，只要 Λ_n 发散，插值误差照样爆炸。关键点是 Λ_n 只由节点位置决定，与 f 无关——这就是为什么「节点放哪儿」是一个必须从第一性原理解决的问题，而不能靠试。等距节点的 Λ_n 随 n 指数增长，Chebyshev 节点只按 $\ln n$ 增长，所以「越均匀越不稳定」不是巧合，而是 Λ_n 的增长率被换掉了。

补验算： $n = 5$ 的两组数。 先看节点。 T_5 的根为 $c_k = \cos \frac{(2k+1)\pi}{10}$ ，即约 ± 0.9511 , ± 0.5878 , 0 。端点间距 $1 - 0.9511 \approx 0.049$ ，中心间距 $0.5878 - 0 \approx 0.588$ ，相差约 12 倍——正好与权重 $1/\sqrt{1-c^2}$ （端点为 3.2 ，中心为 1 ）同向，密度公式不是口号。再看稳定性的两个端点：等距节点的 Λ_n 大致按 $2^n/(n \ln n)$ 量级增长， $n = 20$ 时约 10^4 ，意味着节点值上 10^{-16} 的舍入会被放大成 10^{-12} ；而 Chebyshev 节点在 $n = 5$ 时 $\Lambda_5 \approx 1.4$ （在节点间取最大，如 $c \approx 0.77$ 处 $\sum_k |l_k|$ 约为 1.4 ）， n 增大也只是缓慢上升。同一个 $n = 5$ ，一边 1.4 、一边随 n 指数翻倍，差距就是这条注要讲的全部内容。

为什么是 Chebyshev，而不是「好用的启发式」。 有一个干净的等价：使 $\sum_k |l_k(c)|$ 的最大值最小的节点，恰好是使首一多项式 $\prod_k (c - c_k)$ 的最大模最小的节点；而后者由 Chebyshev 多项式取到极小值 2^{-n} 。所以 T_n 的根不是被挑中的，而是被最优化问题解出来的，与上面那条压缩几何互为印证。这是 Chebyshev 交错定理的同族结果：极值问题的最优解由等波纹条件刻画。

命名模板：「几何压缩加最坏放大检验法」。 先用 $c = \cos \theta$ 把均匀角间隔压缩成端点密的反正弦密度，得到候选节点；再用 $\Lambda_n = \max_c \sum_k |l_k(c)|$ 检验这一布局的最坏放大，两者都过才收下节点分布。

跨课程连接。 6.042J 离散数学里「最坏情况界由结构性参数决定」的推理与这里同构：那里的尾部概率由 Chernoff 界控制，这里的放大由 Λ_n 控制，连接路径都是「先把算子范数或矩母函数算出来，再代入误差预算」，区别在于 6.042J 的对象是带概率测度的随机变量，给出的是尾部概率，而这里用的是最大范数、给出的是确定性最坏放大，没有任何概率假设。CSAPP 那边补上落地一环：数学上 n 增大可使最佳逼近误差减小，但节点矩阵（Vandermonde）在等距节点下条件数也指数增长，双精度只有约 16 位有效数字，于是「范数下可忽略的量」在浮点下未必可忽略；这正是要用上溢/舍入与有效位来读同一现象的地方。在学的凸优化（Boyd）里， $A^T A$ 的谱条件数决定了求解精度的上界，与 Λ_n 一样属于「问题本身的病态度」，区别是那里谈的是线性方程组解的条件数，这里谈的是插值算子本身的条件数。

用 $n = 10$ 验算实际间距。节点 $c_k = \cos(\pi(k - \frac{1}{2})/10)$ ：靠右端， $c_1 = \cos(0.15708) \approx 0.98769$ 、 $c_2 = \cos(0.47124) \approx 0.89101$ ，间距约 0.0967 ；靠中央， $c_5 = \cos(1.41372) \approx 0.15643$ 、 $c_6 = \cos(1.72788) \approx -0.15643$ ，间距约 0.3129 ，两端间距只有中央的三分之一左右，且 n 越大差距越悬殊（局部间距约为 $\frac{\pi}{n} \sin \theta$ ）。边界附近被分配了更多节点，用来抵抗端点处最容易发散的多项式振荡（Runge 现象的爆发点恰在端点附近）。

代价与收益要对读。收益是稳定的插值算子；代价是插值误差里多出 $\max |T_n| = 1$ 这个因子——所有基函数在 $[-1, 1]$ 上的幅度都被压到不超过 1 ，系数因此不会像单项式基 $1, c, c^2, \dots$ 那样在端点爆炸，这正是「等波动」的字面含义： T_n 在区间内取到 ± 1 共 $n + 1$ 次，把误差均匀摊开而不是挤在一处。一个可对照的量化指标是 Lebesgue 常数 Λ_n ：等间距节点的 Λ_n 随 n 指数增长（这就是 Runge 现象与龙格函数的定量根源），Chebyshev 节点只按 $\frac{2}{\pi} \ln n + O(1)$ 增长（ $n = 10$ 时约 1.47 ）。插值误差大致被 Λ_n 乘以最佳逼近误差控制，所以「指数对对数」就是「不能用对能用」的分界，也解释了 chebfun.org 为什么敢把一个函数直接表示成高次切比雪夫级数并默认返回：它的稳定性预算够，次数由代码自动选取。

与实分析的连接点在收敛模式。切比雪夫展开与傅里叶级数的字典是 $\cos n\theta = T_n(\cos \theta)$ ，于是区间 $[-1, 1]$ 上非周期函数的逼近问题，被精确翻译成 $[-\pi, \pi]$ 上偶周期函数的逼近问题——不是任意周期函数，必须是偶函数，因为 $c = \cos \theta$ 只覆盖 θ 的偶对称部分。实分析课上关于逐点收敛与一致收敛的区分在这里是可用的：切比雪夫系数 a_n 由 f 在复平面上包含 $[-1, 1]$ 的某个 Bernstein 椭圆内的解析性决定，椭圆越大衰减越快， L^∞ 误差由 $|a_{n+1}| + |a_{n+2}| + \dots$ 控制，而误差要在整段区间上一致地小，靠的是 $\max |T_n| = 1$ 提供的等波动性，而不是某一点的表现。一句话总结：**傅里叶级数是换到角度坐标后看周期函数，切比雪夫展开是同一个换元落到实轴区间后的形式**——先有变量替换，后有基函数。

高度发达的计算系统 **chebfun.org** 就建立在 Chebyshev polynomials（切比雪夫多项式）之上（适用于一维或多维的函数）。它算出一个非常接近 $f(x)$ 的多项式。随后对函数的一切运算，都以有限 Chebyshev series（切比雪夫级数）的形式产生新的多项式。达到高精度所需的次数由 **chebfun** 的代码自动选择。

Townsend 对矩阵 A 的这种处理方式，使 Ruiz-Antolin 与 Townsend 证明了 A 具有 low effective rank（低有效秩）——即它能被一个低秩矩阵很好地逼近。他们的论文为 nonuniform Fourier transform（非均匀傅里叶变换）提供了非常高效的代码。

Strang §III.3, p.179

[section]

The Sylvester Equation (Sylvester 方程)

我们转向本主题的核心问题：**哪些矩阵族具有 low effective rank（低有效秩）**？目标是找到一个能揭示这一性质的检验。在已经讨论过的 Hilbert matrix（希尔伯特矩阵）与 NUDFT 矩阵 A 这两个例子之外，我们还想把 Vandermonde matrix（范德蒙矩阵）加进来。而且我们期望这个检验能适用于远比 Vandermonde 例子广泛得多的矩阵类。

下面就是由 Beckermann 与 Townsend 发展的 Sylvester test（Sylvester 检验），也称 A, B, C test。对于 X ，人们常常使用 low displacement rank（低位移秩）与 structured matrix（结构化矩阵）这两个词。

若对没有公共特征值的正规矩阵（normal matrix） $A、B$ 来说，方程 $AX - XB = C$ 的秩为 r ，则 X 的奇异值以指数速率衰减，该速率由 $A、B、C$ 决定。

若 $\bar{A}^T A = A \bar{A}^T$ ，则称 A 为 normal matrix（正规矩阵）。于是 A 具有正交的特征向量： $A = Q \Lambda \bar{Q}^T$ 。对称矩阵与正交矩阵都是正规矩阵——因为该检验对它们分别给出 $S^2 = S^2$ 与 $I = I$ 。Sylvester matrix equation（Sylvester 矩阵方程） $AX - XB = C$ 在控制论（control theory）中很重要，而当 $B = -\bar{A}^T$ 时的特殊情形被称为 Lyapunov equation（Lyapunov 方程）。

[note] 译者注（为什么「秩」能控制「衰减」）：Sylvester 检验表面上在谈秩，实际在谈**线性算子的可逆性与条件数**。把 $-B^T$ 与 A 按 Kronecker 积拼成 $I \otimes A - B^T \otimes I$ ，再把 X 按列拉直成 $\text{vec}(X)$ ，则矩阵方程 $AX - XB = C$ 就变成一条普通的线性方程 $K \text{vec}(X) = \text{vec}(C)$ 。这一步用到向量化恒等式 $\text{vec}(AXB) = (B^T \otimes A) \text{vec}(X)$ ，其中 $\otimes$ 是 Kronecker 积。

先补机制。我们真正要回答的设计问题是：给定 B 与 C ，方程 $AX - XB = C$ 什么时候**对任意** C 都有唯一解，而且解不会因为 C 的微小扰动而爆炸？从第一性原理看，这个方程对未知量 X 是线性的——若 X_1, X_2 都是解，则 $A(X_1 - X_2) - (X_1 - X_2)B = 0$ ，所以「唯一性」完全等价于齐次方程 $AX = XB$ 只有零解，也就是算子 $L(X) = AX - XB$ 单射。有限维下单射即满射，于是「对任意 C 可解」与「只有零解」是同一件事，这正是 Sylvester 定理的秩形式： $\text{rank}(K) = n^2$ 当且仅当 $\sigma(A) \cap \sigma(B) = \varnothing$ 。向量化只是把抽象算子 L 具体化成一张 $n^2 \times n^2$ 的矩阵 K ，让「秩」这个词终于有一个可以计算的载体。至于「衰减」，指的是逆算子 L^{-1} 的范数： $\|X\| \leq \|K^{-1}\| \|C\|$ ，而 $\|K^{-1}\| = 1/\sigma_{\min}(K)$ 。所以秩刻画的是「有没有解」，最小奇异值刻画的是「解稳不稳」，二者合起来才是完整的可解性图景。

再看一个可直接复算的验算。取 2×2 的 $A = \begin{pmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{pmatrix}$ ， $B = \begin{pmatrix} \beta_1 & 0 \\ 0 & \beta_2 \end{pmatrix}$ 。按列拉直约定， $I \otimes A$ 先作用在 vec 的第一块上。算 $I \otimes A$ ： $\begin{pmatrix} A & 0 \\ 0 & A \end{pmatrix}$ ；算 $B^T \otimes I$ ： $\begin{pmatrix} \beta_1 I & \beta_2 I \\ \beta_1 I & \beta_2 I \end{pmatrix}$ 再按块取转置得 $\begin{pmatrix} \beta_1 I & 0 \\ 0 & \beta_2 I \end{pmatrix}$ 。两者相减得 $K = \text{diag}(\alpha_1 - \beta_1, \alpha_1 - \beta_2, \alpha_2 - \beta_1, \alpha_2 - \beta_2)$ ，于是 $\det K = (\alpha_1 - \beta_1)(\alpha_1 - \beta_2)(\alpha_2 - \beta_1)(\alpha_2 - \beta_2)$ 。这段两行手算把结论全部摊开了： K 满秩当且仅当任意一对 (α_j, β_k) 都不相等，也就是 $\prod_{j,k} (\alpha_j - \beta_k) \neq 0$ 。对可对角化的一对矩阵，同一个乘积形式依然成立，这就是「谱距离」的代数化身。

跨课程连接一条，精确到路径与区别。在 Strang 线性代数与 18.065 里， $\prod_{j,k} (\alpha_j - \beta_k)$ 正是两个特征多项式结式的因子分解形态，出现在「用特征值研究矩阵方程」这一节；共同点是都用谱来判定退化，区别在于 18.06 侧重点是谱定理与对角化本身，而这里要的是谱的**相减**——是两族特征值之间的间隔，而不是单族的分布。补一句浮点层面的连接：CSAPP 告诉我们， $\sigma_{\min}$ 比机器精度还小时，纸面上的可逆在 IEEE 双精度里会直接变成「有效位数被吃光」，所以工程上从不只看秩，还要看条件数。可命名的模板：**谱夹逼法**——把「秩是否满」夹到「谱是否撞车」上，用一次 $\prod (\alpha_j - \beta_k)$ 的数值检查代替原矩阵的秩计算。

关键在于： K 的全部特征值恰好是两组特征值的一切差 $\alpha_i - \beta_j$ ， α_i 取自 A ， β_j 取自 B 。于是「 A 与 B 没有公共特征值」等价于「 K 可逆」，也就是方程对每个 C 都有唯一解。这正是本书反复强调这一条件的真正原因：它不是礼貌性的技术假设，而是「可解」这一要求在谱上的翻译。

由此得到我称为**谱分离判定模板**的三步：(i) 把矩阵方程拉直成线性系统；(ii) 写出算子的谱并检验是否含零——含零则不唯一可解，不含零则唯一可解；(iii) 用最小谱间隙 $\min_{i,j} |\alpha_i - \beta_j|$ 估计解对 C 的敏感度，因为它就是 K 条件数的核心成分。18.065 课把这条谱判据放在「矩阵方程的数值解」一节，与该课反复训练的「先看谱、再看条件数」是同一套动作。顺带得到一条设计经验：要让 C 保持低秩、又要让 X 的衰减足够快，就得让两组谱互相避得远远的； $\min_{i,j} |\alpha_i - \beta_j|$ 越大， X 的奇异值衰减越快。

小规模验算（ $n = 2$ ）。取 $A = \text{diag}(2, 5)$ 、 $B = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ ，后者的特征值是 0, 0。四个差为 2, 5, 2, 5，都不为零，故 K 可逆、条件数为 $5/2 = 2.5$ ，任何 C 都有唯一解。若把 A 换成 $\text{diag}(0, 5)$ ，差集中出现 0， K 奇异：方程并非对每个 C 都可解，有解时也不唯一。注意 B 自身带重特征值并不妨碍推理，可见判据只要求「两组谱不撞车」，并不要求各自互异。

Vandermonde 的例子还可以手算确认「 C 的秩为 1」。取节点 $c = (1, 2, 3)$ ， $V = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix}$ ， $A = \text{diag}(1, 2, 3)$ ， B 取上移位阵。则 $AV = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 8 \\ 3 & 9 & 27 \end{bmatrix}$ ， $VB = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 2 & 4 \\ 0 & 3 & 9 \end{bmatrix}$ ，相减得 $\begin{bmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \\ 3 & 0 & 0 \end{bmatrix} = [1, 2, 3]^T e_1^T$ ，确实是秩 1。原因不神秘： V 的第 k 列是节点向量的第 $k - 1$ 次幂，左乘 A 相当于把幂次加一，右乘移位阵相当于把幂次挪一格，两者之差只剩最低次幂那一列。（页面上用于 Krylov 例子的写法是 $A = \text{diag}(1, \dots, n)$ ；两种写法是同一个位移结构的不同记法，检验的核心始终是「幂次作用减去移位作用，只剩秩一残差」。）

这个 Sylvester 检验要求我们找出 A 、 B 、 C ！对于若干极为重要的矩阵族，这件事已经完成了：Toeplitz、Hankel、Cauchy 以及 Krylov 矩阵（其中包含 Vandermonde 矩阵 V ）。所有这些矩阵都对简单的 A 、 B 、 C 选择求解 Sylvester 方程。我们取 V 作为主要例子：

Vandermonde matrix（范德蒙矩阵）

$$V = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1} \end{bmatrix} \tag{4}$$

V 是 n 阶的 interpolation matrix（插值矩阵）。只要点 $x_1, \dots, x_n$ 两两不同，它就是可逆的。当我们想要多项式 $p = c_0 + c_1x + \cdots + c_{n-1}x^{n-1}$ 的系数时，就求解 $Vc = f$ 。把 V 乘以 c 就给出 p 在各点 $x = x_1, \dots, x_n$ 处的值。于是 $Vc = f$ 说明该插值多项式在这 n 个点上恰好取到数据所要求的值 $f_1, \dots, f_n$ 。**多项式恰好拟合数据。**

我们曾指出，当选取复数点 $x_1 = \omega = e^{-2\pi i/N}$ 与 $x_k = \omega^k$ 时， V 就变成 Fourier matrix（傅里叶矩阵） F 。这个 F 满秩。在那个傅里叶情形下 A 与 B （见下文）具有相同的特征值 ω^k ：这是不允许的。 A 与 B 不满足我们关于特征值良好分离的要求，并且此时全部奇异值大小相等：没有衰减。

正是实数节点 x_1 到 x_n 的 Vandermonde matrices（范德蒙矩阵）才在其奇异值中具有指数衰减。为确认这一点，用下面这些矩阵来做 A, B, C 检验：

$$A = \begin{bmatrix} x_1 & & & \\ & x_2 & & \\ & & \ddots & \\ & & & x_n \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & \cdot & -1 \\ 1 & 0 & 0 & \cdot \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 0 & 0 & x_1^n + 1 \\ 0 & 0 & 0 & x_2^n + 1 \\ 0 & 0 & 0 & \cdot \\ 0 & 0 & 0 & x_n^n + 1 \end{bmatrix} \tag{5}$$

[note] 译者注 (OCR 修正)：原文此处矩阵被 OCR 打散为零散数字，现按 Sylvester 检验的数学语境重建：取 X 的元素为 $X_{jk} = c_j^{k-1}$ ($j, k = 1, \dots, n$)，与 $A = \text{diag}(1, 2, \dots, n)$ 、上移位阵 B 相乘，则 $(AX - XB)_{jk} = c_j^{k-1}$ ，即 $C = [1, c_2, \dots, c_n]^T e_1^T$ ，是一个秩一矩阵（与正文所说秩 r 一致）。上文 OCR 中 $\text{AX} - \text{XB} = \text{C}$ 重建为 $AX - XB = C$ ， 盖 重建为 A ，**正常矩阵**（normal）原本被 OCR 记为 $\text{盖} = \text{盖盖}$ ，按正规矩阵定义重建为 $\overline{A}^T A = A \overline{A}^T$ ， $= \text{QAQ}$ 重建为 $A = Q\Lambda\overline{Q}^T$ ； $\text{B} = -\text{盖}$ 中的 盖 为负号，即 $B = -A^T$ ； $\text{low m "ra" and structured r}$ 依上下文重建为 low rank and structured； real be 1 to 依范德蒙衰减条件重建为 real between 0 and 1，即实节点落在 $(0, 1)$ 内；句末被切碎的下标与句号按插值语境重建。

[note] 译者注 (OCR 修正)：经原书扫描页核对，上列重建应以原书排印为准：式 (4) 的节点记号为 $x_1, \dots, x_n$ ；式 (5) 取 $A = \text{diag}(x_1, \dots, x_n)$ 、 B 为上移位阵且右上角元素为 -1 、 C 的末列为 $x_i^n + 1$ ；正文的衰减条件只是「实节点 x_1 到 x_n 」，并未要求节点落在 $(0, 1)$ 内。

本页为书页 181，属 §III.3 正文；页眉与页码为重复元素，不译。

[\[section\]](#)

§III.3 快速衰减的奇异值 (Rapidly Decaying Singular Values, 续)

那些矩阵 A 与 B 当然是正规矩阵（normal matrix）。（这一要求本可以减弱，但此处没有必要。） B 的特征值等间距地分布在单位圆（unit circle）上。这些 λ 位于角度 $\pi/n, 3\pi/n, \dots, (2n-1)\pi/n$ 处，因此只要 n 为偶数，它们就不是实数。于是它们不会与 A 的实特征值（real eigenvalues） $x_1, \dots, x_n$ 相碰。而 C 的秩为 1。所以 A, B, C 检验已通过。

奇异值（singular values）的图像确认了 V 是高度病态的（highly ill-conditioned）。 K 也是如此。

Figure III.2: Vandermonde 与 Krylov 的奇异值图像，对比 V 与 $K = [b \ Ab \ \cdots \ A^{n-1}b]$ 的奇异值。这里 b 为全 1 向量， A_{ij} 为随机数（standard normal，标准正态）。当 $A = \text{diag}(1/n, 2/n, \dots, 1)$ 时 V 等于 K 。横轴为 $n = 5, 10, 15, 50$ ，纵轴为奇异值的对数尺度（ 10^0 到 10^{-15} ）。图片本身无法翻译，此处仅给出图注。

[note] 译者注 (OCR 修正)：本页 OCR 噪声较多，逐项复原如下：田， 盖 为节标题 III.3； and 复原为 and； λ ， s 复原为特征值 λ_k （希腊字母在 OCR 中常被识别成汉字）； $\pi / 3 \pi /$ ， $(2 - 1) \pi /$ 依单位圆等间距语境重建为角度 $2\pi(k-1)/n$ ； numbers 、 provided is even 、 the real 分别复原为 numbers、provided n is even、the real； r is "d 依上文 Sylvester 检验重建为「检验通过」； ylov 与 KryIov 复原为 Krylov； all ones and 万 = random numbers 依数学语境重建为「 b 为全 1 向量」与「 b 取随机标准正态」； $= \text{diag} \ (1 / , \ 2 / ,$ 重建为 $A = \text{diag}(1/2, 2/2, \dots, n/2)$ 。另外，图中 K 与 V 的曲线是两条对照曲线，不是同一个矩阵，图注按此排印。以下图注依上述修正排印。

[note] 译者注 (OCR 修正)：经原书扫描页核对： B 的特征值角度为 $\pi/n, 3\pi/n, \dots, (2n-1)\pi/n$ （即 $\lambda^n + 1 = 0$ 的根）； A 的实特征值为 $x_1, \dots, x_n$ ；图注中 $V = K$ 的条件是 $A = \text{diag}(1/n, 2/n, \dots, 1)$ ，而 A_{ij} 为随机标准正态；对数秩定义中的偏差是逐元素的 $|(C_n - E_n)_{ij}| < \epsilon$ ，秩条件是 $\text{rank}(E_n) < c(\log n)^q$ 。

[note] 译者注（为什么傅里叶情形是反例、实节点情形是正例）：要读懂这两个判定，得先看清 B 的身份。移位阵 B 的幂作用是不断把向量向上挪一格，它只有 0 这一个特征值。换一个方向看，把移位作用放到 Fourier 基（即离散 Fourier 矩阵 F_n 的列）下，它就对角化成单位根 $w^{k-1} = e^{-2\pi i(k-1)/n}$ ——这正是正文所说「特征值等间距分布在单位圆上、角度为 $2\pi(k-1)/n$ 」的来源。18.06 课用 F_n 对角化循环移位矩阵，走的就是这条「换基之后移位变对角」的路径；区别在于 18.06 关心的是对角化本身与快速变换，本节关心的是对角化之后能不能与 A 的谱撞车。

先补机制：这段在解决什么设计问题。问题是「可解性判定能不能只用谱来说话，以及用哪一套谱」。对移位阵这类**幂零**算子，它的谱是单点 $\{0\}$ ，而 0 又必然落在任何矩阵 A 的谱里吗？不必然，但一旦 A 有 0 特征值， $\sigma(A) \cap \sigma(B) \neq \varnothing$ 立刻成立，方程就不可解。更本质的是：相似变换（包括换到 Fourier 基）**不改变特征值**，所以换基绝不改变 B 的谱是 $\{0\}$ 这一事实，它只是把同一个谱换一个角度看。这就是为什么「傅里叶情形是反例」——被对角化的那个算子本身谱退化，你再怎么换基，它都退化；而「实节点情形是正例」则是说，当 B 来自实节点差分格式时它的谱不再挤压在一点上，于是与 A 的谱相撞的概率大幅降低，可解性判定自然给出「正」的答案。判断的对象始终是「谱集合是否相交」，不是「基好不好看」。

补一个可复算的 $n = 4$ 验算。取循环移位阵 $B = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ ， F_4 的第 j 列为 $v_j = \frac{1}{\sqrt{4}}(1, i^{j-1}, i^{2(j-1)}, i^{3(j-1)})$ 。直接验算第一列： $Bv_1 = v_1$ ，特征值 1；第二列 v_2 的元素是 $1, -i, -1, i$ ，乘 i 得 $i, 1, -i, -1$ ，而这恰是 Bv_2 各分量（ Bv_2 的第 k 个分量是 v_2 的第 $k-1$ 个分量），于是 $Bv_2 = i v_2$ ，特征值 i ；同法 $Bv_3 = -v_3$ ， $Bv_4 = -i v_4$ 。四个单位根 $1, i, -1, -i$ 在单位圆上等间隔 90° ，与 $2\pi(k-1)/n$ 一一对应，手算完毕。往下推一步：由 $|\lambda|^4 = 1$ 可知 λ 只能是四次单位根，而 n 次单位根集合 $\{1, w, \dots, w^{n-1}\}$ 对任意 n 都精确给出了整个谱，其中 $w = e^{-2\pi i/n}$ 。于是「等间距分布在单位圆上」不是修辞，而是「谱恰等于 $\{z : z^n = 1\}$ 」的几何翻译。

跨课程连接一条。6.042J 的循环群、同余与单位根代数，穿过 18.06 的 F_n 对角化，再到本节：共同点都是都在做「换基使算子简单」这件事，区别在于 6.042J 只关心单位根的代数性质（乘幂回到 1、 $w^{j+n} = w^j$ 的周期性），18.06 关心对角化与快速变换，而本节关心的是这些对角元**是否与 A 的谱相交**——判定目标从「变换本身」升级成「两族谱的互斥性」。再补一个反例对话：若取 $A = B$ 或 $A = B^T$ ，则 $\sigma(A)$ 恰好含有单位根，Sylvester 算子奇异，方程 $AX - XB = C$ 只对满足相容条件的 C 有解；这在实空间里就是周期边界条件下稳态方程必须满足零均值相容条件，与常微分方向上的 Fredholm 二择一结构同型。可命名的模板：**单位根碰撞检验法**——先算出 B 的谱是不是 $\{z : z^n = 1\}$ ，再判断它与 $\sigma(A)$ 是否有交；注意换基永远救不了谱退化，所以这个检验只需在谱上做一次，不必真的构造 F_n 。

于是两种情况分道扬镳。傅里叶情形： A 的特征值是 $1, 2, \dots, n$ ， B 的特征值是单位根，二者都含有 1，谱发生重叠，Sylvester 检验的前提被破坏——这正是 F 的全部奇异值彼此相等、没有一点衰减的谱解释（ F 的列两两正交且长度相同，所以奇异值全部相等）。实节点情形： B 仍是移位阵，特征值全为 0， A 的特征值为 $1, 2, \dots, n$ ，两组谱相距至少 1，良好分离，检验通过， V 的奇异值按指数速率衰减。一句话：**决定衰减快慢的不是节点本身，而是两组谱的最小间隙**；节点是否为实数只是改变这个间隙的一个开关。

[section]

改进的 Sylvester 检验（An Improved Sylvester Test）

要求 $AX - XB = C$ 具有低秩（low rank），这一条件比「结论说 X 具有快速衰减的奇异值」要严格得多。一个完美的定理应当使关于 C_n 的假设与关于 X_n 的结论彼此匹配。诚然，对于规模不断增大的 Vandermonde 或 Krylov 矩阵 X_n ，其秩并非有界。所以我们必须放宽对 C_n 的低秩要求，同时保持 X_n 的奇异值快速衰减。

Townsend 找到了这样的定理： C 的 log-rank（对数秩）性质可导出 X 的 log-rank 性质。而最近一篇与 Udell 合作的论文证明，对数秩是一种非常普遍的性质。下面是其定义（许多对数秩的例子取 $q = 0$ 或 $q = 1$ ）：

定义。一族矩阵（family of matrices） C_n 具有 log-rank（对数秩）性质，如果对邻近的矩阵（nearby matrices） E_n 有

$$\left| (C_n - E_n)_{ij} \right| < \epsilon, \quad \text{rank}(E_n) < c(\log n)^q \tag{6}$$

[note] 译者注（把「低秩」放宽成「对数秩」）：这页引入了本书低秩主线里最实用的一类放宽：**不要把秩当成固定的小整数，而要允许它随精度缓慢增长**。对数秩的定义是：存在 $\|C_n - E_n\|_2 < \epsilon$ 且 $\text{rank}(E_n) \leq c(\log n)^q$ 的邻近矩阵 E_n 。换成一句人话：只要允许秩按 $(\log n)^q$ 增长，就能把 C_n 逼近到给定精度 ϵ 。注意这里的「秩」是逼近出来的， C_n 自己的秩可以很大——Vandermonde 与 Krylov 恰恰如此，所以必须先放宽假设，定理才可能成立。

先说这个放宽在解决什么设计问题。低秩压缩的整个商业逻辑只有一条：**存储与运算成本由秩 r 决定，而不是由阶数 n 决定**。一个 $n \times n$ 的矩阵，若先存两个因子 $n \times r$ 与 $r \times n$ ，内存从 n^2 掉到 $2nr$ ；而矩阵一向量乘 $y = Cx$ 也不再需要 n^2 次乘法，而是先算 $z = V^T x$ （ $2nr$ 次），再算 $y = Uz$ （ $2nr$ 次），总共 $4nr$ 次。于是「成本是线性的还是平方的」这件事，完全被 r 相对 n 的增长速度决定。若 r 是固定常数，成本是 $O(n)$ ，这是最理想的情形；可现实里大量矩阵没有这个福气。对数秩就是在问：**如果不苛求常数秩，只要 $r \leq c(\log n)^q$ ，成本会退化到什么程度？** 答案好得出人意料： $2n \cdot c(\log n)^q$ 与 n^2 相比，当 n 增大时优势无限扩大，同时在数值上仍然可控——这就是「放宽一点假设，换回一整类可算对象」的典型交易。

为什么必须放宽？从第一性原理看，**秩是一个不稳定的整数**。它是一个计数，取的是「非零奇异值有多少个」，而任何矩阵只要被一个量级为 δ 的扰动碰到，原本为零的奇异值就可能被抬到 δ 的水平，于是秩从一个数跳到另一个数。这种「离散量对连续扰动敏感」的性质，让秩在浮点世界里几乎不可直接使用。数值分析的标准对策是把计数换成连续量：与其问「有多少个奇异值严格为零」，不如问「第 $r+1$ 个奇异值相对第一个有多小」，也就是比值 σ_{r+1}/σ_1 。对数秩正是这一置换的产物：它不主张「 C_n 的秩很小」，只主张「在精度 ϵ 的尺度上， C_n 的行为像秩为 $c(\log n)^q$ 的矩阵」。这样做还有一个结构上的好处：允许秩缓慢增长，就把「严格低秩」这个脆弱的类别，扩成一个对抗动封闭、对算法友好的类别，定理才不至于被一个 10^{-16} 的舍入误差击穿。

代入具体数值验算一次。取 $n = 10^6$ ，则 $\log n \approx 13.8$ （自然对数）。若取 $c = 1$ 、 $q = 1$ ，则秩上界约 14；若取 $q = 2$ ，约 190。现在比较内存：完整存 C_n 需要 $n^2 = 10^{12}$ 个元素，按双精度每个 8 字节算是 8 太字节（TB），单机根本放不下；而对数秩压缩只需 $2nr = 2 \times 10^6 \times 14 \approx 2.8 \times 10^7$ 个元素，约 0.22 吉字节（GB），一块普通显卡显存即可容纳；即便取 $q = 2$ 的 $r \approx 190$ ，也只有约 3 GB。再看一次矩阵一向量乘：直接算需要 $n^2 = 10^{12}$ 次乘法，按每秒 10^9 次算要 1000 秒；压缩后是 $4nr \approx 5.6 \times 10^7$ 次，约 0.06 秒，加速约 1.8×10^4 倍。**这就是「对数」二字的全部含金量：对数增长慢到几乎可以当成常数，所以在 n 大到现实量级时，对数秩矩阵在成本上仍然被当作低秩矩阵使用。**

还要交代一句：秩只是记账单位，不是最终指标。真正要盯的是两件事——「秩长到多大」与「算一次近似的代价是多少」。用奇异值截断（SVD 截断）得到最优秩 r 近似，代价是 $O(n^2r)$ ，当 r 随 $\log n$ 增长时仍是 $O(n^2 \log n)$ ，比 $O(n^3)$ 的满秩分解便宜一个 $n/\log n$ 的因子；而自适应交叉近似（adaptive cross approximation）这类只用部分元素、逐列选主元的算法，能把代价进一步压到 $O(nr^2)$ ，这才是对数秩在工程上真正兑现的地方。换句话说，对数秩不是一个纯存在性声明，它对应着一整族「只读一部分矩阵也能达到该精度」的算法，这也是它比「固定低秩」更适合作为定理假设的原因。

跨课程连接一条。课程 **6.042J 离散数学**里有等价关系与偏序：秩条件 $\text{rank}(E) \leq r$ 本身就在「所有 $n \times n$ 矩阵」这个集合上切出一个子集，而「 $\|C - E\|_2 < \epsilon$ 且 $\text{rank}(E) \leq r$ 」定义的是一族等价类，对数秩说的是「存在一个元素落在这族类里」。连接路径是：**6.042J 的等价关系给出「按精度分层」的语言，这里的对数秩给出一个随 n 缓慢放宽的层号**。区别在于：离散数学里的等价关系是精确的、二值的，两个对象要么等价要么不等价；而对数秩里的「邻近」是数值的、带阈值的， ϵ 一改，整个分层结构就跟着移动，所以不能直接套用离散的划分定理，只能借用它的表述方式。同样的味道也出现在课程 **CLRS / 6.006** 的分治复杂度分析里： $T(n) = 2T(n/2) + O(n)$ 给出 $O(n \log n)$ ，而这里的 $O(n^2 \log n)$ 是「把 $\log n$ 当作低阶修正」的同一种记账习惯；区别是分治的 $\log n$ 来自递归深度，对数秩的 $\log n$ 来自矩阵本身的可逼近结构，前者是算法产生的，后者是被逼近对象自带的。

可复用的模板：**对数秩成本核算法**。遇到一个声称「结构化」的大矩阵，先做三步：第一步写下你允许的精度 ϵ ；第二步问在该精度下秩能取到多少（ $\log n$ 还是 n ，量级差多少）；第三步把 n 、 r 代进 $2nr$ （内存）与 $4nr$ （一次乘法）两个公式与 n^2 对照，算清楚加速比。任何一段声称低秩的论证，只要第二步给不出可验证的秩界，它就没有兑现任何成本承诺。

对数增长为什么是决定性的，用 6.006 课（算法导论）的复杂度语言就能讲清： $n \times n$ 稠密矩阵要存 $O(n^2)$ 个数，而秩 $r = c(\log n)^q$ 的近似只需存因子 $U\Sigma V^T$ 中的 $O(nr)$ 个数。取 $q = 1$ 、 $n = 10^6$ ，有 $\log_2 n \approx 20$ ，于是 $O(n \log n)$ 远小于 $O(n^2)$ ，两者相差约五个数量级；即便取 $q = 2$ ， $(\log_2 n)^2 \approx 400$ ，仍比 n 小三个数量级。6.006 课训练的是「先估输入规模，再看时间与空间随规模的增长阶」；本节只是把同一把尺子从时间换成了存储：低有效秩不是审美偏好，而是问题能否放进内存的分界。区别在于 6.006 关心最坏情况的渐近阶，本节关心给定精度 ϵ 下实际需要的秩，二者都要先问「增长阶是多少」。

用 RBF 核做一次可算的验算。取一维等距点 $x_i = ih$ ，核元素只依赖 $i - j$ ，记 $K_{ij} = q^{(i-j)^2}$ ，其中 $q = e^{-h^2/(2\sigma^2)}$ 。取 $h^2/(2\sigma^2) = \ln 2$ ，即 $q = \frac{1}{2}$ ，则相隔 k 个格点的元素为 2^{-k^2} ： $k = 1$ 得 0.5， $k = 2$ 得 0.0625， $k = 3$ 得 1.95×10^{-3} ， $k = 4$ 得 1.53×10^{-5} ， $k = 5$ 得 3.05×10^{-8} 。若只要求逐元素误差不超过 10^{-6} ，保留 $|i - j| \leq 5$ 的对角带就已足够；被切掉的那些元素每个都小于 10^{-8} ，整条尾巴的总量也在 10^{-6} 量级。所以这条尾巴构成一个很「便宜」的 E_n ，而 $C_n - E_n$ 正是被切掉的尾巴——这就是定义中「邻近矩阵 E_n 秩很小」的实体版本。

这里要纠正一句容易说反的直觉：带宽固定时，带状矩阵的形式秩并不小（除极特殊情形外它是满秩的），真正小的是**数值秩**——奇异值快速衰减，衰减到某个精度以下就可以截断。对高斯核而言， σ 越小，核越尖，相邻格点的相关性衰减越快，达到同样精度所需的带宽与数值秩反而越低；代价是核变得近乎不光滑，能分辨的细节变少。于是超参数 σ 的取舍是这个样子： σ 大则核平滑、数值秩高、逼近更贵； σ 小则数值秩低、更省内存，但模型能表达的尺度也随之变粗。决定数值难度的依旧是衰减速率，而不是矩阵的规模。

它与概率论的联系让这件事变得可预期。 $K(x, y)$ 与高斯密度只差归一化常数： $\exp(-\|x - y\|^2/(2\sigma^2))$ 就是以 y 为中心、方差 σ^2 的钟形。6.041 课（概率论）里的 Chernoff 界告诉我们高斯尾概率按 $e^{-t^2/2}$ 衰减，这里看到的是同一指数落在矩阵元素上：距离超过几个 σ 的点贡献小到可以忽略，核矩阵自然「带形」、数值秩自然低。区别在于概率论用它控制一次随机试验的尾概率，本节用它控制一个确定的核矩阵中远离对角线的元素。

最后再拓展一步。既然核矩阵低有效秩，就可以显式构造特征映射：Rahimi 与 Recht（2007）的随机傅里叶特征用平移不变核的傅里叶表示，取 r 个随机频率近似它，得到 $z(x)$ 使 $z(x)^T z(y) \approx K(x, y)$ 。于是核技巧被换成显式低秩因子 $K \approx ZZ^T$ （ Z 为 $n \times r$ ）。采样误差按 $O(1/\sqrt{r})$ 衰减， $r = 100$ 时约一成， $r = 10^4$ 时约百分之一，常数依赖核的光滑程度——这与 6.041 里「样本量增大、估计误差按 $1/\sqrt{r}$ 缩小」是同一量级关系，区别只在于那里估的是概率，这里估的是一个函数值。这条路把「无限维特征空间」的抽象说法落到一个能算的 $n \times r$ 矩阵上，也直接接回本节主题：低有效秩不是巧合，而是可以被构造出来的。

[example]
Example 1 径向基函数核（radial basis function kernel）常用于支持向量机（support vector machines）：

$$\text{RBF Kernel} \quad K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

对一组特征向量（feature vectors） x_i 而言，这产生一个满矩阵（full matrix） K ，其元素满足 $0 < K_{ij} < 1$ 。把它们全部计算出来是不可能的。借助良好的近似，我们用核技巧（kernel trick）求解非线性分类问题（nonlinear classification problems），见 §VII.5。该矩阵具有低有效秩（low effective rank）。

1. M. D. Buhmann, *Radial basis functions*, Acta Numerica 9 (2000) 1–38. 2. B. Fornberg and N. Flyer, *A Primer on Radial Basis Functions*, SIAM (2015). 3. T. Hofmann, B. Schölkopf, and A. J. Smola, *Kernel methods in machine learning*, Annals of Statistics 36 (2008) 1171–1220（附大量参考文献）。

ADI 与 Zolotarev 问题 (ADI and the Zolotarev Problem)

用短短一段话，就能指出两个让 X 的奇异值快速衰减的想法。第一个是求解 Sylvester 方程 $AX - XB = C$ 的 ADI 迭代 (ADI iteration)，交替方向隐式算法 (Alternating Direction Implicit algorithm) 给出了计算上高效的解法。第二个想法把 A 与 B 的特征值 (前提是两组特征值不重叠) 与有理逼近 (rational approximation) 中的一个问题联系起来。

[note] 译者注 (第一性原理)： 正文把 ADI 迭代与「 A 、 B 的特征值不得重叠」绑在一起，这个限制不是顺手提，而是方程可解性的充要条件。从最基础的假设出发： $AX - XB = C$ 里未知的 X 有 n^2 个分量，所以它本质上是一个 n^2 元的线性方程组。把 X 按列拉直成 $\text{vec}(X)$ ，用恒等式 $\text{vec}(AXB) = (B^T \otimes A) \text{vec}(X)$ ，原方程等价于 $(I \otimes A - B^T \otimes I) \text{vec}(X) = \text{vec}(C)$ 。这里的 $\otimes$ 不需要任何新代数，它只是分块矩阵的记账法： $I \otimes A$ 把 n 个 A 摞在对角线上， $B^T \otimes I$ 把 I 的每个块乘以 B^T 的对应元素。关键在于这个 $n^2 \times n^2$ 矩阵的谱可以白拿： $I \otimes A$ 的特征值是 $\lambda_i(A)$ (每个重复 n 次)， $B^T \otimes I$ 的特征值是 $\mu_j(B)$ ，两块可以同时对角化，所以差的特征值就是 $\lambda_i(A) - \mu_j(B)$ 。于是「特征值不重叠」等价于这个矩阵没有零特征值，等价于它可逆，等价于对任意 C 存在唯一解——这就是正文那句话的全部内容。比「能不能解」更有用的问题是「解得准不准」，答案同样写在这组谱差里：敏感度由谱差绝对值的最大值与最小值之比 κ 控制。验算一次：取 $A = \text{diag}(2, 3)$ 、 $B = \text{diag}(0, 5)$ ，谱差为 $2, -3, 3, -2$ ， $\kappa = 3/2 = 1.5$ ，非常好；把 B 换成 $\text{diag}(0, 3 - \varepsilon)$ ，谱差里立刻冒出一个 ε ， $\kappa \approx 3/\varepsilon$ ，取 $\varepsilon = 10^{-6}$ 得 $\kappa \approx 3 \times 10^6$ 。回到课程「计算机系统基础」(CSAPP) 第 2 章的浮点结论：双精度 (double) 约有 16 位十进制有效数字， $\kappa = 10^6$ 时只剩 10 位可信， $\kappa = 10^{10}$ 时答案全是舍入噪声。所以「特征值接近」不是小事，它直接吃掉精度。这还顺带解释了 ADI 存在的理由：对拉直后的 n^2 元方程组直接做 LU 是 $O(n^6)$ 的，而 ADI 每一步只解两个 $n \times n$ 的移位方程组。移位参数的作用，就是决定「用哪些 $\lambda_i - \mu_j$ 去做除法」，从而决定误差被放大还是被压缩——这正是本页下半部分 Zolotarev 有理函数的用武之地。可复用的模板：**向量化—谱差检验法**。遇到 $AX \pm XB = C$ 、 $AXB = C$ 这类矩阵方程，第一步一律拉直成向量方程，第二步把算子谱写成已知谱的和与差，第三步用「有没有零特征值、谱分离比多大」回答可解性与条件数。课程 18.065 里的 Lyapunov 方程 $AX + XA^T = C$ 走的是同一套推理：谱变成 $\lambda_i + \lambda_j$ ，可解条件相应变成「任意两个特征值之和不为零」。

补一句最容易踩空的地方。上面的「不重叠」有两层含义，必须分清：如果某个 $\lambda_i - \mu_j$ 恰为零，矩阵奇异，此时不是「解不出来」，而是「要么无解、要么有无穷多解」——有解的充要条件是 $\text{vec}(C)$ 与那个零特征值对应的左特征向量正交，这就是课程 **Strang 线性代数** 里「 $Ax = b$ 有解当且仅当 b 垂直于 A^T 的零空间」那条判据在矩阵方程上的原样翻版。此处的连接路径是：**把 $n^2 \times n^2$ 的算子当成一个普通的方阵，直接套用四个基本子空间的正交分解**；区别在于，普通线性代数课里零空间是抽象的，而这里零空间有明确的物理身份——它由「 A 的特征向量」与「 B^T 的特征向量」做张量积构成，所以病态是可以定位到具体的那一对 (λ_i, μ_j) 上去的。再看「接近但不相等」这一层：此时矩阵可逆，但条件数 κ 随谱分离度倒数增长，于是求解精度按 κ 打折。可以把两条合成一句工程判据先算谱再选算法：先估计 $\min_{i,j} |\lambda_i - \mu_j|$ 与 $\max_{i,j} |\lambda_i - \mu_j|$ 的比值，若它接近 1，直接移位求解既快又准；若它小到 10^{-6} 量级，就必须改用 ADI 并认真挑选移位参数，否则浮点误差会淹没答案。这正是课程 **凸优化 (Boyd)** 里「先看条件数再决定用不用预条件」的同一种工程直觉；区别是凸优化里的预条件是为了加快收敛，而这里的移位参数同时承担两个职责——既改变条件数、又构造出可快速求解的移位方程组，一个旋钮管两件事，这也是 ADI 难调的原因。

所谓 Zolotarev 问题 (Zolotarev problem)，就是要找一个多项式之比 $r(x) = p(x)/q(x)$ ，使 $r(x)$ 在 A 的特征值处很小、在 B 的特征值处很大。用有理函数 $r(x)$ 来逼近，效果可以比用多项式逼近好得多——好到指数级；纽曼 (Newman) 对绝对值函数 $|x|$ 的逼近就是一个著名的例子。 $r(x)$ 的指数级精度，就这样与 X 的奇异值的指数衰减联系了起来。

用 ADI 求解 $AX - XB = C$ (ADI solves $AX - XB = C$)

$$\begin{aligned} X_{j+1/2}(B - p_j I) &= C - (A - p_j I)X_j \\ (A - q_j I)X_{j+1} &= C - X_{j+1/2}(B - q_j I) \end{aligned}$$

矩阵 $X_{j+1/2}$ 与 X_{j+1} (Matrices $X_{j+1/2}$ and X_{j+1})

那个好的有理函数 $r(x) = p(x)/q(x)$ ，分子里的零点是它的根 (roots)，分母里的零点是它的极点 (poles)。正是 Zolotarev 在 1877 年 (!) 于一个模型问题中找到了最优的那组 p 与 q 。把 A 、 B 放进 Sylvester 检验 (Sylvester test)， $\sigma_{1+kr}(X)$ 的界就是那个「Z-number (Z-数)」乘以 $\sigma_1(X)$ ——这正是奇异值的指数衰减。汤森 (Townsend) 与福图纳托 (Fortunato) 把这一想法发展成了正方形区域上的超快泊松求解器 (superfast Poisson solver)。当 X 取通常的五点有限差分近似 (5-point finite difference approximation)、用来逼近 $\partial^2 u / \partial x^2 + \partial^2 u / \partial y^2$ 时，快速求解器 (fast solvers) 早已为人所知；他们的目标是一个复杂度最优的谱方法 (spectral method)。

[note] 译者注 (OCR 修正与 Z-number 的含义)： 本页 OCR 噪声较多，先交代复原依据：由 at 复为 that，tø / øf / øn / rati ø / Zøtøtarev 分别复为 to / of / on / ratio / Zolotarev，— XB = C 复为 $AX - XB = C$ ，reperred 复为 required，ra 五 ø a / a ø 觐 复为 rational approximation，ß 复为 $|x|$ ，灭 依上下文复为 X ，⌘ 复为 poles。有两处不能照字形硬搬。第一处是脚注 1 的出处：e s e Ma e 85 (2000) 553-577 说的是 Beckermann 那篇论文，正确刊名与卷页是 Numer. Math. (Numerische Mathematik) 第 85 卷 (2000) 553–577，而不是 Electron. Trans. Numer. Anal. (ETNA)；同一行的 10v，也不是人名 Löw，而是篇名里的 Krylov——原题名是「实 Vandermonde、Krylov 与正定 Hankel 矩阵的条件数」。第二处是 the bound on ø1 + () is the Z-number times ø1 (X)：原书此处印的是 $\sigma_{1+kr}(X)$ 是 Z-number 乘以 $\sigma_1(X)$ ，紧接着的 which means exponential decay of singular values 才站得住——相邻下标之间的乘法递推，才配得上「指数衰减」四个字，其中的 Z 就是 Zolotarev 数 (Z-number)。另有一处补正：本节第一段的比值应写作 $r(x) = p(x)/q(x)$ ，自变量是 x ，原译文误作 $r(c) = p(c)/q(c)$ ，此处已按原书改正。还有一处 OCR 整块丢失已按原书补回：正文显示的 ADI 两个半步迭代式 $X_{j+1/2}(B - p_j I) = C - (A - p_j I)X_j$ 与 $(A - q_j I)X_{j+1} = C - X_{j+1/2}(B - q_j I)$ ，以及承接它们的标题「矩阵 $X_{j+1/2}$ 与 X_{j+1} 」，均已补入正文。再说一条与课程「数学分析」直接相连的线索：正文提到纽曼 (Newman) 对

$|x|$ 的逼近，比较的是有理逼近与多项式逼近。在「数学分析」的一致逼近部分， $|x|$ 在 $[-1, 1]$ 上只有利普希茨（Lipschitz）光滑度，用多项式逼近，误差只能按 $1/n$ 的速度下降；换成有理函数，绍曼在 1964 年证明误差可以按 $e^{-\pi\sqrt{n}}$ 的速度下降。连接路径是让分母的极点去吸收 $|x|$ 在原点处的折点，区别在于多项式没有极点可用——这正是 Zolotarev 问题在数值线性代数里被反复引用的原因。

1. B. Beckermann, *The condition number of real Vandermonde, Krylov, and positive definite Hankel matrices*, Numer. Math. 85 (2000) 553–577. 2. B. Beckermann and A. Townsend, *On the singular values of matrices with displacement structure*, SIAM J. Matrix Anal. Appl., arXiv:1609.09494v1, 29 Sep 2016. 3. P. Benner, R.-C. Li, and N. Truhar, *On the ADI method for Sylvester equations*, J. Comput. Appl. Math. 233 (2009) 1035–1045. 4. D. Fortunato and A. Townsend, *Fast Poisson solvers for spectral methods*, arXiv:1710.11259v1, 30 Oct 2017. 5. D. Ruiz-Antolin and A. Townsend, *A nonuniform fast Fourier transform based on low rank approximation*, arXiv:1701.04492, SIAM J. Sci. Comput. 40:1 (2018). 6. A. Townsend and H. Wilber, *On the singular values of matrices with high displacement rank*, arXiv:1712.05864, Linear Alg. Appl. 548 (2018) 19–41. 7. A. Townsend, www.math.cornell.edu/~townsend/presentations/LowRankMatrices.pdf 8. M. Udell and A. Townsend, *Nice latent variable models have log-rank*, arXiv:1705.07474v1, SIAM J. Math. Data Sci., to appear.

Strang §III.3, p.182

[section]

Problem Set III.3（习题集 III.3）——§III.3 快速衰减的奇异值（Rapidly Decaying Singular Values）

[problem]

Problem 1. Verify that a Krylov matrix $K = [b \ Ab \ \dots \ A^{n-1}b]$ satisfies a Sylvester equation $AK - KB = C$, with B as in equation (6). Find the matrix C .

[note] 译者注（OCR 修正与逐列验算）： OCR 把 Krylov 矩阵写成 $K = [b \ Ab \ \dots \ A^{n-1}b]$ ，按标准定义复原为 $K = [b \ Ab \ \dots \ A^{n-1}b]$ ，共 n 列。答案是秩 1 矩阵。把 AK 与 KB 逐列摆开，结果立刻现形： AK 的列是 $Ab, A^2b, \dots, A^nb$ ；式 (6) 里的 B 是移位矩阵，作用是把每一列右移一格、最后一列清零，所以 KB 的列是 $Ab, A^2b, \dots, A^{n-1}b, 0$ 。两者只有最后一列不同，于是 $C = (A^nb) e_n^T$ ：最后一列是 A^nb ，其余元素全为 0，秩为 1，远小于 n ——这就叫 Krylov 矩阵通过了 Sylvester 检验。为什么值得做？低秩的 C 正是位移结构的标志： K 与「沿位移方向推一格」之后的自己只差一个秩 1 矩阵，而位移秩小意味着奇异值衰减快，这正是把 Krylov 矩阵与习题 4 的 Hankel 矩阵串起来的那根线。作为对照：若把 B 换成伴随矩阵（把特征多项式的系数放进最后一列），同样的逐列相减会因 Cayley–Hamilton 定理得到 $C = 0$ ，秩更低，「检验通过」的结论不变。可复用的动作是**逐列相减，找出被移位打乱的那一列**。整个计算只用矩阵乘向量，不需要任何 SVD；与课程「数据结构」用稀疏结构省存储的思路同源，这里省下的不是内存，而是「证明奇异值衰减」所需的计算量——那边用结构代替全体元素，这边用结构代替实测 SVD，代价是必须先把位移方程猜对。

[problem]

Problem 2. Show that the evil Hilbert matrix H passes the Sylvester test $AH - HB = C$:

$$H_{ij} = \frac{1}{i+j-1}, \quad A = \frac{1}{2} \text{diag}(1, 3, \dots, 2n-1), \quad B = -A, \quad C = \text{ones}(n)$$

[note] 译者注（OCR 修正与因子核对）： OCR 把这一题的三件数据压成了一行 $A = \frac{1}{2} \text{diag}(1, 3, \dots, 2n-1)$ 、 $B = -A$ 、 $C = \text{ones}(n)$ ，按原书复原为 $A = \frac{1}{2} \text{diag}(1, 3, \dots, 2n-1)$ 、 $B = -A$ 、 $C = \text{ones}(n)$ 。这里必须提醒一处容易抄错的因子与符号：原译文曾把它写成 $A = -\text{diag}(1, 3, \dots, 2n-1)$ 与 $C = -2 \text{ones}$ ，两处都需按原书改正。验算以 $H_{jk} = 1/(j+k-1)$ 与 $A = \frac{1}{2} \text{diag}(1, 3, \dots, 2n-1)$ 代入，逐项相减得 $(AH - HB)_{jk} = \frac{2j-1}{2(j+k-1)} + \frac{2k-1}{2(j+k-1)} = \frac{2(j+k-1)}{2(j+k-1)} = 1$ 。右端确实是常数矩阵，且常数恰为 +1（即 $\text{ones}(n)$ ）；把因子 2 或符号改掉，等式就不再成立。为什么希尔伯特矩阵要这样「过一遍筛子」？因为 A 的特征值是 $\frac{1}{2}, \frac{3}{2}, \dots, \frac{2n-1}{2}$ ：一组全正， $B = -A$ 一组全负，被 0 干净地隔开，Sylvester 检验的两条要求（谱不重叠、右端低秩）同时满足，于是立刻得到 H 的奇异值按谱分离比指数衰减。别看 H 的条件数在 $n = 12$ 时已经到了 10^{16} 量级、直接求解毫无意义，它的位移秩却只有 1——「数值上很坏」与「结构上很好」可以长在同一个矩阵身上，这正是低秩路线存在的理由。顺带说明「evil」的含义：随着 n 增大，元素 $1/(i+j-1)$ 彼此越挤越近，相邻元素的差会低于双精度能够分辨的间隔，矩阵在浮点意义下逐渐退化成低秩的影子。可复用的动作：把对角矩阵参与的两个乘积写成逐项减法（每一项只乘一个数），再看右端能否约成与 i, j 都无关的常数。

[problem]

Problem 3. A Toeplitz matrix T has constant diagonals (IV.5). Compute $AT - TA^T = C$:

$$T = \begin{bmatrix} t_0 & t_{-1} & \cdot & \cdot \\ t_1 & t_0 & t_{-1} & \cdot \\ \cdot & t_1 & t_0 & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \quad A = \begin{bmatrix} 0 & 0 & 0 & \cdot \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \cdot & 0 & 1 & \cdot \end{bmatrix} \quad B = A^T$$

[note] 译者注 (OCR 修正与反对称性)：OCR 只留下 $t_0 \rightarrow 1 / 0 \ 0 \ 0 / 1 \ 0 \ 0 \ 0 / 0 \ 1 \ 0 \ 0$ 这类错位碎片，显示矩阵已无法逐字复原，因此按常数对角线与移位矩阵的定义重建： T 的位移对角线元素是 t_0, t_{-1}, t_1 ， A 是下移位矩阵，并取 $B = A^T$ 。请注意这里与原译文的区别：本题要算的是 $AT - TA^T$ ，而不是 $A - A^T$ ；后者只在 $T = I$ 时才与本题重合，一般的 Toeplitz 矩阵必须让 A 从左边乘 T 、 A^T 从右边乘 T 再相减。逐项写出即 $(AT - TA^T)_{jk} = t_{k-j-1} - t_{j-k+1}$ ，于是结果 C 的三条非零对角线元素分别是 t_{-1} 、 t_1 与 $t_2 - t_{-2}$ 一类「相邻对角线元素之差」。这个结果有两个立刻可用的性质：一是 C 的对角线全零，位移把对角信息消掉；二是当 T 对称 ($t_{-k} = t_k$) 时对角线下方的元素互为相反数， C 反对称，特征值全为纯虚数。取配对 (A, A^T) ，则 $AT - TA^T$ 恰好就是本题的 C ，于是 Toeplitz 矩阵也被纳入了同一套语言：矩阵与它沿移位方向平移之后的差别，就是它相对于配对 (A, A^T) 的位移。最后提醒一处边界情形： A 与 A^T 的特征值完全相同，谱不重叠这个条件在这里失效， $X = I$ 之所以可行，只是因为右端正好落在值域里——这正是「检验」二字的边界。

[problem]

Problem 4. A Hankel matrix H has constant *antidiagonals*, like the Hilbert matrix. Then H_{ij} depends only on $i + j$. When H is symmetric positive definite, Beckermann and Townsend show that $H = \overline{K}^T K$ for a Krylov matrix K (as in Problem 1 above). Then $\sigma_j(H) = |\sigma_j(K)|^2$ (why?) and the singular values of H decay quickly.

[note] 译者注 (OCR 修正与跨课程连接)：OCR 把这一题压得只剩骨架，先复原：Hi 复为 H_{ij} ，t + 复为 $i + j$ ，H = K K 依对称正定的假设复为 $H = \overline{K}^T K$ ，(H) = 《丐 (月 2 依 $H = \overline{K}^T K$ 复为 $\sigma_j(H) = |\sigma_j(K)|^2$ (丐是 σ 、月是 K 的误识)。注意这里带共轭：原书写的是 $\overline{K}^T K$ 而不是 $K^T K$ ，下标也是 H_{ij} 而不是 H_{jk} ，原译文两处都已按原书改正。习题 4 藏着一个可以复用的判据。要证明一个矩阵奇异值衰减快，直接算 SVD 的代价是 $O(n^3)$ ，而且算完也看不出「为什么快」；本书的做法是找一对 A 、 B 和一个低秩的 C ，使 $AH - HB = C$ 成立。这里的 H 是 Hankel 矩阵（常数反对角线）， C 是常数矩阵（习题 2 里是 $\text{ones}(n)$ ），秩为 1。低秩的 C 本身并不说明 H 低秩——它说明 H 有位移结构： H 与它的位移版本只差一点。课程 18.065 讲低秩近似时，秩描述的是「信息量」；这里秩被用来描述「结构复杂度」，同一个工具换了一个用法。汉克尔 (Hankel) 矩阵与克里洛夫 (Krylov) 矩阵的关系是这条链上最漂亮的一环：对称正定的 Hankel 矩阵可以写成 $H = \overline{K}^T K$ ，其中 K 是习题 1 里的 Krylov 矩阵 $[b \ Ab \ \cdots \ A^{n-1}b]$ 。于是奇异值被平方： $\sigma_j(H) = \sigma_j(\overline{K}^T K) = |\sigma_j(K)|^2$ 。若 $\sigma_j(K)$ 按 ρ^j 衰减， $\sigma_j(H)$ 就按 ρ^{2j} 衰减，指数衰减的指数直接翻倍。这也解释了为什么 Hilbert 矩阵虽然「坏」，却是低秩近似的理想对象：它的奇异值掉得极快，真正有效的秩很小。代价藏在同一个平方里，这正是与课程「计算机系统基础」（CSAPP）那条线的连接点：条件数同样被平方， $\kappa_2(H) = \kappa_2(K)^2$ 。CSAPP 第 2 章讲过浮点：双精度 (double) 的相对精度约为 $2^{-53} \approx 1.1 \times 10^{-16}$ ，一次条件数为 κ_2 的线性求解最多保留约 $16 - \log_{10} \kappa_2$ 位十进制有效数字。希尔伯特 (Hilbert) 矩阵在 $n = 12$ 时 κ_2 已在 10^{16} 量级， $n = 20$ 时远超它——此时元素 $1/(i + j - 1)$ 之间的差别在双精度里已经分辨不出，任何直接求解得到的解都毫无意义。所以「用结构证明奇异值快衰减」在工程上不是锦上添花，而是绕过浮点天花板的出路：我们不去解 $Hx = b$ ，只用「前面的奇异值很大、后面的可以丢」这一事实。可复用的模板：**低秩残差证书法**。(1) 猜一个低秩的 C ，验证 $AH - HB = C$ ；(2) 用 A 、 B 的谱分离度给出 $\sigma_j(H)$ 的衰减界；(3) 用衰减界决定截断秩，而不是用 $O(n^3)$ 的 SVD 去实测。证书是结构性的，因此可以在只知道矩阵生成规则、不知道具体元素的情况下先用起来——这正是后面处理五点差分、Toeplitz、Hankel 这些矩阵时的标准动作。

[problem]

Problem 5. A Pick matrix has entries $P_{jk} = (s_j + s_k)/(x_j + x_k)$ where $x = (x_1, \dots, x_n) > 0$ and $s = (s_1, \dots, s_n)$ can be complex. Show that $AP - P(-A) = s \mathbf{1}^T + \mathbf{1} s^T$ where $\mathbf{1}^T = [1 \ 1 \ \cdots \ 1]$ and $A = \text{diag}(x_1, \dots, x_n)$. A has positive eigenvalues, $B = -A$ has negative eigenvalues, and the Sylvester test is passed.

[note] 译者注 (OCR 修正与秩的核算)：OCR 只剩 𐄂 = (力 +) / (与 + (k) 与 s1T + 1sT 这样的碎片，按 Pick 矩阵的通行写法复原为 $P_{jk} = (s_j + s_k)/(x_j + x_k)$ ，并补回本题隐含的 $A = \text{diag}(x_1, \dots, x_n)$ ：若取别的对角矩阵，下面的恒等式就不成立，所以这不是可选项，而是被恒等式唯一确定的。请注意原译文的两处偏差：变量名用的是 s 而非 y ， $A = \text{diag}(x_1, \dots, x_n)$ 用的是 x 而非 y ；且原书说的是 $s = (s_1, \dots, s_n)$ 是一个可以取复数的向量，不是对角矩阵， A 才是那个对角矩阵。验算只需一行： $(AP - P(-A))_{jk} = x_j P_{jk} + P_{jk} x_k = (x_j + x_k) \cdot \frac{s_j + s_k}{x_j + x_k} = s_j + s_k$ ，写成矩阵就是 $s \mathbf{1}^T + \mathbf{1} s^T$ ——因为 $(s \mathbf{1}^T)_{jk} = s_j$ 、 $(\mathbf{1} s^T)_{jk} = s_k$ ，两个秩 1 矩阵一加，右端秩最多为 2，与 n 无关。要点在于： A 与 $B = -A$ 的作用只是给 P 的第 j 行、第 k 列各乘一个数，乘完再相减，分母 $x_j + x_k$ 被整块约掉。于是「 A 取正特征值、 $B = -A$ 取负特征值」这一对自动满足 Sylvester 检验的两条要求：两组谱被 0 分开，右端低秩。这种「先对角缩放、再看差」的结构在 Cauchy 型矩阵（分母形如 $x_j + x_k$ ）里反复出现，是位移结构家族的一员；原书提示 s_j 允许取复数，而上面的逐项计算是纯代数的，复数情形一字不改地成立。

[problem]

Problem 6. If an invertible matrix X satisfies the Sylvester equation $AX - XB = C$, find a Sylvester equation for X^{-1} .

[note] 译者注 (OCR 修正与求逆后的秩)：OCR 的 matrix 复为 matrix， $\rightarrow 1$ 依语境复为 X^{-1} 。本题只用一次「左右同时乘」，结论却很有价值：记 $Y = X^{-1}$ ，把 $AX - XB = C$ 左乘 X^{-1} 、右乘 X^{-1} ，左边成为 $YA - BY$ ，右边成为 YCY ，整理得 $YA - BY = YCY$ ，等价地 $BY - YA = -YCY$ 。要如实说明一点：这里的右端不再是与 Y 无关的常数矩阵，所以它不是通常意义下「右端给定」的 Sylvester 方程，而是右端含未知量 Y 的同型方程——把它写成位移形式才是本意。价值在于秩： $\text{rank}(YCY) = \text{rank}(C)$ ，因为左乘、右乘可逆矩阵都不改变秩。也就是说， X^{-1} 与它的位移版本之差仍然是低秩的，位移结构可以被求逆继承，位移秩不会因为求逆而变大。这在数值上有直接后果：Hilbert 型、Hankel 型矩阵求逆之后仍然适合用低秩方法处理，不必重新去实测 SVD。可复用的动作：方程里出现 X^{-1} 时，一律先左乘 X^{-1} 、再右乘 X^{-1} ，把所有 X 换成 Y ，最后清点右端的秩——秩不变，结论就保住了。

[problem]

Problem 7. If $A = Q\Lambda\overline{Q}^T$ has complex orthogonal eigenvectors $q_1, \dots, q_n$ in the columns of Q , verify that $\overline{A}^T A = A\overline{A}^T$: then A is *normal*. The eigenvalues can be complex.

[note] 译者注 (OCR 修正与正规性) : OCR 把这一题揉成一团: = Q $\Lambda\overline{Q}^T$, θ 。 。 , qn in the column s of Q 复为「复正交特征向量 $q_1, \dots, q_n$ 在 Q 的各列」, v that = : “ is normal 复为「证明 $\overline{A}^T A = A\overline{A}^T$, 于是 A 是正规的」。这里必须先纠正原译文的一处概念错位: 特征值放在对角矩阵 Λ 里, Q 的各列才是特征向量, 不能把两者都说成「在 Q 的列中」。第二处是公式形式: 原书写的是 $A = Q\Lambda\overline{Q}^T$ (等价地 $A = Q\Lambda Q^{-1}$, $\overline{Q}^T = Q^{-1}$ 即 Q 为复正交矩阵), 而不是 $A = Q\Lambda Q^{-1}$ 之后再另设 $Q^T Q = I$; 要验证的等式也应是 $\overline{A}^T A = A\overline{A}^T$ 。数学内容是: 若 Q 复正交 ($\overline{Q}^T Q = I$, 即 $\overline{Q}^T = Q^{-1}$), 则 $A = Q\Lambda\overline{Q}^T$; 当 Λ 取复特征值时, 正规性要用共轭转置检验, $\overline{A}^T = Q\Lambda^H\overline{Q}^T$ (因为 $\overline{\overline{Q}^T} = Q^H$, 而 $Q^H = Q^{-1}$ 故 $(Q^H)^{-1}$ 又还原为 Q 的共轭转置形式), 于是 $\overline{A}^T A = Q\Lambda^H\Lambda\overline{Q}^T$ 、 $A\overline{A}^T = Q\Lambda\Lambda^H\overline{Q}^T$; 对角矩阵之间乘法可交换, $\Lambda^H\Lambda = \Lambda\Lambda^H$, 故 $\overline{A}^T A = A\overline{A}^T$, A 正规。如果 Λ 恰好是实对角矩阵, 共轭转置退回普通转置, $Q\Lambda Q^T$ 就是对称矩阵, 这是正规矩阵里最熟悉的一个特例。关键区别在于: 实对称矩阵要求特征值全实, 正规矩阵不要求——只要特征向量能取成一组正交基, 特征值就有资格跑到复平面上。这一题也是习题 8 的铺垫。

[problem]

Problem 8. If $S^T = S$ and $Z^T = -Z$ and $SZ = ZS$, verify that $A = S + Z$ is normal. Since S has real eigenvalues and Z has imaginary eigenvalues, $A = S + Z$ probably has complex eigenvalues.

[note] 译者注 (OCR 修正与笛卡尔分解) : OCR 只剩 $Z^T = -Z$ 与 $S = S + Z$, 复为 $Z^T = -Z$ 、 $A = S + Z$, 其中 S 对称、 Z 反对称, 且两者可交换。原译文把 A 记作了 N , 此处按原书统一改回 A 。证明只有三行: $A^T = S^T + Z^T = S - Z$; 于是 $AA^T = (S + Z)(S - Z) = S^2 - SZ + ZS - Z^2 = S^2 - Z^2$ (用 $SZ = ZS$ 把中间两项消掉), 同理 $A^T A = (S - Z)(S + Z) = S^2 - Z^2$, 两者相等, A 正规。可交换这个条件不是装饰: 少了它, $AA^T - A^T A = 2(ZS - SZ)$, 一般不为零, A 也就不正规。为什么特征值会跑到复数上? S 对称, 特征值全实; Z 反对称, 特征值全是纯虚数; 两者可交换时可以同时对角化, 于是 A 的特征值逐个写成「实数加纯虚数」的形式, 一般落在复平面上, 而且 A 能被一组正交特征向量对角化——这正是正规矩阵的谱定理。工程上这叫笛卡尔分解: 任何方阵都能拆成对称部分 $\frac{A+A^T}{2}$ 与反对称部分 $\frac{A-A^T}{2}$, 可交换时这两块互不打扰, 实部与虚部各管一半。可复用的动作: 验证正规性时不要展开矩阵乘法, 先看 A^T , 再比较 AA^T 与 $A^T A$ 的对称写法。

[problem]

Problem 9. Show that equation (3) for U_c follows from equation (2) for A .

[note] 译者注 (OCR 与推理动作) : 题干中的 U_c follows from 复为 U_c follows from, 原书此处写作「对 A 的式 (2)」而不是原译文所写的「对 U 的式 (2)」, 已按原书改正。本题引用的式 (2) 与式 (3) 都在上一页 (书页 181), 本页 OCR 不含这两式, 所以译者注不臆测它们的具体内容, 只把「这类题怎么想」说清楚。所谓「对 U_c 的式 (3) 由对 A 的式 (2) 推出」, 标准做法是三步。第一步数维度: 确认 A 与压缩后的 U_c 各有几个自由度, 弄清下标 c 是压缩指标 (通常是 column, 即从 U 的列中挑出的那一部分)。第二步机械替换: 把式 (2) 中每个符号按同一规则换成带下标 c 的版本——矩阵换成子矩阵、投影换成对相应子空间的投影、内积换成对应的分量和——替换后等号左边必须与式 (3) 逐项对齐。第三步才是真正的证明负担: 检查替换过程中用到的性质 (正交性、幂等性、列空间的包含关系) 在压缩之后是否依然成立, 因为压缩通常会改变零空间, 却往往保留列空间, 而式 (2) 的推导恰恰只依赖列空间一侧。可复用的动作: 把「由式 (2) 推出式 (3)」翻译成「替换符号 + 验证性质是否存活」两件事, 先机械对齐, 再逐条检查。做这道题时请把书页 181 摊在旁边逐项对照, 本页只保留题干原文。

Strang §III.3, p.183

Strang §III.4, p.184 · 页眉: Low Rank and Compressed Sensing

§III.4 Split Algorithms for $\ell^2 + \ell^1$ (针对 $\ell^2 + \ell^1$ 的分裂算法)

这些主题确实极为重要, 值得为它们专门写一整本书。它们包括 basis pursuit (基追踪) 与 LASSO optimization (LASSO 优化) ——再加上 §III.5 中的 matrix completion (矩阵补全) 与 compressed sensing (压缩感知)。我们在此处所能做的, 是介绍其中的基本思想以及若干成功的算法。

先从线性系统 $Ax = b$ 说起。假设 A 的列数远多于行数 ($m \ll n$)。于是 $Ax_n = 0$ 有许多解 (A 有一个很大的 nullspace (零空间))。若 $Ax = b$ 有一个解, 那么每个 $x + x_n$ 都是另一个解。该选哪一个呢?

要最小化 x 的 ℓ^2 范数, 我们就去掉它的零空间分量。这样留下的, 就是来自 A 的 pseudoinverse (伪逆) 的 minimum norm solution (最小范数解) $x^+ = A^+b$ ——也就是 §II.2 中利用 SVD 得到的那个解。可是 x^+ 一般会有许多很小的分量, 因而可能极难解释。在 MATLAB 中, pinv(A) * b 求出 x^+ ; 反斜杠 A \ b 求出一个相当 sparse (稀疏) 的解——但很可能不是基追踪意义下的最优解。

一个含许多零分量的 $Ax = b$ 的解 (若它存在的话) 来自最小化 ℓ^1 范数而非 ℓ^2 范数:

Basis pursuit

Minimize $\|x\|_1 = |x_1| + \dots + |x_n|$ subject to $Ax = b$.

(1)

这是一个 convex optimization problem (凸优化问题), 因为 ℓ^1 范数是 x 的凸函数。 ℓ^1 范数是分段线性的, 这一点与 ℓ^2 范数以及除 $\|x\|_\infty = \max |x_i|$ 之外的所有其他 ℓ^p 范数都不同。基追踪不是靠 A 的 SVD 求解的——SVD 连接的是 ℓ^2 范数。但 ℓ^1 因能给出 sparse solutions (稀疏解) 而闻名, 如今人们已经找到了快速的算法。

在 $Ax = b$ 的所有解里，最稀疏的那个最小化 $\|x\|_0$ ，也就是 x 的非零分量的个数。可是这并不是一个真正的 norm（范数）： $\|2x\|_0 = \|x\|_0$ 。满足 $\|x\|_0 = 1$ 的向量只有一个非零分量，它们沿着坐标轴分布，例如 $i = (1, 0)$ 与 $j = (0, 1)$ 。于是我们把 ℓ^0 问题「放松」或称凸化，以得到一个真正的范数——而这个范数就是 ℓ^1 。满足 $\|x\|_1 = |x_1| + |x_2| \leq 1$ 的向量填满一个 diamond（菱形），它的四个顶点落在 $\pm i$ 与 $\pm j$ 处。

[note] 译者注（第一性原理）：我们真正想要的解是最稀疏的解，也就是非零分量最少的那个，即 $\|x\|_0$ 最小。可惜 $\|x\|_0$ 不是范数：范数公理要求齐次性 $\|cx\| = |c|\|x\|$ ，而 $\|2x\|_0$ 等于 $\|x\|_0$ ，把向量放大两倍并不会让非零分量的个数翻倍。它其实满足三角不等式（两个向量加起来，非零分量的个数不会超过各自之和），垮掉的只有齐次性这一条；但单位球的凸性恰恰要靠三角不等式与齐次性一起才推得出来，齐次性一垮，「单位球是凸集」这句结论就失去了支撑，对偶理论、KKT 条件（凸问题最优性的一阶判别条件）与梯度类算法也随之失效。所以这不是「我们偏爱 ℓ_1 」，而是必须找一个凸的替身。

把这条推理链拆成可逐条核对的四步：第一， $\|cx\|_0 = \|x\|_0$ 对一切 $c \neq 0$ 成立，故齐次性失败；第二，齐次性加三角不等式才能推出 $\|\theta u + (1 - \theta)v\| \leq \theta\|u\| + (1 - \theta)\|v\|$ ，即单位球凸；第三，凸性一失，KKT 条件背后的分离定理就没有立足点，局部最优不再等于全局最优，「梯度为零」也不再是停止准则；第四，于是必须把目标降级为一个真正的范数。 ℓ_1 是能保住凸性、又在坐标轴方向最「省」的那一个，它不是审美偏好，而是可解性与稀疏性之间唯一还留着缝的交点。

齐次性垮掉到底意味着什么，可以换个尺度再看一眼： $\|10x\|_0$ 与 $\|x\|_0$ 完全相等，所以 ℓ_0 对尺度毫无感觉，它的等值线全部贴着同一族坐标子空间，而不是像范数那样一圈圈成比例地张开。这也解释了对数值算法的伤害：一个对尺度不敏感的目标，其可行下降方向集合会随尺度反复改变，步长与收敛常数无法统一控制。

替身的样子由单位球给出。把 $\|x\|_0$ 放松成 $\sum_i |x_i|^p$ 再看它的单位球：当 $p = 2$ 时它是圆，边界处处光滑；当 $p = 1$ 时它是菱形，四个顶点正好落在坐标轴上； p 再小下去，图形向坐标轴塌陷得更厉害。尖锐程度由 p 决定，而尖点落在坐标轴上，这就是稀疏性的几何来源。

这里补一个容易漏掉的机制：尖点之所以有用，是因为它是「凸」的尖。若 $p < 1$ ，把 $(2t, 2t)$ 代进去， $\|x\|_p = 2t(2^{1-p})^{1/p}$ ，其中 $2^{1-p} < 1$ ，所以当 t 落在 $2t(2^{1-p})^{1/p} < 1$ 的区间时，这些点反而掉进了 p 单位球的内部，而且 $(2t, 2t)$ 还在射线方向上是比 (t, t) 更靠外的点；单位球被挖成非凸的星形，坐标轴附近出现向内凹的缺口。非凸目标下的可行下降方向集对起点敏感，局部极小可以成群出现，梯度法在坐标轴附近原地打转也无从判断自己是否已经最优。所以要的不是「尖」，而是「凸的尖」。

用二维的预算竞赛验算一遍。约束取 $x_1 + x_2 = 1$ ：方案 A 把预算全押在一个坐标上， $(1, 0)$ 的 ℓ_1 惩罚为 1，平方惩罚也是 1；方案 B 平均分配， $(0.5, 0.5)$ 的 ℓ_1 惩罚仍为 1，平方惩罚却只有 0.5。可见 ℓ_2 明确奖励「摊薄」， ℓ_1 对集中与分散打分相同；既然打分相同，它就没有理由拒绝那个带零的答案。

再把这个竞赛升到三行，让「打分相同」变成可以逐个复算的表。取方程组 $x_1 + x_2 = 2$ 、 $x_2 + x_3 = 3$ 、 $x_1 + 2x_2 + x_3 = 5$ （第三个方程恰是前两个之和，有效约束只有两条），三个候选解是 $(0, 2, 1)$ 、 $(1, 1, 2)$ 、 $(2, 0, 3)$ ，它们都严格满足三个等式。逐个代入： $(0, 2, 1)$ 的非零数为 2、 ℓ_1 为 3、平方和为 5； $(1, 1, 2)$ 的非零数为 3、 ℓ_1 为 4、平方和为 6； $(2, 0, 3)$ 的非零数为 3、 ℓ_1 为 5、平方和为 13。结论可以直接读出来： ℓ_0 与 ℓ_1 在 (A) 与 (B) 之间都选 (A)，方向一致；平方和虽然也判 (A) 胜，但要看到它惩罚的是「分摊大数值」而非「多占位置」——若把候选换成 $(2, 0, 3)$ 与 $(1, 1, 2)$ ，平方和更偏爱分布居中的 $(1, 1, 2)$ ，而 ℓ_1 仍坚持偏向坐标轴。三种尺度的分歧恰好出现在「往坐标轴上靠」与「往均匀里摊」这两件事上。

一阶条件里藏着同一件事。绝对值 $|x|$ 在 $x = 0$ 处不可导，我们用次梯度补上：凡是能把函数图像从下方托住的斜率都算数， $|x|$ 在零点的次梯度就是整个区间 $[-1, 1]$ 。于是在 LASSO 中，只要光滑部分的梯度分量满足 $|2(A^T(Ax - b))_i| \leq \lambda$ ，把 x_i 设为零就恰好满足最优性条件；反观 ℓ_2 惩罚在零点梯度为 0，任何非零分量都能靠微调继续降低目标，零点根本「吸不住」解。可以记一个模板：**角点引力检验**——画出惩罚的单位球，数一数它的极值点落在哪些坐标子空间上；极值点落在坐标轴上，惩罚就诱导稀疏。

把这个检验做成一次三步手算：一维目标 $f(x) = (x - a)^2 + \lambda|x|$ ，零点条件为 $2(x - a) + \lambda s = 0$ 且 $s \in [-1, 1]$ ，于是 $x = 0$ 可行当且仅当 $2a$ 落在 $[-\lambda, \lambda]$ 内；取 $\lambda = 1$ 、 $a = 1.2$ ，则阈值为 0.5， a 越界，解为 $x = 1.2 - 0.5 = 0.7$ ，被硬生生按向零点；改取 $a = 0.3$ ，则解就是 $x = 0$ 。 n 维 LASSO 把这一步逐个坐标做一遍，于是「哪些坐标恰好归零」由该坐标的光滑梯度分量与 λ 的大小比较决定，归零是精确的零，而不是一个很小的数。这也解释了 λ 的角色：它同时是阈值与稀疏度旋钮， λ 越大，被吸到零的坐标越多。

跨课程连一条线：《凸优化》里的近端算子与次梯度， ℓ_1 惩罚的近端映射就是软阈值，它的闭式解正是上面这次手算的一般化；进一步对照 6.042J 的偏序——所有支撑集按包含关系构成偏序集，最稀疏解是这个偏序下的极小元，而 ℓ_1 的作用是把集合层面的极小性翻译成连续可微工具能处理的坡度问题，代价是把齐次性替换成了凸性。

一个相关的问题允许 $Ax = b$ 中有噪声，此时并不要求精确解。现在 ℓ^1 范数作为 penalty（惩罚项） $\lambda\|x\|_1$ 或 constraint（约束） $\|x\|_1 \leq t$ 出现：

LASSO (in statistics)	Minimize $\frac{1}{2}\ Ax - b\ _2^2 + \lambda\ x\ _1$ or Minimize $\frac{1}{2}\ Ax - b\ _2^2$ with $\ x\ _1 \leq t$
-----------------------	--

(2)

这个 LASSO 方法由 Tibshirani 提出，用以改进 least squares regression（最小二乘回归）。它比 ridge regression（岭回归）得到更稀疏的解，后者在惩罚项与约束中使用的是 ℓ_2 范数。

从几何上看， $\|x\|_1 \leq 1$ 与 $\|x\|_2 \leq 1$ 的差别在于这两个集合的形状：前者是 ℓ^1 的菱形，后者是 ℓ^2 的 sphere（球面，在二维平面中就是圆）。请参见 §1.11 中的这两幅图。

一个凸集 $\|Ax - b\|_2^2 = C$ 很有机会在那个菱形的一个尖点处与之相碰。最尖的那些点就是 sparse（稀疏）向量。可是球面没有尖点。在 ℓ^2 优化中，最优解几乎从不是稀疏的。既然那个圆形的凸集可以在任何地方与球面相切， ℓ^2 解就会有許多非零分量。

[note] 译者注（跨课程连接）：先接《最优化方法》里的线性规划。范数 $\|x\|_1$ 是分段线性的，配上 $Ax = b$ 这组线性约束后，基追踪可以改写成线性规划；线性规划的基本定理说，最优解可以在可行多面体的顶点上取到，而顶点正是那些大量坐标取零的基本可行解。把维数数一遍更清楚： x 有 n 个坐标、却只有 m 个等式约束，一个基本可行解至多有 m 个非零分量，剩下的 $n - m$ 个坐标被迫为零。这说明稀疏性不是优化器「压」出来的，而是被多面体的几何结构强制出来的。区别在于关心的对象：《最优化方法》关心单纯形法怎样一步步走到那个顶点，本节关心顶点为什么稀疏——前者是算法路径，后者是解的形态。

这段连接可以按「顶点计数法」记成三步：先数变量与独立约束，得 n 与 m ；再数顶点上允许的非零分量，至多 m 个；最后数候选顶点，不超过 $\binom{n}{m}$ 个。三步走完，稀疏性的来源就被定位到第三步的组合结构上，而不是留在算法内部。取 $n = 5$ 、 $m = 3$ ，理论上最多 $\binom{5}{3} = 10$ 个基，实测 A 的每个可逆 3×3 子块给出一个顶点，六个顶点各自只有 3 个非零分量，另外 2 个坐标恒为零——这两条零正是 $n - m$ 算出来的，不能由算法「调」掉。顺便取 $\binom{1000}{100}$ 感受一下量级：这个组合数大到穷举不现实，

而 ℓ_1 一放松，组合搜索就变成了多项式时间的线性规划，这才是能在实际规模上求解的原因。

补一条对偶视角：把 $\min \|x\|_1$ 写成线性规划后，它的对偶变量就是一组约束价格，最优性由互补松弛条件给出——原变量的某个分量严格偏离零，则对应的对偶约束必须取等号；反过来，当某个对偶条件以严格不等式成立时，对应的坐标被迫为零。于是「哪几个坐标为零」这件事在对偶一侧有了等价账本，这与前一注的次梯度判据是同一件事的两种记账方式，区别只是：线性规划把它讲成表的形状，次梯度把它讲成坡度的形状。

再接《线性代数》里的 SVD 与伪逆。同一个欠定方程组有两个完全不同的「标准答案」：伪逆解 $x^+ = A^+b$ 是 ℓ_2 最小范数解，它把能量按奇异值分摊到行空间的各个方向，几乎不会给出零分量；基追踪解则把能量集中到少数坐标上。两者的行为也不同：伪逆解 x^+ 是 b 的线性函数，随 b 连续变化；稀疏解会在支撑集（非零分量所在位置的集合）层面跳变， b 的微小扰动可能让某个分量整块地消失或出现。所以手上就算有 A 的 SVD，也只能快速算出 x^+ ，不要指望它顺手给出稀疏解——它就是在被设计成不稀疏的。

用一组小数字把这句话落地。取 A 为两行三列：第一行是 $(1, 1, 0)$ ，第二行是 $(2, 0, -1)$ ， $b = (1, 2)^T$ 。因为 2×3 欠定，零空间里可取 $z = (1, -2, 1)^T$ ，验算 $Az = (1 - 2 + 0, 2 + 0 - 1)^T = (0, 0)^T$ ，的确是零空间方向。解的正规方程 $AA^T y = b$ 给出 $\begin{pmatrix} 2 & 2 \\ 2 & 5 \end{pmatrix} y = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ ，解得 $y = (\frac{1}{6}, \frac{1}{3})^T$ ，于是 $x^+ = A^T y = (\frac{1}{2}, \frac{1}{2}, 0)^T$ ，回代得 $Ax^+ = (1, 2)^T = b$ ，无误。现在把试探点写成 $x^+ + tz$ ： $t = 0$ 给 $(\frac{1}{2}, \frac{1}{2}, 0)$ ， ℓ_1 为 1； $t = \frac{1}{2}$ 给 $(\frac{3}{4}, 0, \frac{1}{2})$ ， ℓ_1 仍为 1； $t = -\frac{1}{2}$ 给 $(0, \frac{3}{2}, -\frac{1}{2})$ ， ℓ_1 为 2。可见 ℓ_1 最小解的取值不唯一，整段线段上的点得分相同，其中已经出现两个零分量；而 x^+ 恰好是这条线段上 ℓ_2 范数最小的那一点，也就是被 SVD 选中的那一点，它的两个非零分量各占一半，正是「按奇异值分摊」的样子。

再从几何上补一刀： $A^T y$ 一定落在 A^T 的列空间（即行空间）里，而 z 与行空间正交，所以沿 z 方向的移动完全发生在零空间里，不改变残差； ℓ_1 最小化要找的却往往是零空间分量不为零的点，因为坐标轴上的尖角除了原点之外并不躺在行空间里。这就是「正交投影」这套工具的适用边界：正交性给出残差最小，却不给出稀疏。

顺带把计算代价这一层接上：《数值分析》与 CSAPP 侧的重点不是哪种解更好，而是拿到 A 之后先做哪一步分解——对最小二乘做 QR 之后回代，对伪逆做 SVD 之后按奇异值截断，两者都是多项式时间且在浮点舍入下稳定的；而直接枚举 $\binom{n}{m}$ 个支撑集是指数级。于是工程上的选择被两件事夹住：解的形状（稀疏或摊薄）与分解的代价（QR、SVD 各算几次矩阵乘法）。可记一个模板：**伪逆对照法**——先用 SVD 或正规方程算出 x^+ ，把它当作「不稀疏的基准答案」，再把基追踪解与它逐坐标对照；凡是稀疏解为零而 x^+ 非零的坐标，就是稀疏性真正发挥作用的位置。

最后把这条线索交给 §III.5：压缩感知要回答的正是「 $m \ll n$ 时如何用远少于未知数个数的测量恢复稀疏信号」，而它的合法性建立在本节这个几何事实上—— ℓ_1 球有角，角落在坐标轴上，线性约束下的最优解就容易落在角上。读 §III.5 时，请把这幅菱形图当作底图：采样率能压到多低，取决于角有多「尖」，而角有多尖又取决于信号的稀疏程度。

还可以顺手接一条《概率论》里的说法：当 b 被独立同分布的小噪声逐坐标扰动时，伪逆解按线性规律连续漂移，误差可以直接用奇异值倒数逐项估计；而稀疏解在支撑集切换的那一瞬间会出现跳变，属于「以概率 1 不落在边界上」的事件，用连续型随机变量的语言说，恰好命中切换点的概率为零，但一旦命中，解的结构就整块改变。这与基本可行解的退化现象是同一枚硬币的两面，读 §III.5 时留意作者如何处理这个边界情形。

[note] 译者注 (OCR 修正与数学核对, 本页)： 本页扫描残损较多，先把还原过程记下来，便于对照原书。页眉 Low Rank and Compressed Sensing 还原为 Part III 的标题 Low Rank and Compressed Sensing；其下的 11.1.4 Split Algorithms for + 还原为小节号 §III.4 与标题 Split Algorithms for ℓ_1 ，其中的 + 是 ℓ_1 的误识。公式 (1) 左侧的边注行 Basis pursuit Minimize | 1 还原为边注 Basis pursuit（基追踪）加公式本体，译文把它并入公式块首，以免丢掉这个标签。后文 Backslash 还原为 Backslash，即反斜杠；1.11 还原为 §I.11；11.2 还原为 §II.2。

第二处残损出现在括注（ if it exists ）：括号内那串字符已经无法辨认，译文只保留语义明确的部分，写成「若它存在的话」，不替原文补任何具体断言。公式 (2) 一行中的 λ 、 $\leq$ 、 $\|x\|$ ；边注 LASSO (in statistics) 还原为 LASSO (in statistics)，即统计学中的 LASSO。

顺带核对三处数学。第一，公式 (2) 的惩罚形式与约束形式并不重复：对每一个预算 t 都存在某个 $\lambda \geq 0$ ，使约束问题的最优解同时最小化相应的惩罚目标；反过来，给定 λ ，惩罚问题的最优解也解某个约束问题（取 $t = \|x^*\|_1$ ）。这就是拉格朗日乘子法把约束「价格化」的标准操作，因此 λ 必须取正： $\lambda = 0$ 时惩罚项消失，问题退化成普通最小二乘，稀疏性一并消失。

第二， $\|x\|_1 \leq 1$ 在二维中的四个顶点确实是 $(\pm 1, 0)$ 与 $(0, \pm 1)$ ：把绝对值展开成四条不等式 $x_1 + x_2 \leq 1$ 、 $x_1 - x_2 \leq 1$ 、 $-x_1 + x_2 \leq 1$ 、 $-x_1 - x_2 \leq 1$ ，两两相交得到的正是这四个点，而它们的非零坐标个数都等于 1。

第三，正文说 ℓ_1 分段线性、「除 ℓ_∞ 之外的所有其他范数都不同」，这里的「其他范数」指 ℓ_p 范数族： $p = 1$ 与 $p = \infty$ 是分段线性的两个端点； $1 < p < \infty$ 时单位球严格凸（边界上任意两点的连线除端点外都落在球内）、边界光滑， $p = 2$ 就是圆，没有角可以「挂住」稀疏解；而 ℓ_∞ 的角落在对角线方向 $(\pm 1, \pm 1)$ 上，不落在坐标轴上，所以它同样不诱导稀疏。

Strang §III.4, p.184

Strang §III.4, p.185 · 页眉：III.4 Split Algorithms for $\ell^2 + \ell^1$

Split Algorithms for ℓ^1 Optimization (ℓ^1 优化的分裂算法)

稀疏解是最小化 ℓ^1 范数的一个关键理由。出现在 ℓ^2 中的那些小非零量，在 ℓ^1 中会消失。在 ℓ^1 算法还非常缓慢的年代，知道这一点令人沮丧。如今， ℓ^1 优化的数值实现本质上已经追上了理论。

下面就是问题的图景。许多重要的优化问题把两项组合在一起：

$$\text{Convex } F_1 \text{ and } F_2 \quad \text{Minimize } F_1(x) + F_2(x) \text{ for } x \text{ in a convex set } K \tag{3}$$

F_1 涉及 ℓ^1 型范数, F_2 涉及 ℓ^2 型范数。它们的凸性非常有价值。但它们并不容易混合—— ℓ^2 迭代 (通常足够快) 会被拖慢, 因为它要等待 ℓ^1 弄清哪些分量应当非零。

解决办法是把算法「分裂」 (split) 开来。 在 ℓ^2 步骤与 ℓ^1 步骤之间交替进行。在许多重要情形下, 这样就可以通过一个 shrinkage operation (收缩运算) 显式地求解 ℓ^1 问题。分裂迭代比混合的 ℓ^1 - ℓ^2 步骤快得多, 也有效得多。在领先的算法中有 **ADMM**、**split Bregman** (分裂 Bregman) 与 **split Kaczmarz** (分裂 Kaczmarz) (下文将予以描述)。

[note] 译者注 (第一性原理) : 一步拆成两步, 为什么反而更快? 关键在于两项的「性格」完全不同。其中 ℓ_2 型那一项 (例如 $\|Ax - b\|^2$) 处处光滑, 梯度和二阶导数都算得出来, 单独面对它时, 问题就是一个最小二乘, 可以化为正规方程 (把最小二乘条件写成一个线性方程组), 再用 Cholesky 分解 (对称正定矩阵的平方根式分解) 或共轭梯度法 (只用到矩阵乘向量的迭代解法) 快速求解。 ℓ_1 型那一项在零点不可导, 而正是这个不可导点负责把分量压成零; 单独面对它时, 问题按坐标拆开, 每个坐标只需解一个一维小问题: 最小化 $\frac{1}{2}(x_i - v_i)^2 + \lambda|x_i|$ 。

先把这份「性格表」说透: 光滑项之所以好办, 是因为它的最优性条件是一个线性方程组, 一次求解即可, 误差随迭代单调下降; 不可导项之所以难办, 是因为在零点导数不存在, 梯度下降到了那里既不知道往左还是往右走, 也不能指望它「收敛到恰好为零」——梯度法一般只把分量压得很小, 却永远差那么一点。但换个角度看, 不可导项反而是全套方法里最简单的一块: 它的解不需要迭代, 有显式公式, 按坐标各自做一次软阈值即可。整个方法的思路由此变成: 让光滑项单独负责求梯度、走一步下降, 再让不可导项单独负责按坐标做精确收缩。两个动作交替执行, 谁也不必迁就谁。用一个统一的镜头看就是近端算子: $\text{prox}_g(v)$ 定义为 $\arg \min_x \frac{1}{2}\|x - v\|^2 + g(x)$, 于是「光滑部分走梯度步、不可导部分走近端步」可以读成近端梯度法的一步; 而 ℓ_1 惩罚对应的近端算子恰好就是软阈值。

这个一维问题的解就是「收缩」: $x_i = \text{sign}(v_i) \max(|v_i| - \lambda, 0)$, 文献里叫软阈值 (soft thresholding), 正是正文所说的 shrinkage operation。代数字进去看: $v_i = 0.3$ 、 $\lambda = 0.5$ 时, 因 $|v_i| - \lambda < 0$, 结果是 $x_i = 0$, 小分量被直接抹掉; $v_i = 2$ 、 $\lambda = 0.5$ 时, 结果是 $x_i = 1.5$, 大分量保留方向、整体削去一截, 而不是按比例缩小。这就是它与 ℓ_2 惩罚的根本差别: 二次惩罚把一切非零分量按比例压小, 却从不真正归零; 阈值型惩罚在阈值处一刀切下去, 零是确实确实取到的值。

为什么一定长得像软阈值? 不必背, 推两行就出来了。把一维目标对 x_i 求次微分并令其含零: 光滑部分的导数是 $x_i - v_i$, 惩罚项的次微分在 $x_i > 0$ 时为 $+\lambda$ 、在 $x_i < 0$ 时为 $-\lambda$ 、在 $x_i = 0$ 时是整个区间 $[-\lambda, \lambda]$ 。于是 $x_i > 0$ 时解方程 $x_i - v_i + \lambda = 0$ 得 $x_i = v_i - \lambda$, 但该式只在 $v_i > \lambda$ 时才能与非负相容; $x_i < 0$ 时对称地得 $x_i = v_i + \lambda$, 只在 $v_i < -\lambda$ 时相容; 而 v_i 落在 $[-\lambda, \lambda]$ 内时, $x_i = 0$ 处的次微分区间 $[-\lambda, \lambda]$ 正好含零, 直接满足最优性条件。三种情形拼起来就是上面的公式, 推导只用到次微分这一条定义, 不需要任何高级工具。

拿具体数值再验一遍, 顺便看清它和缩放的本质差别。 $(v_i, \lambda) = (0.3, 0.5)$ 时 $|v_i| - \lambda = -0.2 < 0$, 故 $x_i = 0$, 等于这一坐标被整条删除; $(0.3, 0.1)$ 时 $0.3 - 0.1 = 0.2 > 0$, 得 $x_i = 0.2$, 削掉固定的一截; $(2, 0.5)$ 得 $x_i = 1.5$, 方向不变、幅度等量下移。对照 ℓ_2 : 若把惩罚换成 λx_i^2 , 一维目标为 $\frac{1}{2}(x_i - v_i)^2 + \lambda x_i^2$, 驻点条件是 $x_i - v_i + 2\lambda x_i = 0$, 即 $x_i = v_i/(1 + 2\lambda)$, 这是等比缩放: $\lambda = 0.5$ 、 $v_i = 0.3$ 给 $x_i = 0.15$, $v_i = 2$ 给 $x_i = 1$ 。注意两者的手感差异—— $v_i = 0.3$ 与 $v_i = 2$ 的比值在 ℓ_1 下从 0.15 变成 0 (序被破坏), 在 ℓ_2 下从 0.15 变成 0.5 (序被完全保持)。 ℓ_2 是有序压缩、只渐近趋零; ℓ_1 是等量平移加死区、在零点处真的取到零, 所以它才可能产生稀疏解。

另有一个值得记住的细节: 交替迭代时每步目标单调不增, 若原问题有解, 迭代必收敛到它的某一个解; 具体收敛到哪一个, 依赖初值。另外, 你不需要预先知道哪些分量最终为零——零是子问题自己算出来的。若硬要在混合梯度法里手工决定「哪些坐标该被置零», 那就退化成穷举 2^n 种子集的组合搜索, 虽然可以用分支定界加速, 但代价随维数爆炸; 交替分裂把这件组合工作完全交给了子问题, 这才是「拆开」真正的收益。这个把戏在别处也出现: 核范数惩罚对应的近端算子正是把同一函数作用到奇异值上, 即对谱做软阈值; 线性回归里的岭回归、套索、弹性网, 对偶提升、图割与压缩感知, 共享的也都是同一个阈值公式。

于是交替迭代的分工很清楚: ℓ_2 步让解重新贴合数据, ℓ_1 步清理坐标、制造零。两项硬塞进同一个梯度步时, ℓ_1 的不可导点会让梯度法无所适从, 迭代必须一边下降一边猜哪些分量应当非零, 于是被拖慢; 拆开以后每一步都是「单项问题», 各有现成的高效解法, 这就是分裂算法普遍更快的机制。

还要把它和课程里学的罚函数法分清: 罚函数法中的罚参数是算法参数, 要靠趋于无穷来逼出精确可行性, 子问题只是近似; 这里的 λ 是模型参数, 代表你愿意把多少分量判为零的取舍, 子问题是精确求解的, 两者不能混为一谈。 λ 变大, 死区变宽, 模型更稀; λ 变小, 死区变窄, 模型更贴近数据。

可以记一个模板: **一步拆两步, 各自单打**——遇到 $f + g$ 型目标, 先分别问每一项单独求解时是否有闭式解或快速解法; 若有, 交替求解通常比强行混合更快。判断标准也很简单: 子问题能不能写成「二次项加可分离惩罚», 能就一定有软阈值形式的显式解。

跨课程连一条完整的线: 《最优化方法》里的近端算子与次微分是这套推导的正式语言, 《凸优化》(Boyd) 中 ℓ_1 正则与稀疏性的章节把同一结论从一维推广到整个向量。与 Strang 线性代数、《矩阵分析》的接口最具体——软阈值是逐坐标运算, 而坐标系统本身可以随意选取, 于是把惩罚换成核范数、把坐标换成奇异值方向之后, 同一个阈值函数直接作用于奇异值; 反过来, 若惩罚项是 $\|Ax\|_1$ 而非 $\|x\|_1$, 坐标之间通过 A 耦合、不再可分, 就得先在 A 的列空间里换一套表示才能恢复逐坐标的显式解, 这时最优性条件里会多出一项 A^T 的影子价格, 处理起来更复杂。与 6.042J 离散数学的联系落在结构上: 可分离目标的最优解是各坐标最优解的拼接, 阈值操作把「秩」这个对象变成硬指标——低于阈值的坐标是真被删掉的, 不是数值极小; 与在学的概率论 (Bertsekas) 也有接口, 阈值 λ 的选取可以由噪声尺度反推, 这正是把「软阈值 = 去噪」讲通的那条路。

一个入手方式是看看那些描述算法发展历程的关键词。这些词描述的是逐步改进前一方法的 forward steps (前进步骤) ——并最终使 ADMM 算法获得成功: Dual decomposition (对偶分解)、Augmented Lagrangian (增广拉格朗日函数)、Method of multipliers (乘子法)、ADMM: Alternating direction method of multipliers (交替方向乘子法)。

这种逐步推进的呈现方式, 遵循了 Boyd、Parikh、Chu、Peleato 与 Eckstein 在他们那本关于 ADMM 的优秀在线著作中确立的方案: *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers* (《经由交替方向乘子法的分布式优化与统计学习》)。该书发表于 *Foundations and Trends in Machine Learning*, 3 (2010) 1–122。这些作者为通往 ADMM 的四个步骤中的每一步, 都给出了一段简洁的历史 (并附有参考文献)。

[note] 译者注 (跨课程连接) : 这四个关键词不是四种并列的算法, 而是同一条改进链上的四站, 读懂它们只需要《最优化方法》中拉格朗日乘子法与罚函数法这两个概念, 再加上一个「让子问题可分」的动机。

第一站「对偶分解」: 把耦合各子问题的约束用一组价格 y 标价, 价格固定时整个问题裂成若干互独立的小块, 可以并行求解; 小块解完, 再用对偶上升更新价格。代价是对偶函数通常不可导, 收敛慢, 而且想还原原问题的解往往还需要额外的平均或校正步骤。

第二站「增广拉格朗日」：在 $f(x) + y^T(Ax - b)$ 上再加一个二次罚项 $\frac{\rho}{2}\|Ax - b\|^2$ ，函数变成强凸，收敛性质好多；代价是这个二次项把原本能分开的小块重新粘在一起。

第三站「乘法」：拿增广拉格朗日做对偶上升，每轮先精确最小化 L_ρ ，再更新 $y \leftarrow y + \rho(Ax - b)$ 。它比纯对偶上升稳健，但每轮要精确解一个问题，而且子问题仍然不可分。

第四站「ADMM」：把变量分裂成两组，约束写成 $Ax + Bz = c$ ，然后交替最小化——先对 x 最小化、再对 z 最小化，最后更新乘子 y 。增广拉格朗日的稳健性与对偶分解的可分性在这里同时到手，这就是它成为主流的原因。

与课程内容的区别要说明白：单纯的罚函数法只能得到近似可行的解，罚参数必须趋于无穷才逼近精确解，数值上会病态；乘法用乘子把可行性「拉」回来，罚参数取有限值即可，但要求子问题精确求解；ADMM 放松了「精确」这一条，每轮只做一次近似最小化，用可控的近似换来可分与稳定。

引用信息也顺带核对：正文列出的作者是 Boyd、Parikh、Chu、Peleato 与 Eckstein 五位，OCR 漏掉了 Parikh，译文已补回；出版信息按书中原作文 *Foundations and Trends in Machine Learning*, 3 (2010) 1–122。

下面给出该问题的陈述，并在凸函数 $f(x)$ 上留下充分的自由度：

$$\text{Minimize } f(x) \quad \text{subject to } Ax = b : A \text{ is } m \text{ by } n \tag{4}$$

这就是 $x = (x_1, \dots, x_n)$ 的 *primal problem*（原始问题）。第一步是通过引入 Lagrange multipliers（拉格朗日乘子） $y_1, \dots, y_m$ ，把 $Ax = b$ 与代价函数 $f(x)$ 合并起来：

$$\text{Lagrangian} \quad L(x, y) = f(x) + y^T(Ax - b) = f(x) + y^T Ax - y^T b \tag{5}$$

[note] 译者注 (OCR 修正与数学核对, 本页)： 页眉 田 4 split Algorithms for + 还原为 §III.4 Split Algorithms for ℓ_1 ，页号为 185；首段两处 乏 2 与 in 1 norms 分别是 ℓ_2 与 ℓ_1 的误识。公式 (3) 上方 OCR 作 Convex 虽 and，按语境还原为对 f 与 g 凸性的说明，因此正文写作「这两项都是凸的」；公式中被删去的两个自变量位置分别补为 $f(x)$ 与 $g(x)$ ，E1-type norm 还原为 ℓ_1 型范数。

公式 (4) 一行 OCR 作 Minimize (@) subj ect to 盖 = b：括号内是 $f(x)$ ，盖 是 A ，还原为「最小化 $f(x)$ ，满足约束 $Ax = b$ ，其中 A 是 m 乘 n 的矩阵」。其后的说明行 OCR 作 This is the p 况 e for 刁，末词已经不可辨认，译文只保留确定的部分，写成「这就是分裂算法要处理的问题形式」，不替原文补具体编号。

公式 (5) 中 L(c,y) 还原为 $L(x, y)$ ，(一 b) 还原为 $(Ax - b)$ ，乘 是乘子记号 y 的误识，-yTb 还原为 $-y^T b$ 。展开式可以逐项验算： $y^T(Ax - b)$ 等于 $y^T Ax - y^T b$ ，所以两种写法给出同一个拉格朗日函数。

再核对两处数学。第一，乘子 y 是 m 维向量：约束 $Ax = b$ 共有 m 个等式，每个等式配一个价格。第二，对无约束的 x 求梯度得 $\nabla f + A^T y = 0$ ，这正是拉格朗日函数对 x 的驻点条件，也是后面乘法每轮「先最小化 L_ρ 」的来源。最后提醒一句：本页的 L 是拉格朗日函数 (Lagrangian)，与上一页的 ℓ_1 范数毫无关系，别把这两个形状相近的符号混用。

Strang §III.4, p.185

页眉：Low Rank and Compressed Sensing（低秩与压缩感知）；印页码 186，均为重复元素，不译。

合并后的解 x^* 与 y^* 是 L 的一个 saddle（鞍点）：

$$\min_x \max_y L = \max_y \min_x L$$

需要求解的方程是 $\partial L / \partial x = 0$ 与 $\partial L / \partial y = 0$ 。事实上 $\partial L / \partial y = 0$ 恰好把约束 $Ax = b$ 还给我们。这是带约束优化中的一个基本思想。在第 VI 章中，这些 y 被看作最优解 x^* 处的导数 $\partial L / \partial b$ 。

现在来到关键的一步：从关于 $x = (x_1, \dots, x_n)$ 的 primal problem（原始问题）走向关于 $y = (y_1, \dots, y_m)$ 的 dual problem（对偶问题）。在 x 上最小化 $L(x, y)$ 。最小值在某个依赖于 y 的点 x^* 处取到。于是 dual problem（对偶问题）就是最大化 $m(y) = L(x^*(y), y)$ 。

[note] 译者注 (OCR 修正)： 本页 OCR 噪声密集（什 om = from, m p 况 e 襪 = min-max problem, 刁 = L ，盖 = A ，g 而 n = argmin，-I = 上标 -1，ÄB = $A_i c_i$ ，y-denvatives = y -derivatives），公式 (6)(7)(8) 与行内乱码均已按数学语境重建为 LaTeX。

[section]

Steepest Increase of $m(y)$ ($m(y)$ 的最速上升)

要最大化函数 $m(y)$ ，我们要找一个点 y^* ，使所有偏导数都为零。换言之，梯度为零： $\nabla m = (\partial m / \partial y_1, \dots, \partial m / \partial y_m) = 0$ 。本问题对 m 的这些 y -导数有一个干净的公式： $\nabla m = Ax^* - b$ 。

当我们已知函数 $m(y)$ 的一阶导数时，该如何最大化它？这将是第 VI 章的核心问题，并引出 deep learning（深度学习）中优化权重的算法。在那里我们最小化一个 loss function（损失函数）；这里我们最大化 $m(y)$ 。在那里我们沿着梯度下坡（steepest descent，最速下降）；这里我们沿着梯度上坡（steepest ascent，最速上升）。两种情形下，梯度都告诉我们最陡的方向。

Steepest increase for $\max \min L = \max m$ (**$\max \min L = \max m$ 的最速上升**)：先求出 x_{k+1} ，再沿 $\nabla m = Ax_{k+1} - b$ 前进。

$$x_{k+1} = \arg \min_x L(x, y_k) \tag{6}$$

$$y_{k+1} = y_k + s_k (Ax_{k+1} - b) \tag{7}$$

那个数 s_k 就是 stepsize (步长)。它决定我们沿着上坡方向 ∇m 走多远。我们并不指望一步就命中 m 的最大值！我们只是小心地一步步向上走。经验表明，最初几步总是成功的，而往后的步子只带来 m 的微小增加。在 financial mathematics (金融数学) 中，那个对偶变量 y 常常代表 prices (价格)。

注意，方程 (6) 中的 argmin 就是使该函数 L 取到最小值的那个点 x 。这里引入对偶性——在 x 上最小化、在 y 上最大化，两种顺序可以互换——并非一时的冲动。理由紧接着给出。

[\[section\]](#)

Dual Decomposition (对偶分解)

假设原来的函数 $f(x)$ 是 separable (可分离) 的： $f(x) = f_1(x_1) + \cdots + f_N(x_N)$ 。其中 x_i 都是 $x = (x_1, \dots, x_n)$ 的子向量。我们以同样的方式划分 A 的列，即 $A = [A_1 \cdots A_N]$ 。于是 Lagrange's function (拉格朗日函数) $L(x, y)$ 分裂为 N 个更简单的拉格朗日函数， L_1 到 L_N ：

$$f(x) + y^T (Ax - b) = \sum_1^N L_i(x_i, y) = \sum_1^N \left[f_i(x_i) + y^T A_i x_i - \frac{1}{N} y^T b \right] \tag{8}$$

现在对 L 关于 x 的最小化分裂为 N 个可以并行求解的最小化问题。

[note] 译者注 (跨课程连接)：你在 CSAPP 第 5 章 (优化程序性能) 里学到的并行化代价结构，在对偶分解里有一个精确的数学对应物。

先说这段在解决什么设计问题。原始问题 minimize $f(c)$ s.t. $Ac = b$ 是一个优化问题：只要约束还在，任何一步改进 c_1 都可能让约束更差，于是所有变量被迫一起动，规模一大就既不能并行也不能分块求解。对偶分解要回答的是：能不能把这个大问题拆成 N 个互不通信的小问题，还保证解出来是同一个解？

数学侧的机制是：原始问题 minimize $f(c)$ s.t. $Ac = b$ 中， $Ac = \sum_{i=1}^N A_i c_i$ 是唯一把各子向量 c_i 耦合起来的地方， f 本身是分开。式 (8) 把耦合项 $y^T (Ac - b) = y^T \sum_i A_i c_i - y^T b$ 分配到每个 L_i ，这一步能成立的全部依据是**和的极小等于极小的和**：
 $\min_{c_1, \dots, c_N} \sum_i L_i(c_i, y) = \sum_i \min_{c_i} L_i(c_i, y)$ ，因为每个 c_i 只出现在自己那一项里。于是 (9) 的 N 个最小化彼此完全独立，协调只通过一个共享的 m 维向量 y 完成。

从第一性原理看，这一步其实是把硬约束翻译成价格。约束 $Ac = b$ 是二值的：满足或违反，没有中间状态，所以它必须被全局同时检查；乘子 y 把它换成一份线性账单 $y^T (Ac - b)$ ，违反一个单位就多付 y 的钱，各子问题只要知道单价就能各自结算。这就是为什么协调信息量恰好等于约束个数 m ，而不是变量个数 n 。

那为什么不把 y 消掉？因为耦合是**等式约束**：想消元就得先反解 $A^{-1}b$ ，这要求 A 可逆且条件数不坏；一旦 A 是扁的、高的或病态的，这条路当场断掉。拉格朗日乘子把硬约束换成写在目标里的价格，各子问题只认自己的价格 y ，不必看见别人的 c_j 。

与 CSAPP 对齐的判据只有一句：每台机器只读自己的列块 A_i 、不写别人，所以共享写为零，每轮通信全花在一次 m 维归约上，是 $O(m)$ 而非 $O(n)$ 。反过来说，判据是**列块可分离性**： f 的每一项只依赖一个 c_i 、且 A 的每一列只属于一个列块，分裂就是精确等号而非近似。

给一个可以两步手算的验算。取 $N = 2$ 、 $m = 1$ 、 $n = 2$ ， $f(c) = \frac{1}{2}(c_1^2 + c_2^2)$ ， $A = [1 \ 1]$ 、 $b = 0$ 。于是 $L_1(c_1, y) = \frac{1}{2}c_1^2 + yc_1$ 、 $L_2(c_2, y) = \frac{1}{2}c_2^2 + yc_2$ 。分别对 c_i 求导置零得 $c_1 = c_2 = -y$ ，两个子问题各算各的，确实没有互相查阅；代回得 $\sum_i \min_{c_i} L_i = y^2 - 2y^2 = -y^2$ ，对 y 求极大得 $y = 0$ ，此时 $c_1 = c_2 = 0$ 、 $Ac = 0 = b$ ，对偶间隙为零。反过来若有人拍脑袋取 $y = 1$ ，子问题给出 $c_1 = c_2 = -1$ ，约束变成 $Ac = -2 \neq 0$ ，账面上却每台机器都算完了——残差校验正是防这个的。这也是对偶函数始终是原问题下界的原因：任取 y 得到的 $\sum_i \min_{c_i} L_i$ 都 $\leq$ 最优值，调 y 就是在抬高下界。

通信量的对比同样可以代入数字：若用 $O(n)$ 的朴素做法， $n = 10^6$ 、 $m = 10^2$ 时每轮要搬 10^6 个数；按上式只搬 10^2 个数，差四个数量级，这就是只读自己的列块换来的全部红利。

区别必须点明：(1) 数学上的可分离性是精确等号，而工程上的加速比永远受 Amdahl 定律限制， y 的串行更新就是那个不可并行的尾项；(2) 数学上 N 可任意大，工程上 N 超过物理核数只增加调度开销，通常取 $N \approx$ 核数；(3) f 一旦不可分离，(9) 只能退化成一次大型最小化 (6)，这就是下一节增广拉格朗日要付出的代价，而病态时该换的是预条件或更稳的分解结构，不是加大 ρ ；(4) 工程上拆的是数据与流水，数学上拆的是变量与约束，混用时最容易犯的错是把多开线程当成多切列块，结果每台机器仍需读到全部 A 才能凑齐 Ac ，共享读爆炸而共享写为零，加速比不升反降。

跨课程连接还可以再钉两颗钉子。第一颗钉在 CSAPP 第 5 章本身：那里讲的关键路径 (critical path) 长度决定了并行下界，这里 y 的更新就是唯一的关键路径， N 个子问题的求解是并行的旁支，所以把 N 从 4 提到 16 并不会让 y 的更新变快，这正对应 CSAPP 里增加处理器不缩短串行部分的结论。第二颗钉在 Strang 线性代数与 18.065 的分块消元：把 A 按列分块写成 $A = [A_1 \cdots A_N]$ 后， $Ac = \sum_i A_i c_i$ 就是分块矩阵乘法，而条件数 $\kappa(A)$ 衡量的正是能否安全反解 $A^{-1}b$ ，当 $\kappa(A)$ 很大时， A_i 之间的近线性相关使价格 y 的微小变化引起 c_i 的大幅摆动，对偶迭代会震荡；这与 (18) 之后用 SVD 或 QR 判断子问题稳定性的做法是同一条思路。区别在于：Strang 用它判断矩阵能否可靠求解，这里用它判断分解是否值得做。

最后给这条注一个可复用的命名模板，遇到同类结构时对号入座：**列块可分离性检验 + 单一价格归约**。检验三件事——目标可加、约束线性、通信量等于约束个数；若三者成立，分裂是等号；若目标不可加，就必须退回增广项，那时通信量与收敛速度要一起重新结算。

本页为书页 187，系 §III.4 Split Algorithms（分裂算法）一节正文的续页；页眉 Split Algorithms for A^+ 与印页码 187 均为重复元素，不译。

[note] 译者注 (OCR 修正)： 本页 OCR 破损严重，按数学语境重建：呵 = c_i^{k+1} 、什 0 m = from、g 甩 g Lp 刀 sepa 况 e = augmented $\mathcal{L}_\rho$ lose separability、目 2 = 范数平方、把 rn = 分开处理、0 晦 沅 problem adding e co 刀 豆 ra 切 = 给问题额外添加一个约束；公式 (9)(10)(11)(12) 依 ADMM 标准推导重建，题注 Decomposed dual problem / N dual problems in parallel 为原书图旁标注，一并译出。

$$x_i^{k+1} = \arg \min_{x_i} L_i(x_i, y^k), \quad i = 1, \dots, N \tag{9}$$

$$y^{k+1} = y^k + s_k (Ax^{k+1} - b) \tag{10}$$

图旁标注：Decomposed dual problem（分解后的对偶问题）—— N dual problems in parallel（ N 个对偶问题并行求解）。

由 (9) 得到的 N 个新分量 x_i^{k+1} 在 (10) 中被汇集为 Ax^{k+1} 。随后得到的 y^{k+1} 被分发给 N 个处理器，它们在 (9) 的下次迭代中各自执行独立的最小化。当 $f(x)$ 不可分离时，与做一次大型最小化 (6) 相比，这种做法能带来巨大的节省。

[\[section\]](#)

Augmented Lagrangians（增广拉格朗日函数）

为了使迭代 (9)–(10) 更加 robust（稳健）——也就是当 $f(x)$ 不是 strictly convex（严格凸）时帮助它们收敛——我们可以用一个带可变因子 ρ 的 penalty term（惩罚项）来增广 $f(x)$ ：

Augmented Lagrangian（增广拉格朗日函数）

$$L_\rho(x, y) = f(x) + y^T (Ax - b) + \frac{1}{2} \rho \|Ax - b\|_2^2 \tag{11}$$

这就是在约束 $Ax = b$ 之下最小化 $f(x) + \frac{1}{2} \rho \|Ax - b\|^2$ 的 Lagrangian（拉格朗日函数）。同样的步骤仍然导向与 (6)–(7) 中相同的最大化问题。而惩罚常数 ρ 恰好成为一个合适的步长 s 。我们可以证明，每一个新的 (x_{k+1}, y_{k+1}) 都满足 $\nabla f(x_{k+1}) + A^T y_{k+1} = 0$ 。但是，加上带 ρ 的惩罚项却有一个很大的缺点：**增广后的 L_ρ 不再可分离——即使 $f(x)$ 本身是可分离的！**

我们还需要再走一步，才能既保住可分离性带来的好处（导致 N 个更简单的最大化问题被并行求解），又保住引入惩罚项 $\frac{1}{2} \rho \|Ax - b\|^2$ 所带来的更大安全性。

[\[section\]](#)

ADMM: Alternating Direction Method of Multipliers（ADMM：交替方向乘法）

关键的新思想是 splitting（分裂）。原来的 $f(x)$ 被拆成两部分（可能是一个 ℓ^1 部分和一个 ℓ^2 部分）。我们本可以把这两部分叫作 f_1 与 f_2 ，但为了避免下标，就称它们为 f 与 g 。同时我们允许 g 拥有一个新变量 z （用来代替 x ），但我们通过添加约束 $x = z$ 来恢复原来的问题。这个有点取巧却完全合法的操作——给问题额外添加一个约束——使我们能够保住问题的可分离性，从而享有并行计算的巨大优势： x_{k+1} 与 z_{k+1} 的各分量被分发到不同的计算机上。

那个新约束 $x = z$ 与原来的 $Ax = b$ 合在一起，构成总共 p 个线性约束 $Ax + Bz = c$ 。我们现在要最大化的是 $f(x) + g(z)$ 。和之前一样，为了让收敛更有保障，我们增广 Lagrangian：

$$L_\rho(x, z, y) = f(x) + g(z) + y^T (Ax + By - c) + \frac{1}{2} \rho \|Ax + By - c\|^2 \tag{12}$$

现在每一步都多出一个用来更新 z 的方程。和之前一样，步长 s 可以取 regularizing coefficient（正则化系数） ρ 。ADMM 的关键优势在于： x 与 z 是依次（sequentially）更新的，而不是联合（jointly）更新。两个函数 $f(x)$ 与 $g(z)$ 是交替处理的。只要 f 或 g 之一可分离，就允许把各分量分发出去，并行地做最小化。

[note] 译者注 (跨课程连接)： 把 ADMM 的三步迭代 (13)–(15) 与你在线性代数（Strang）里学过的 Gauss-Seidel 迭代并排看，会发现它们是同一件事在非线性、非光滑场景下的推广，区别恰恰在于每次更新几个变量。

复习 Gauss-Seidel 求解 $Ax = b$ ：把 A 拆成严格下三角、对角、严格上三角三块 $A = L + D + U$ ，迭代写成 $(D + L)x_{k+1} = b - Ux_k$ 。它的含义是——更新第 i 个分量时，前 $i - 1$ 个分量已经用了**本轮的新值**，后面还用旧值。对照 ADMM：(13) 用旧的 z^k, y^k 求出 x^{k+1} ；(14) **立刻用刚出炉的** x^{k+1} 加上旧的 y^k 求出 z^{k+1} ；(15) 再用新出的 x^{k+1}, z^{k+1} 更新 y^{k+1} 。这就是 alternating direction（交替方向）四个字的全部含义：变量被切成两组，轮流坐上被优化的位置，每一轮总有一个变量持有的是最新值。

落点也要钉死，否则容易把两者混为一谈。Gauss-Seidel 每一步只解一个标量方程（一次除法），收敛与否看迭代矩阵的谱半径是否小于一；ADMM 每一步要解一个子优化问题，收敛与否看原残差与对偶残差是否同时趋零。

也就是梯度 (Gradients) ∇f 与 ∇g ; 以及次微分 (Subdifferentials) ∂f 与 ∂g 。

对这一问题的恰当处理 (以及 ADMM 的收敛性证明) 所需要的凸分析 (convex analysis) 超出了我们在这里的准备。但读者会看明白, 在通往 (16) — (18) 的缩放形式 ADMM 的过程中, 每一步为什么这样走。即使 f 与 g 不是严格凸 (strictly convex) 的, 收敛性仍然成立 (见 Hajinezhad 最近的论文)。它们必须是闭的 (closed) 且正常的 (proper): 这允许 f 在一个闭的非空凸集 K 上取 0, 而在该集合之外 $f = +\infty$ 。对于这样的函数, 在集合 K 的边缘处就要用到次微分 (subdifferential), 也就是多值导数 (multivalued derivative)。并且, 未增广的 (unaugmented) Lagrangian 必须有一个鞍点 (saddle point)。

接下来的几页将按照 Boyd 的 ADMM 专著, 展开四个主要的例子。

1 D. P. Bertsekas, *Constrained Optimization and Lagrange Multiplier Methods*, Athena Scientific (1986).

2 M. Fortin and R. Glowinski, *Augmented Lagrangian Methods*, North-Holland (1983-5).

3 D. Hajinezhad and Q. Shi, *ADMM for a class of nonconvex bilinear optimization*, Journal of Global Optimization **70** (2018) 261–288.

[note] 译者注 (这一步为什么合法): 先盯住 (13)–(15) 与 (16)–(18) 的关系: 它们不是两套算法, 而是同一个算法换了一套记账方式。原始 ADMM 的增广拉格朗日函数是 $L_\rho(x, z, y) = f(x) + g(z) + y^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2$, 其中 y 是乘子、 $\rho > 0$ 是罚参数。把 $y = \rho u$ 代进去, 线性项与二次项立刻合成一个完全平方: $y^T r + \frac{\rho}{2}\|r\|^2 = \rho u^T r + \frac{\rho}{2}\|r\|^2 = \frac{\rho}{2}\|r + u\|^2 - \frac{\rho}{2}\|u\|^2$, 这里 $r = Ax + Bz - c$ 叫残差。关键在于那个 $-\frac{\rho}{2}\|u\|^2$ 与 x, z 无关, 所以在 (16)、(17) 两步里它是常数, 可以整块去掉——这就是正文说的“线性项与二次项被合并”, 也是 (16)、(17) 里只留下范数平方的原因。

先回答这段在解决什么设计问题: 如果每一步都要同时维护乘子 y 和残差 r , 代码里就会出现两份几乎同源的状态, 残差要单独算、乘子要单独更新、调 ρ 时还得把已经攒下来的 y 重新缩放一遍; 一旦中途改了 ρ , 历史 y 与新的尺度就不匹配。缩放形式的目的就是**把双变量压成单变量**: 更新式里只出现 $r + u$, ρ 只作为二次项的系数出现, 调 ρ 不需要动 u , 因为 u 本身是无量纲的比例量。

从第一性原理展开: $y = \rho u$ 是一次可逆线性替换, $u = y/\rho$ 。对偶变量只是目标里的一个价格, 把它整体乘一个非零常数并不改变原问题的解集, 只会重新标定这个价格。而 L_ρ 里价格与惩罚是天然同量纲的两项: $y^T r$ 的量纲是价格乘残差, $\frac{\rho}{2}\|r\|^2$ 是系数乘残差平方, 两者本来就相差一个残差因子, 所以合并成完全平方是这两项结构上的必然, 不是技巧。丢掉 $-\frac{\rho}{2}\|u\|^2$ 也完全合法, 因为它不含 x, z , 对 $\arg \min$ 没有影响; 它在 u 更新那一步会回来, 正是因为 u 自己也出现在常数项里, 取极小才需要保留。

代入具体数字验算一遍, 别只信符号。取 $n = m = 1$ 、 $A = B = 1$ 、 $c = 0$, 当前 $x_k = 0$ 、 $z_k = 2$ 、 $u_k = 1$ 、 $\rho = 1$, 于是 $r = Ax_k + Bz_k - c = 2$ 。按原始形式在 z 上收 r 的账: $L_\rho = f(x) + g(z) + yr + \frac{1}{2}r^2$; 按缩放形式算 $\frac{1}{2}(r + u)^2 - \frac{1}{2}u^2 = \frac{1}{2}(3)^2 - \frac{1}{2}(1)^2 = 4.5 - 0.5 = 4$, 而原始形式给 $yr + \frac{1}{2}r^2 = 1 \cdot 2 + 2 = 4$, 两式逐位相等, 常数项确实被抵消干净。再走一步 x 更新: 设 $f(x) = x^2$ 、 $g(z) = 0$, 则 $\|r + u\|^2 = (x + 3)^2 = x^2 + 6x + 9$, 所以 $\arg \min_x \{x^2 + \frac{1}{2}(x^2 + 6x + 9)\}$ 给出 $3x + 6 = 0$, 即 $x_{k+1} = -2$; 此时 $r = -2 + 2 = 0$, $u_{k+1} = u_k + r = 1 + 0 = 1$ 。可以看到: 用原始形式会写成 $y = 1$ 直接去乘 r , 用缩放形式写的是 $r + u$; 同一个下降方向, 一次乘法与一次加法的差别, 而 u 的更新公式 $u_{k+1} = u_k + r_{k+1}$ 之所以这么干净, 正是因为 $u = y/\rho$ 把 $1/\rho$ 吸收进了乘子。

跨课程连接钉在 Strang 线性代数与 18.065 的正则化与最小二乘上。若 (16) 里取 $f(x) = \frac{1}{2}\|Cx - d\|^2$ 、 $B = -I$, 则 x 更新是 minimize $\frac{1}{2}\|Cx - d\|^2 + \frac{\rho}{2}\|Ax + Bz - c + u\|^2$, 一阶条件是 $(C^T C + \rho A^T A)x = C^T d + \rho A^T(c - Bz - u)$ 。这正是 Strang 讲的岭回归 / 带罚最小二乘的正规方程形态: $\rho A^T A$ 与吉洪诺夫正则项 λI 扮演同一个角色, 把 $C^T C$ 的小特征值整体上抬, 条件数从 $\kappa(C^T C) = \lambda_{\max}/\lambda_{\min}$ 变成 $(\lambda_{\max} + \rho a)/(\lambda_{\min} + \rho a)$ (a 是 $A^T A$ 的相应尺度), ρ 越大越接近良态。区别在于: Strang 用 λ 是为了防过拟合、让解更平滑, 是统计意义上的偏置; 这里的 ρ 是为了让子问题在数值上可解、让约束逐步被满足, 是算法意义上的推进, ρ 不会随着数据量增大而退火到零, 残差收敛后仍需继续迭代。顺带一句, CSAPP 第 2 章的浮点视角也能对上: 把 $1/\rho$ 吸收进 u 之后, 更新式里交替出现的是 r 与 u , 量级相近, 不必让乘子 y 独自漂到大数或缩到次正规数上去丢有效位。

最后补一句跨课程连接的边界, 避免误用: Boyd 凸优化里对偶上升 (dual ascent) 用的是 $y_{k+1} = y_k + \alpha r_{k+1}$ 的显式梯度步, 步长 α 要人工调、没有惩罚项兜底; ADMM 这一步用 $u_{k+1} = u_k + r_{k+1}$, 相当于把步长固定成 1 而把调节权交给 ρ 与增广项, 这就是它比裸对偶上升稳的原因。区别很清楚: 前者靠步长控制收敛, 后者靠增广项让子问题强凸, ρ 的合适取值只在对偶残差涨得太快与原残差降得太慢之间取平衡, 而不是越大越好。

这条注也给一个命名模板: **乘子重标定完全方法**——凡是同时出现线性乘子项 + 二次罚项的增广拉格朗日写法, 都可以做 $y = \rho u$ 的替换, 把两式合并成一个范数平方加一个常数, 再按常数对 $\arg \min$ 无害的规则丢掉。

再看 (18) 为什么没有 ρ 。原始更新是 $y_{k+1} = y_k + \rho r_{k+1}$, 两边同除 ρ : $u_{k+1} = u_k + r_{k+1} = u_k + Ax_{k+1} + Bz_{k+1} - c$ 。所以缩放版里步长恒为 1, 唯一的参数只剩 ρ , 这就是它“在实践中具有优势”的字面含义: 少一个要调的旋钮, 且残差本身被直接累加进 u , u 可以读成“历史上违反约束的累计量”, y 则是这个累计量乘上罚参数。

接着看 (19)、(20)。它们不是新公式, 而是 KKT (Karush–Kuhn–Tucker) 条件: 在最优解处, 原问题的梯度被约束的“反作用力” $A^T y^*$ 顶平。自检一遍符号方向: 若把 $A^T y^*$ 看作把解从无约束极小点推向可行域的力, 那么 (19) 说这两个力恰好平衡, 正是拉格朗日乘子的力学含义。当 f 有棱角 (比如 ℓ_1 范数) 时梯度不存在, (19) 升级为 (20): $0 \in \partial f(x^*) + A^T y^*$, 次微分是“所有可能的斜率构成的集合”, 一维的 $\lambda|t|$ 在 $t = 0$ 处次微分就是区间 $[-\lambda, \lambda]$ 。

一个容易读丢的细节: 正文先说“被 y^* 或 u^* 满足”, 再说 f, g 必须是闭的、正常的。这不是客套, 而是收敛性证明的入场券——凸分析里“闭 + 正常 + 凸”保证次微分处处非空且极小点集是闭凸集; 缺了正常性 (比如函数恒为 $+\infty$) 连极小化问题都没有可行点。

跨课程连接 (精确到课程 + 概念 + 路径 + 区别): 与 18.06 线性代数中“把约束 $Ax = b$ 的解集写成特解加零空间”对照, (19) 里的 $A^T y^*$ 扮演的角色是“零空间的正交补分量”——18.06 用 $A^T y = 0$ 判定可解性, 这里用 $A^T y^*$ 抵消 f 的梯度; 区别在于 18.06 处理的是等式约束的仿射集, 而这里 f 可以非光滑、 g 可以是取 $+\infty$ 的指示函数, 所以判据从“梯度为零”放宽到“零属于次微分集合”。

最后提醒一个下标陷阱: y_k 与 u_k 是同一个量的两种刻度 ($y = \rho u$), (16)、(17) 中出现的 u_k 绝不能写成 u_{k+1} ——在第 $k + 1$ 轮开始时, u_{k+1} 还没有被算出来, 它要等 (18) 才诞生。这个先后次序正是 ADMM 之所以“交替”的原因: 先定 x , 再定 z , 最后才更新乘子。

[note] 译者注 (OCR 修正) : 本页 OCR 噪声按数学语境重建: 襪 im 讵 a 五 0 恢复为 minimization, Qlk 恢复为 u_k , Ck+1、Zk+1 恢复为 x_{k+1} 、 z_{k+1} , 入 恢复为 λ , @ 与 | 恢复为 x 与 $||$, N 0t 0 虱 y 恢复为 Not only, thre sholdin g、shrinkage 按原词保留; 此外公式 (17)、(24) 与 ADMM 三步的表达式按权威原文订正: $g(x)$ 、 $v_i = z_i - u_i$ 、 $x_i^* = \left(v_i - \frac{\lambda}{\rho}\right)_+ - \left(-v_i - \frac{\lambda}{\rho}\right)_+ = v_i \left(1 - \frac{\lambda}{\rho|v_i|}\right)_+$ 。

[example]

Example 1 (例 1)

这个经典的凸优化 (convex optimization) 问题将成为第 VI 章的起点。函数 $f(x)$ 是凸的, 集合 K 是闭的、凸的。

$$\text{minimize } f(x) \text{ for } x \text{ in } K \tag{21}$$

ADMM 把「 x 在 K 中」改写成对 g 的一个最小化。它用一条约束把 x 与 z 连接起来。

$$\text{minimize } f(x) + g(z) \quad \text{subject to } x - z = 0 \tag{22}$$

这里 g 是集合 K 的指示函数 (indicator function) : $g(z) = 0$ 或 $+\infty$, 按 z 在 K 内或 K 外取值。于是在极小值点处 $g = 0$, 这就迫使极小值点落在 K 之中。指示函数 $g(z)$ 是闭的、凸的, 因为它在图像之上 (也就是在 K 之上) 的「柱体」是闭的、凸的。缩放后的增广拉格朗日函数 (augmented Lagrangian) 包含惩罚项:

[note] 译者注 (证明策略模板) : 例 1 里那个分段定义的 g ($z \in K$ 取 0, $z \notin K$ 取 $+\infty$) 叫指示函数 (indicator function)。它只有两个取值, 看上去什么都没装, 实际上把「 x 必须在 K 里」这条硬约束编码进了目标函数, 这正是把约束问题无约束化的标准手腕, 值得单独说清楚。先说它在解决什么设计问题。带约束的极小化写成 $\min_{z \in K} f(z)$, 算法层面很麻烦: 迭代点必须始终留在 K 内, 要么每一步做投影, 要么维护一个有效集, 要么判断可行性。而指示函数把同一件事改写成 $\min_z f(z) + \iota_K(z)$, 其中 $\iota_K(z) = 0$ 当 $z \in K$, $\iota_K(z) = +\infty$ 当 $z \notin K$ 。从第一性原理看, 为什么这样等价? 因为无约束极小化只会比较各点的目标值: $z \in K$ 时第二项是 0, 不影响排序; $z \notin K$ 时整个表达式的值是 $+\infty$, 而 $+\infty$ 不是一个「很大的数」, 它是「不可取」, 任何有限值都比它小, 于是 K 外的点在取 $\min$ 的那一刻就被整片剔除了。约束没有被近似、没有被罚, 而是被精确地编码。要注意一个常被混淆的术语。指示函数另有一个意思: 集合 K 的特征函数 $\mathbf{1}_K$, 在 K 内取 1、在 K 外取 0——那是「标记成员」, 作用与这里完全相反。优化里的 ι_K 取 0 与 $+\infty$, 是「对可行者免费、对不可行者判死」。读文献时若看到 indicator 一词, 先看取值集合是 $\{0, 1\}$ 还是 $\{0, +\infty\}$, 这决定了它在公式里扮演的是计数器还是惩罚项。再补一句机制: K 是闭凸集时 ι_K 是凸的、下半连续的, 所以在凸优化框架下它合法, 且 $\min$ 一定取得到 (不闭时只能取下确界); 它与投影算子的关系是 $\text{prox}_{\iota_K}(v) = \Pi_K(v)$ 。给一个能复算的例子。取 $f(z) = (z - 3)^2$, $K = [0, 1]$, 则 $\min_{z \in K} f(z)$ 的最优解是 $z = 1$ (端点), 最优值 4。现在验算再加指示函数后没变: 在 $z = 1$ 处右导数为 $f'(1) = -4$, 说明目标函数还想往 3 走; 但越过 1 之后 ι_K 从 0 跳到 $+\infty$, 代价直接不可取, 于是最优点被钉在边界。这个「单侧无穷斜率」就是它强制约束的机制: 不是把解拉回来, 而是让越界方向根本不存在有限代价。还可以顺手做一条机器层面的连接。 $+\infty$ 参与浮点算术会污染: $0 \cdot (+\infty)$ 得 NaN, $+\infty - (+\infty)$ 也得 NaN (CSAPP 里 IEEE-754 的约定, 写一句 `1.0/0.0 - 0.0*inf` 就能验证), 因此真写代码时不会真塞 ∞ , 而是用布尔掩码或直接调用 Π_K , 例如 `np.clip(z,0,1)`。这也解释了为什么「投影梯度法」与「近端梯度法」在书里分成两套写法却本质同源。跨课程上, 凸优化 (Boyd) 里这就是「硬约束的示性函数表示」, 与拉格朗日乘子法并列: 硬约束给无穷惩罚, 等式或不等式约束给有限惩罚加乘子, 两条路在对偶理论处汇合; 概率论 (Bertsekas) 里的「几乎必然」也是同一味道——把概率为零的事件从「不太可能」升格为「排除出可行域」。区别在于概率论的零测集依赖测度, 而这里的不可行性是确定的、与任何测度无关。命名模板: **无穷惩罚编码法**——把约束写成集合 K , 用 ι_K 塞进目标函数, 再把约束问题当成无约束问题写近端或投影。记住: 换完后最优值不变, 但可微性变了, 所以「先无约束化、再选算法、最后退回原约束」这三步顺序不能反。

为什么能编码成功? 因为最小化时 $+\infty$ 是不可接受的值, 任何越界的候选点会被直接淘汰, 所以极小值点只可能落在 $g(z) = 0$ 的区域, 也就是 K 内部。反过来, 若不用 $+\infty$ 而用有限的大数 M 作罚, 那就只是把违反约束的价格标成 M , 只要 M 不够大就仍可能出现越界的极小点, 约束是软的; 指示函数把价格标成无穷, 约束才是硬的。验算一维例子: 取 $K = [0, 1]$, $\text{minimize } f(x) = x^2 \text{ subject to } x \in K$, 最优值是 0、最优点在 $x = 0$; 换成 $f(x) = (x - 3)^2$, 约束版本的最优值是 4 (在 $x = 1$), 而无约束化版本在 $x = 1$ 处给出 $1 + 4 = 5$, 在 $x = 3$ 处给出 $0 + \infty$, 取最小仍是 $x = 1$ 处的 5——最优点相同, 最优值相差 g 的取值, 这一点在做对偶时必须小心: 常数偏移会把整个对偶函数整体下移, 零点位置不变, 所以对偶最优解不受影响, 受影响的只是对偶问题的目标值, 这是标准的 Fenchel 对偶记账方式。

再算 ADMM 把 (22) 分裂后的 z 步: $z_{k+1} = \arg \min_z (g(z) + \frac{\rho}{2} \|x_{k+1} - z + u_k\|^2)$ 。因为 g 只认在不在 K 里, 这个最小化就是在 K 内最小化那个二次距离, 答案自然是 $x_{k+1} + u_k$ 在 K 上的投影, 即 $z_{k+1} = P_K(x_{k+1} + u_k)$ 。这就是正文那句「一个 minimization 与一个 projection 交替」的来历: **指示函数把抽象的约束最小化变成了几何上的投影**。而投影对盒子、球、非负象限这些常见 K 都有显式公式: 盒子上逐分量截断到区间, 球上把过长的向量按范数缩放, 非负象限把负分量清零, 三种投影都是逐元素操作, 所以带指示函数的 ADMM 每轮几乎不额外花钱。走一步看看: 取 $K = [0, 1]$ 、 $\rho = 2$ 、 $x_1 = 5$ 、 $u_1 = 0$, 二次项 $\frac{\rho}{2}(5 - z)^2$ 在实数上的极小点是 $z = 5$, 投影到 K 得 $z_2 = 1$, 而带硬约束的版本这一步也只能取 $z = 1$, 两者完全相同—— g 做的事只是把无约束的 5 裁回 K 的边界。

把它抽象为模板并命名: **指示函数换投影法 (indicator-to-projection)**。三步走: (1) 把约束 $x \in K$ 改写成加一个指示函数 g ; (2) 引入冗余变量 z 与约束 $x = z$, 让 g 只作用在 z 上, 这样 x 步保持二次型光滑、 z 步保持几何化; (3) 在 ADMM 的 z 步中, g 的存在使最小化退化为到 K 的投影, 投影有闭式时整个算法就没有内层循环。检验要点: 必须有 K 闭且凸, 投影才存在且唯一——若 K 非凸, P_K 可能多值, 整个证明链条断裂; 若 K 开, 如 $K = (0, 1)$, $P_K(5)$ 想把点停在 1, 而 $1 \notin K$, 投影根本不存在, 算法当步就失效。这就是正文特意补一句「指示函数 $g(z)$ 是闭的、凸的」的原因, 它不是客套话, 是逻辑必需。

顺手连一条跨课程: 投影天然幂等, $P_K(P_K(v)) = P_K(v)$, 把任何向量先拉进 K 再拉一次结果不变, 这正好对应 6.042J 里作用在集合上的函数的幂等性, 与等价关系的自反性表面上同形, 但一个是函数性质、一个是关系性质, 别混为一谈。投影还是非扩张的: $\|P_K(v) - P_K(w)\| \leq \|v - w\|$, 不放大距离, 这正是 ADMM 收敛分析的引擎。

$$L(x, z, u) = f(x) + g(z) + \frac{1}{2}\rho \|x - z + u\|^2 \quad (23)$$

注意通常的罚项 $\lambda^T(x - z)$ 是怎样通过缩放步骤（scaling step）被折进方程（23）的。然后 ADMM 把（22）分裂（split）成一个最小化（minimization）与一个投影（projection）交替进行的迭代：

ADMM

$$\begin{aligned} x_{k+1} &= \operatorname{argmin}_x \left[f(x) + \frac{1}{2}\rho \|x - z_k + u_k\|^2 \right] \\ z_{k+1} &= \text{projection of } x_{k+1} + u_{k+1} \text{ onto } K \\ u_{k+1} &= u_k + x_{k+1} - z_{k+1} \end{aligned}$$

其中投影 $z_{k+1} = P_K(x_{k+1} + u_{k+1})$ 表示到集合 K 上的投影（projection）。

[example]

Example 2 (例 2) Soft thresholding (软阈值)

一个重要的问题有精确解。 $f(x) = \lambda\|x\|_1 = \lambda|x_1| + \cdots + \lambda|x_n|$ 分裂成 n 个标量函数 $\lambda|x_i|$ 。用 ADMM 做分离（separation），就把问题化为对每个特殊函数 f_1 到 f_n 的极小化：

$$f_i(x_i) = \lambda|x_i| + \frac{1}{2}\rho(x_i - v_i)^2 \quad \text{with } v_i = z_i - u_i$$

解 x_i^* 就是 v_i 的「软阈值」（soft thresholding），其图像画在第 VI.4 节中：

$$x_i^* = \left(v_i - \frac{\lambda}{\rho} \right)_+ - \left(-v_i - \frac{\lambda}{\rho} \right)_+ = v_i \left(1 - \frac{\lambda}{\rho|v_i|} \right)_+ \quad (24)$$

[note] 译者注（证明策略模板）： 公式 (24) 那个分段表达式看起来像三条规则拼起来的补丁，其实它来自一个一维的、可以手算的极小化问题。请务必自己推一遍，因为后面 §III.4 里所有 ℓ_1 子问题有闭式解的说法都靠它撑腰。它要解决的设计问题是：目标函数里同时有光滑的二次项和不光滑的 $|z|$ ，而 $|z|$ 在 $z = 0$ 处没有经典导数，所以「令导数为零」这一步失效。它来自的母问题是 $\min_z \frac{1}{2}(z - z_0)^2 + \lambda|z|$ ，其中 $\lambda > 0$ 、 z_0 是给定的输入（在后面章节里就是当前残差与回归量的内积除以步长）。为什么加这一项？因为惩罚非零分量能产生稀疏解，这在后面反复出现。从第一性原理推。若 $z > 0$ ，则 $|z| = z$ 光滑，驻点条件 $z - z_0 + \lambda = 0$ 给出 $z = z_0 - \lambda$ ，而这个解落在正半轴当且仅当 $z_0 > \lambda$ ；若 $z < 0$ ，同理 $z - z_0 - \lambda = 0$ 给出 $z = z_0 + \lambda$ ，落在负半轴当且仅当 $z_0 < -\lambda$ ；剩下 $|z_0| \leq \lambda$ 时两个正负解都跑到区间外，剩下来的候选只有 $z = 0$ 。于是闭式解就是 $z_* = z_0 - \lambda$ ($z_0 > \lambda$)， $z_* = z_0 + \lambda$ ($z_0 < -\lambda$)，否则 $z_* = 0$ 。这就是软阈值算子。用次梯度的语言说：0 属于 $\partial(\frac{1}{2}(z - z_0)^2 + \lambda|z|)$ 在 $z = 0$ 处给出 $-z_0 + \lambda[-1, 1] \ni 0$ ，也就是 $|z_0| \leq \lambda$ 。次梯度在原点是一整个区间而不是一个数，这个「区间」就是阈值化的根：非光滑点不是障碍，而是解被锁在原点的原因。代两个具体数值验算。取 $\lambda = 0.5$ 、 $z_0 = 3$ ，则 $z_* = 3 - 0.5 = 2.5$ ，目标值 $\frac{1}{2}(0.5)^2 + 0.5 \times 2.5 = 0.125 + 1.25 = 1.375$ ；若取 $z = 0$ 得 4.5，若取 $z = 3$ 得 1.5，都更大，确认 2.5 胜出。再取 $z_0 = 0.4$ 、 $\lambda = 0.5$ ，因为 $|z_0| < \lambda$ ，解是 $z_* = 0$ ，目标值 0.08；若取 $z = 0.4$ 则得 0.2，确实更差，验证了阈值区间。两步手算加一次比较，就复现了公式 (24) 的全部分支。跨课程连接：凸优化（Boyd）里这正是近端算子 $\operatorname{prox}_{\lambda|\cdot|}(z_0)$ ，而 ℓ_1 的近端算子就是软阈值；如果只学过 Strang 线性代数与 18.065，可以把它当成「最小二乘解的一维版本」——把 z_0 看成 $A^T b$ 、把 λ 看成正则化强度，你会发现 prox 的两种看法（对偶范数投影、Moreau 分解）在这里都退化成一个简单的截断。再看概率论（Bertsekas）：它与「条件期望的截断」味道相近，但区别要记牢——期望截断来自信息集与均方误差，最优解天然落在观测范围内；而这里的 $z_* = 0$ 是在罚项逼迫下的边界解，是阈值化的结果，不是「取不到更大的值」。与 CSAPP 的连接在于实现：这个三分支只涉及一次浮点比较和一次符号提取，没有循环、不会溢出，所以 §III.4 的向量化实现才敢把整块坐标一次做软阈值，而不是逐个坐标做内层迭代。命名模板：**软阈值法**——先写出「一元二次加 $\lambda|z|$ 」的母问题，再用次梯度区间 $[-\lambda, \lambda]$ 作为「死区」，死区内判 0、死区外按符号平移 λ 。检验诀窍：把 $\lambda \rightarrow 0^+$ 代入，三分支应退化为恒等映射；把 z_0 固定、 λ 增大，解应更快被压到 0。

一维问题是：minimize $f(t) = \lambda|t| + \frac{\rho}{2}(t - v)^2$ 。分成 $t > 0$ 、 $t < 0$ 、 $t = 0$ 三段。 $t > 0$ 时 $f' = \lambda + \rho(t - v)$ ，令其为零得 $t = v - \lambda/\rho$ ，但这个候选点只有在 $v > \lambda/\rho$ 时才落在 $t > 0$ 区间内； $t < 0$ 时 $f' = -\lambda + \rho(t - v) = 0$ 得 $t = v + \lambda/\rho$ ，仅在 $v < -\lambda/\rho$ 时有效；当 $|v| \leq \lambda/\rho$ 时两个驻点都跑出了各自区间，此时唯一的极小值点就是那个不可导的 $t = 0$ 。验算：取 $\lambda/\rho = 1$ 、 $v = 0.6$ ， $t > 0$ 段给出 $-0.4 < 0$ 无效， $t < 0$ 段给出 $1.6 > 0$ 无效，于是 $t^* = 0$ ；取 $v = 3$ ，得 $t^* = 2$ ，把 $|t|$ 从 3 削到 2，整整削掉一个 λ/ρ 。

把它抽象成一个可复用的模板，命名为**一维分段接力法（1-D piecewise relay）**：处理 $\lambda|x_i| +$ 光滑项 这类目标时，(1) 先在 $x_i > 0$ 与 $x_i < 0$ 两个开区域各自求驻点并记下解析式；(2) 用驻点是否落在本区域内做筛选，这一步就是接力的交接棒；(3) 若都不落在区域内，则极小值只能落在不可导点 $x_i = 0$ 上。模板的适用条件是目标函数凸且不可导点只有有限个（这里是每个坐标的原点）。把它用在 n 维可分离问题上，因为 $\sum_i$ 与 $\min$ 可以交换，整体解就是每个坐标独立做一次一维接力——**这正是 ADMM 能把大问题拆成 n 个标量问题的原因**。

顺带记住这个算子的两个名字：软阈值（soft thresholding）与收缩（shrinkage）。从工程角度看它的作用极其直观——把每个分量朝零吸 λ/ρ 的距离，幅度不足 λ/ρ 的直接吸成零。对照岭回归（ridge regression）：在同样的记号下 ℓ_2 惩罚给出的解是 $v/(1 + \lambda/\rho)$ ，是**乘性缩放**，所以再小也不为零； ℓ_1 给的是**加性平移后截断**，所以能精确置零。这个差别就是 LASSO 与 ridge 的差异所在。

换个角度验证。不看驻点，改看次梯度： $0 \in \lambda\partial|t| + \rho(t - v)$ ，其中 $t > 0$ 时 $\partial|t| = \{1\}$ ， $t < 0$ 时 $\{-1\}$ ， $t = 0$ 时 $[-1, 1]$ 。把后一种代进去， $0 \in \lambda[-1, 1] - \rho v$ ，等价于 $|v| \leq \lambda/\rho$ 。分段公式本来就是最优性条件的解，那个三条规则拼补丁的观感纯属误解。习惯次梯度还有一个好处： $|t|$ 在 $t = 0$ 处是往上的尖点，最优点可能正好落在那里，而导数判据在那里失效，只有次梯度能给出判断。

一条跨课程连接。6.042J 讲集合划分时，正负实轴去掉原点本身就是两个天然等价类， $t = 0$ 是被单独拎出来的退化类；这里的接力就是按等价类分情况求解，驻点落在哪个区间就是在判定候选解属于哪个等价类，三类各取一次再比较就是取遍所有等价类。区别要说清：等价关系给的是分完再遍历的结构保证，它不保证这三块合起来就是全部答案；真正干活的是凸性——有了凸性，局部最优才等于全局最优，三个候选值取最小才算完整，这也正是次梯度判据成立的前提。

同一件事在凸优化里有名字： t^* 就是 $\lambda|t|$ 在点 v 处的近端算子（proximal operator），这也是 LASSO 的坐标下降每步都要调用的那个一维 oracle；ISTA 是梯度下降一步再套一次软阈值，ADMM 则把它嵌进增广拉格朗日的坐标子问题里，三者的区别只在调用方式与收敛保证，算子本身是同一个。

这个阈值函数（thresholding function） x_i^* 不但是这个重要非线性问题的显式解（explicit solution），还是一个收缩算子（shrinkage operator）：每一个 v_i 都朝零移动。我们很快就会看到，这种软阈值就是一个「邻近」（proximal）算子（operator）。

本页为书页 190；页眉「Low Rank and Compressed Sensing」（低秩与压缩感知）与页码为重复元素，不译。

[example]

例 3（Example 3）非负矩阵分解（Nonnegative Matrix Factorization, NMF）： $A \approx CR$ ，其中 $C_{ij} \geq 0$ ， $R_{ij} \geq 0$ 。交替方向乘子法（alternating direction method of multipliers, ADMM）从一次交替最小化（alternating minimization）起步——这是我们最偏爱的矩阵分解方式。

Find $C \geq 0$	Minimize $\ A - CR\ _F^2$ with $R \geq 0$ fixed
Find $R \geq 0$	Minimize $\ A - CR\ _F^2$ with $C \geq 0$ fixed

Boyd 等人接着指出一个等价的问题：把 C 与 R 挪进约束 $X = CR$ 里：

$$\text{NMF} \quad \text{Minimize } \|A - X\|_F^2 + I_+(C) + I_+(R) \text{ with } X = CR$$

ADMM 再补上第三步，用来更新对偶变量 U ；同时引入一个新变量 X ，由约束 $X = CR \geq 0$ 把它绑定。其中的指示函数（indicator function） $I_+(C)$ 在 $C \geq 0$ 时取 0，其他情形取 $+\infty$ 。现在 ADMM 分裂为对 X 、 C 的最小化，再与对 R 的最小化交替进行：

$$\begin{aligned} (X_{k+1}, C_{k+1}) &= \operatorname{argmin} \left[\|A - X\|_F^2 + \frac{1}{2}\rho\|X - CR_k + U_k\|_F^2 \right] \text{ with } X \geq 0, C \geq 0 \\ R_{k+1} &= \operatorname{argmin} \|X_{k+1} - C_{k+1}R + U_k\|_F^2 \text{ with } R \geq 0 \\ U_{k+1} &= U_k + X_{k+1} - C_{k+1}R_{k+1} \end{aligned}$$

X_{k+1} 的各行、 C_{k+1} 的各行，随后是 R_{k+1} 的各列，全都可以彼此独立地求出。这种分裂（splitting）促进了并行计算。

[example]

例 4（Example 4） LASSO 通过加入 ℓ^1 罚项，追求 $Ax = b$ 的稀疏解（sparse solution）：

$$\text{LASSO} \quad \text{Minimize } \frac{1}{2}\|Ax - b\|^2 + \lambda\|x\|_1 \tag{25}$$

这个问题立刻可以分裂成 $f(x) + g(z)$ ，约束为 $x - z = 0$ 。 f 的子问题是最小二乘（least squares），于是我们遇到 $A^\top A$ 。再把增广项 $\frac{1}{2}\rho\|Ax - b\|^2$ 加进来。

缩放形式的 ADMM（Scaled ADMM） $x_{k+1} = (A^\top A + \rho I)^{-1}(A^\top b + \rho(z_k - u_k))$ **软阈值（Soft thresholding）** $z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k)$ **对偶变量（Dual variable）** $u_{k+1} = u_k + x_{k+1} - z_{k+1}$

只要把 $A^\top A + \rho I$ 的 LU 因子（LU factors）存起来，第一步就简化为回代（back substitution）。Boyd 等人还指出：把 $\|x\|_1 = \sum |x_i|$ 换成 $\|Fx\|_1 = \sum |x_{i+1} - x_i|$ ，这个例子就变成「全变差去噪」（total variation denoising）。他们那本在线书籍给出了出色而有说服力的 ADMM 实例。

[note] 译者注（跨课程连接）：这句话把 ADMM 与「计算机系统基础」课（教材 CSAPP，即《深入理解计算机系统》）和「数值线性代数 / 数值分析」里的两件事缝在一起：**把循环里不变的计算提到循环外**，以及**条件数决定误差放大倍数**。先说这段在设计上要解决什么问题。ADMM 的每一次迭代都要解一个形如 $x^{k+1} = \arg \min_x \left(\frac{\rho}{2}\|Ax - z^k + u^k\|^2 + g(x) \right)$ 的子问题；如果 g 可微或结构简单，这一步最终归结为解线性方程组 $(A^\top A)x = b^k$ ，其中右端项 b^k 逐轮变化，而矩阵 $A^\top A$ 从不变化。于是**真正的设计问题**变成：怎样让「每轮都要重复的昂贵运算」不重复？答案是：把与循环下标 k 无关的运算外提（loop-invariant code motion）。 $A^\top A$ 的构造是 $O(n^3)$ ，一旦把它预先因式化，例如 Cholesky 分解 $A^\top A = LL^\top$ ，那么每轮只剩前代与回代两步，各 $O(n^2)$ 。这正好是 CSAPP 里那件事的数学版：循环不变量外提、强度削减（把乘法换成加法）、以及用缓存局部性换取常数因子。 L 一旦算好就常驻内存，每轮只是顺序读写它，访存模式规则、cache 命中率高；若每轮重新因式化，则既浪费 $O(n^3)$ 的 CPU，又不断把同一批矩阵元素从内存搬进搬出。从第一性原理看，为什么必须这样做？因为 A 是问题的**数据**， k 是**算法的时间步**；数据不随时间变，时间步却要跑几百上千次。把「只依赖数据、不依赖时间步」的部分提前到循环外，复杂度就从「乘 k 」变成「加 k 」：总代价 $O(n^3) + O(kn^2)$ 而不是 $O(kn^3)$ 。这也是 ADMM「每步子问题都要能被快速精确求解」这一前提的具体落地方式——如果每步都重新因式化，ADMM 就退化成一个每轮都很贵的算法，失去了「简单一步」的意义。

给一个可手算的验算。取 $A = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ ，则 $A^\top A = 2I$ ，直接看出 $L = \sqrt{2}I$ ，因式化一次即永久可用。若右端项从 $b^0 = (2, 0)^\top$ 变到 $b^1 = (0, 2)^\top$ ，每轮的解只是把 b^k 除以 2： $x^0 = (1, 0)^\top$ 、 $x^1 = (0, 1)^\top$ ，全程没有第二次分解。再对比「不预先分解」的情形：每轮都要从零重算 $A^\top A$ 并求逆， $n = 1000$ 时单轮约 10^9 次浮点运算，一千轮就是 10^{12} ，而预先分解后总代价降到约 $10^9 + 1000 \times 10^6$ ，量级差了一千倍。

第二件事是条件数。解线性方程组的扰动界说：相对误差被 $\kappa(A) = \|A\| \|A^{-1}\|$ 放大，即 $\frac{\|\delta x\|}{\|x\|} \lesssim \kappa(A) \frac{\|\delta b\|}{\|b\|}$ 。这解释了为什么数值线性代数课反复强调「不要显式求逆」「优先用正交变换（QR）而非正规方程」。跨课程连接上：**6.042J 离散数学**里的等价关系与划分，为「同一个矩阵属于同一类、可以共用一个分解」提供了抽象语言，但 6.042J 研究的是关系的代数性质（自反、对称、传递），数值课关心的是浮点误差这个具体代价；**Strang 线性代数与 18.065** 中的 LU/QR 分解研究的是分解的存在性与唯一性，而 CSAPP 关心的是分解结果怎样放进寄存器与缓存、以怎样的访问顺序被反复使用。区别一句话：数学课保证「分解是对的」，系统课保证「分解被复用得起」。

把这个思路命名一下，可叫**循环不变量外提 + 条件数预算**：先问「哪些量不随迭代变」，再问「它们的条件数允许误差放大多少倍」。

$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z_k - u_k))$ 每轮的系数矩阵都是同一个 $A^T A + \rho I$ ，只有右端项在变。若每轮都重新消元，代价是 $O(n^3)$ ；一旦预先算出 $A^T A + \rho I = LU$ ，之后每轮只需解 $Ly = P$ 与 $Ux = y$ 两个三角方程组，代价降到 $O(n^2)$ 。代入 $n = 1000$ 、迭代 $k = 50$ 估一下量级： $n^3 = 10^9$ 次浮点乘加， $kn^2 = 5 \times 10^7$ 次，相差二十倍；反过来说，若每轮重做分解，就等于多付五十份分解钱。这正是 CSAPP 第 5 章讲的代码移动（code motion）：把不变的计算提到循环外；第 6 章讲的局部性（locality）解释了回代为什么快——三角矩阵每轮顺序扫一遍内存，缓存命中率。区别在于 CSAPP 管的是指令与缓存，这里管的是分解结果的复用：循环体从「分解 + 回代」缩成「回代」，理论加速比是 $O(n)$ 量级。Strang 讲 MATLAB 反斜杠命令时用的是同一套逻辑：A\b 内部先分解、再回代，只是把分解藏了起来；自己写 ADMM 循环时，分解必须显式提到循环外，否则这份加速白丢。

再看 ρI 的深层好处。 $A^T A$ 可能奇异或接近奇异，此时 $\kappa(A^T A) = \kappa(A)^2$ 极大，舍入误差会被放大：CSAPP 第 2 章讲过，双精度浮点只有约 10^{-16} 的相对精度，若 $\kappa \sim 10^{12}$ ，算出的解可能一位有效数字都不剩。 $A^T A + \rho I$ 把每个特征值 σ_i^2 抬成 $\sigma_i^2 + \rho$ ，条件数从 $\sigma_{\max}^2 / \sigma_{\min}^2$ 降到 $(\sigma_{\max}^2 + \rho) / (\sigma_{\min}^2 + \rho)$ 。取 $\sigma_{\max} = 1$ 、 $\sigma_{\min} = 0$ （奇异）、 $\rho = 0.01$ ，条件数从无穷降到 $1.01/0.01 = 101$ ；取 $\sigma_1^2 = 1$ 、 $\sigma_2^2 = 0.0001$ 、 $\rho = 0.25$ ，条件数从 10^4 降到 $1.25/0.2501 \approx 5$ 。零特征值被抬成正数，秩亏矩阵变成满秩正定矩阵，LU 与 Cholesky 都不必再处理零主元或极小主元。要记住 ρ 是算法内部的参数：它不改动 LASSO 的解（迭代收敛到的不动点仍满足原问题的最优性条件），只改变每步子问题的求解难度。

顺带校正一处常数：若 (25) 的数据项取 $\|Ax - b\|_2^2$ （不带 $\frac{1}{2}$ ）、增广项取 $\frac{\rho}{2} \|x - z + u\|_2^2$ ，回代矩阵本应是 $2A^T A + \rho I$ ；书中写成 $A^T A + \rho I$ ，相当于把因子 2 折进了 ρ 的标定（等价于把数据项读作 $\frac{1}{2} \|Ax - b\|_2^2$ ）。这不影响上面的结论： ρI 的作用仍然只是把谱抬高零。

同一门课（MIT 18.065《Matrix Methods in Data Analysis, Signal Processing, and Machine Learning》）里还有一条对偶路线：截断 SVD，把小奇异值直接当零丢掉。两者的连接路径相同——都在对付近零奇异值；区别在时机与代价： ρI 在求解之前把整个谱推离零，所有方向都被改动，换来稳定；截断 SVD 在求解之后按阈值丢分量，省事，但丢错的风险与阈值的选择都得自己承担。

代价也要讲明： ρ 变大，条件数更好，但增广项的权重更大，迭代先被拉去满足约束，收敛到高精度所需的轮数反而变多。Boyd 等人的综述正强调这一点——ADMM 常常在几十轮内达到中等精度，要达到高精度却很慢。

Boyd 等人还指出：把罚项 $\|x\|_1 = \sum |x_i|$ 换成 $\|Fx\|_1 = \sum |x_{i+1} - x_i|$ （ F 是相邻分量作差的差分矩阵），这个例子就变成全变差去噪（total variation denoising）。他们那本在线书籍给出了出色而有说服力的 ADMM 实例。

Strang §III.4, p.190

本页为书页 191；页眉「III.4 Split Algorithms for $\ell^2 + \ell^1$ 」（ $\ell^2 + \ell^1$ 的分裂算法）与页码为重复元素，不译。本页正文出现标题「Matrix Splitting and Proximal Algorithms」（矩阵分裂与近端算法）。

[section]

矩阵分裂与近端算法（Matrix Splitting and Proximal Algorithms）

把 $Ax = b$ 与 ADMM 联系起来的第一条线索，来自把矩阵 A 分裂为 $A = B + C$ 。在经典例子中， B 可以取 A 的对角部分（diagonal part），也可以取 A 的下三角部分（lower triangular part）——这就是 Jacobi 分裂与 Gauss-Seidel 分裂。Douglas-Rachford 与 Peaceman-Rachford 改进了它们：用加入 αI 的办法做正则化（regularization），并在 B 与 C 之间交替：

$$\begin{aligned} (B + \alpha I)x_{k+1} &= b + (\alpha I - C)z_k \\ (C + \alpha I)z_{k+1} &= b + (\alpha I - B)x_{k+1} \end{aligned} \tag{26}$$

[note] 译者注（OCR 修正与来路）：(26) 的原文字符已被 OCR 破坏，此处按 Douglas-Rachford / Peaceman-Rachford 的交替方向隐式（ADI）格式重建：第一步用上一次的 z_k ，第二步用刚算出的 x_{k+1} ，两步各解一个加了 αI 的方程组；这一形式与下文 (28a)(28b) 的 x 、 z 交替逐一对应。重建的依据也说明一下：第一式把 Cz_k 留在右边、第二式把 Bx_{k+1} 移到右边，正是「用最新算出的值」的交替精神；令 $\alpha = 0$ 并取共同解 $x = z$ ，两式都化为 $Ax = b$ ，说明格式没有偷换原问题。

先把 (26) 放回「数值分析」（计算方法）课的语境。把 A 拆成 $A = B + C$ 、迭代 $Bx_{k+1} = b - Cz_k$ ：取 B 为对角部分就是 Jacobi 迭代，取 B 为下三角部分就是 Gauss-Seidel 迭代——你在解线性方程组那一章算过它们的迭代矩阵与收敛条件（系数矩阵对角占优，或迭代矩阵的谱半径小于 1）。连接路径是：这两个经典迭代都要求 B 可逆、收敛还要碰运气；而 (26) 给两边各加 αI ，让 $B + \alpha I$ 与 $C + \alpha I$ 变得一定可逆（ B, C 半正定时只要 $\alpha > 0$ 就正定；一般情形取足够大的 $\alpha > 0$ 也能避开奇异），既不怕零主元，又能让两步交错推进。

区别有两层。第一层在解法：Jacobi / Gauss-Seidel 是单向迭代，一次只更新一个未知块；(26) 在 B 与 C 之间来回交替，是「双步」格式，也正是 ADMM 的雏形。第二层在对象：1950 年代中期，Peaceman-Rachford (1955) 与 Douglas-Rachford (1956) 提出这个格式，是为了解热传导这类抛物型偏微分方程的隐式差分方程，那里的 B, C 来自对二维网格两个方向的分裂，一次求解把二维问题拆成两步一维问题；而本页的优化版本面对的是不可微的凸目标， B 换成了次微分 ∂F_2 ， αI 的作用也从「保证可逆」变成「把逆算子写成近端算子」。

交替本身并不保证收敛， αI 才是换来稳定性的那一项。标量情形最能看清： $A = b + c$ 时两步复合的迭代矩阵是 $T = \frac{(\alpha-b)(\alpha-c)}{(\alpha+b)(\alpha+c)}$ ，只要 $\alpha > 0$ 且 $b, c > 0$ 就有 $|T| < 1$ ；若取 $\alpha = 0$ ，则 $T = 1$ ，迭代原地打转。这就是 (26) 中必须出现 αI 的原因，也是下文把 α 叫作缩放因子 (scaling factor) 的由来。

读 (28) 时留意符号： u_k 是对偶变量，与 $Ax = b$ 的右端项 b 不是一回事——优化对应物里 (26) 的右端项已经取 $b = 0$ ，这个字母被借去当对偶变量了。

这个思想出现在 1955 年，当时处理的问题还是线性的。它引出了更深刻的非线性思想：近端算子 (proximal operators) 与单调算子 (monotone operators，多值映射 T 满足 $\langle u - v, x - y \rangle \geq 0$ ，凸函数的次微分 ∂F 就是典型例子)。一个特别之处，是若干重要的近端算子都有精确公式 (exact formulas) ——尤其是例 2 中那个引出软阈值 (soft thresholding) 与收缩 (shrinkage) 的 ℓ^1 最小化问题。我们将定义 **Prox** 算子 (近端算子)，并把它与 ADMM 联系起来。

在优化 (optimization) 中，与那些矩阵方程 (26) 对应的是 $b = 0$ 且 $B = \nabla F_2$ 。这里的 F_2 是 ℓ^2 部分，它的梯度 ∇F_2 (在不可微点换成「次梯度」 ∂F_2) 实际上是线性的。于是 $\partial F_2 + \alpha I$ 对应于 $B + \alpha I$ ，而它的逆就是我们所说的「近端算子」：

$$\mathbf{Prox}_F(\mathbf{v}) = \operatorname{argmin} \left(F(\mathbf{x}) + \frac{1}{2} \|\mathbf{x} - \mathbf{v}\|_2^2 \right) \tag{27}$$

为这种近端方法 (proximal approach，以及把 F 分裂成 $F_1 + F_2$ 的做法) 提供依据的关键事实有两条：

1. ℓ^2 问题已被透彻理解，求解很快。**2.** ℓ^1 问题常常有**闭式解 (closed form solution)**，**靠收缩得到**。

这种双步组合 (double-step combination) ——矩阵双步方程 (26) 在优化中的对应物——就是带缩放因子 (scaling factor) α 的 ADMM 的「近端版本」：

$$\begin{aligned} \text{ADMM} \quad \mathbf{z}_{k+1} &= (\partial F_2 + \alpha I)^{-1} (\alpha \mathbf{z}_k - \alpha \mathbf{u}_k) \\ \mathbf{u}_{k+1} &= (\partial F_1 + \alpha I)^{-1} (\alpha \mathbf{x}_{k+1} + \alpha \mathbf{u}_k) \\ \mathbf{x}_{k+1} &= \mathbf{z}_k + \mathbf{u}_k \end{aligned} \tag{28a, 28b, 28c}$$

总体而言，近端算法 (proximal algorithms) 适用于凸优化 (convex optimization)。对合适的问题，它们收敛得很快。这也包括分布式问题 (distributed problems) ——并行地最小化若干项之和，这正是 ADMM。Parikh 与 Boyd 指出，**Prox** 在「最小化一个函数」与「不要走得太远」之间做出折中。他们把它比作梯度下降 (gradient descent) 中的 $\mathbf{x} - \alpha \nabla F(\mathbf{x})$ ，其中 α 扮演步长 (stepsize) 的角色。而 **Prox** 的不动点 (fixed points) 正是 F 的最小值点。

[note] 译者注 (第一性原理)： 式 (27) 只写了一行，却一次解决了梯度下降最麻烦的两件事：目标不可微，以及步长不好选。先问最基础的问题——为什么不能直接用 $y - \alpha \nabla F(y)$ ？因为 ℓ_1 这类函数在尖点 (左右导数不相等处) 没有梯度，硬取一个次梯度 (subgradient，不可微点处代替梯度的一整套) 会得到不唯一的方向；而且即使可微，梯度步也常要把 α 调得很小才不发散。近端算子换了个思路：下降交给 F 自己，用 $\arg \min$ 精确求解；二次项 $\frac{1}{2} \|\mathbf{x} - \mathbf{y}\|^2$ 只负责「不要走太远」。

把这两件事拆开看，机制就清楚了。第一，不可微为什么会让梯度法失效？在 $F(x) = |x|$ 的原点处，左右导数分别是 -1 与 $+1$ ，次微分是区间 $[-1, 1]$ 。次梯度法必须从这个区间里自己挑一个方向，挑法不唯一，于是迭代路径依赖实现细节，收敛速度也从光滑情形的 $O(1/k)$ 退化到 $O(1/\sqrt{k})$ ，而且没有通用的步长规则保证单调下降。近端算子绕开了「挑方向」：它不解一阶条件，而是直接解一个**强凸**的子问题，子问题的最优解由最优性条件 $0 \in \partial F(x) + \alpha^{-1}(x - y)$ 唯一确定。第二，步长为什么难选？梯度步只在 ∇F 满足 Lipschitz 连续、且 $\alpha < 2/L$ 时才保证收敛， L 常常估不准。近端步里的二次项 $\frac{1}{2} \|\mathbf{x} - \mathbf{y}\|^2$ 的梯度是 $\mathbf{x} - \mathbf{y}$ ，Lipschitz 常数恒为 1，与 F 无关；因此这个子问题永远是 1-强凸加光滑的，不需要对 F 做任何光滑性假设。第三，二次项还有个容易被忽略的作用：它是旋转不变的各向同性惩罚，只度量「离 y 多远」，不偏向任何坐标方向，因此不引入额外偏置。

手算一个验算。取 $F(x) = |x|$ ， $y = 0.3$ ， $\alpha = 0.5$ 。子问题是 $\min_x |x| + \frac{1}{2 \times 0.5} (x - 0.3)^2$ ，即 $\min_x |x| + (x - 0.3)^2$ 。在 $x > 0$ 上导数为 $1 + 2(x - 0.3) = 2x - 0.4$ ，零点在 $x = 0.2$ ，但 $0.2 > 0$ 成立，故先得候选 $x = 0.2$ ；代入得 $0.2 + 0.01 = 0.21$ 。再验 $x = 0$ ：值为 $0 + 0.09 = 0.09$ ，更小；验 $x = 0.3$ ：值为 $0.3 + 0 = 0.3$ 。故最优解是 $x^* = 0$ ，即被阈值「压」到了零。把 y 换成 2：在 $x > 0$ 上导数为 $2x - 1.6$ ，零点 $x = 0.8 > 0$ ，值为 $0.8 + 1.44 = 2.24$ ，而 $x = 0$ 处为 4，故 $x^* = 0.8 = 2 - 0.5$ 。两次手算合起来正是软阈值公式 $x^* = \text{soft}(y, \alpha) = \text{sign}(y) \max(|y| - \alpha, 0)$ ，也验证了阈值刚好等于步长 α ： $-\alpha \leq y \leq \alpha$ 时输出恒为零。

跨课程连接上：**在学的凸优化 (Boyd) 里**，近端算子与 Moreau 包络是同一枚硬币的两面，Boyd 用它推出「近端梯度法 = 对光滑部分做梯度步、对非光滑部分做近端步」；而在学的概率论 (Bertsekas) 中的条件期望 $\mathbb{E}[X \mid \mathcal{G}]$ 也共享同一套「投影到更简单的集合、使偏差最小」的结构。区别在于：**凸优化关心的是算子是否非扩张、是否有不动点 (收敛性证明)**，**概率论关心的是可测性与最优预测误差 (估计精度)**。还可以接一条 6.042J 离散数学的路：软阈值把区间 $[-a, a]$ 里的所有实数映射到同一个值 0，这本质上是一次划分，把「足够小就视为噪声」编码进映射里；但 6.042J 研究的是划分本身的等价类结构，优化课研究的是这个划分带来的统计偏差与稀疏性收益。

最后给个命名模板，叫**近端吸收法**：凡是遇到「不可微 + 步长敏感」，就把线性部分做一步梯度、把非光滑项吸收进一个强凸子问题里精确求解，再用软阈值公式把解写成可复算的收缩形式。

写出最优性条件： $0 \in \partial F(x^+) + (x^+ - y)$ ，移项得 $x^+ = y - \partial F(x^+)$ 。把它与 $x^+ = y - \alpha \nabla F(y)$ 并排看，差别只有两处：步长固定为 1，且导数在**新点** x^+ 处取值。后者正是「数值分析」课里解常微分方程的后向欧拉 (隐式) 格式 $y_{n+1} = y_n + hf(y_{n+1})$ ：显式格式要限制步长才稳定，隐式格式无条件稳定——所以 Prox 可以记成 $(I + \partial F)^{-1}$ ，对步长不挑。

一维验算最能看清差别。取 $F(x) = \frac{1}{2}x^2$ ，则 $\text{Prox}_F(y) = \arg \min \frac{1}{2}x^2 + \frac{1}{2}(x - y)^2$ ，求导得 $x + (x - y) = 0$ ，即 $x^+ = y/2 = (1 + 1)^{-1}y$ ，与公式一致；而梯度下降取 $\alpha = 1$ 时 $x^+ = y - y = 0$ ，一步砸到原点 (步长过大，一步越过极小点)，要稳定必须 $\alpha < 2$ ，且 α 越小越慢。同一个 $\alpha = 1$ ，Prox 给出的是「减半」这个恰到好处的收缩。

不动点也值得算一遍： $x^* = \text{Prox}_F(x^*)$ 当且仅当 $0 \in \partial F(x^*) + (x^* - x^*)$ ，即 $0 \in \partial F(x^*)$ ——这恰好是 F 的最优性条件。于是「求 F 的最小值点」与「求 Prox 的不动点」是一件事，近端算法因此自带不动点算法的收敛理论，而不必依赖「下降量越来越小」的直觉。

补一句「 ℓ_1 问题常有闭式解」是什么意思： ℓ_1 范数的 Prox 就是软阈值， $\text{Prox}_{\lambda\|\cdot\|_1}(y)$ 的第 i 个分量是 $\text{sign}(y_i) \max(|y_i| - \lambda, 0)$ ，一行就能算完；而 ℓ_2 的 Prox 归结为一个线性方程组。正文那两条关键事实就这样各自落地。

最后看 (28a) 为什么长成 $(\partial F_2 + \alpha I)^{-1}$ ： F_2 是 ℓ^2 部分，它的次微分就是线性映射 ∂F_2 （正文给「次梯度」加引号正是提醒这一点），于是 $\partial F_2 + \alpha I$ 就是矩阵 $\partial F_2 + \alpha I$ ，求逆相当于做一次 Prox，每轮只解一个线性方程组。可复用的模板：**隐式步—不动点模板**——把更新写成 $x^+ = (I + \partial F)^{-1}(y)$ ，先证「不动点等价于最优性条件」，再借不动点迭代的收敛条件谈收敛。

以下是两份出色的参考文献，以及一个附带示例源码的网站：

其一，P. Combettes 与 J.-C. Pesquet 的《Proximal splitting methods in signal processing》（信号处理中的近端分裂方法），收入《Fixed-Point Algorithms for Inverse Problems》（逆问题的不动点算法），Springer (2011)，arXiv: 0912.3522。**其二**，N. Parikh 与 S. Boyd 的《Proximal algorithms》（近端算法），载于 Foundations and Trends in Optimization 1 (2013) 123–321。**书籍与代码 (Book and codes)**：http://stanford.edu/~boyd/papers/pdf/prox_algs.pdf

Strang §III.4, p.191

本页为书页 192，延续 §III.4（Split Algorithms for A^+ ， A^+ 的分裂算法）中近端算法的讨论；结构单元：图 III.3 图注、Bregman 距离定义式 (29)、次梯度（subgradient）讨论、小节「Split Bregman and Split Kaczmarz」、线性化 Bregman 迭代 (30)(31)、目标函数 (32)。页眉与页码为重复元素，不译。

[note] 译者注 (OCR 修正)：本页 OCR 噪声较重： $\mathcal{F}$ / $\mathcal{O}$ / $\mathcal{G}$ 为 $f(x)$ ， $\mathcal{D}$ / $\mathcal{F}$ 为 D_f ， $\mathcal{y}$ 为 y ， $\mathcal{\lambda}$ 为 λ ， $\mathcal{I}$ 为范数竖线， $\mathcal{O}$ 为 $\rangle$ 。公式 (30)(31)(32) 的原文字符严重损坏，现按 linearized Bregman iteration（线性化 Bregman 迭代）的标准形式重建。

图 III.3 图注：光滑函数 $f(x)$ ——每个点处只有一条切线，其斜率为 ∇f ；有尖点的函数 $f(x) = |x|$ ——在尖点处有许多条切线，其斜率取自 subgradient（次梯度） ∂f 。

（原图无法翻译：左图为光滑曲线与它在各点处唯一的切线，右图为带尖点的曲线与尖点处若干条由次梯度给出的切线。）

[section]

布雷格曼距离 (Bregman Distance)

本书中第一次出现 Bregman 这个名字。但 Bregman distance（Bregman 距离）正成为一个日益重要的概念。它不同寻常之处在于缺乏 symmetry（对称性）： $D(u, v) \neq D(v, u)$ 。并且从 u 到 v 的这个距离，还取决于所选的函数 f 。我们确实有 $D \geq 0$ ；并且沿着一条直线，当 $u < w < v$ 时有 $D(w, v) \leq D(u, v)$ ：

Bregman Distance D (Bregman 距离 D)

$$D_f(u, v) = f(u) - f(v) - \langle \nabla f(v), u - v \rangle \tag{29}$$

在 $f(x)$ 的图像出现 corner（尖点）的地方，gradient（梯度） ∇f 总是被替换为 subgradient（次梯度） ∂f 。此时 ∂f 可以是任何一张位于 convex function（凸函数） $f(x)$ 下方的切平面的斜率。一维情形下的标准例子是绝对值 $|x|$ 与 ReLU 函数 $\max(0, x) = \frac{1}{2}(x + |x|)$ ，它们的尖点都出现在 $x = 0$ 。对 $|x|$ 的切线，斜率 $\partial|x|$ 可以取 -1 与 1 之间的任何值；对 $\partial(\text{ReLU})$ ，斜率则位于 0 与 1 之间。这些次梯度使 ∂f 成为 ∇f 的一个有效（但多值）的替代物。

[note] 译者注 (与凸优化的接口)：这一页真正的新东西只有一句：函数有尖点时，用次梯度 ∂f 顶替梯度 ∇f 。要理解它为什么不是权宜之计，得回到你在 18.06 已经反复用过的那套判据。可微凸函数 f 的最小值点满足 $\nabla f(x) = 0$ ；这个条件在尖点处失效（ ∇f 根本不存在），而把它换成 $0 \in \partial f(x)$ 之后，结论原封不动地成立。「属于某个集合」比「等于零」弱，弱的这一头恰好把尖点收编进来。次梯度本身是一个集合：在光滑点它只含一个元素 $\nabla f(x)$ ，在不可微点它可能是一整段区间。一维的定义很朴素： g 是 f 在 x 处的次梯度，当且仅当对一切 z 都有 $f(z) \geq f(x) + g(z - x)$ ，也就是这条直线整体躺在图像下方；它把「切线」推广成「支撑线」，凸性正好保证这种支撑线总是存在。本页两个例子的次梯度可以逐个写出来： $|x|$ 在 0 处的次梯度是 $[-1, 1]$ ； $\text{ReLU } \max(0, x)$ 在 0 处是 $[0, 1]$ 。注意两者都不是单点集，这正是原文说「多值」的意思，也正是下一条注里分段求导法能成立的原因——中段取 $x = 0$ 时，我们用最优性条件 $0 \in \partial f(x)$ 代替了求导。接上你在大二上「凸优化初步」里会遇到的 KKT 条件： ℓ_1 范数不可微，所以基追踪的最优性条件只能写成次梯度形式；而线性化 Bregman 迭代之所以要那个步长条件 $s \lambda_{\max}(A^T A) < 1$ ，是因为每做一步，目标函数下降的幅度受 $\frac{1}{2}\|Ax - b\|_2^2$ 的曲率控制，步长太大就会跨过尖点来回振荡、不再下降。

先回答设计问题：这一段到底在解决什么。 可微情形下，判断这个点是不是最小点只需要一件事：看梯度是否为零。可一旦目标里出现 $|x|$ 、 $\max(0, x)$ 、 $\|x\|_1$ 这类带尖点的项， ∇f 在尖点处连定义都没有，于是整套判据断线。工程上常见的做法是什么？绕过尖点、改用光滑近似（比如把 $|x|$ 换成 $\sqrt{x^2 + \epsilon}$ ）——但这样就把问题改了：解会随 ϵ 漂移，稀疏性也不再精确，只有让 $\epsilon \rightarrow 0$ 才回到原问题，这时又变回不可微。次梯度的思路完全相反：不改问题，改判据。与其问斜率是多少，不如问哪些斜率可以充当斜率——把单个向量放宽成一个集合，让尖点也能报出一组合法的候选斜率。

从第一性原理看为什么必须这样做。 凸函数的图像是碗形，碗底那一点的特征不是水平切线存在，而是存在一条过该点的直线，整条直线不在图像上方。光滑点的这条直线唯一，就是切线；尖点的这条直线不唯一，可以左右摆动，摆动的整个角度范围就对应一个斜率区间。所以支撑线是更本质的对象，切线只是它退化成单点的特例。这也解释了为什么凸性是前提：非凸函数的尖点可能一条支撑线都没有（比如 $-|x|$ 在 0 处），此时次梯度集合是空集，判据重新失效。 $0 \in \partial f(x)$ 的含义因此可以一句话说清：存在一条通过 $(x, f(x))$ 的水平支撑线，即 x 处没有可行的下降方向。

用两个小数字把两个例子验算一遍。 取 $f(x) = |x|$ 、 $x = 0$ ：要 $g \in \partial f(0)$ ，就得对一切 z 有 $|z| \geq gz$ 。若 $|g| \leq 1$ ，则 $gz \leq |g||z| \leq |z|$ ，成立；若 $g > 1$ ，取 $z = 1$ 得左边 1、右边 $g > 1$ ，不等式反向；若 $g < -1$ ，取 $z = -1$ 同样反向。所以 $\partial |x|(0) = [-1, 1]$ 的端点是刚好卡住的临界值。再看 $\text{ReLU } f(x) = \max(0, x)$ 、 $x = 0$ ：要求对一切 z 有 $\max(0, z) \geq gz$ 。 $0 \leq g \leq 1$ 时， $z < 0$ 一侧左边为 0 而右边为负，自动成立， $z \geq 0$ 一侧化为 $z \geq gz$ ，由 $g \leq 1$ 成立； $g > 1$ 时取 $z = 1$ 得 $1 \geq g$ ，失败。于是 $\partial f(0) = [0, 1]$ ，与原文一致。注意 $[0, 1]$ 的宽度正是 ReLU 两侧斜率的差 $1 - 0$ 。

多维时它是集合的笛卡尔积。 对 ℓ_1 范数 $f(x) = \sum_i |x_i|$ ，在原点 $\partial f(0) = [-1, 1]^n$ ，是 n 维立方体而非某个向量；在 $x = (0, 5)$ 这类混合点， $\partial f(x) = [-1, 1] \times \{1\}$ ，只有真正取零的那些坐标才贡献区间。代进本页问题 $\min_x \frac{1}{2} \|x - b\|_2^2 + \lambda \|x\|_1$ 检验：最优性条件是 $0 \in (x - b) + \lambda \partial \|x\|_1$ 。取 $b = (3, 0.5)$ 、 $\lambda = 2$ 与候选 $x = (2, 0)$ ，第一坐标残差为 -1 ， $x_1 = 2 > 0$ 光滑，需 $-1 + 2 \cdot 1 = 1 \neq 0$ ？注意此处 ℓ_1 项系数与目标配比给出的是 $x_1 - b_1 + \lambda \text{sign}(x_1) = 2 - 3 + 2 = 1$ ——不满足，说明 λ 与 b_1 的搭配没到最优；改取 $x = (1, 0)$ 得 $1 - 3 + 2 = 0$ ，第二坐标残差为 -0.5 ，其绝对值 $0.5 \leq \lambda = 2$ ，由 ∂ 提供 -0.5 使条件成立。于是 $x^* = (1, 0)$ ：第二个坐标被区间吸收而恰好置零，第一个坐标停在等式条件上。这正是基追踪里 ℓ_1 不可微时 KKT 只能写成次梯度形式的原因——判据从等式升级为包含关系，零解那一支才被允许存在。

跨课程连接（精确到路径）。 18.06 里判断 $Ax = b$ 有解用的是 b 落在 A 的列空间内，判据里 A 不必可逆；6.042J 里把相等放宽成等价关系下的同类，都属于同一个动作：用更宽的包含/同类关系替换严格的等号，从而把边界情形纳入同一套语言。区别在于：列空间是线性子空间，判定是无损的硬条件；次微分是集值映射，它给的是一组合法斜率，因此判据本身变成存在性命题——只要集合里有一个元素能被抵消就够了。把这套动作命名下来，可叫 **等号升级法**：凡遇某量等于零的判据，先问它是否由某量属于某集合退化而来；若是，则在尖点处保留集合、只把点取成区间，结论自动延续。

[section]

分裂 Bregman 与分裂 Kaczmarz（Split Bregman and Split Kaczmarz）

对于带 ℓ_1 penalty term（罚项）的最小化问题（例如 basis pursuit（基追踪）），有两个迭代算法必须看清楚。从 $Ax = b$ 出发：

Linearized Bregman iteration with parameter $\lambda \geq 0$ （带参数 $\lambda \geq 0$ 的线性化 Bregman 迭代）

$$y_{k+1} = y_k - s_k A^T (Ax_k - b) \tag{30}$$

$$x_{k+1} = S(y_{k+1}) = \text{sign}(y_{k+1}) \max(|y_{k+1}| - \lambda, 0) \tag{31}$$

式 (30) 是一个常规的可调整步长的梯度步，用来减小 $\|Ax - b\|_2^2$ ，其步长为 $s > 0$ 。接着，soft thresholding function（软阈值函数） S 逐分量地施加到向量 y_{k+1} 上。若 $y_{k+1} = (1, -3)$ 且 $\lambda = 2$ ，则由 (31) 中这个非线性函数 S 得到的输出 x_{k+1} 将是向量：

$$x_{k+1} = ((1) \max(1 - 2, 0), (-1) \max(3 - 2, 0)) = (0, -1)$$

[note] 译者注（证明策略模板）： (31) 那个看起来随手写下的分段函数，其实是 ℓ_1 范数的近端算子，可以完整推出来，而推导过程本身就是一个可反复使用的模板。要解的问题是 $\min_x \lambda |x| + \frac{1}{2}(x - y)^2$ （一维，然后逐分量作用于 y_{k+1} ）。 $|x|$ 在 $x = 0$ 处不可微，于是按这个不可微点把实轴切成三段，每段内部用普通求导：若 $x > 0$ ，导数为 $\lambda + (x - y) = 0$ ，得 $x = y - \lambda$ ，要求它真的落在 $x > 0$ 内，即 $y > \lambda$ ；若 $x < 0$ ，导数为 $-\lambda + (x - y) = 0$ ，得 $x = y + \lambda$ ，要求 $y < -\lambda$ ；剩下的中段只能取 $x = 0$ ，检查最优性条件 $0 \in \partial(\lambda|x|) + (0 - y)$ ，即 $|y| \leq \lambda$ 。三段拼起来正是 (31) 的 $\text{sign}(y) \max(|y| - \lambda, 0)$ 。这里的精髓不是「按比例缩小 λ 」而是**截断**： $|y| \leq \lambda$ 的分量被直接压成 0，这才是 ℓ_1 能产生稀疏解的机制，而 ℓ_2 的近端算子只会整体缩小、永远不给零。把这个事实放回 (30)(31) 的组合，算法就清楚了：(30) 在原始空间做一步可调步长的梯度下降，用来减小 $\|Ax_k - b\|_2^2$ ；(31) 在梯度（对偶）空间做一次 ℓ_1 的近端步。往后看一步：把绝对值换成奇异值，就得到矩阵补全里的奇异值阈值算法——先做 SVD $M = U\Sigma V^T$ ，对 Σ 逐元素做同一个 $\max(\sigma - \lambda, 0)$ ，再拼回 $U\Sigma_\lambda V^T$ 。这一步之所以自然，是因为向量的 prox 逐分量做，而 SVD 正是矩阵的「分量分解」，18.065 的 SVD 在这里从分析工具变成了算法的一步。可复用的模板：**分段求导—边界拼接法**。面对「一个不可微的凸函数加一个二次项」的极小化，先按不可微点切段，在每段内用普通求导得到候选点，再检验候选点是否落在自己那一段内（这一步最容易被跳过，而它恰恰决定了截断区间的大小），最后用几个具体数值代回目标函数验算。这套动作对任意分段线性的 F 都适用。

它要解决的设计问题：为什么必须有「二次项」。 如果没有 $\frac{1}{2}(x - y)^2$ ， $\min_x \lambda |x|$ 的答案平凡得毫无用处——只要 $\lambda > 0$ 就是 $x = 0$ ，跟输入 y 完全脱钩，算法每一步都把变量清零。二次项的作用是把**前一步的信息锚住**：它说「我既想压小 $|x|$ ，又不想离上一轮给出的 y 太远」。正是这个拉扯产生了阈值： y 小的时候二次项赢，解被拉到 0； y 大的时候线性项压不住，解停在 y 旁边偏移 λ 。近端算子（proximal operator）这个名字的含义就在这里， $\text{prox}_{\lambda|\cdot|}(y)$ 是「在 y 附近找 f 的折中解」的算子。这也解释了为什么近端算法能把不可微项和可微项分开处理：可微项交给梯度步，不可微项交给 prox，两者各管一段。

代数字复算一遍（务必亲手走这两步）。 取 $\lambda = 1$ 、 $y = 3$ ：第二段判据 $y > \lambda$ 成立， $x = y - \lambda = 2$ ，代回目标函数得 $1 \cdot 2 + \frac{1}{2}(2 - 3)^2 = 2 + 0.5 = 2.5$ ；若改取 $x = 3$ 得 $3 + 0 = 3$ ，若改取 $x = 0$ 得 $0 + 4.5 = 4.5$ ，2.5 确实最小，验算通过。再取 $y = 0.4$ ：中段判据 $|0.4| \leq 1$ 成立， $x = 0$ ，目标值 $0 + 0.08 = 0.08$ ；若硬取第一段候选点 $x = y - 1 = -0.6$ ，它连自己那一段（要求 $x > 0$ ）都不满足，说明候选点越界必须作废，此时目标值 $0.6 + 0.5 = 1.1$ ，比 0.08 大，截断赢了。这两个数字就是「稀疏从哪来」的全部证据： $|y|$ 没有超过阈值 λ 的分量被精确地推成零，而不是变成一个小数。

与 ℓ_2 的对照，以及它在 18.065 的位置。 ℓ_2 版本解 $\min_x \lambda x^2 + \frac{1}{2}(x - y)^2$ 得 $x = \frac{y}{1 + 2\lambda}$ ：无论 λ 多大，只要 λ 有限， x 就永远不为零，只是被整体缩放。这就是岭回归与 Lasso 的分水岭，也是「软阈值」这个词的由来—— $\max(|y| - \lambda, 0)$ 是在数轴上把每个分量朝原点推固定距离 λ ，推过了头就直接落地为零，而 ℓ_2 是乘一个小于 1 的因子，越来越近但永不到达。跨课程连接（18.065 矩阵方法 $\rightarrow$ 本页）：连接路径是「向量逐分量操作 $\rightarrow$ 矩阵谱操作」。SVD $M = U\Sigma V^T$ 把矩阵分解成一族秩一层的和， Σ 的对角元就是每层的「分量大小」，因此对向量逐分量做软阈值，自然对应到对 Σ 逐元素做软阈值，再拼回 $U\Sigma_\lambda V^T$ 。区别要讲清楚：向量情形各分量彼此正交、软阈值互不干扰，矩阵情形奇异向量一般也正交，但奇异值必须**按大小排序**才能谈「保留前 k 个」；而且矩阵补全里观测是稀疏采样的， M 有缺失项，SVD 只能在已观测位置上交替做，收敛性比 (30)(31) 这种全观测情形弱得多。把奇异值阈值记作**谱软阈值**，就能一眼看出它是本页公式的矩阵推广。

可复用的动作清单。 命名模板仍用**分段求导—边界拼接法**，但执行时按四步：切段（按不可微点划分定义域）、求候选（段内令导数为零）、回代验边界（候选点是否落在本段，这一步决定阈值大小）、数值抽检（挑一个越界点和一个内点代回目标函数）。你对任意分段线性的 F 都能跑这套流程，比如 hinge 损失的 prox 会得到另一个带阈值形式的分段函数，形状不同而骨架完全一样。

若 $Ax = b$ 有解，并且 $s \lambda_{\max}(A^T A) < 1$ ，则这些 Bregman 迭代向量 x_k 将收敛到最优向量 x^* ，它最小化：

$$x^* \text{ minimizes } \lambda \|x\|_1 + \frac{1}{2} \|x\|_2^2 \text{ subject to } Ax = b. \tag{32}$$

本页为书页 193，是 §III.4 Split Algorithms for A^+ 的续页；结构单元：Kaczmarz 迭代的标准步骤 (33)、稀疏 Kaczmarz 步骤 (34)、步长与在线学习讨论、三条脚注、小节 Bounded Variation: L1 Norm for the Gradient、BV 范数公式、一维阶跃函数例子、图像去噪应用与 Rudin-Osher-Fatemi 文献。页眉与页码为重复元素，不译。

[note] 译者注 (术语校正)： 本页公式按 Kaczmarz 迭代的数学语境重建：丐 = bt 为 $a_i^T x = b_i$ ，第 i 个方程，扣 2 为范数平方 $\|a_i\|_2^2$ ，凭 为 y ，5 为步长 s ；衍 B g 赅 为 split Bregman；沅 i w 方 为 ℓ_1 minimization；Bounded Variation：L1 Norm 为 Bounded Variation: L1 Norm。

先回答一个问题：这一页的符号为什么会碎成这样？扫描件把下标、方括号和希腊字母一起丢掉了，OCR 只留下字形轮廓相似的中文字符。所以「重建」不是猜谜，而是**按机制反推**：只有当 丐 = bt 被读成 $a_i^T x = b_i$ 时，后面的 扣 2 才可能是有量纲意义的分母——因为 $(a_i^T x - b_i)$ 是标量、 a_i 是向量，想让商仍是向量，唯一自然的除法就是除以 $\|a_i\|_2^2$ 。量纲（更准确说是形状）一致性是判据，而不是对页面的记忆。

这套公式在解决什么设计问题？给定 $Ax = b$ ，如果 A 是 $m \times n$ 、 $m > n$ 且列满秩，通常没有精确解，只能最小化 $\|Ax - b\|_2^2$ ，代价是要求解正规方程 $A^T Ax = A^T b$ ，也就是一次矩阵分解；如果 A 大到放不进内存（CT 重建、层析成像里的 A 就是几十万乘几十万），你根本做不起分解。Kaczmarz 的答案是：**永远不组装整体，只逐行使用**。第 k 步取一行 a_i ，把当前点 x_k 沿 a_i 方向平移到超平面 $\{x : a_i^T x = b_i\}$ 上，写成一行就是 $x_{k+1} = x_k + s \frac{b_i - a_i^T x_k}{\|a_i\|_2^2} a_i$ 。第一性原理只有一条：如果一个点已经落在该超平面上，残差为零，公式就把它原地不动，所以「不动点 = 解」这件事是构造出来的，不是事后验证的。凭 为 y 说明中间量 $b_i - a_i^T x_k$ 被单独命名过；5 为步长 s 说明这一步带一个松弛因子， $s = 1$ 是严格投影， $s \in (0, 2)$ 是过松弛/亚松弛， $s = 1$ 时误差范数单调不减。至于 衍 B g 赅 沅 i w 方、Bounded Variation：L1 Norm 三处，正是 Kaczmarz 路线的下游：当你要解的已经不是 $Ax = b$ 而是 $\min \|x\|_1$ 或 $\min \|\nabla x\|_1$ （有界变差 + L1，图像去噪与压缩感知的标准能量），split Bregman 与 ℓ_1 minimization 就登场了——split Bregman 把「可微的数据项」和「不可微的 L1 项」拆成两个变量交替求解，而 Kaczmarz 正好提供其中一个子问题的廉价内层迭代。所以本页不是名词罗列，而是一条链：线性投影 $\rightarrow$ 稀疏正则 $\rightarrow$ 算子分裂。

代一个能复算的数： $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ ， $b = (1, 2)^T$ ，真解 $x^* = (1, 1)^T$ 。取 $x_0 = (0, 0)^T$ ，按 $i = 1, 2, 1, 2, \dots$ 顺序扫描。第 1 步： $\|a_1\|_2^2 = 1$ ，残差 $b_1 - a_1^T x_0 = 1$ ，于是 $x_1 = (0, 0)^T + 1 \cdot (1, 0)^T = (1, 0)^T$ 。第 2 步： $\|a_2\|_2^2 = 1^2 + 1^2 = 2$ ，残差 $2 - (1 + 0) = 1$ ，修正量 $\frac{1}{2}(1, 1)^T$ ，得 $x_2 = (1.5, 0.5)^T$ 。第 3 步：残差 $1 - 1.5 = -0.5$ ， $x_3 = (1, 0.5)^T$ 。第 4 步：残差 $2 - 1.5 = 0.5$ ， $x_4 = (1.25, 0.75)^T$ 。误差 $e_2 = (0.5, -0.5)$ 、 $e_3 = (0, -0.5)$ 、 $e_4 = (0.25, -0.25)$ ， $\|e_k\|_2^2$ 依次为 0.5, 0.25, 0.125——每两步恰好减半。这里 a_1, a_2 正交，所以收缩是干脆的；若把矩阵换成 $A = \begin{bmatrix} 1 & 0 \\ 1 & \varepsilon \end{bmatrix}$ （两行几乎平行），同样的顺序扫描就会退化：残差里几乎不含 x 的第二个分量信息，迭代原地打转。这正好解释了第 2 条注为什么必须把行的选择随机化。

跨课程连接，挑最直接的一条：**CSAPP「存储层次与缓存」+ 数组布局**。Kaczmarz 每步只碰一行，属于典型的带宽受限内核；行主序存储时 a_i 连续，缓存行利用率高，列主序（Fortran 式）则每取一个元素就浪费一条缓存行， m 很大时这一步之差就是数倍墙钟时间。连接路径是：按行存储 $\rightarrow$ 单行遍历命中连续的缓存行 $\rightarrow$ 内核内存带宽决定每秒能扫过多少行 $\rightarrow$ 决定 Kaczmarz 每小时能做多少次投影。区别在于：CSAPP 关心的是「同一份数在存储层次之间搬动的代价」，而数值线性代数关心的是「同样的浮点运算次数下误差收缩多少」；Kaczmarz 恰好是两者的交点——它的收敛率由 $\rho = \sigma_{\min}^2 / \|A\|_F^2$ 这类纯数学量决定，而它的实际速度由缓存与内存带宽决定，两者互不替代。

命名模板：**投影不动点检验法**——拿到任一逐行更新公式，先令残差为零代入，看该点是否被公式固定；被固定，则公式至少是「一致的」，再谈收敛率。

Kaczmarz iteration (Kaczmarz 迭代) 对含稀疏向量的大数据问题非常有效。它的标准步骤求解 $Ax = b$ ：按顺序、或者随机地循环经过这 n 个方程 $a_i^T x = b_i$ （其中 a_i^T 是 A 的第 i 行）。每一步都用 A^T 的某一行 a_i 的一个倍数 ca_i 来调整 x_k ，使第 i 个方程 $a_i^T x_{k+1} = b_i$ 得到满足：

$$x_{k+1} = x_k + \frac{b_i - a_i^T x_k}{\|a_i\|_2^2} a_i \quad \text{solves} \quad a_i^T x_{k+1} = a_i^T x_k + a_i^T a_i \frac{b_i - a_i^T x_k}{\|a_i\|_2^2} = b_i. \tag{33}$$

****Sparse Kaczmarz (稀疏 Kaczmarz)** **步骤把 (33) 与 (31) 中的 soft threshold（软阈值）结合起来：

$$y_{k+1} = x_k - s_k a_i \quad \text{and} \quad x_{k+1} = S(y_{k+1}). \tag{34}$$

步长 s_k 始终有选择的自由。取 $s_k = \frac{a_i^T x_k - b_i}{\|a_i\|_2^2}$ 就与 Kaczmarz 迭代一致。Lin 与 Zhou 针对同一个问题提出了一种 online learning（在线学习）算法。这意味着，一旦新的观测值 b_k 与新的方程 $a_k^T x = b_k$ 到达并加入计算，就立刻执行 $k \rightarrow k + 1$ 这一步。于是学习算法 (34) 立即算出 y_{k+1} 与 x_{k+1} 。

[note] 译者注 (证明策略模板)： 译文这里说一旦新观测值到达就立刻执行 $k \rightarrow k + 1$ ，把 Kaczmarz 变成了在线学习算法。要理解为什么这条路线值得单独提，先把单步误差看清楚：设 x^* 是满足 $Ax = b$ 的真解，令 $e_k = x_k - x^*$ ，则 $b_i - a_i^T x_k = -a_i^T e_k$ ，代入 (33) 得 $e_{k+1} = e_k - \frac{a_i^T e_k}{\|a_i\|_2^2} a_i$ ，也就是从 e_k 中减去它在 a_i 方向上的投影。取平方得 $\|e_{k+1}\|_2^2 = \|e_k\|_2^2 - \frac{(a_i^T e_k)^2}{\|a_i\|_2^2} \leq \|e_k\|_2^2$ ：误差范数单调不减，而且整个过程没有任何步长参数可调，这是 Kaczmarz 最讨人喜欢的性质。但单调不减不等于收敛快。如果 a_i 反复选中那些与当前误差几乎垂直的行，损失项 $\frac{(a_i^T e_k)^2}{\|a_i\|_2^2}$ 会很小，迭代就会原地踏步；顺序扫描遇到病态的行序时完全可能发生这种事。随机化正是为此而生：按概率 $p_i = \|a_i\|_2^2 / \|A\|_F^2$ 抽取第 i 行，则每步的期望损失为 $\sum_i p_i \frac{(a_i^T e_k)^2}{\|a_i\|_2^2} = \frac{\|Ae_k\|_F^2}{\|A\|_F^2}$ ，再用 SVD 给出下界 $\|Ae_k\|_2^2 \geq \sigma_{\min}^2 \|e_k\|_2^2$ ，便得到期望收缩 $\mathbb{E} \|e_{k+1}\|_2^2 \leq \left(1 - \frac{\sigma_{\min}^2}{\|A\|_F^2}\right) \|e_k\|_2^2$ ，收缩因子 $\rho = \sigma_{\min}^2 / \|A\|_F^2$ 只依赖于矩阵本身，不再依赖行的顺序。

先把「为什么必须是这个概率」讲透，因为这是整条证明里唯一看起来像魔法的地方。已知每步的损失是 $\frac{(a_i^T e_k)^2}{\|a_i\|_2^2}$ ，我们想让它对随机抽取的期望尽量大、而且最好与 e_k 的**方向**无关。写 $q_i = \|a_i\|_2^2 / \|A\|_F^2$ ，则 $\sum_i q_i \frac{(a_i^T e_k)^2}{\|a_i\|_2^2} = \frac{1}{\|A\|_F^2} \sum_i (a_i^T e_k)^2 = \frac{\|Ae_k\|_2^2}{\|A\|_F^2}$ ，而 $\sum_i (a_i^T e_k)^2$ 恰恰是 $\|Ae_k\|_2^2$ 按行展开的结果。换句话说， $p_i \propto \|a_i\|_2^2$ 之所以正确，是因为它把权重 $\|a_i\|_2^2$ 与损失里的分母 $\|a_i\|_2^2$ **精确约掉**，剩下的量只通过 A 作用于 e_k 体现。若改用均匀抽样 $p_i = 1/m$ ，期望损失会变成 $\frac{1}{m} \sum_i \frac{(a_i^T e_k)^2}{\|a_i\|_2^2}$ ，长行与短行的贡献被人为拉平，界就退化成含 $\max_i \|a_i\|_2^2 / \min_i \|a_i\|_2^2$ 这类条件数的式子——多出一个可被数据尺度操纵的因子。这就是「按行范数平方加权」的全部道理：不是启发式，而是把分母消掉的唯一权重。另需注意随机化只改变**期望**，单步仍可能有损失接近零的倒霉抽籤，所以结论是一条期望收缩，不是逐步收缩。

代一个完整可复算的 2×2 ： $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ ， $b = (1, 2)^T$ ，真解 $x^* = (1, 1)^T$ ，从 $e_0 = (1, -1)^T$ 出发。行范数平方为 $\|a_1\|_2^2 = 1$ 、 $\|a_2\|_2^2 = 2$ ，故 $\|A\|_F^2 = 3$ ，抽样概率 $p_1 = 1/3$ 、 $p_2 = 2/3$ 。两种行各自带来的收缩：抽到第 1 行，损失 $\frac{(a_1^T e_0)^2}{\|a_1\|_2^2} = \frac{e_{01}^2}{\|e_0\|_2^2}$ ；抽到第 2 行，损失 $\frac{(e_{01} + e_{02})^2}{2}$ 。代入 $e_0 = (1, -1)$ ： $a_1^T e_0 = 1$ ，损失 1， $\|e_0\|_2^2 = 2$ ，收缩因子 $1 - \frac{1}{2} = 0.5$ ； $a_2^T e_0 = 0$ ，损失 0，该步原地不动，因子 1。所以顺序最坏时（反复抽第 2 行）会卡住，这正是「单调不增 $\neq$ 收敛快」的数值证据。而按 $p_1 = 1/3, p_2 = 2/3$ 求期望： $\frac{1}{3} \cdot 0.5 + \frac{2}{3} \cdot 1 = \frac{0.5+2}{3} = \frac{2.5}{3} \approx 0.8333$ 。再用公式核一遍： $\sigma_{\min}$ 为 $A^T A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ 的最小特征值之平方根，特征值满足 $(2 - \mu)(1 - \mu) - 1 = 0$ ，即 $\mu^2 - 3\mu + 1 = 0$ ， $\mu_{\min} = \frac{3-\sqrt{5}}{2} \approx 0.381966$ ，故 $\rho = \frac{0.381966}{3} \approx 0.12732$ ， $1 - \rho \approx 0.87268$ 。这个界比实测的 0.8333 略松——因为 SVD 下界对所有 e 统一成立，属于最坏情况保证，而刚才那个具体的 e_0 恰好对第 2 行的损失为零，所以一步的实测不如平均。这个「界略松」的差额本身就是一次有价值的手算：它告诉你定理保证的是上界而非精确值。

再补一层机制：为什么 $p_i \propto \|a_i\|_2^2$ 在实践中也合理？长行携带更多「能量」，一次投影能移动的距离更大，给它更高被抽中的机会，等于把每次迭代的计算预算按信息量分配。这也解释了它为什么等价于把 A 的每一行归一化后的均匀抽样，只是用概率吸收掉了尺度差异。

跨课程连接，两条。第一条：**Strang 线性代数 (18.06/18.065) 的 SVD 与最小二乘**。连接路径是： $\|Ae\|_2^2 = e^T A^T A e \geq \sigma_{\min}^2 \|e\|_2^2$ 正是把 e 在 $A^T A$ 的特征基下展开后的瑞利商下界， ρ 就是 $\frac{\sigma_{\min}^2}{\sum_i \sigma_i^2}$ ，即「最小奇异值平方占总能量之比」。区别在于：Strang 用 SVD 刻画解与最优低秩逼近（伪逆、 A^+ 、条件数 $\sigma_{\max}/\sigma_{\min}$ ），是**静态**的结构定理；这里用的是同一批奇异值，但目的是给一个**动态**迭代的每步收缩配一个与行序无关的常数，奇异值从「描述解的样子」变成「描述迭代的速度」。第二条：**Boyd 凸优化（在学）的增量梯度法 / SGD**。连接路径是：把每步只看一行的思想搬到 $\min f(x) = \sum_i f_i(x)$ 上，就是增量梯度法 $x_{k+1} = x_k - s \nabla f_i(x_k)$ ；Kaczmarz 正是 $f_i(x) = \frac{1}{2}(a_i^T x - b_i)^2$ 的特例，且此时 $\nabla f_i(x) = a_i(a_i^T x - b_i)$ 、步长自动取 $1/\|a_i\|_2^2$ 。区别在于：Boyd 关心凸性与步长选取（ L -光滑、强凸参数 μ ），一般要用递减步长或小批量才能收敛；而 Kaczmarz 在精确算法下不需要递减步长，因为它是投影不是下降，误差的单调性来自几何而非凸性假设。再加一条轻量的：**在学概率论 (Bertsekas) 的条件期望**—— $\mathbb{E}\|e_{k+1}\|_2^2 \leq \dots$ 的推导只用全期望公式，不涉及分布的具体形状，这正是该方法对行分布稳健的原因。

命名模板：**残差投影三步法**。模板固定为三步——(1) 写 $e_k = x_k - x^*$ ，用 $Ax^* = b$ 把残差改写成 $-a_i^T e_k$ ，消去 b ；(2) 代入更新式取范数平方，得到「不增 + 显式损失项」；(3) 对随机选择取期望，用 $\|Ae_k\|_2^2$ 与 $\sigma_{\min}$ 夹出只依赖矩阵的收缩因子。凡是逐行、逐样本更新的算法（Kaczmarz、SGD、随机坐标下降），证明都可以照这个模板默写。

先说这段在解决什么设计问题。Kaczmarz 面对的处境是：方程组按行陆续到达，甚至 A 大到根本装不进内存，你只有一行 a_i 和右端 b_i ，却想尽快逼近 x^* 。从第一性原理看，第 i 个方程 $a_i^T x = b_i$ 在几何上就是一张超平面，满足全部方程的解只能是所有超平面的公共交点，于是最朴素也最稳的动作就是：当前点 x_k 若不在第 i 张平面上，就沿该平面的法向 a_i 垂直走到平面上，因为法向是到这张平面距离最短的方向。这就是 (33) 的来历，而上面的推导说明这个几何动作等价于把误差在 a_i 上做正交投影再减掉，所以误差范数平方的减少量恰好正比于 $\cos^2 \theta$ ，其中 θ 是 e_k 与 a_i 的夹角：行几乎与误差平行时收益最大，几乎垂直时收益趋零。这条几何图像也解释了为什么不需要步长：投影算子本身已经是正交投影，放大或缩小都会破坏“落在平面上”这个不变量。

代入具体数值走两步，把上面的公式变成可复算的手算。取 $A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ ， $b = (1, 1)^T$ ，真解是 $x^* = (1, 0)^T$ 。从 $x_0 = (0, 0)^T$ 出发，此时 $e_0 = (-1, 0)^T$ ， $\|e_0\|_2^2 = 1$ 。若先抽到第 1 行 $a_1 = (1, 0)^T$ ： $t = \frac{b_1 - a_1^T x_0}{\|a_1\|_2^2} = \frac{1}{1} = 1$ ，得 $x_1 = (1, 0)^T = x^*$ ，一步到位， $\|e_1\|_2^2 = 0$ 。若先抽到第 2 行 $a_2 = (1, 1)^T$ ： $t = \frac{1}{2} = 0.5$ ，得 $x_1 = (0.5, 0.5)^T$ ， $e_1 = (-0.5, 0.5)^T$ ， $\|e_1\|_2^2 = 0.5$ ，损失正好是 $\frac{(a_2^T e_0)^2}{\|a_2\|_2^2} = \frac{1}{2}$ 。第二步再抽第 1 行： $t = \frac{1 - 0.5}{1} = 0.5$ ，得 $x_2 = (1, 0.5)^T$ ， $\|e_2\|_2^2 = 0.25$ ，又是减半。注意 $p_1 = \frac{1}{3}$ 、 $p_2 = \frac{2}{3}$ （因为 $\|a_1\|_2^2 = 1$ ， $\|a_2\|_2^2 = 2$ ， $\|A\|_F^2 = 3$ ），可以验算 $\sum_i p_i \frac{(a_i^T e_0)^2}{\|a_i\|_2^2} = \frac{1}{3} \cdot 1 + \frac{2}{3} \cdot \frac{1}{2} = 1 \cdot \frac{\|Ae_0\|_2^2}{\|A\|_F^2} = \frac{1}{3}$ 里的系数对齐关系；再算最小奇异值： $AA^T = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$ ，特征值为 $\frac{3}{2} \pm \frac{\sqrt{5}}{2}$ ，故 $\sigma_{\min}^2 = \frac{3-\sqrt{5}}{2} \approx 0.382$ ， $\rho \approx 0.127$ ，也就是说这个 2×2 例子给出的界只承诺每步期望收缩约 13%，而实测单步常常减半——这说明界是安全但保守的，保守恰恰是它能不依赖行序的原因。

跨课程连接一：Strang 线性代数与 18.065 的 SVD 与最小二乘。Kaczmarz 用到的下界来自 $\|Ae_k\|_2 \geq \sigma_{\min} \|e_k\|_2$ ，这正是 SVD 把矩阵看成旋转—伸缩—旋转的推论：最坏方向被拉伸得最少，拉伸倍数就是 $\sigma_{\min}$ 。连接路径是：18.065 里用 $A = U \Sigma V^T$ 分析条件数与伪逆，这里直接把同一个 $\sigma_{\min}$ 搬进收敛率；区别在于最小二乘用 QR 或正规方程一次性求出 x^* ，而 Kaczmarz 是逐行、可中断、内存只需一行的迭代法，代价是收敛率被 $\sigma_{\min}^2 / \|A\|_F^2$ 这个可能很小的数拖住。

跨课程连接二：CSAPP 的缓存与内存层次。为什么“只存一行”不是学术洁癖？因为现实中的 A 可能大到远超内存，能顺序流式读取、算完即弃的算法才能和磁盘带宽对齐，这与 CSAPP 讲缓存局部性时把“访问模式决定实际吞吐”当作第一原则是同一件事；区别在于 CSAPP 关心的是各级存储的命中延迟，这里关心的是迭代次数与每步访存量的乘积。

跨课程连接三：CLRS 与 6.006 的分治/复杂度视角，以及 6.042J 的 Chernoff 界。前者的连接点是：一次性直接法（如 LU 或 QR）的复杂度是 $O(n^3)$ 量级且必须看到整个 A ，而 Kaczmarz 把问题切成 $O(1/\rho)$ 次 $O(n)$ 的行级操作，属于典型的“用多次廉价操作换取无需全局”的权衡；区别在于分治追求的是把 $O(n^3)$ 的指数压下来，这里追求的是让在线、单遍的场景可行。后者则是最后那段高概率升级的来源：单步损失的随机性来自“抽到哪一行”，各轮独立，于是 log 之后的求和恰好落在 Chernoff 界的适用前提上，把期望收缩翻译成“这一次运行大概率收缩”。

命名模板：**期望收缩** $\rightarrow$ **乘积形式** $\rightarrow$ **集中不等式**。使用时按三步走：第一步写出 $\mathbb{E}[Z_k] \leq 1 - \rho$ 这样的单步期望收缩；第二步把多步误差写成 $\|e_k\|_2^2 = \|e_0\|_2^2 \prod_{j < k} Z_j$ ，取 $\log$ 化成随机变量之和；第三步对这个和用 Chernoff 界（本质是对独立随机变量和做指数矩估计），得到以 $1 - \delta$ 的概率把误差压到 ϵ_0 以下只需 $k = O\left(\frac{1}{\rho} \log \frac{1}{\epsilon_0 \delta}\right)$ 步。这套模板的适用条件必须记牢：单步收缩因子要有与当前误差无关的期望上界、各轮随机性独立、且有 $\log$ 可乘性。只要这三条成立，任何“每步随机、整体收敛”的迭代都能套用，这也是为什么随机数值线性代数与压缩感知的定理总是以“以压倒性的概率”收尾。

把这段推导代入一组小数字，你会更容易感到随机化不是装饰。取 $A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ ， $b = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ ，真解 $x^* = (2, 1)^T$ ，从 $x_0 = (0, 0)^T$ 出发交替扫第 1、2 行： $x_1 = (2, 0)^T$ ， $x_2 = (2.5, 0.5)^T$ ， $x_3 = (2, 0.5)^T$ ，误差平方从 5 一路降到 1、0.25，每两步减半；而按上面的公式， $\|A\|_F^2 = 3$ 、 $\sigma_{\min}^2 \approx 0.382$ ，给出的收缩因子 $\rho \approx 0.127$ ，即期望误差平方每步乘以约 0.873，两步约 0.76——理论上明显比这次实际轨迹保守，这正是集中不等式要接管的地方。

一条跨课程连接可以把这套语言钉牢：Strang 线性代数（18.06/18.065）里最小二乘是 $\min \|Ax - b\|_2^2 \Rightarrow A^T Ax = A^T b$ ，一次性用全部行，解是 $A^\dagger b$ ；Kaczmarz 每次只用一行，解的轨迹始终落在超平面上，而且它的极限点就是 $A^\dagger b$ ——因为 $A^\dagger b$ 落在 A 的行空间里，每次修正又都在行空间内进行，两部分误差因此互相正交、可以分开看。区别在代价：最小二乘一次性付出分解 $A^T A$ 的代价，Kaczmarz 每步只需一次内积，行稀疏时极便宜，但收敛速度由 $\rho = \sigma_{\min}^2 / \|A\|_F^2$ 控制，条件数大就慢。最后补一个 CSAPP 视角的工程权衡：随机挑行在缓存眼里是最坏访问模式，每次跳到一个新内存块；实践中常改成先随机置换行序、再顺序扫描，用一个置换保留随机性的统计保证，同时恢复顺序访问的局部性优势——这是数学保证与缓存层次之间一次非常典型的谈判。

脚注 1：T. Goldstein 与 S. Osher，*The split Bregman method for L1 regularized problems*，SIAM Journal on Imaging Sciences 2 (2009) 323–343。

脚注 2：W. Yin、S. Osher、D. Goldfarb 与 J. Darbon，*Bregman iterative algorithms for ℓ_1 -minimization with applications to compressed sensing*，SIAM J. Imaging Sciences 1 (2008) 143–168。

脚注 3：Y. Lei 与 D.-X. Zhou，*Learning theory of randomized sparse Kaczmarz method*，SIAM J. Imaging Sciences 11 (2018) 547–574。

[section]

Bounded Variation: L^1 Norm for the Gradient (有界变差：梯度的 L^1 范数)

自然图像都有 edges（边缘）。在这些边缘的两侧，defining function（定义函数） $u(x, y)$ 会出现 jump（跳跃）。它的 gradient（梯度） $\nabla u = (\partial u / \partial x, \partial u / \partial y)$ 沿边缘方向可以是光滑的，但在垂直方向上 ∇u 含有一个 delta function（ δ 函数）。 u 的 energy norm（能量范数）是无穷大，而它的 bounded variation norm（有界变差范数，BV norm）却是有限的：

$$\|\nabla u\|_2^2 = \iint (u_x^2 + u_y^2) \, dx \, dy = \infty \quad \text{but} \quad \|u\|_{\text{BV}} = \|\nabla u\|_1 = \iint \sqrt{u_x^2 + u_y^2} \, dx \, dy < \infty$$

在一维情形下， $u(x)$ 可以取 unit *step function*（单位阶跃函数）。它的导数是 delta function（ δ 函数） $\delta(x)$ 。 $\delta(x)^2$ 的积分是无穷大，而 $\delta(x)$ 的积分等于 1。我们无法用导数 $\delta(x)$ 的 L^2 norm（ L^2 范数）来工作，但 $\delta(x)$ 的 L^1 norm（ L^1 范数）很好用。在 image denoising（图像去噪）的应用中，这个 BV norm（BV 范数）是一个非常成功的 penalty term（惩罚项）。我们用它来拟合数据，并用 BV 范数来防止剧烈的振荡。图像可以是 smooth 或 piecewise smooth（分片光滑）的，但不能含有随机的斑点噪声。

L. Rudin、S. Osher 与 E. Fatemi，*Nonlinear total variation based noise removal algorithms*，Physica D 60 (1992) 259–268。（这篇论文是把 BV 范数应用于图像处理的奠基性工作。）

本页为书页 194，是 §III.4 Problem Set（习题 III.4），配套 §III.4 Split Algorithms for A^+ （ A^+ 的分裂算法）。按本系列惯例，习题题干保留英文原文；题干中的 OCR 乱码按数学语境重建为 LaTeX，并以绿色 note 说明。页眉（Low Rank and Compressed Sensing）、页码 194 与左侧竖排题号噪声不译、不收录。

[section]

§III.4 Problem Set（习题 III.4）

[note] 译者注（OCR 噪声校勘）： 本页由 Windows OCR 输出，噪声极重： $\mathcal{A}$ / 鼠 多为函数名与积分号之误，什 om / 丘 om 为 from，mnimize / mmimize / nuzeS 为 minimize， ℓ / 11 为 ℓ 或 $\|\cdot\|$ ，訓 为 $\|Ac - b\|_1$ 之误，@ / 日 为系数与 γ 之误，之 为 $\geq$ ， π 为 π ，咒 为 R ，一 常为下标或负号之误。各题的重建均依据本页数学语境与所在节（§III.4 的例 1—例 4 与式 (27)）给出；因习题页题干须保留英文原文，重建只落在英文题干上，不作整句改写。（一）机制：OCR 是闭合词表上的最近邻投影 Windows OCR 并不先判断「这是不是数学」，它把切分出的每个连通域送进一个字符分类器，而分类器的输出空间只有它训练时见过的那几千个汉字、拉丁字母与标点。数学排版用的斜体衬线字形几乎不在这个分布里，于是识别器只能吐出「最像的那个词表项」： π 与 π 的横画加两只竖脚几乎同形，之 与 $\geq$ 共享一条斜画， ℓ 与 ℓ 以及 $\|\cdot\|$ 的竖线本来就是同一批像素。第二级更狠：识别器后面挂着语言模型做束搜索，它按「这个字后面接什么字更像中文」给候选打分，英文单词

from 的字母证据太弱，中文双字词的证据强，于是 from 被掰成 什 om；minimize 的尾巴只剩几笔高对比笔画，再被读出一个大写 S，就成了 nuzeS。（二）设计问题：为什么只能重建，不能重写 习题页有一条硬约束：英文题干必须保留原文，读者才能与影印本逐词核对。于是译者手里的信道是有损的，任务不是「把句子翻译通顺」，而是「在本页数学语境的约束下把它还原成唯一可信的式子」。这把改写降级成一个判定问题，判据有三条：语境证（同一符号在 §III.4 正文何处已定义）、类型证（这个位置要的是函数名、积分号还是范数）、量纲证（等号两侧是否同类量）。三条齐备才落笔，缺一条就宁可存疑标注。（三）一次可复算的验算以 训 还原成 $\|Ac - b\|_1$ 为例。取 A 为 2×2 单位阵、 $b = (1, 3)$ 、 $c = (0, 0)$ ，则 $Ac - b = (-1, -3)$ ：按 ℓ_1 读法该值为 $1 + 3 = 4$ ，按 ℓ_2 读法该值为 $\sqrt{10} \approx 3.162$ 。两种候选读数给出不同数值，可见此处修复不能凭手感，必须由本节残差语境定夺： ℓ_1 把误差按绝对值累加，解偏向稀疏、带中位数味道； ℓ_2 把误差按平方累加，解是最小二乘。在这里，类型证就等于追问一句「这个数是范数还是范数的平方」。（四）命名模板：三证定位法 把流程压成一句可复用的口诀：语境证，再类型证，最后量纲证，三证齐备才动笔；任何一证缺失，就写成「原文疑为某式，待定」而不是硬猜。（五）跨课程连接：与 CSAPP 的浮点表示对照 CSAPP 讲浮点时给过一个可迁移的直觉：实数轴上有无穷多个值，机器只表示其中有限多个，于是每次运算都要就近投射到可表示集合里， $0.1+0.2$ 因此不等于 0.3 。OCR 的闭合词表投影是同一件事的离散版本：真实符号落在词表之外，识别器只能投到最近邻的那个汉字上。连接路径是 CSAPP 的「可表示集合加就近投射」到本页的「汉字词表加最近邻字形」。区别在于 CSAPP 的投射带度量保证（误差不超过半个 ulp，舍入规则单调、可预测），所以能写出误差界；OCR 的最近邻没有度量保证，错得可以毫无上界，因此只能靠语境反推，不能靠「差得不多」来免检。

Problem 1. On the line $x = 1$, what vector c minimizes the function $F(c) = \frac{1}{2}\|c\|_2^2$?

[note] 原文 What vector C mini nuzeS 歹 （ ） = 1 ? 有两处缺字：目标函数 歹 （ ） 依本节例 1 的能量型目标重建为 $F(c) = \frac{1}{2}\|c\|_2^2$ ，约束 on the line = 1 依例 1 重建为直线 $x = 1$ 。这里要注意「被最小化的变量」是向量 c ，而约束是一个方程 $x = 1$ ：它把二分量向量 $c = (c_1, c_2)$ 钉在一条竖直直线上，可行集不是整个平面，而是一维的线。于是目标 $\frac{1}{2}(c_1^2 + c_2^2)$ 中 c_1 已被固定，只剩 c_2^2 可变，最小值在 $c_2 = 0$ 处取到，即 $c^* = (1, 0)$ ——这既是最小化问题的解，也正是原点 $(0, 0)$ 到直线 $x = 1$ 的垂足。本页后面 Problem 2 会把这个几何事实改写成 ADMM 可以处理的形式， $g(z)$ 只报告「点是否落在直线上」，而把「离得多远」交给 $\frac{1}{2}\|z\|_2^2$ 去度量。顺带说明符号：本页 F 一律指被最小化的目标函数， c 、 x 、 z 是向量变量， u 是 ADMM 中的对偶变量（也叫乘子），四者的角色在 §III.4 正文中固定，翻译时不改写。

Problem 2. Following Example 1 in this section, write Problem 1 as a minimization of $\frac{1}{2}\|z\|_2^2 + g(z)$, where $g(z)$ is the 0-1 indicator function of the line. Describe that 0-1 indicator function $g(z)$, and take one ADMM step starting from $x = 0$, $z = 0$, $u = 0$.

[note] 译者注（机制与验算）：原文 What vector C mini nuzeS 歹 （ ） = 1 ? 有两处缺字：目标函数 歹 （ ） 依本节例 1 的能量型目标重建为 $F(c) = \frac{1}{2}\|c\|_2^2$ ，约束 on the line = 1 依例 1 重建为直线 $x = 1$ 。这里要注意「被最小化的变量」是向量 c ，而约束是一个方程 $x = 1$ ：它把二分量向量 $c = (c_1, c_2)$ 钉在一条竖直直线上，可行集不是整个平面，而是一维的线。于是目标 $\frac{1}{2}(c_1^2 + c_2^2)$ 中 c_1 已被固定，只剩 c_2^2 可变，最小值在 $c_2 = 0$ 处取到，即 $c^* = (1, 0)$ ——这既是最小化问题的解，也正是原点 $(0, 0)$ 到直线 $x = 1$ 的垂足。本页后面 Problem 2 会把这个几何事实改写成 ADMM 可以处理的形式， $g(z)$ 只报告「点是否落在直线上」，而把「离得多远」交给 $\frac{1}{2}\|z\|_2^2$ 去度量。顺带说明符号：本页 F 一律指被最小化的目标函数， c 、 x 、 z 是向量变量， u 是 ADMM 中的对偶变量（也叫乘子），四者的角色在 §III.4 正文中固定，翻译时不改写。（一）机制：这一步在解决什么设计问题 ADMM 的套路是把一个难问题拆成两块轮流最小化，而本节先给一个能手算的玩具：在没有拆分的世界里，把 $\frac{1}{2}\|c\|_2^2$ 在约束 $x = 1$ 下最小化。为什么偏挑这个目标？因为 $\frac{1}{2}\|c\|_2^2$ 是严格凸的二次函数，Hessian 是单位阵，等高线是以原点为心的同心圆，沿任意方向离开原点函数值都单调上升。于是「在可行集上最小化它」与「在可行集上找离原点最近的点」是同一个问题：答案是原点在可行集上的投影，也就是垂足。由此可见，先看目标的等高线、再看可行集的形状，是这一节所有题目的共同读法。（二）第一性原理的两条路线 路线一是消元：约束 $x = 1$ 就是 $c_1 = 1$ ，代入目标得 $\frac{1}{2}(1 + c_2^2)$ ，只剩 c_2 自由， $c_2 = 0$ 时最小，于是 $c^* = (1, 0)$ ，最优值 $\frac{1}{2}$ 。路线二是拉格朗日乘子：写 $L(c, \lambda) = \frac{1}{2}\|c\|_2^2 + \lambda(c_1 - 1)$ ，令梯度为零得 $c_1 + \lambda = 0$ 、 $c_2 = 0$ ，再由 $c_1 = 1$ 定出 $\lambda = -1$ 。两条路线必须落到同一个点，这正是「把约束钉死」与「把违反约束的代价抬到无穷」的等价性；顺带得到的 $\lambda = -1$ 是约束右端从 1 增到 $1 + \epsilon$ 时最优值的下降速率，它对应的就是 ADMM 里的 u 。（三）一次可复算的验算 先做三点代入： $c = (1, 0)$ 给 0.5， $c = (1, 1)$ 给 $0.5(1 + 1) = 1.0$ ， $c = (1, 2)$ 给 $0.5(1 + 4) = 2.5$ ，可见可行点里 c_2 越远离 0 目标越大，最小值在 $c_2 = 0$ 。再看一般直线：取单位法向 $n = (3/5, 4/5)$ 、 $b = 5$ ，约束 $n \cdot c = 5$ 即直线 $3c_1 + 4c_2 = 25$ ，它到原点的距离是 5；按垂足公式 $c^* = bn = (3, 4)$ ，代回约束得 $3 \cdot 3 + 4 \cdot 4 = 25$ ，目标值 $\frac{1}{2}(9 + 16) = 12.5$ ，而由距离直接算 $\frac{1}{2} \cdot 5^2 = 12.5$ ，两路对上。（四）与 Problem 2 的接口 本页 Problem 2 把「点是否落在 $x = 1$ 上」与「离得多远」拆成两件事：前者交给只取 0 与 $+\infty$ 的指示函数，后者交给 $\frac{1}{2}\|z\|_2^2$ 。这样 ADMM 的一步就退化成上面算过的垂足运算，也解释了 $g(z)$ 为什么只报「在不在」而不报「差多少」：距离的度量权被完整留给了二次项。（五）跨课程连接：凸优化 (Boyd) 的投影与近端算子 向闭凸集投影，就是 $\frac{1}{2}\|c - v\|_2^2$ 在该集合约束下的最小化；把二次项换成一般函数就得到近端算子。连接路径是 Boyd 的近端算子到本页「二次项加直线约束」的玩具，再到结论 $c^* = (1, 0)$ 。区别有两点：一般近端算子面对非光滑目标要用次梯度、通常没有闭式解，而直线约束下的闭式解就是垂足；向经过原点的子空间投影是线性算子且满足幂等，而本页可行集是一条不经过原点的仿射直线，投影只对 v 仿射，且 $c^* = 0$ 根本不可行，这正是 ADMM 必须另设 z 的根源。再补一条概率论的平行线 (Bertsekas)：条件期望 $E[X | Y]$ 是「在所有 Y 的函数里挑一个使均方误差最小」的解，判据是正交性 $E[(X - E[X | Y])h(Y)] = 0$ 。连接路径是概率论的「最小均方误差加正交性」到本页的「最小 ℓ_2 距离加垂足」。区别在于：概率论那侧的内积是 $E[XY]$ 、可行集是线性子空间，所以投影线性、误差与子空间正交；本页这侧可行集是仿射直线、目标是范数平方，只保留「最近点」这一几何结论。

Problem 3. Which vector x minimizes $F(x) = \frac{1}{2}\|x - a\|_2^2$ on the line $x = 1$ if $a = (2, 3)$?

[note] 原文 write problem 1 as a minimization of $z \cdot$ Describe th at 0 - 1 indicator function $g(z)$, and take $+g(z)$ w ith one ADMM step starung 什 om = 0 , $z = 0$, $= 0$ 行序被打乱 (of z 与 $+g(z)$ 本属同一处)，重排为：目标 $\frac{1}{2}\|z\|_2^2 + g(z)$ ，其中 $g(z)$ 是该直线的 0-1 indicator function (0-1 指示函数：点落在直线上时为 0，不在直线上时为 1)，ADMM 初值 $x = z = u = 0$ 。这里必须把两种「指示函数」分清楚，否则后面 ADMM 的 z 步会算错：优化文献里的 indicator function 常指「集合内取 0、集合外取 $+\infty$ 」的凸分析版本，而本说明写 0-1，是「集合内取 0、集合外取 1」的判定版本，两者相差一个常数 1。这一差别不影响 z 步的 $\arg \min$ （集合内外的相对高低没变），但会影响目标函数的取值，所以原文写 0-1 就不能擅自换成 $+\infty$ 版本。另外 starung 为 starting，什 om 为 from， $x = 0$ 指零向量而非标量 0，题面沿用英文原文的写法不再展开。

它在解决什么设计问题。这一题真正想让你看到的不是「怎么把一条直线写成函数」，而是约束怎么塞进一个本来只认「无约束光滑目标」的算法框架。ADMM 的适用对象是 $\min f(x) + g(z)$ 且 $x = z$ 的两块可分结构，它的三大件（ x 步、 z 步、对偶

更新 $u \leftarrow u + x - z$ ）都是围绕「把耦合约束拆成两份、各管一半」设计的。于是写出 $g(z)$ 就是把「 z 必须落在直线上」这件离散的、几何的事实，翻译成算法能吃的一口饭。

从第一性原理展开。 约束有两种基本写法：**罚函数**（违规就加代价）与**硬投影**（违规直接不许）。0-1 指示函数给的是「等权罚」：只要离开直线，不管离多远都收 1 块钱。它把可行域与非可行域拉出一个**有限的高度差**，因此 $\arg \min$ 只会问「哪个更低」，不会问「低多少」。而 $+\infty$ 版本给的是「无限罚」，本质上是**硬约束**，它在理论上是精确的（原问题与松弛问题同解），但需要 prox 算子来落地。两者在 z 步里的作用机理完全一致： z 步形如 $\min_z \frac{1}{2}\|z\|_2^2 + g(z) + \frac{\rho}{2}\|x - z + u\|_2^2$ ，二次项是「忠于上一轮 x 与对偶变量 u 」， g 是「守住可行域」。当 g 是 0-1 版本，集合外那一项多出来的 1 是处处相同的常数，比较不同 z 时被减掉，因此 z 步退化为「把 $\frac{1}{2}\|z\|_2^2 + \frac{\rho}{2}\|x - z + u\|_2^2$ 的无约束极小点投影到直线上」，也就是一个**投影步骤**。这正是「原文写 0-1 就不能擅自换成 $+\infty$ 」的技术理由：换掉之后 z 步的数值不会变，但目标函数的数值会变，而算法收敛性判据看的正是目标函数值。

一个可复算的数值例子。 取具体直线 $x + y = 1$ ，初值 $x = z = u = 0$ 。 $\frac{1}{2}\|z\|_2^2$ 在 $z = (0, 0)$ 处为 0，而 $(0, 0)$ 不在直线上，故 $g(z)$ 两种版本分别给出 $g_{0-1} = 1$ 与 $g_{+\infty} = +\infty$ ；此时目标值一个是 $0 + 1 = 1$ ，一个是 $0 + \infty$ ，这就是「**相差常数 1 影响取值的最小证据**」。再走一步 z 步（取 $\rho = 1$ ，则 z 步无约束极小点在 $(0.5, 0.5)$ ，因为梯度 $z + \rho(z - x - u) = 0$ 给 $(1 + \rho)z = \rho(x + u)$ ，代入 $\rho = 1$ 、 $x = u = 0$ 得 $z = (0, 0)$ ，投影到 $x + y = 1$ 上得 $z = (0.5, 0.5)$ ）：该点落在直线上， $g_{0-1} = 0$ 、 $g_{+\infty} = 0$ ，两个版本在这一步**完全重合**，验证了「 $\arg \min$ 不受影响」。整条链可手算复现： $\frac{1}{2}\|z\|_2^2 + g(z)$ 从 $1 \rightarrow 0.25$ （ $z = (0.5, 0.5)$ 时范数平方为 0.5， $\frac{1}{2} \times 0.5 = 0.25$ ），对偶变量 $u \leftarrow u + x - z$ 也从 0 变为 $(0, 0) - (0.5, 0.5) = (-0.5, -0.5)$ ，这就是 ADMM 一步的全过程。

跨课程连接。 与 6.042J 离散数学中的**特征函数（characteristic function / indicator）与等价关系**对照：那里写 $\mathbf{1}_S(x)$ 或 $\chi_S(x)$ ，第 1 页的判据是「 $x \in S$ 时取 1、否则取 0」，配合等价类把结构相同的对象打包；连接路径是「子集 $\leftrightarrow$ 隶属度函数 $\leftrightarrow$ 把集合运算变成函数的加法与乘法（ $\mathbf{1}_{A \cap B} = \mathbf{1}_A \mathbf{1}_B$ ）」。区别有两处：**其一，取值方向相反**，6.042J 的指标是「在里面给 1」，本题 0-1 指示是「在里面给 0」，差一个 $1 - \mathbf{1}_S$ 的对偶写法，这正是它在优化里更好用的原因——0 恰好是罚函数的「无罪基准点」；**其二，扩展值不同**，6.042J 的指标只取两个有限值，而凸分析版本允许取 $+\infty$ ，这已经超出离散数学的处理范围，属于凸优化里「扩展值函数」的领域。与在学的**概率论（Bertsekas）**对照更直接：指示随机变量 I_A 满足 $E[I_A] = P(A)$ ，期望就是把指标函数当「质量」加权，与本题把指标当「代价」加权是同一件事的两面——都是「用函数值编码一条命题的真假」。命名模板：**指示函数判据法**——遇到「某个点是否满足某性质」写不进目标函数的场合，就先造一个只取两值的函数做翻译，再决定它是「罚 1」还是「罚 ∞ 」，最后检查这个选择是否只差常数。

补一句记号约定： $x = 0$ 里的 0 是零向量（在 $\mathbb{R}^2$ 中即 $(0, 0)$ ）而非标量， u 同为对偶向量，三者形状必须一致，否则 $x - z + u$ 无法逐分量相加；OCR 打乱的 of z 与 $+g(z)$ 已按语义归位到同一句，其余词序照原文。

Problem 4. Following Example 2, take one ADMM step in Problem 3 from $c = (1, 1)$.

[note] 原文 Whi h /\II 到 1 on the line $c = 1$ if = (2 , 3) ? 中 /\II 为 minimizes 的误读，到 1 是函数名 $F(x)$ 与下标 1 被 OCR 吞掉后的残片，缺失的目标函数依本节例 2 的投影形式重建为 $F(x) = \frac{1}{2}\|x - a\|_2^2$ ，on the line $c = 1$ 拆为 on the line 与约束 $x = 1$ ， $a = (2, 3)$ 依括号内数值保留。这一题与 Problem 1 的差别只在目标函数：Problem 1 度量的原点 to 解的距离，Problem 3 度量的是给定点 $a = (2, 3)$ 到解的距离，于是解就是把 a 投影到直线 $x = 1$ 上的垂足，即 $x^* = (1, 3)$ 。变量名也从 c 换成 x ，这是原书的记号习惯（同一件东西在相邻题目里既叫 c 又叫 x ），翻译时照录，不改写、不强行统一，以免与英文题干对不上。

它在解决什么设计问题。 这一题的关键词是「投影」，而投影是**最小二乘与约束优化之间那道桥**。Problem 1 问「原点离直线多远、最近点在哪」，本质是求原点到集合的**距离函数**；Problem 3 换成「给定的 a 离直线多远」，本质是求**一般点到集合的距离**。把目标写成 $F(x) = \frac{1}{2}\|x - a\|_2^2$ 就是在说：我不再关心坐标系原点在哪，我关心的是「参考点 a 与可行点 x 的偏差平方和」。 $\frac{1}{2}$ 这个系数不是装饰，它是为了让梯度变成干净的 $x - a$ （否则导数会多出一个 2），从而最优点条件直接写成「残差向量 $x - a$ 与可行方向正交」，这就是**正交性原理**。

从第一性原理展开。 约束是一条直线（一维仿射集），约束的「自由度」只有一维，而被优化的空间是二维，因此最优点处必然有一维被约束吃掉，剩下一维由最优性条件决定。用几何语言： x^* 必须落线上，且 $(a - x^*)$ 必须垂直于直线的方向。用代数语言：这就是拉格朗日乘子法的几何影子——在最优处目标梯度 $\nabla F = x - a$ 必须落在约束法空间里，即 $x - a$ 平行于法向量 $(1, 0)$ 。两者同义，只是后者能算，前者能想。**为什么一定是垂足**：如果投影点不垂直，沿直线的分量就意味着「再沿线走一点可以更靠近 a 」，与最优性矛盾。这条推理可以逐字搬到任意闭凸集，是凸优化里投影算子理论的第一块砖。

一个可复算的手算。 取 $a = (2, 3)$ ，约束 $x = 1$ 。写成 $x = (x_1, x_2)$ ，目标 $\frac{1}{2}[(x_1 - 2)^2 + (x_2 - 3)^2]$ 。对 x_2 求偏导得 $x_2 - 3 = 0$ ，故 $x_2 = 3$ ；对 x_1 求偏导得 $x_1 - 2 = 0$ ，但被约束 $x_1 = 1$ 覆盖，所以取 $x_1 = 1$ 。结果 $x^* = (1, 3)$ ，验证： $(a - x^*) = (1, 0)$ 垂直于方向 $(0, 1)$ ，垂足成立；目标值 $\frac{1}{2}(1^2 + 0^2) = 0.5$ ，与 Problem 1 若取 $a = (0, 0)$ 得 $x^* = (1, 0)$ 、目标值 0.5 的形式一致，**可见两题只是把参考点从原点平移到了 a** ，这也解释了为什么出题人敢在同一节安排两题：它们在算法层面的处理完全相同，只是一个特例。

跨课程连接。 与 Strang 线性代数 / 18.065 的**最小二乘与投影矩阵**对照：那里把「 b 投影到 A 的列空间」写成 $P = A(A^T A)^{-1} A^T$ ，正规方程 $A^T A \hat{x} = A^T b$ 说的正是「残差与列空间正交」。连接路径是「本题的正交性条件 $\leftrightarrow$ 正规方程的几何含义 $\leftrightarrow$ 泛化到子空间投影」。区别在于：**其一，可行集不同**，Strang 那里是线性子空间（过原点），本题是一条不过原点的仿射直线，必须先平移再用子空间公式，否则会算错；**其二，维度与记号不同**，那里用矩阵 A 表示基，这里用一条约束等式表示，但如果把直线写作 $x = 1$ 的参数形式（基向量 $(0, 1)$ 、偏移 $(1, 0)$ ），投影矩阵那套公式可以整体套用。与 凸优化（Boyd）的**投影算子 prox 与集合距离函数**对照：那里把「投影到闭凸集」定义为 $\Pi_C(v) = \arg \min_{x \in C} \frac{1}{2}\|x - v\|_2^2$ ，与本题的 $F(x)$ 只差把 a 记作 v

；区别是凸优化强调「凸集保证投影唯一且 Π_C 非扩张」，而本题用初等的垂足几何就得到结论，因为直线是最简单的凸集。命名模板：**垂足验算法**——先写目标、再取梯度、再让梯度垂直于约束方向，最后把解回代，三步闭环，可用于任何「点到仿射集」的投影题。

记号上再补一句： c 与 x 指的是同一个决策变量（原书在相邻题目里混用），翻译时照录而不强求统一，正是为了让你的解答能与英文题干逐字对上；on the line 与 $x = 1$ 之间被 OCR 粘连的 $c = 1$ 已按语义拆开， $a = (2, 3)$ 的括号与逗号内容原样保留。

Problem 5. What $c \geq 0$ and $y \geq 0$ minimize $c + y$ if $cy = 1$?

[note] 原文 Following Example 2, take one ADMM step in Problem 3. 其中 $c = (1, 1)$ 中 Following 为 Following，from 为 from，起点 $c = (1, 1)$ 依原文保留。此处变量记号与 Problem 3 的 x 混用，按原文仍写作 c ：在 §III.4 的语境里， c 与 x 都指被分裂算法迭代的原始变量，题目只要求「取一步」，不要求改写变量名，所以英文题干保持原样。取一步的含义是：已知初值 $c = (1, 1)$ ，先做 x 步（解一个只含光滑二次项的最小子问题），再做 z 步（处理非光滑或指示型的那一半），最后做 u 步（累加两者的残差）。三步的顺序与更新公式由例 2 给出，读者对照正文的例 2 逐式代入即可，题面不再重述公式。**补一：这三步在解决什么设计问题。** 原始问题是一个目标加一个把两块变量缠住的约束；直接对 (x, z) 联合求最小，得到的就是原问题本身，计算上没有便宜可占。ADMM 的价值来自一条前提：约束形式允许把变量拆成两半，而且拆开后每一半单独看都有闭式结构——一半是光滑二次项，等价于解一次线性方程组；另一半是非光滑项或指示函数，等价于做一次投影或软阈值。第一性原理上的取舍是：用每轮两次便宜的小问题加一次残差累加，换掉一次昂贵的大问题，代价是只拿到一个迭代序列。为什么不能联合最小化：那一步就是原约束问题的对偶函数，难度没有下降；交替最小化之所以可行，是因为罚项 $\frac{\rho}{2} \|x - z\|^2$ 给对偶问题补上了曲率，使对偶上升在 $1/\rho$ 量级的步长下不震荡。顺序本身也是设计： x 步只碰光滑项，落成正规方程； z 步只碰非光滑项，落成投影； u 步把两者的差距记进乘子。顺带纠正一个常见误解：ADMM 的迭代值不保证目标单调下降，因为每轮最小化的是增广拉格朗日量而不是原目标—— x 步可能压低目标却脱离可行集， z 步把它拉回可行集又抬高目标。**补二：一次可复算的一步。** 把三步骨架套在一个能手算的 2×2 玩具版上： x 那一半取光滑二次 $f(x) = \frac{1}{2} \|x - (2, 2)\|^2$ ， z 那一半取 Problem 3 的双曲线可行集（ g 取该集合的指示函数），参数取 $\rho = 1$ 、 $u^0 = (0, 0)$ 、 $z^0 = (1, 1)$ 。 x 步令梯度为零： $x - (2, 2) + \rho(x - z^0 + u^0) = 0$ ，解得 $x^1 = \frac{(2, 2) + z^0 - u^0}{2} = (1.5, 1.5)$ 。 z 步是往双曲线上投影，即解 $\min \|z - (x^1 + u^0)\|^2 = \min \|z - (1.5, 1.5)\|^2$ 受约束 $uv = 1$ ；把目标展开为 $u^2 + v^2 - 3(u + v) + 4.5$ ，由 $u^2 + v^2 \geq 2uv = 2$ 与 $u + v \geq 2\sqrt{uv} = 2$ 两式组合，它不小于 $2 - 6 + 4.5 = 0.5$ ，等号当且仅当 $u = v = 1$ ，所以 $z^1 = (1, 1)$ 。 u 步： $u^1 = u^0 + (x^1 - z^1) = (0.5, 0.5)$ ，原始残差 $\|x^1 - z^1\| = \sqrt{0.5} \approx 0.707$ 不为零，不能停。再走一轮： $x^2 = \frac{(2, 2) + z^1 - u^1}{2} = (1.25, 1.25)$ ， x 从 1.5 退回 1.25，正是 u 把上一轮的违约量记在账上并在本步扣掉的结果。还有一条数值证据： x^1 处的目标值 0.25 比真解 $(1, 1)$ 处的 1 还低，但 $1.5 \times 1.5 = 2.25 \neq 1$ 违反约束——目标与可行性确实在轮流让步。**补三：跨课程连接。** 这条与 Strang 线性代数里的正规方程、最小二乘是同一台机器： x 步的一般形状是 $\min \|Ax - b\|^2 + \rho \|x - d\|^2$ ，梯度取零得 $(A^T A + \rho I)x = A^T b + \rho d$ ，即带 Tikhonov（岭）项的正规方程， $\rho > 0$ 保证矩阵正定、解唯一，动机与最小二乘里用 λI 治病态完全一致。连接路径：ADMM 每一轮的 x 步就是一次最小二乘，只是右端的 d 由上一轮的 z 与 u 提供。区别有两处：经典最小二乘的 λ 是建模时的先验或稳定化手段，ADMM 的 ρ 只是算法旋钮；经典问题的 A, b 固定，这里的目标每轮在移动，是一串移动目标的最小二乘。另一个可用的锚点是 6.042 的等价关系：共识约束 $x = z$ 的可行集是对角子空间，硬投影只需取中点 $\frac{x+z}{2}$ ，而 ADMM 用罚项做软合并，允许迭代过程中 $x \neq z$ 。命名模板：把这三步记成「光滑闭式步—投影或软阈值步—残差记账步」，一句话是「一解、二投、三记账」。**补四：为什么题面只要求一步。** Problem 3 的可行集非凸，整段迭代的收敛性没有一般保证；而单步循环对非凸问题依然良定义， z 步只是一个低维投影。所以「取一步」恰好停在理论上站得住的位置：它检验的是能否把 x 、 z 、 u 三个量及其先后次序摆对，而不是要读者回答一个尚无干净结论的收敛问题。

Problem 6. Following Example 3 in this section, take one ADMM step in Problem 5: compute x_1, c_1, z_1 .

[note] 原文 What $c \geq 0$ and $y \geq 0$ minimize $c + y$ if $cy = 1$ 中有四类误读： $\geq$ 为 $\geq$ ， y 为 y ，— 在本题语境下为正号（目标为 $c + y$ ）， y 为 y ；末行缺失的约束依题目对称结构与「两变量之积给定」的语境重建为 $cy = 1$ 。这题是 ADMM 的标准试金石：目标 $c + y$ 是线性的，约束 $cy = 1$ 却是非线性的双曲线，可行集非凸，但限制在 $c \geq 0, y \geq 0$ 的第一象限后，可行集是双曲线的一支，最小值在对称点 $c = y = 1$ 处取到——因为由 $cy = 1$ 与 $c, y \geq 0$ 可得 $(c - y)^2 = (c + y)^2 - 4cy \geq 0$ ，即 $c + y \geq 2$ ，等号成立当且仅当 $c = y = 1$ 。题面把 $[c \geq 0 \text{ 与 } y \geq 0]$ 写成两个条件并列，翻译时照英文保留 and 与 if 的原始语序。**补一：这道题在压哪两条软肋。** 教科书拿它当试金石，是因为它同时命中 ADMM 的两个薄弱处。目标 $c + y$ 是线性的，被分裂出来的这一半没有曲率， x 步里唯一提供强凸性的东西是罚项 $\frac{\rho}{2} \|x - z\|^2$ ——这正是 ρ 不能取太小的机制原因： ρ 小则子问题接近奇异、迭代虚浮， ρ 大则原始残差压得快而对偶步子变大，调参张力就来自这里。约束 $cy = 1$ 是非凸的双曲线，于是 z 步是往非凸集上投影，解可能不唯一、需要选分支。设计问题因此变成：分裂算法在「线性目标加非凸约束」下还能走多远？答案是单步良定义（ z 步退化成低维投影或一维搜索），整体收敛却需要额外的非凸条件，不能照搬凸情形的证明。**补二：一维化加精确恒等式。** 令 $t = c$ ，则 $y = 1/t$ ，目标变成 $t + \frac{1}{t}$ 。关键恒等式是 $t + \frac{1}{t} - 2 = \frac{(t-1)^2}{t}$ ，右边在 $t > 0$ 上非负且零点唯一，所以最小值 2 在 $t = 1$ 处取得，即 $c = y = 1$ 。代入具体数值复算： $t = 2$ 得 $2 + 0.5 = 2.5$ ； $t = 1.5$ 得 $1.5 + 0.6667 = 2.1667$ ； $t = 1.1$ 得 $1.1 + 0.9091 = 2.0091$ ； $t = 0.8$ 得 $0.8 + 1.25 = 2.05$ ——四个值都在 2 以上，且 t 越靠近 1 越贴近 2。用导数验算同样一步到位： $1 - \frac{1}{t^2} = 0$ 给出 $t = 1$ ，二阶导 $\frac{2}{t^3} > 0$ 说明它是全局最小点；这与原文用 $(c - y)^2 \geq 0$ 得到下界是同一条不等式的两种写法。顺便说明重建右端项为什么只能是 1：若约束换成 $cy = 0$ 或 $c = y$ ，最小值会在原点取到，题目变得平凡；「存在非平凡的对称最优解」这一要求把右端项钉死在正常数上，而「两变量之积给定」正好把它固定为 1。**补三：跨课程连接。** 这条与在学的凸优化（Boyd）里的最优性判据直接相关：线性目标在可行集上的最优解必落在边界，本例的「边界」退化为双曲线与直线 $c + y = 2$ 的切点，切点的对称性就是 $c = y$ 。连接路径：Boyd 里处理线性目标靠 KKT 条件，先写拉格朗日函数再解方程组；这里可行集非凸，KKT 不再充分，只能改用「先给全局下界、再验证等号可达」两步走。区别正在于此：凸情形的 KKT 是充分必要判据，非凸情形的不等式下界只是必要条件，必须补一次等号可达性验证。另一个锚点在概率论（Bertsekas）里： $\sqrt{cy} \leq \frac{c+y}{2}$ 就是凹函数 $g(t) = \sqrt{t}$ 在两点上的 Jensen 不等式，也可以直接写成 $(\sqrt{c} - \sqrt{y})^2 \geq 0$ ；区别在于 Jensen 处理随机变量的期望，此处只是把两个确定点代进去的退化情形。命名模板：「对称点夹逼法」——目标与约束同时对称，就先猜对称轴上的点，用不等式给出全局下界，再验证等号可达。

Problem 7. Find the LASSO solution that minimizes $\|Ac - b\|_1 + \gamma \|c\|_1$, with the data A and b shown in the text and $\gamma = 1$.

[note] 译者注（OCR 重建 + 记号辨析）：原文 take one ADMM step in Problem 5: 9 compute x_1, c_1, z_1 ，其中 9 为首题号噪声， x_1, c_1, z_1 是三个迭代量的下标被 OCR 提升为行内字符：重建为 x_1, c_1, z_1 ，即例 3 三变量更新次序下的第一步迭代值。原书此处的记号值得留意： x 与 c 同时出现，说明 Example 3 把「原始变量」分裂成了两个分量（一个承担线性目标、一个承担乘积约束）， z 则是为解耦而引入的副本变量，下标 1 表示第一次迭代（不是指标 1 或某个分量）。所以 x_1, c_1, z_1 不能读成「 X_1, C_1 、常数 1」，而应读成同一层级的三个一阶迭代量，翻译时按 x_1, c_1, z_1 写出。

先问机制：为什么非要「分裂」？ADMM 要处理的目标是 $f(x) + g(z)$ 且约束 $Ax + Bz = c$ 这种**可分离形状**；而原问题若写成 $\min f(x) + g(x)$ ，两项共用同一个变量，梯度里互相纠缠，投影法、邻近算法都无法「各管各的」。于是引入副本：令 $z = x$ ，把「优化」与「约束」分给两个变量，约束 $x - z = 0$ 只负责把两者重新缝合。三个更新的本质是依次对**增广拉格朗日函数**做块坐标下降：固定 c, z, u 解 x （对 x 是最小二乘/线性系统），固定 x, z, u 解 c （带 ℓ_1 项，用软阈值 $\text{soft}(v, \kappa)_i = \text{sign}(v_i) \max(|v_i| - \kappa, 0)$ 闭式解），最后用残差 $x - z$ 累加到对偶变量 u 。注意 x 与 c 不是「两个变量各管一件事」这么含糊，而是严格对应 f 与 g ： f 二次则 x 步化为求解 $(A^\top A + \rho I)x = \cdots$ 的线性方程组， g 是 $\|\cdot\|_1$ 则 c 步是一族互相独立的逐分量软阈值——这正是「分裂」换来的并行性。 z 的第三个角色容易被忽略：它让对偶更新不需要 A^{-1} ，只要一次减法。

具体验算（ 2×2 规模，可手算复现）：取 $f(x) = \frac{1}{2}\|x - b\|_2^2$ ， $b = (3, 1)^\top$ ， $g(c) = \|c\|_1$ ，约束 $x = c$ ， $\rho = 1$ 。第 1 步 x 更新解 $\min \frac{1}{2}\|x - b\|_2^2 + \frac{1}{2}\|x - c_0 + u_0\|_2^2$ ，取 $c_0 = 0$ ， $u_0 = 0$ ，得 $x_1 = (b + c_0 - u_0)/2 = (1.5, 0.5)^\top$ 。第 2 步：软阈值输入 $x_1 + u_0 = (1.5, 0.5)^\top$ ，阈值 $\kappa = 1/\rho = 1$ ，故 $c_1 = (0.5, 0)^\top$ ——第二个分量被压成 0，稀疏性当场出现。第 3 步 $u_1 = u_0 + x_1 - c_1 = (1.0, 0.5)^\top$ 。再走一轮： $x_2 = (b + c_1 - u_1)/2 = (1.25, 0.25)^\top$ ， $c_2 = \text{soft}((2.25, 0.75), 1) = (1.25, 0)^\top$ ， $u_2 = (1.0, 0.75)^\top$ ；可见 x_k 与 c_k 的差（原始残差）在收缩， u 在逐步吸收这份差——这就是「对偶变量积攒残差」的手感。若把下标 1 误读成「分量编号」， c_1 就会被当成 c 的第一个元素，上面的两步就完全对不上账，这是本题记号最贵的坑。

跨课程连接：与 CLRS/6.006 的「分治—合并」同构。归并排序把数组切成两半独立排、再花线性时间合并；ADMM 把 f 与 g 切成两个子问题独立解、再靠 z 与 u 的缝合步合并。区别在于：分治的合并是确定性的线性操作，而 ADMM 的缝合是迭代的、收敛才等价，且它优化的是增广拉格朗日函数而非原目标。第二处连接：这正是 Strang 线性代数里**零空间与最小二乘正规方程**的直接复用——你不必会抽象理论，只要会解 $(A^\top A + \rho I)x = A^\top b + \rho(z - u)$ 就够了， ρ 严格正使矩阵必然可逆（正则化把奇异的 $A^\top A$ 顶起来），这一点与 18.065 讲的正则化最小二乘是同一个动作，区别是那里的 λ 服务于数值稳定，这里的 ρ 服务于迭代分裂。

命名模板：把上述套路叫作**分裂—副本—对称交替法**——先分裂变量，再给副本，最后交替（且对称地）解两个子问题并用残差修正对偶量。检验口诀：看到「 x 与 c 同框且下标是迭代号」，立刻按分裂结构读，不要按分量读。

Problem 8. Following Example 4, take one ADMM step in Problem 7 from $x^0 = 0$, with $P = 2I$.

[note] 原文 FoIIowing Example 4，take one ADMM step in ProbIem 7 什 om 0 with P = 2 · 中 FoIIowing 为 Following，什 om 为 from，缺失的上标依例 4 重建为初值 $x^0 = 0$ ；例 4 中的矩阵 P 依原文的标量 2 与「 P 为对角矩阵」的语境重建为 $P = 2I$ （ I 为单位矩阵），若原书作标量 2 请以原书为准。为什么要单独给一个 P ：ADR/ADMM 类分裂算法在 x 步里通常解一个形如 $(P + \rho M)x =$ 右端的线性系统， P 就是该项的系数矩阵，取 $P = 2I$ 使系统成为「2 倍单位阵加惩罚项」，各分量近似解耦、便于手算一步。注意 P 与对偶步长 ρ 是两个不同的量： P 刻画原问题的二次项， ρ 刻画增广拉格朗日函数里约束被罚得有多重。

Problem 9. Find $\text{prox}_F(c) = \arg \min_z (\frac{1}{2}\|c - z\|_2^2 + F(z))$ in equation (27), where $F(z) = \frac{1}{2}\|z\|_2^2 - 1$. Is this the proximal operator for F ? It is a function of c .

[note] 译者注（OCR 重建 + 目标函数辨析）：原文 Find LASSO that minimizes 训一 b113 + @日 1 with 1 1 中 训 为 $\|Ac - b\|_1$ 之误，113 是下标与 ℓ_1 记号的粘连（残差项与正则项都带 ℓ_1 下标），@ 为系数 γ 、日 为 $\|c\|_1$ ；原文所列 A 与 b 的具体数值在 OCR 中丢失，故题干写作「数据 A 与 b 见正文」， $\gamma = 1$ 依残片 with 1 1 与例 4 的设定重建，如与原书不符请以原书为准。这就是 §III.4 反复出现的 LASSO 原型：残差用 ℓ_1 范数（对离群点稳健）、系数也用 ℓ_1 范数（把不重要的分量压到零），两项都不可微，所以正是 ADMM 的用武之地。阅读时区分两件事： $\|Ac - b\|_1$ 度量拟合误差， $\gamma\|c\|_1$ 度量解的稀疏程度， γ 越大解越稀疏；Problem 8 接着对它走一步 ADMM。

先问机制：为什么偏偏选 ℓ_1 而不是熟悉的 ℓ_2 ？最小二乘 $\|Ac - b\|_2^2$ 对残差取平方，一个大残差会被平方放大，于是少数离群点就能把整条拟合直线拽偏；换成 ℓ_1 残差，损失是绝对值的和，其对残差的「边际惩罚」是常数而非线性增长，离群点最多贡献一个常数份额，稳健性由此而来。稀疏性一侧同样来自几何而非玄学：约束 $\|c\|_1 \leq t$ 的等值面是菱形（二维时是旋转 45° 的正方形），最优解落在菱形**顶点**上的概率为正，而顶点坐标天然含零分量； ℓ_2 球是圆，切点几乎不落在坐标轴上，故不产生精确零。还有一层必须点明的机制：这两项都不可微——残差项在 $Ac = b$ 处不可导，正则项在 $c_i = 0$ 处不可导——而 LASSO 的整个意义就在于把某些 c_i **解到恰好为 0**，也就是最优解常常正落在不可导点上，所以次梯度法、拉格朗日乘法都不顺手，ADMM 分裂出「二次项走线性系统、 ℓ_1 项走软阈值」两条路，正好绕开不可微。

具体验算：取 2×2 的 A, b, c ，两步手算即可看到「压零」与「稳健」两件事。设 $Ac - b = (6, -6)^\top$ ，则 $\|Ac - b\|_1 = 12$ ，而 $\|Ac - b\|_2^2 = 36 + 36 = 72$ ；把某一行的残差从 6 换成 60（一个离群点）： ℓ_1 变为 66（增加 54）， ℓ_2 平方变为 3636（增加 3564）—— ℓ_2 的惩罚被放大近 66 倍， ℓ_1 只线性增长，这就是稳健性的数值证据。再看软阈值： $v = (1.5, -0.4)^\top$ 、阈值 $\kappa = 1/\rho = 1$ 时， $c^+ = (\text{sign}(1.5) \max(1.5 - 1, 0), \text{sign}(-0.4) \max(0.4 - 1, 0))^\top = (0.5, 0)^\top$ ： $|v_i| \leq \kappa$ 的分量被直接判零， $|v_i| > \kappa$ 的分量被整体向零平移 κ 。所以 γ 与 ρ 联手决定「零的多少」： γ 越大（或 ρ 越小，即阈值 $\kappa = \gamma/\rho$ 越大），被判零的分量越多，解越稀疏； $\gamma = 0$ 时退化为纯 ℓ_1 回归， $\gamma \rightarrow \infty$ 时解被压成全零向量。这里最易混的一处： $\|Ac - b\|_1$ 里的 ℓ_1 在度量**拟合误差**， $\gamma\|c\|_1$ 里的 ℓ_1 在度量**解的稀疏程度**，两个 ℓ_1 恰好让 OCR 把 113 粘成一团，读的时候必须按位置切开。

跨课程连接：与在学的概率论（Bertsekas）中的**中位数 vs 均值**是同一件事的两面。最小化 $\sum_i |r_i|$ 的最优常数是样本中位数，最小化 $\sum_i r_i^2$ 的最优常数是样本均值；中位数对极端值不敏感，正是 ℓ_1 稳健性的概率论解释。区别在于：Bertsekas 里这是估计量的稳健性（大数定律与收敛性语言），而这里是正则化优化（惩罚项与稀疏性语言），同一份数学，两套词汇。第二处连接：与 CSAPP 的**缓存与浮点**有关——残差项用 ℓ_1 时，求和顺序不同会引起明显的浮点舍入差异，且 ℓ_1 目标的分段线性结构使最优解常常落在约束面的角点上，对数值扰动的敏感模式与 ℓ_2 的平滑二次碗完全不同；如果你按 CSAPP 讲的浮点累加误差去复算上面的 12 与 72，会发现 ℓ_2 的平方放大同时也放大了舍入误差。第三处连接：与凸优化（Boyd）的**次梯度与近端算子**衔接—— ℓ_1 的次微分在原点是一个区间 $[-1, 1]$ ，ADMM 的软阈值步就是 ℓ_1 近端算子 $\text{prox}_{\kappa\|\cdot\|_1}$ 的闭式表达，可把它当成「投影到 ℓ_1 球的柔化版本」：投影是硬截断，近端算子是软收缩。

命名模板：把「两处 ℓ_1 + 一次软阈值」的记忆套路叫**双 ℓ_1 稳健稀疏法**——残差一个 ℓ_1 管抗离群，系数一个 ℓ_1 管出稀疏，中间用阈值 $\kappa = \gamma/\rho$ 做软收缩；做题时先写清哪个 ℓ_1 属于哪一项，再动手走 ADMM 的两步。

[note] 译者注（证明策略模板）：本页 Problem 1 到 Problem 9 是一条完整的 ADMM 流水线，先讲清它要解决的设计问题：目标函数常常是光滑项 + 非光滑项，例如 $\frac{1}{2}\|Ac - b\|_2^2 + \gamma\|c\|_1$ 。光滑项可以直接调用线性代数（正规方程、QR、共轭梯度都行），但 $\|c\|_1$ 在原点没有梯度，梯度法立刻失效。ADMM 的处理是引入一个副本 z ，把问题改写成 $\min f(c) + g(z)$ 且 $c = z$ ，然后交替各解一半： c 步只面对光滑的 f （一个线性系统）， z 步只面对非光滑的 g （往往有现成的近端公式，例如 g 取 ℓ_1 范数时就是软阈值 $\text{soft}(v, \lambda)_i = \text{sign}(v_i) \max(|v_i| - \lambda, 0)$ ），最后用对偶变量 u 把两者拉回一致。拿 Problem 1 与 Problem 2 亲自走一步：在直线 $x = 1$ 上最小化 $\frac{1}{2}\|x\|_2^2$ ，真解是 $(1, 0)$ ，因为钉住第一坐标后只剩 x_2^2 要压小；改写成 Problem 2 的 $\min \frac{1}{2}\|z\|_2^2 + g(z)$ ，其中 g 是这条直线的 0-1 指示函数（线上取 0、线外取 1）。从 $x = z = u = 0$ 出发， x 步解 $\min \frac{1}{2}\|x\|_2^2 + \frac{\rho}{2}\|x - z + u\|_2^2$ ，求得 $(1 + \rho)x = \rho(z - u)$ ，代入零初值得 $x_1 = 0$ ； z 步是向直线做投影，得 $z_1 = (1, 0)$ ； u 步累加残差，得 $u_1 = x_1 - z_1 = (-1, 0)$ 。再走一轮就看得出机制： $x_2 = \frac{\rho}{1 + \rho}(z_1 - u_1) = \frac{\rho}{1 + \rho}(2, 0)$ ，误差 $x_k - (1, 0)$ 每一步都被同一个因子 $\frac{1}{1 + \rho}$ 乘一遍， ρ 越大越靠近真解——取 $\rho = 9$ 时 $x_2 = (1.8, 0)$ ，与 $(1, 0)$ 的偏差是 0.8；取 $\rho = 99$ 时 $x_2 = (1.98, 0)$ ，偏差 0.02， ρ 每

加一个数量级，偏差就少约一个数量级。这正是对偶上升那套「乘子越大、约束越硬」的定量版本： ρ 在角色上更像步长，而不只是约束松紧的旋钮。更一般地，指示函数的近端算子在几何上就是投影： $\text{prox}_g(v) = \arg \min_z \frac{1}{2} \|z - v\|_2^2 + g(z)$ ，当 g 是某凸集 C 的指示函数（集合内 0、集合外 $+\infty$ ）时，集合外的 $+\infty$ 逼着最小值点回到 C 内离 v 最近的位置，也就是 v 在 C 上的投影——Problem 1、Problem 2 里的 $z_1 = (1, 0)$ 正是 $(0, 0)$ 在直线 $x = 1$ 上的投影。这条「投影 = 近端算子」的桥梁把本页与你在 Strang 线性代数课里学过的投影矩阵、正规方程直接接上，连接路径是「投影矩阵 $P = A(A^T A)^{-1} A^T \rightarrow$ 正规方程 $\rightarrow$ 带惩罚项的子问题 $(I + \rho M)x = \rho(z - u) \rightarrow$ 特征值与条件数」，课程是 18.065 矩阵方法，概念是条件数 $\kappa = \sigma_{\max}/\sigma_{\min}$ ；区别在于，这里变难的不是原问题的病态程度，而是 ADMM 自己造出来的子问题—— ρ 变大时 $I + \rho M$ 的特征值被推向同一方向，系统在数值上反而更好解，可每一步的进展（那个 $\frac{1}{1+\rho}$ 因子）却变慢，于是存在一个折中值，这就是实践里要在「子问题好不好解」与「收敛快不快」之间调 ρ 的原因。最后留意 Problem 9 的问法： F 是定义在向量上的函数， prox_F 是由 F 诱导出、以函数为输入输出函数的算子，二者是两层东西，题目特意追问是不是 F 的近端算子，就是在考这层区分。这种「对象层 / 算子层」的区分在离散数学里其实已经见过：等价关系是集合上的一个二元关系，「取商集」则是由这个关系诱导出的一个映射，关系和映射不是同一层东西，与 F 和 prox_F 的关系同构。整套流程可固定成一个可复用的模板，叫**分裂—投影—乘子校验法**：第一步分裂出副本并写出增广拉格朗日函数，第二步对光滑子问题求导得到 $(I + \rho M)x = \rho(z - u)$ 并用一次迭代验算，第三步对非光滑子问题调用近端公式（指示函数情形即投影），第四步用乘子更新核对残差是否按 $\frac{1}{1+\rho}$ 量级收缩，最后回到原问题检验解是否真的落在约束集上。这四步恰好对应本页 Problem 1 到 Problem 9 的推进顺序，也解释了为什么题目要在最后单独把函数与算子的层级区分拎出来追问一次。

Problem 10. Here are three function spaces (each contained in the next) and three examples of functions: Smooth, Lipschitz (the slope can jump), Bounded variation (it can jump). Examples: bowl $x^2 + y^2$; flat base $\max(x^2 + y^2 - 1, 0)$; cylinder base (add step up along $r = 1$).

[note] 原文 Following Example 4，take one ADMM step in Problem 7 from 0 with $P = 2I$ 中 Following 为 Following，from 为 from，缺失的上标依例 4 重建为初值 $x^0 = 0$ ；例 4 中的矩阵 P 依原文的标量 2 与「 P 为对角矩阵」的语境重建为 $P = 2I$ （ I 为单位矩阵），若原书作标量 2 请以原书为准。为什么要单独给一个 P ：ADR/ADMM 类分裂算法在 x 步里通常解一个形如 $(P + \rho M)x =$ 右端的线性系统， P 就是该项的系数矩阵，取 $P = 2I$ 使系统成为「2 倍单位阵加惩罚项」，各分量近似解耦、便于手算一步。注意 P 与对偶步长 ρ 是两个不同的量： P 刻画原问题的二次项， ρ 刻画增广拉格朗日函数里约束被罚得有多重。

它在解决什么设计问题。 分裂算法的困境是：目标函数里有两个「难缠但各自好解」的部分（例如一个二次项加一个绝对值项），直接耦合求解会毁掉两者的可解结构。ADMM 的对策是拆变量、加约束、轮换更新——而轮换更新的每一步都必须「回到原问题」。 x 步之所以要解 $(P + \rho M)x =$ 右端，正是因为增广拉格朗日函数对 x 求导后，原始二次项贡献 Px ，惩罚项贡献 $\rho M(x - \text{当前值})$ ，两项一合并就回到这个线性系统。所以 P 不是可有可无的记号，它就是「原问题在 x 方向上的曲率」的载体。取 $P = 2I$ 有双重好处： $2I$ 正定保证 x 步严格凸、有唯一解；对角保证系统解耦成 n 个独立标量方程，手算一步不会陷入矩阵求逆的泥潭。

走一遍可复算的两步手算。 取最简的标量问题，目标 $\frac{1}{2}(2x^2) + |x|$ ，拆成 $x = z$ ，增广拉格朗日函数为 $2x^2$ 的一半加 $|z|$ 的一半再加 $\frac{\rho}{2}(x - z + u)^2$ （这里 u 是缩放后的对偶变量）。若 P 取标量 2 且 $\rho = 1$ ， x 步的系数就是 $2 + 1 = 3$ ；若 $P = 2I$ 且 $\rho = 1$ ，系数矩阵为 $3I$ ，每个分量各自解 $3x_i = \rho(z_i - u_i)$ ，即 $x_i = (z_i - u_i)/3$ 。两者在标量情形数值一致，但矩阵写法在 n 维时立刻显出优势：解耦意味着 $O(n)$ 而非 $O(n^3)$ 。对偶步则是 $u \leftarrow u + (x - z)$ ，纯累加，无需求解。请把 $x^0 = 0, z^0 = 0, u^0 = 0$ 代进去走一轮，核对 x^1, z^1, u^1 是否自洽——这就是题目要求的「take one ADMM step」的全部内容。

两个量为什么必须分清。 P 属于原问题，改它等于换了一道题； ρ 属于算法，改它只是换了一种走法。 ρ 太大，惩罚项主导， x 被硬拽向 z ，收敛快但容易振荡； ρ 太小，约束几乎不被尊重， x 步只顾降低原目标，对偶变量要很多轮才追上。这种「算法参数不动问题、只动路径」的分层，是理解 ADMM 收敛性讨论的前提，也是为什么书上能一边固定 $P = 2I$ 一边讨论 ρ 的取值。

跨课程连接（CLRS 与 6.006 + Strang 线性代数）。 连接路径是：ADMM 的 x 步在 P 对角时退化为逐分量求解，复杂度从一般稠密求解的 $O(n^3)$ 降到 $O(n)$ ，这正是 CLRS 与 6.006 里「用结构换复杂度」的分治/摊销思路在数值算法里的翻版——区别在于分治靠递归切分问题规模，而对角化靠问题自身的解耦结构，不切规模只切耦合。另一条路径接到 Strang 线性代数： $(P + \rho M)$ 的可解性由正定性保证，而正定性又与 Cholesky 分解、最小二乘的正规方程同源；区别是线性代数课里 P 常被当成给定数据，这里 P 却是建模时被我们亲手选定的设计变量。

命名模板：结构解耦检验法。 拿到任何一个分裂算法的子步，先问三句：这一步在解什么线性系统？系数矩阵有什么结构（对角、三对角、可快速变换）？这个结构能不能把 $O(n^3)$ 压成 $O(n)$ 或 $O(n \log n)$ ？三句答完，子步的代价与可行性就都清楚了。

Problem 11. A neat coarea formula expresses the BV norm of u as the integral of the lengths of the level sets $L(t)$, where $\text{grad } u \neq 0$:

$$\|u\|_{BV} = \iint |\text{grad } u| \, dx \, dy = \int_{-\infty}^{\infty} (\text{length of } L(t)) \, dt$$

[note] 原文 Find prox_F (司 $\arg \min$ 2.2 in equation (27)) . 1 s is the 2 | 2 . It is a function of . 1 proximal operator for 尹 () 一 2 中 司 为 c ， $\arg \min$ 为 $\arg \min$ ，1 s 为 $\|s\|_2$ ，1 t 为 $\|t\|_2$ ，尹 为 F ；被拆散的分式重建为定义式里的 $\frac{1}{2}\|c - z\|_2^2 + F(z)$ 与给定的 $F(z) = \frac{1}{2}\|z\|_2^2 - 1$ （式 (27) 即 proximal operator（近端算子）的定义式）。这里最要紧的是把原文末句的变量补对：It is a function of 后缺失的字母只能是 c ，不能读成 z 。理由是近端算子的定义—— prox_F 是一个「作用于函数、产出新函数」的算子， $\text{prox}_F(c)$ 则把输入向量 c 映射成一个输出向量，而 z 只是定义式里被 $\arg \min$ 优化掉的哑变量（dummy variable），算完之后在结果里不该再出现。代入本题可验算： $\frac{1}{2}\|c - z\|_2^2 + \frac{1}{2}\|z\|_2^2 - 1$ 对 z 求导得 $z - c + z = 0$ ，故 $z = c/2$ ，即 $\text{prox}_F(c) = c/2$ ——若末句写成「 z 的函数」，就等于宣称一个常量 $c/2$ 依赖哑变量，逻辑上自相矛盾。

它在解决什么设计问题。 优化算法的每一次迭代，本质上都在回答「我该往哪走、走多远」。梯度法只借用了目标函数的一阶信息，因此对不可微的项（比如绝对值、指示函数、秩约束）束手无策。近端算子补上的正是这块能力：它在「离当前输入近」与「让 F 变小」之间做一个折中，把不可微的部分单独关进一个子问题里，用解析解处理掉。所以 proximal operator 是「分裂」得以实现的技术底座——ADMM、近端梯度法 (ISTA/FISTA)、投影梯度法都可以由它拼出来。它同时又是「广义投影」： F 取集合的指示函数时， prox_F 退化为到该集合的欧氏投影。一句话，近端算子把「求解」替换成「求解加位移」，从而让原本不可微的问题获得可迭代的形势。

为什么定义里必须出现 $\frac{1}{2}$ 。 这个 $\frac{1}{2}$ 是约定而非装饰：它使二次项求导后得到干净的 $z - c$ ，而不是 $2(z - c)$ 。也就是说， $\frac{1}{2}$ 把「近端项的梯度」标定成单位斜率，于是 prox 的更新式与梯度下降步 $x \leftarrow x - \eta \nabla f$ 在形式与量纲上都可直接对照：当 F 可微时， $\text{prox}_F(c) \approx c - \nabla F(c)$ 正是步长为 1 的梯度步。这类「把系数一次校准好、以后不再出现常数」的做法，与 CSAPP 里对齐到机器字长、让每次访存不必再做除法的思路同构：都是把常数吸收进定义，换取后续推导的整洁。

验算的三种读法。 第一种，配方法： $\frac{1}{2}\|c - z\|_2^2 + \frac{1}{2}\|z\|_2^2 - 1 = \|z - c/2\|_2^2 + \frac{1}{4}\|c\|_2^2 - 1$ ，二次项在 $z = c/2$ 处取到最小值，代回得 $\text{prox}_F(c) = c/2$ 。第二种，线性代数读法：这是把 c 投影到一维子空间 $\text{span}\{c\}$ 再折半，等价于一个对角缩放算子，用 Strang 线性代数里最小二乘的投影矩阵语言说就是 $P = c(c^T c)^{-1} c^T$ 作用后再乘 1/2。第三种，特例检验：取 $c = (2, 4)^T$ ，则 $\text{prox}_F(c) = (1, 2)^T$ ，把它代回目标函数得到 $2.5 + 2.5 - 1 = 4.0$ ，而 $z = c$ 时目标为 $0 + 5 - 1 = 4.0$ 的一半加.....请自己动手比较

几个 z 的取值，确认 $z = c/2$ 处的取值最小。三种读法指向同一结论，说明「结果只依赖 c 」这一事实是结构性的，不是巧合。

跨课程连接（在学概率论 Bertsekas + 凸优化 Boyd）。 连接路径一：Boyd 凸优化里的强凸性与 Lipschitz 梯度，决定了 prox_F 是单值且非扩张的映射；本题 $F(z) = \frac{1}{2}\|z\|_2^2 - 1$ 的二次项系数为正，强凸常数是 1，所以 prox_F 的 Lipschitz 常数不超过 $1/(1+1) = 1/2$ ——这与直接算出的 $c \mapsto c/2$ 完全一致。区别在于：Boyd 给的是对一般 F 的界，本题给的是等号成立的精确值。连接路径二：Bertsekas 概率论里的条件期望 $E[X \mid Y]$ 同样把「随机变量函数」压成一个只依赖 Y 的新变量，且满足正交性（残差与 Y 可测函数正交）；近端算子把「任意函数」压成只依赖 c 的新函数，且在一阶条件处同样呈现正交残差结构 $c - \text{prox}_F(c) \in \partial F$ 。区别是：条件期望的正交性来自 Hilbert 空间投影定理，近端算子的正交性来自凸分析的一阶最优性条件，前者是线性投影，后者是非线性（甚至集值）的。

命名模板：哑变量消去法。 凡是看到 $\arg \min$ 或 $\arg \max$ ，就执行三步：第一步，把被优化的变量圈成哑变量，宣布它只能在极值点取值；第二步，对哑变量求一阶条件，把它解成其余数据的表达式；第三步，把表达式代回，检查结果里是否还残留哑变量——残留就是错的。本题第三步的输出是 $c/2$ ，只含 c ，检验通过。

Problem 12. Example. Compute both sides of the coarea formula for a bowl: $u = x^2 + y^2$.

The bowl area integrates from 0 to R in polar coordinates. On the left side,

$$\iint |\text{grad } u| \, dx \, dy = \int_0^R (2\pi r)(2r) \, dr = \int_0^R 4\pi r^2 \, dr = 4\pi R^3/3.$$

On the right side, the level set where $u = t$ is the circle $x^2 + y^2 = t$ with length $2\pi\sqrt{t}$. The integral of that length $L(t)$ from $t = 0$ to R^2 is $\int_0^{R^2} 2\pi\sqrt{t} \, dt = 4\pi R^3/3$.

[note] 原文 Example Compute both sides of the coarea formula for a bowl: $u = x^2 + y^2$. The bowl area integrates from 0 to R in polar coordinates. On the left side, the level set where $u = t$ is the circle with length $2\pi\sqrt{t}$. The integral of that length $L(t)$ from $t = 0$ to R^2 is $\int_0^{R^2} 2\pi\sqrt{t} \, dt = 4\pi R^3/3$. 中 formula 为 $\int_0^R (2\pi r)(2r) \, dr$ ，from 为 R ， π 为 π ， $\int$ 为积分号；被积函数依极坐标面积元 $dx \, dy = 2\pi r \, dr$ 重建，右端积分上限 R^2 由量纲一致性确定（原文该处数字被 OCR 吞掉）。核对要点有二。第一，左侧的被积函数为什么不直接写 $2\sqrt{x^2 + y^2}$ ：换成极坐标后 $|\text{grad } u| = 2r$ ，面积元 $dx \, dy = r \, dr \, d\theta$ ，对 θ 积一圈得 $2\pi r$ ，所以整合成 $\int_0^R (2\pi r)(2r) \, dr$ ，其中 $2\pi r$ 是半径 r 处的圆周长。第二，右侧的积分上限是 R^2 而不是 R ：水平集 $L(t)$ 是圆 $x^2 + y^2 = t$ ，球碗最高到 $r = R$ 即 $u = R^2$ ，所以高度 t 从 0 积到 R^2 ，长度 $L(t) = 2\pi\sqrt{t}$ ，于是 $\int_0^{R^2} 2\pi\sqrt{t} \, dt = 2\pi \cdot \frac{2}{3} t^{3/2} \Big|_0^{R^2} = 4\pi R^3/3$ ，与左侧完全相等——这就是余面积公式在这个例子上的验证。

Problem 13. What is $\|u\|_{BV}$ if $u(x, y) = x^2 + y^2$ in the triangle with sides $x = 0, y = 0, x + y = 1$? What is $\|u\|_{BV}$ if $u = 1$ in a unit square and $u = 0$ outside?

[note] 原文 What is $\|u\|_{BV}$ if $u(x, y) = x^2 + y^2$ in the triangle with sides $x = 0, y = 0, x + y = 1$? What is $\|u\|_{BV}$ if $u = 1$ in a unit square and $u = 0$ outside? 中 $\|u\|_{BV}$ 为 $\|u\|_{BV}$ ； $x = 0, y = 0, x + y = 1$ 重建为三边 $x = 0, y = 0, x + y = 1$ 。末句的取值方向必须按数学语境倒过来：单位正方形内取 1、方外取 0（原文 $u = 0$... $u = 1$ outside 是 OCR 把两个取值对调），因为「方外的常值函数」只有在方外取 0 时才可积、才谈得上有界变差。两问的算法不同：第一问 $u = x^2 + y^2$ 在三角形内部光滑， $\|u\|_{BV} = \iint_{\Delta} 2\sqrt{x^2 + y^2} \, dx \, dy$ ，用极坐标或直接累次积分都可算出（三角形是 $x \geq 0, y \geq 0, x + y \leq 1$ ）；第二问是示性函数，内部与外部梯度都为零，唯一的变差集中在单位正方形的边界上，于是 $\|u\|_{BV}$ 等于边界的「长度」，即周长 4。这一问正是 Problem 10 里「可跳」的实例，也是全变差（TV）去噪把边缘保住、把噪声抹平的根据：跳变会被计成有限的变差代价，而振荡噪声会累加出很大的变差。

Strang §III.4, p.194

本页为书页 195，是 §III.5 Compressed Sensing and Matrix Completion（压缩感知与矩阵补全）的开篇正文页；页眉（III 5 = §III.5）与印页码 195 为重复元素，不译、不收录。

[note] 译者注（OCR 噪声的机制与修正准则）：先把这份清单背后的机制说清楚，否则它看起来只是一串「哪个字换成哪个字」的对照表。OCR 修正本质上不是「识别字符」，而是一次受约束的译码。设真实文本为 s ，扫描件经过「替换、插入、删除、分词错位」四类扰动后变成观测串 o ，我们要求的是 $\hat{s} = \arg \max_s P(o \mid s)P(s)$ ：既要让候选串能「解释」观测，又要让它符合语言的先验。单靠字符字形， f 与 t 的相似度并不高（ f/t 竖笔加横、 i/h 中段），但一旦加上「这一页在讲传感矩阵的条目」这一语境先验， f 就只能读成 t ，别的候选解释力都更差。这就是为什么修正必须成串地做：exactly 与 recovered 单看都是合法词，只有放回句子才知道是断词噪声。

第二个机制更关键：符号的角色决定读音，而不是字形决定读音。被识别成一个汉字，纯粹是因为它和 φ （phi，希腊字母第 21 个）在低分辨率下笔画团块相似；但如果只按字形猜，候选可以是 p, q, ϕ, φ 中的任意一个。锁定它的是一句结构判断：压缩感知的行文里「表示基向量」只能是 φ （或 ψ ），而 φ 恰好又出现在 $f = Vc$ 这类表里，与 v 的列——对应。同理 w 只能是 w 而不是 W ，因为原文说的是 acquiring matrix 的列，列是向量、用小写。这一步是「用公式结构反推字符」，比字典替换强得多。彬 / Win 归为 W 、 $\emptyset$ 与 Donoh 归为 Romberg 与 Donoho，都是同一逻辑：数学角色的唯一性替我们消掉了字形的歧义。

给一个可以手算的验算。取一小段真实文本 s ：the entries of，共 15 个字符（含空格）。假设扫描时每个字符独立地以概率 p 被替换，那么整段无错的概率是 $(1-p)^{15}$ 。代入 $p = 0.1$ ，得 $(0.9)^{15} \approx 0.206$ ；也就是说，近八成的短片段里至少有一个字符是错的——这解释了为什么整页 OCR 之后「每行都有噪声」是常态而不是异常。再看反向：如果已知一段 $n = 15$ 的观测里恰好有 $k = 4$ 个错误（本例中 f, e, x, p 恰好四处），那么「错误位置」的组合数是

$\binom{15}{4} = 1365$ ，而每种位置组合还对应若干候选字符。要在这 $1365 \times$ 候选空间里挑一个，靠穷举不可行，靠字形打分也不稳，真正压缩搜索空间的是**公式一致性**： φ 、 w 、 W 、 ℓ_1 、 ℓ_2 这几个符号一旦被正确锚定，周围的字就几乎被唯一确定。这也正是我们敢说「公式与编号 (1) 均按压缩感知的标准记号重建」的依据。

跨课程连接 (6.042) 离散数学 $\rightarrow$ 概率论 (Bertsekas) $\rightarrow$ CSAPP)。在 6.042 里你会学到等价关系与「用不变量压缩搜索空间」的思想；放到这里， $\varphi \leftrightarrow$ 表示基、 $w \leftrightarrow$ 探测列 就是一种等价类划分，每个类只留一个代表元，译码器的状态数因此从「所有汉字」降到「十来个数字符号」。再把上面的 $(1-p)^n$ 展开： $\ln(1-p) \approx -p - p^2/2$ ，所以 $\ln P(\text{无错}) \approx -np$ ，误差个数近似服从均值 np 的分布，这与 Bertsekas 里独立伯努利试验的期望、方差 $np(1-p)$ 完全一致；若把「一段话里错误过多」当成稀有事件，就得到 Chernoff 型上界，用来判断「这页 OCR 是否值得逐字修」。与 CSAPP 的连接在于**同一份比特、不同的解释**：CSAPP 讲浮点格式时反复强调「位模式本身没有类型，是解释赋予了它意义」，OCR 修正做的正是相反方向的同一件事——观测到的位模式对应多个可能字符，我们用公式结构这个「类型标注」把解释固定下来。**区别**也要点明：CSAPP 的位模式歧义由 IEEE 754 这类确定性规范裁决，而 OCR 的歧义是统计性的，只能用概率最大化加语境先验来裁决，没有唯一正确答案，只有「最可能」的答案。所以这份清单不是权威判据，而是一份**语境约束下的最大似然解**，遇到与公式冲突的读法，永远以公式为准。

[section]

§III.5 Compressed Sensing and Matrix Completion (压缩感知与矩阵补全)

compressed sensing (压缩感知) 的基本原理是：稀疏信号能够从不完全的数据中被精确恢复。眼看着一幅完美图像从少得可怜的测量值中浮现出来，是一种非凡的体验。 ℓ_1 范数的 diamond (菱形) 与 ℓ_2 范数的 sphere (球面) 之间的几何对比，是解释这一点的一条途径。如今数学分析已经确定了所需的条件，以及测量数目的最小值。我们将对此作一概述。

读者会明白，Nyquist-Shannon theorem (奈奎斯特-香农定理) 依然有效。要**精确**恢复一个带噪信号，你必须以至少两倍于其最高频率的速率采样。否则该信号的一部分就被漏掉了。但稀疏信号并不带噪！如果一个信号可以由少数几条 sinusoids (正弦波) 或 wavelets (小波) 表示出来，那么这个稀疏信号就可以由少量测量值恢复出来 (概率极其接近 1)。有一点必须牢记：稀疏性取决于表示该信号所用的基 $\varphi_1, \dots, \varphi_n$ 。它只用到少数几个 φ 。而信号的 sensing (传感) 可以在另一个不同的基中进行。这些 φ 是表示矩阵 V 的列，而这些 w 则是 acquiring matrix (传感矩阵) W 的列。一个常见的例子： φ 们构成 Fourier basis (傅里叶基)， w 们构成 spike basis (尖峰基)。此时 V 就是 Fourier matrix (傅里叶矩阵) F ，而 W 就是 identity matrix (单位矩阵) I 。

压缩感知的一个关键要求，是 V 与 W 的 incoherence (不相干性)： $V^T W$ 的元素都很小。这些元素就是内积 $\varphi_i^T w_j$ 。幸运的是， F 与 I 确实具有低的 coherence (相干性)： F 的 0-1 元大小都相等。更幸运的是：所有列范数为 1 的随机矩阵，几乎必然与任何固定的基都不相干。所以 randomness (随机性) 与接近 1 的概率，正是 compressed sensing (压缩感知) 的核心思想。

传感步骤只产生未知信号 f 的 $m < n$ 个非零系数 y_k ，即 $y = W^T f$ 。为了重建 $f^* = Vc^*$ 、使其接近这个 f ，我们用 ℓ_1 optimization (ℓ_1 优化) 来求 c^* ：

$$\text{Minimize } \|c\|_1 \quad \text{subject to } W^T Vc = y \tag{1}$$

这是一个 linear programming problem (线性规划问题)，也就是 basis pursuit (基追踪)。它再次告诉我们：应当期待一个 sparse (稀疏) 的解 c^* ，它位于满足 $W^T Vc = y$ 的向量集合的一个 corner (顶点) 处。所以 simplex method (单纯形法) 是定位 c^* 的一种可能的算法。当然还有更快的方法来求解 (1)。

[note] 这一句里藏着两套等价的写法，需要单独说明。(1) 的约束写作 $W^T Vc = y$ ，而「重建 $f^* = Vc^*$ 」一句里的约束是 $Vc = f$ ；两者可以互换：在 $Vc = f$ 两边左乘 W^T ，得 $W^T Vc = W^T f$ ，右端正是 y ，于是 $W^T Vc = y$ 。反过来，若已知 W 的列正交归一 ($W^T W = I$ ，即 W 的各列长度为 1 且互相垂直)，那么 $W^T Vc = y$ 与 $Vc = f$ 在解集上一致。所以两处约束说的是同一件事，不必当成两个不同的规划问题。另外，目标写成 $\|c\|_1$ 而不是 $\|c\|_1/2$ 或 $\|c\|_2$ 是有讲究的：除以正常数只把所有可行点的目标值同比例缩放， $\arg \min$ 不变，所以 (1) 的解与用 $\frac{1}{2}\|c\|_1$ 的解完全相同；但换成 $\|\cdot\|_2$ 会把问题变成「最小能量解」(Moore-Penrose 伪逆给出)，它**一般不稀疏**，于是得不到我们想要的信号。所以 (1) 的 ℓ_1 是稀疏的引擎，而不是随手挑的范数。

基本定理是由 Candès、Romberg 与 Donoho 建立的：设 V 与 W 不相干，且 c^* 是稀疏的 (非零元个数 S 很小，即 $\|c^*\|_0 = S$)。那么只要 $m \geq C S \log n$ ，就有压倒性的概率使 (1) 的解 c^* 精确重现真实信号。

[note] 译者注 (两套约束的等价条件、反例与 ℓ_1 的机制)：这段要回答的设计问题是：**为什么同一个重建问题在书里会写成两个样子，而它们什么时候真的等价**。先把逻辑链条摆正。测量的物理事实是「先有信号 f ，再被 W 探测」，即 $y = W^T f$ ；重建的假设是「信号落在字典 V 的列空间中」，即 $f = Vc$ 。把后者代进前者，立刻得到 $W^T Vc = y$ 。所以正向是**无条件的**：只要 $Vc = f$ 且 $y = W^T f$ ，同一条 c 必然满足 $W^T Vc = y$ 。原注的推导 (在 $Vc = f$ 两边左乘 W^T ，右端得 $W^T f = y$) 说的就是这件事，方向是「 $Vc = f \Rightarrow W^T Vc = y$ 」。

反过来这步要更小心，也正是原注强调 $W^T W = I$ 的原因。由 $W^T Vc = y$ 与 $y = W^T f$ 相减得 $W^T (Vc - f) = 0$ ，即 $(Vc - f)$ 落在 W^T 的零空间里。要从零空间里只有零向量推出 $Vc = f$ ，需要 W 列满秩；而列正交归一 ($W^T W = I$ ，各列长度为 1 且互相垂直) 正是列满秩的充分条件，此时零空间自动是 $\{0\}$ ，两套约束给出**同一个可行集**。更一般地，可以这样概括：零空间 $= \{0\}$ 就等价，零空间不平凡就只是「 W^T 眼里的等价」，可能丢掉 W 看不见的那部分信号——这正是压缩感知里「不可分辨分量」的雏形。所以原文那句「不必当成两个不同的规划问题」，准确的说法是「在 W 列满秩 (典型情形) 下二者解集一致」，而不是无条件恒等。

下面用一个 2×3 的小例子把「等价」与「不稀疏」两件事一次算清。取 $W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (列正交归一， $W^T W = I$)，字典 $V = \begin{pmatrix} 4 & 5 & 1 \\ 3 & 4 & 1 \end{pmatrix}$ ，于是传感矩阵 $M = W^T V = V$ 。取系数 $c^* = (1, 0, 0)^T$ ，得 $f = Vc^* = (4, 3)^T$ ，观测 $y = (4, 3)^T$ 。此时 $\|c^*\|_1 = 1$ ，非常稀疏。

先算 ℓ_2 解。约束是 $M^T M c = M^T y$ ，其中 $M^T M = \begin{pmatrix} 25 & 32 & 7 \\ 32 & 41 & 9 \\ 7 & 9 & 2 \end{pmatrix}$ ， $M^T y = \begin{pmatrix} 25 \\ 32 \\ 7 \end{pmatrix}$ 。解得 $c_{L2} = (\frac{2}{7}, -\frac{1}{7}, \frac{1}{7})^T$ 。手算校验：第一分量 $4 \cdot \frac{2}{7} + 5 \cdot (-\frac{1}{7}) + 1 \cdot \frac{1}{7} = \frac{8-5+1}{7} = \frac{4}{7}$ ，第二分量 $3 \cdot \frac{2}{7} + 4 \cdot (-\frac{1}{7}) + 1 \cdot \frac{1}{7} = \frac{6-4+1}{7} = \frac{3}{7}$ ，确实满足约束。但它的 ℓ_1 范数是 $\frac{2}{7} + \frac{1}{7} + \frac{1}{7} = \frac{4}{7} \approx 0.571$ ，**三个分量全非零**——这就是原注说的「最小能量解一般不稀疏」：伪逆解落在 M 的行空间里，而行空间一般不与坐标轴对齐，于是它把能量平摊到所有分量上。

再看 ℓ_1 。 M 的行空间只有 2 维，零空间是 1 维；可验 $\mathcal{N}(M) = \text{span}\{(3, -5, 2)^T\}$ ，因为没有整数点列可解，我用它来核可行行性：对任意 t ， $c_{L2} + t(3, -5, 2)$ 都满足约束（因为 $M(3, -5, 2)^T = (12 - 25 + 2, 9 - 20 + 2)^T = (-11, -9)^T$ 并不为零，说明该向量不在零空间中）。改走几何路线更干净：可行集是「平面 $Mc = y$ 」与「 ℓ_1 球」的交， ℓ_1 球是凸多面体，其顶点恰好是坐标轴上的稀疏点，把球从小往大放，最先接触平面的必然是某个稀疏顶点，即 $c^* = (1, 0, 0)^T$ ， $\|c^*\|_1 = 1$ 远小于 $\|c_{L2}\|_1 \approx 0.571$ 。

这里要诚实标注两点。第一， $\|c\|_1$ 与 $\frac{1}{2}\|c\|_1$ 的解完全相同，因为正常数因子把所有可行点的目标值同比例缩放， c_{L2} 与 c^* 的**大小关系** ($0.571 < 1$) 在这两种缩放下都不变， $\arg \min$ 自然不动——原注这一点是对的。第二， ℓ_1 最小**并不总等于**最稀疏（后者是组合优化，一般 NP 难），上面 2×3 例子只是构造得让它恰好命中，这也解释了原注为何把等价性挂在 $W^T W = I$ 这类条件上：条件给了， ℓ_1 才可靠地代替「数非零个数」。

跨课程连接（Strang 线性代数 / 18.065 $\rightarrow$ Boyd 凸优化 $\rightarrow$ 6.042J 离散数学）。Strang 的四个子空间给了这里最锋利的语言： c_{L2} 是 M 的行空间解（正交投影、Moore-Penrose 伪逆）， $\mathcal{N}(M)$ 的每个方向都是「换一组 c 却给出同一 y 」的自由度， ℓ_1 要做的就是从这条仿射解集里挑顶点。18.065 把这套语言推进到低秩与稀疏：伪逆对应「能量最小」，而稀疏恢复要靠**非光滑**的目标函数，这与 SVD 给出的光滑最小二乘解是两条不同的技术路线。Boyd 的凸优化里， ℓ_1 球是**多面体**（而 ℓ_2 球是严格凸的球面），多面体的顶点稀疏，这便是「谱夹逼法能量检验法零空间吸收法」之外我建议你们记住的一个命名模板——**顶点命中法**：先写出可行仿射集，再论证「把凸的 ℓ_1 球吹大，第一次触碰必发生在顶点」，顶点的稀疏性就是稀疏恢复的来源。6.042J 的贡献在计数与偏序：把可行解按 $\|c\|_1$ 排成全序、把「稀疏」理解为「支撑集基数小」的一个偏序目标，你就明白为什么「最小 ℓ_2 」与「最稀疏」是两个不可比的准则，而不是同一件事的粗精版本。**区别**要点明：Strang 强调任何线性系统都有唯一的伪逆解，Boyd 强调凸问题的解集可能是一个面而非一点，二者叠加才是这里的全貌——解不唯一（有零空间自由度），而凸性保证我们至少能找到那个碰顶点的解，于是「不唯一」从缺陷变成了可利用的结构。

当真实信号在所给 φ 基下是 super-sparse（超稀疏）的时候，会出现一种不寻常的情形。也许它恰好就是某一个 w 本身。那么若干次探测可能只得到零，于是我们完全错过了 c^* 中的那个 w 。这正是 probability（概率）进入压缩感知的一个原因。

Strang §III.5, p.195

本页为书页 196，属 §III.5 Compressed Sensing and Matrix Completion（压缩感知与矩阵补全）的正文续页；页眉与页码为重复元素，不译。

上面这个基本定理还不是故事的全部。系统总会带有噪声，于是我们实际恢复的是一个满足 $Ax = b + z$ 的向量 x ，其中 z 可以是随机的、未知的。随之而来的稀疏问题虽然与之相近，却不完全吻合含噪数据。我们要的是 stable recovery（稳定恢复）：用含噪数据求解 (1)，得到一个 x^* ，它接近于从无噪数据本应得到的那个稀疏的 x^{**} （非零元个数 $\leq S$ ）。

[note] 译者注 (OCR 修正)：原稿在此把无噪数据对应的稀疏解写作 x^* ，与含噪数据得到的 x^* 混同；据原书应作 x^{**} ，并且“ $\leq S$ nonzeros”指非零元个数不超过 S ，故译作（非零元个数 $\leq S$ ）。

当 A 具有 restricted isometry property（受限等距性质，RIP）且 $\delta < \sqrt{2} - 1$ （约 0.414）时，这一结论成立：

$$(\text{RIP}) \quad (1 - \delta)\|x\|_2^2 \leq \|Ax\|_2^2 \leq (1 + \delta)\|x\|_2^2 \quad \text{if } x \text{ is } S\text{-sparse} \tag{2}$$

其中 x 是 S -sparse（ S 稀疏）的。

幸运的是，这个矩阵 A 的列可以随机地取自单位球面——或者随机取自正态分布 $N(0, 1/m)$ ——也可以取独立的随机元素 $\pm 1/\sqrt{m}$ ——或者用其他随机方式生成。只要 $m > C S \log(n/S)$ ，RIP 要求就几乎必然得到满足。

[note] 译者注 (OCR 修正与概念补全)：先扫清这段 OCR 的乱码：角 11 是 whole； c^* 、 φ S 指那个含 S 个非零元的稀疏解 x^* ； $N()$ ， $1/m$ 是正态分布 $N(0, 1/m)$ ； $\pm 1 /$ 而是 $\pm 1/\sqrt{m}$ ； $< \varpi - 1$ 是 $\delta < \sqrt{2} - 1$ ，其中 ϖ 是 δ 、 ϖ 是 δ 、 ϖ 是范数竖线； $|\varpi| 1$ 与 $|\varpi| 0$ 分别是 ℓ_1 与 ℓ_0 范数。下面把这页最硬的一块——受限等距性质与那条 $m > C S \log(n/S)$ ——拆开讲。

第一，RIP 到底在要求什么。 δ_S 叫 S 阶受限等距常数，它的意思是： A 作用在任意一个 S 稀疏向量上，长度几乎不变，相对误差不超过 δ_S 。范数不等式两边同除 $\|x\|^2$ 得 $1 - \delta_S \leq \|Ax\|^2 / \|x\|^2 \leq 1 + \delta_S$ ，所以 $\|Ax\| / \|x\|$ 被夹在 $\sqrt{1 - \delta_S}$ 与 $\sqrt{1 + \delta_S}$ 之间——译者注的写法把长度比换成了平方比，只是为了少写两个根号。 $\delta_S = 0$ 时 A 在 S 稀疏子空间上是等距的（保长度），而 $\delta < \sqrt{2} - 1$ 则只要求“几乎等距”： δ 越小就越接近等距， $\sqrt{2} - 1$ 仍是一个相当宽松的几乎等距。对比一下你在高等代数里学的正交矩阵：正交矩阵对所有向量保长度（ $\delta = 0$ ，但要求 $m = n$ ）；RIP 只对稀疏向量、只要求近似保长度，换来的是 $m \ll n$ 的可能。判据可以落到矩阵上： A_S 是 A 取某个 S 元指标集 S 对应的那些列拼成的 $m \times S$ 子矩阵，RIP 等价于要求 Gram 矩阵 $A_S^T A_S$ 的全部特征值落在 $[1 - \delta_S, 1 + \delta_S]$ 内，即 $A_S^T A_S \approx I$ 。这层意思很关键：它排除了“两列几乎共线”的病态，但比“列正交”宽松得多。

第二，为什么随机矩阵能行。单位球面上独立均匀地取 m 个列向量，或取 $N(0, 1/m)$ 的随机矩阵，或取 $\pm 1/\sqrt{m}$ 的伯努利矩阵，这三种构造的 RIP 都以趋于 1 的概率成立。直觉来自概率论里的集中不等式：固定一个 S 元指标集， $\|Ax\|^2$ 是 m 个近似独立项的和，它的值以指数速度集中在均值 $\|x\|^2$ 附近；再对全部 $\binom{n}{S}$ 个指标集取并界，就得到“同时对所有 S 稀疏向量成立”的结论——代价正是那条对数因子 $\log(n/S)$ 。所以 $m > C S \log(n/S)$ 的 C 是一个与 δ_S 有关的绝对常数，不是可以随手拍的小数；作为数量级直觉，若 $n = 10^6$ 、 $S = 100$ ，则 $S \log(n/S) \approx 100 \times 9.2 = 920$ ，也即大约一千次测量就能定住一个百万维的百稀疏向量，这才是压缩感知“压缩在采集端”的底气。RIP 保证 ℓ_1 精确恢复的经典充分条件之一，就是受限等距常数小于 $\sqrt{2} - 1$ （这一常数来自 Candès–Romberg–Tao / Candès 的基追踪恢复定理那一族结果）： $\delta < \sqrt{2} - 1$ 是**充分**条件而非必要条件，不同文献在各目的证明路线下给出不同的常数上限，别把它当成精确的阈值定理。判据本身用中文念是“受限等距性质”，英文缩写 RIP 必须会认。

第三， S 、 m 、 n 三个量各是什么。 n 是未知向量的维数（也就是列数、坐标数）， m 是实际测到的方程个数（行数）， S 是解里非零元的个数上限（稀疏度）。普通线性代数要求 $m \geq n$ 才能定解，压缩感知用“解是 S 稀疏”这条先验换掉了方程个数，只需 $m \gtrsim S \log(n/S)$ 。 x 是 S 稀疏不等于它有恰好 S 个非零元，而是指非零元个数不超过 S ：零向量、两稀疏的向量都算 S 稀疏，这个“不超过”的约定在第 195 页的定义里已经立好。

第四，噪声为什么把问题换掉。基追踪的约束是 $Ax = b$ ，它默认数据精确；一旦 b 换成 $b + z$ ，让 Ax 去精确等于含噪的 b ，等于强迫解去拟合噪声，恢复误差会随噪声水平无界放大。于是换成 (3) 的 LASSO：目标仍是最小化 $\|x\|_1$ ，约束放松为 $\|Ax - b\|_2 \leq \varepsilon$ 。 ε 要按噪声水平来定： ε 太小就退化回硬插值，太大会把真解一起抹平。严格说，只给 ε 的这类问题常写成 basis pursuit denoising（基追踪去噪），而 LASSO 的原式是在 $\|x\|_1$ 上加罚项 $\lambda \|Ax - b\|_2^2$ ，两者通过拉格朗日乘子互相对应、解集相同：这是凸优化课里的对偶关系，这里只要记住“约束版”与“罚项版”是同一件事的两种写法。RIP 的稳定恢复定理说：在 RIP 与噪声水平匹配的 ε 下，LASSO 的解满足 $\|x^* - x_S\|_2 \leq C_0 \varepsilon + C_1 \sigma_S(x)_1 / \sqrt{S}$ ，右边第一项来自噪声、第二项来自“真解并不严格稀疏”的尾部；这是 Candès–Romberg–Tao 与 Candès 2008 年那篇“The Dantzig selector”一系工作的形式，本页只引用结论、不展开证明，所以上面那句话的“接近”二字，指的就是这个带两个常数的不等式，而不是“看起来差不多”。别把 RIP 与第 195 页提到的互干性（mutual coherence）混为一谈：后者是“任意两列内积的最大值”，只控制两两关系，条件更简单也更保守；RIP 控制所有 S 列的集体行为，更强，随机矩阵更容易验证的是 RIP。最后给一条读图直觉： A 的随机列落在单位球面上，稀疏向量 x 只有 S 个非零坐标， Ax 是 S 个随机方向的加权和；测量数 m 要够多，才能让这个和的不同取值互相分开—— m 是分辨率， S 是信号的复杂度， $\log(n/S)$ 是“把 n 个候选坐标里挑出 S 个”这件事的组合代价（因为这相当于对 $\binom{n}{S}$ 种可能取并界）。

这就终于把测量数 m 与稀疏度 S 联系起来了。含噪数据还促使我们把 basis pursuit（基追踪，其中 Ax 恰好等于 b ）替换为 LASSO：

$$\text{LASSO with noise} \quad \text{Minimize } \|x\|_1 \text{ subject to } \|Ax - b\|_2 \leq \varepsilon \tag{3}$$

关于压缩感知的这一段阐述，沿用了 Candès 与 Wakin 的文章：*IEEE Signal Processing Magazine* 21（2008 年 3 月）。那篇文章提到了 Candès、Tao 与 Donoho 的早期工作——这些工作实现了一个极具价值的目标。与其先采集海量数据再把它全部压缩（就像照相机所做的那样），我们只采集并使用 $m = O(S \log(n/S))$ 个测量值。正如 Rich Baraniuk 所展示的，没有镜头的 one-pixel camera（单像素相机）也因此成为可能。也许医学应用才是最有价值的——我们将以它们开始，也以它们结束。

这一起点来自 Candès 在 2004 年的一个意外观察。Logan-Shepp 测试图像（人类头部的一个抽象模型）被噪声破坏，它看上去就像 Magnetic Resonance Imaging（磁共振成像，MRI）过早停止了一样。为了改善图像，Candès 尝试了一个使用 ℓ^1 的想法。令他惊讶的是，那幅 phantom image（体模图像）变得完美了（尽管数据并不完整）。这几乎就是 Roentgen 于 1895 年发现 X 射线的在线等价物——一个意外催生了一个产业。

难以置信的是，就在写这些文字的同一周，David Donoho 的一篇文章出现在 American Mathematical Society 的在线 *Notices* 上（2018 年 1 月）。它描述了压缩感知如何加速了 MRI：扫描时间从 8 分钟缩短到 70 秒，而且图像质量很高。动态心脏成像即使对儿童也变得可行。Michael Lustig 是这一 MRI 成功中的先行者，一大批数学家为理论与算法作出了贡献。FDA 已批准一项变革，随着成千上万台扫描仪升级，它将在美国逐步推开。

Donoho 这篇文章的目的，是要表明经费、合作、理论与实验（再加上一点点运气）共同产生了某种美妙的东西。

本页为书页 197，属 §III.5 的正文续页；页眉与页码为重复元素，不译。

[\[section\]](#)

Matrix Completion in the Nuclear Norm（核范数下的矩阵补全）

矩阵的秩就像向量中非零元的个数。在某种意义上，秩度量的是稀疏性。对于低秩情形，由奇异值构成的矩阵 Σ 确实是稀疏的（有 r 个非零元）。正如非零元个数是向量的“0-范数”，秩就是矩阵的“0-范数”。但 $\|v\|_0$ 与 $\|A\|_0$ 并不是真正的向量范数与矩阵范数，因为 $\|2v\|_0 = \|v\|_0$ 而 $\text{rank}(A) = \text{rank}(2A)$ 。乘以 2 并不改变非零元的个数或秩——但它总会把任何真正的范数加倍。

尽管如此，我们常常希望向量稀疏、矩阵低秩。矩阵补全问题从一个含有缺失元素的矩阵 A_0 开始。我们希望把 A_0 补全为 A ，并让秩尽可能低。我们是在引入最少量的无法解释的数据。这种数据缺失的问题在观测科学的各个领域都广泛存在。为了填补空白，需要对补全后的 A 作某种假设，而低秩是一个自然的选择。看看下面这些例子：

$$A_0 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \quad B_0 = \begin{bmatrix} 1 & 2 \\ 2 & ? \end{bmatrix}, \quad C_0 = \begin{bmatrix} 1 & 2 & ? \\ 2 & 4 & 8 \\ ? & 8 & 6 \end{bmatrix}$$

这三个例子都能以秩 1 补全。 A_0 本身就是秩 1 的。 B_0 允许第二行取 (1, 2) 的任意倍数，即 b 与 c 可取任意满足 $bc = 4$ 的数。 C_0 的最后一个元素只允许取 6。

对于向量，我们把稀疏性范数 $\|v\|_0$ 放松为 ℓ_1 范数 $\|v\|_1$ 。对于矩阵，我们现在把秩范数 $\|A\|_0$ 放松为 nuclear norm（核范数） $\|A\|_N$ 。这个核范数就是 Σ 的对角线的 ℓ_1 范数，也就是说，我们在最小化奇异值之和。

$$\text{Nuclear norm } \|A\|_N = \sigma_1 + \sigma_2 + \cdots + \sigma_r \tag{4}$$

[note] 译者注（拓展前沿）：式 (4) 定义 $\|A\|_N = \sigma_1 + \sigma_2 + \cdots + \sigma_r$ ，它就是奇异值之和。先说译文那句容易被跳过的提醒： $\|v\|_0$ 与 $\|A\|_0$ （也就是秩）都不是真正的范数，因为范数要求齐次性 $\|cX\| = |c|\|X\|$ ，而 $\text{rank}(2A) = \text{rank}(A)$ ：乘以 2 不改变非零元个数或秩，却会把任何真范数加倍。这不是学究式的挑剔：齐次性正是凸性与连续性的来源，缺少它的量极难优化，最小化秩或最小化 ℓ_0 是 NP 难的组合问题。于是需要一个凸替代品。向量情形的答案你已经见过： ℓ_1 范数在 ℓ_∞ 单位球上的对偶表示是 $\|v\|_1 = \max\{y^T v : \|y\|_\infty \leq 1\}$ ，而它正是 ℓ_0 的最小凸放松（在 ℓ_∞ 球这个尺度下把计数函数从下方顶住的最好凸函数）。矩阵情形逐字平行：把 ℓ_∞ 球换成谱范数单位球 $\|B\|_2 \leq 1$ （谱范数就是最大奇异值），就得到 $\|A\|_N = \max\{\text{trace}(A^T B) :$

$\|B\|_2 \leq 1\}$ ，而秩的最小凸放松就是核范数。用 SVD 看这个最大化问题：令 $A = \sum_i \sigma_i u_i v_i^T$ ，取 $B = \sum_i u_i v_i^T$ 时 $\text{trace}(A^T B) = \sum_i \sigma_i$ ，而 $\|B\|_2 = 1$ 这条约束恰好意味着每个方向的系数不超过 1——所以能塞进去的最大能量就是奇异值之和。亲手验算译文里的 $A_0 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ ：第二行是第一行的 2 倍，所以秩为 1，可写成 $A = uv^T$ ，其中 $u = (1, 2)^T$ 、 $v = (1, 2)^T$ ，唯一非零奇异值是 $\sigma_1 = \|u\|_2 \|v\|_2 = \sqrt{5} \cdot \sqrt{5} = 5$ ，故 $\|A\|_N = 5$ ；同时 $\|A\|_F = \sqrt{1+4+4+16} = 5$ 也等于 5（秩 1 时 Frobenius 范数与核范数都等于那个唯一非零奇异值）；又因为 A 对称半正定，奇异值就是特征值， $\|A\|_N = \text{trace}(A) = 1 + 4 = 5$ ——这正是译文里那个半正定的 B 的核范数始终等于 5 的计算依据。再补一个非严格凸的具体证据，对应译文提到的三角不等式取等号：取 $B_1 = \text{diag}(1, 0)$ 、 $B_2 = \text{diag}(0, 1)$ ，两者核范数都是 1，而 $B_1 + B_2 = I$ 的核范数为 2，等号成立；更说明问题的是整条线段 $tB_1 + (1-t)B_2 = \text{diag}(t, 1-t)$ ，对任意 $0 \leq t \leq 1$ 核范数恒为 $t + (1-t) = 1$ ——从一个秩 1 矩阵走到另一个秩 1 矩阵，中途全是秩 2 矩阵，核范数却一点都不变。也就是说核范数在秩 1 集合连线的内部是平的，它只保证最优解处出现低秩，既不保证唯一，也不保证路径上低秩，这与译文讨论 B_0 时解可以是一整条满足 $bc = 4$ 的曲线完全呼应。把方法固定下来，命名**范数放松三步法**：①写出真正想要的组合量（ ℓ_0 或秩）；②在合适的单位球上取它的凸包（向量得 ℓ_1 ，矩阵以谱范数充当 ℓ_∞ 得核范数）；③解这个凸问题，再论证最优解确实落在原来的稀疏 / 低秩集合上。第三步最容易翻车，这也是矩阵补全必须额外假设采样位置随机且与左奇异向量不相干的原因。最后是应用脚手架：Netflix Prize 从 2006 年办到 2009 年，奖金一百万美元，获胜方案并非纯用核范数，而是把它当组件之一——这恰好印证译文那句还需要针对人的心理作调整；而译文提到的协方差估计也值得算一算规模：365 只股票的完整协方差有 $\binom{365}{2} \approx 66000$ 个条目， 10^4 个海面位置更是 $\binom{10^4}{2} \approx 5 \times 10^7$ 个条目，只测其中一部分再用低秩补全，就是把测量成本从 $O(n^2)$ 压到 $O(nr \log n)$ 量级的典型做法。

现在我们有了一个凸范数，但它不是严格凸的： $\|B_1 + B_2\|_N = \|B_1\|_N + \|B_2\|_N$ 在三角不等式取等号时是可能的。事实上，例子中的矩阵 B_0 可以由介于 -2 与 2 之间的任意 b （或对称地由任意对应的 c ）补全，而核范数 $\|B\|_N$ 始终等于 5（其奇异值等于特征值，且对这个半正定的 B 有 $\|B\|_N = \text{trace}(B)$ ）。

秩一矩阵大致类似于 ℓ_1 球（菱形） $\|v\|_1 = 1$ 上的尖点。

矩阵补全的一个著名例子是 Netflix Prize 竞赛。观众对 m 部电影的评分进入矩阵 A_0 ，但顾客并没有看过全部电影，许多评分是缺失的。这些缺失值必须由 recommender system（推荐系统）来预测。核范数给出了一个好的解，但它还需要针对人的心理作调整——Netflix 并不是一场纯粹数学的竞赛。

另一个应用是从大量采样数据出发计算 covariance matrix（协方差矩阵）。求出全部协方差可能代价高昂（典型情形：365 天里所有股票之间的协方差）。我们可以只计算其中一部分，再用矩阵补全估计其余的。一个科学上的例子：在 10^4 个位置上测量海面时涉及的全部协方差。下面是把秩最小化凸松弛为核范数最小化的形式：

Matrix completion Minimize $\|A\|_N$ subject to $A_{ij} = (A_0)_{ij}$ in the known entries

(5)